



Red Hat OpenShift AI Self-Managed 3.5

Deploy models using Distributed Inference with llm-d

Deploy and serve large language models at scale in Red Hat OpenShift AI

Red Hat OpenShift AI Self-Managed 3.5 Deploy models using Distributed Inference with llm-d

Deploy and serve large language models at scale in Red Hat OpenShift AI

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

As an administrator, you can use distributed inference to deploy and serve large language models at scale on Red Hat OpenShift AI.

Table of Contents

PREFACE	7
CHAPTER 1. DEPLOY MODELS BY USING DISTRIBUTED INFERENCE WITH LLM-D	8
1.1. ENABLE DISTRIBUTED INFERENCE WITH LLM-D	8
CHAPTER 2. DEPLOY MODELS USING THE TOPOLOGY SELECTOR	11
2.1. UNIFIED GENERATIVE MODEL SERVING	11
2.2. INFERENCE WORKLOAD DEPLOYMENT TOPOLOGY PATTERNS	12
2.3. DEPLOYMENT TOPOLOGY CONFIGURATION LABELS	14
2.4. HARDWARE REQUIREMENTS AND CONSIDERATIONS FOR LLM DEPLOYMENT TOPOLOGIES	15
2.5. ROUTER CONFIGURATIONS AND SUPPORTED TOPOLOGIES	17
2.6. SELECTING AN LLM DEPLOYMENT TOPOLOGY	18
2.7. DEPLOYING MODELS WITH THE LLM-D TOPOLOGY SELECTOR	20
2.8. CONFIGURING ADVANCED ROUTING FOR DEPLOYED MODELS	24
2.9. MANAGING TOPOLOGY CONFIGURATION TEMPLATES	25
2.10. MANAGING EXISTING TOPOLOGY CONFIGURATIONS	27
2.11. MANAGING ROUTER CONFIGURATIONS	29
2.12. EDITING ROUTER CONFIGURATIONS	31
2.13. DEPLOY DISAGGREGATED PREFILL/DECODE TOPOLOGY	33
2.13.1. Understanding the automatic scheduler configuration	35
CHAPTER 3. CONFIGURE GATEWAY API FOR DISTRIBUTED INFERENCE WITH LLM-D	37
3.1. GATEWAY DISCOVERY FOR DISTRIBUTED INFERENCE WITH LLM-D	37
3.2. ENABLE GATEWAY DISCOVERY IN THE DASHBOARD	37
3.3. SELECT A GATEWAY FROM THE MODEL DEPLOYMENT WIZARD	38
3.4. SELECT GATEWAY USING YAML	40
3.5. CONFIGURE THE INFERENCE GATEWAY ON OPENSIFT CONTAINER PLATFORM	41
3.6. CREATE A SECURE INFERENCE GATEWAY MANUALLY ON OPENSIFT CONTAINER PLATFORM	43
CHAPTER 4. CONFIGURE AUTHENTICATION FOR DISTRIBUTED INFERENCE WITH LLM-D BY USING RED HAT CONNECTIVITY LINK	45
CHAPTER 5. ENABLE AUTHENTICATION AND AUTHORIZATION FOR AN LLM INFERENCE SERVICE	47
5.1. MAKE AUTHENTICATED INFERENCE REQUESTS TO DISTRIBUTED INFERENCE WITH LLM-D	49
CHAPTER 6. VALIDATE INFERENCE WORKLOAD CHANGES WITH CONTROLLED DEPLOYMENT	52
6.1. CONTROLLED DEPLOYMENT FOR INFERENCE WORKLOADS	52
6.1.1. How the group and weight model works	52
6.1.2. Version isolation	52
6.1.3. Per-member HTTPRoute lifecycle	52
6.1.4. Observability	53
6.1.5. Controlled deployment lifecycle	53
6.1.6. Constraints	53
6.1.7. Model access patterns	53
6.1.8. Scope limitations	54
6.2. DEPLOY A CANARY VERSION OF AN INFERENCE WORKLOAD	54
6.3. MONITOR PER-VERSION INFERENCE METRICS DURING CONTROLLED DEPLOYMENT	57
6.4. PROMOTE OR ROLL BACK A CONTROLLED DEPLOYMENT	61
6.5. AUTHORIZATION MODEL FOR CONTROLLED DEPLOYMENT	63
6.5.1. Instance-level and model-level access patterns	63
6.5.2. The models virtual resource	64
6.5.3. Cross-tenant deny rule	64
6.5.4. Backward compatibility	64

6.5.5. Known limitations	64
6.5.6. Grant model-level access for inference workloads	65
6.6. CONTROLLED DEPLOYMENT API FIELDS	67
6.6.1. Spec fields for controlled deployment	67
6.6.2. Admission validation rules	68
6.6.3. Status fields for routing groups	68
6.6.4. Status conditions for routing groups	69
6.6.5. Weight semantics	69
6.6.6. Status addresses and URL patterns	70
6.7. CONTROLLED DEPLOYMENT METRICS FOR PROMOTION DECISIONS	70
6.7.1. Key metrics for cross-version comparison	70
6.8. MODEL-LEVEL RBAC ROLES FOR INFERENCE ACCESS	72
6.8.1. Aggregate ClusterRoles	72
6.8.2. Virtual resource details	73
6.8.3. AuthPolicy rules for model access	73
6.8.4. Publisher-path URL format	75
6.9. TROUBLESHOOTING CONTROLLED DEPLOYMENT	75
CHAPTER 7. VLLM ARGUMENTS REFERENCE	78
7.1. COMMON USE CASES FOR VLLM ARGUMENTS	78
7.2. CONFIGURE VLLM ARGUMENTS FOR LLM-D DEPLOYMENTS	79
CHAPTER 8. ABOUT VLLM UVICORN ACCESS LOGS IN DISTRIBUTED INFERENCE	82
8.1. CONFIGURING TARGETED VLLM ENDPOINT LOG FILTERING	83
8.2. ENABLE VLLM UVICORN ACCESS LOGS	85
8.3. EXAMPLE USAGE FOR DISTRIBUTED INFERENCE WITH LLM-D	88
8.3.1. Single-node GPU deployment	88
8.3.2. Multi-node deployment	88
8.3.3. Intelligent inference scheduler with KV cache routing	88
CHAPTER 9. TOOL CALLING THROUGH DISTRIBUTED INFERENCE WITH LLM-D	89
9.1. TOOL CALLING COMPONENT RESPONSIBILITIES	89
9.2. CONFIGURE TOOL CALLING FOR DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENTS	90
9.3. SUPPORTED TOOL CALLING PARAMETERS FOR DISTRIBUTED INFERENCE WITH LLM-D	93
9.3.1. Tool calling request parameters	93
9.3.2. Tool choice behavior in vLLM	94
9.3.3. Tool calling response objects	95
9.4. TOOL CALLING TROUBLESHOOTING FOR DISTRIBUTED INFERENCE WITH LLM-D	97
CHAPTER 10. CONFIGURE REQUEST ROUTING FOR {LLM-D}	99
10.1. SCHEDULER CONFIGURATION FOR LLM INFERENCE SERVICES	99
10.2. ENDPOINT PICKER ARCHITECTURE	100
10.2.1. Request flow through the Endpoint Picker	100
10.2.2. Complete request path	101
10.3. CONFIGURE SCHEDULER INLINE IN LLMINFERENCESERVICE	101
10.4. CONFIGURE SCHEDULER USING CONFIGMAP REFERENCES	103
10.5. IMPROVE CACHE HIT ACCURACY WITH TOKEN-LEVEL PREFIX MATCHING	105
10.6. AVAILABLE PLUGINS FOR YOUR INFERENCE WORKLOAD	107
10.7. EPP AND INFERENCE SCHEDULER METRICS FOR LLM-D	110
10.8. MANAGE EPP SCHEDULER CONFIGURATION DURING RHOAI 3.4 TO 3.5 UPGRADES	114
10.8.1. Verify active EndpointPicker scheduler configuration	116
CHAPTER 11. MANAGE MIXED WORKLOADS BY USING PRIORITY QUEUING	118
11.1. FLOW CONTROL AND PRIORITY-BASED QUEUING	118

11.1.1. How flow control works	118
11.1.2. Key flow control concepts	119
11.1.3. Configuration strategies	120
11.2. CONFIGURE FLOW CONTROL FOR DISTRIBUTED INFERENCE WITH LLM-D	121
11.3. INFERENCEOBJECTIVE CUSTOM RESOURCE REFERENCE	126
11.4. VLLM PARAMETER CONFIGURATION FOR FLOW CONTROL	127
11.4.1. Workload tuning profiles	128
11.4.2. Common issues and remedies	129
CHAPTER 12. BATCH INFERENCE WITH DISTRIBUTED INFERENCE WITH LLM-D	131
12.1. BATCH INFERENCE FOR DISTRIBUTED INFERENCE WITH LLM-D	131
12.1.1. When to use batch inference	131
12.1.2. Inference modes in Distributed Inference with llm-d	132
12.1.3. Batch inference architecture	132
12.1.4. OpenAI Batch API compatibility	133
12.1.5. Priority-aware dispatching for batch workloads	133
12.1.6. Multi-tenancy	133
12.2. CONFIGURE BATCH INFERENCE FOR DISTRIBUTED INFERENCE WITH LLM-D	133
12.3. SUBMIT A BATCH INFERENCE JOB	144
12.4. BATCH INFERENCE API REFERENCE	147
CHAPTER 13. AUTOSCALE DISTRIBUTED INFERENCE WITH LLM-D MODEL DEPLOYMENTS	154
13.1. WORKLOAD VARIANT AUTOSCALER	154
13.2. KEY METRICS FOR AUTOSCALING DECISIONS	155
13.3. HOW THE WORKLOAD VARIANT AUTOSCALER WORKS	155
13.3.1. Saturation scaling algorithm	155
13.3.2. LLMInferenceService parallelism and WVA scaling	156
13.4. DISTRIBUTED INFERENCE WITH LLM-D INFERENCE STACK COMPONENTS	158
13.5. ENABLE THE WORKLOAD VARIANT AUTOSCALER FOR DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENTS	159
13.6. REFERENCE WVA CONFIGURATION FOR A MULTI-NODE MODEL DEPLOYMENT	172
13.7. SATURATION SCALING CONFIGMAP REFERENCE	176
13.8. ENABLE THE TOKEN-BASED CAPACITY ANALYZER	177
13.9. AUTOSCALING METRICS	178
13.10. INSTALL WVA PROMETHEUS ALERT RULES	181
13.11. HPA BEHAVIOR DEFAULTS	184
13.12. UPGRADE THE WVA CONFIGMAP FROM VERSION 3.4 TO 3.5	185
13.13. TROUBLESHOOT WORKLOAD VARIANT AUTOSCALER	186
CHAPTER 14. WIDEEP DEPLOYMENT TOPOLOGY AND DP LOAD BALANCING MODES	188
14.1. WIDEEP ONE-POD-PER-NODE TOPOLOGY	188
14.2. DP LOAD BALANCING MODES	189
14.3. WHY EXTERNAL MULTI-PORT MODE ENABLES PREFIX-CACHE-AWARE ROUTING	189
14.4. SUPPORTED LAUNCH PATH: SERVING PORTS AND ADMIN HEALTH	189
14.5. PORT ALLOCATION MODEL	190
14.6. PARALLELISM CONFIGURATION AND DP MODE SELECTION	190
14.7. KNOWN LIMITATIONS	191
CHAPTER 15. LAUNCH A WIDEEP INFERENCE SERVICE WITH DP-AWARE ROUTING	192
15.1. DP SUPERVISOR HEALTH ENDPOINTS AND PROBE CONFIGURATION	196
15.1.1. Health endpoint reference	196
15.1.2. Supervisor internal probe parameters	197
15.1.3. Recommended Kubernetes probe configuration	197
15.1.4. SIGTERM shutdown behavior	199

15.1.5. Health model trade-offs	199
15.2. DP SUPERVISOR CLI FLAGS AND CONFIGURATION OPTIONS	199
15.2.1. Required flags	200
15.2.2. Optional flags	200
15.2.3. Validation rules	201
15.2.4. Incompatible options	202
15.3. KNOWN LIMITATIONS FOR DP-AWARE LOAD BALANCING	202
CHAPTER 16. CONFIGURE DISAGGREGATED PREFILL-DECODE SERVING	204
16.1. NIXLCONNECTOR KV CACHE TRANSFER PARAMETERS	209
CHAPTER 17. MONITOR LLM-D DEPLOYMENTS	212
17.1. METRICS FOR LLM-D INFERENCE DEPLOYMENTS	212
17.2. VERIFY METRICS COLLECTION FOR LLM-D DEPLOYMENTS	213
17.3. BUILT-IN OBSERVABILITY DASHBOARDS FOR LLM-D DEPLOYMENTS	215
17.3.1. Default dashboards	215
17.3.2. Drill-down investigation workflow	215
17.3.3. Common diagnostic patterns	216
17.3.4. Built-in dashboards compared to community Grafana dashboards	216
17.4. ACCESS LLM-D OBSERVABILITY DASHBOARDS	216
17.5. DASHBOARD PANELS AND METRICS FOR LLM-D OBSERVABILITY	218
17.5.1. LLMTraffic dashboard	218
17.5.2. LLM Performance dashboard	219
17.5.3. LLM Utilization dashboard	220
17.5.4. Metric prefixes in dashboard panels	221
17.6. PROMQL QUERIES FOR LLM-D MONITORING	221
17.6.1. Failure and saturation indicators	222
17.6.2. Diagnostic drill-down queries	223
17.7. IMPORT GRAFANA DASHBOARDS FOR LLM-D	225
17.8. VLLM METRICS FOR LLM-D	227
17.9. EPP AND INFERENCE SCHEDULER METRICS FOR LLM-D	228
17.10. MONITOR BATCH INFERENCE WORKLOADS	232
17.11. CAPACITY PLANNING FOR DISAGGREGATED INFERENCE	234
17.12. CONFIGURE DISTRIBUTED TRACING FOR DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENTS	235
17.12.1. Distributed tracing architecture for Distributed Inference with llm-d	235
17.12.1.1. Four-component tracing pipeline	236
17.12.1.2. Span hierarchy	236
17.12.1.3. Why distributed tracing matters for LLM inference	237
17.12.2. Install distributed tracing prerequisite Operators	237
17.12.3. Deploy a Tempo instance for trace storage	238
17.12.4. Enable distributed tracing for LLMInferenceService	240
17.12.5. Distributed tracing sampling strategies	242
17.12.5.1. Why sample traces	242
17.12.5.2. Sampling strategies	242
17.12.5.2.1. Always-on sampling	242
17.12.5.2.2. Probabilistic sampling	243
17.12.5.2.3. Parent-based sampling	243
17.12.5.3. Coordinating sampling across components	243
17.12.6. Configure distributed tracing sampling rates	244
17.12.7. Verify distributed tracing deployment	245
17.12.8. Troubleshoot distributed tracing errors	247
17.12.8.1. No traces appearing in Jaeger UI	247

17.12.9. Distributed tracing sampling parameters	247
17.12.9.1. sampler field	247
17.12.9.2. samplerArg field	248
17.12.9.3. Configuration example	249
CHAPTER 18. COLLECT DISTRIBUTED INFERENCE WITH LLM-D DIAGNOSTIC DATA FROM OPENSIFT ...	250
CHAPTER 19. TROUBLESHOOT DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENT ISSUES 252	
19.1. COMMON ISSUES WITH DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENTS	254
19.1.1. Common issues and solutions	254

PREFACE

As an administrator, you can use Distributed Inference with llm-d to deploy and serve large language models at scale on Red Hat OpenShift AI. You can configure distributed inference and set up authentication and authorization for your model deployments.

CHAPTER 1. DEPLOY MODELS BY USING DISTRIBUTED INFERENCE WITH LLM-D

Distributed Inference with llm-d is a Kubernetes-native, open-source framework designed for serving large language models (LLMs) at scale. You can use Distributed Inference with llm-d to simplify the deployment of generative AI, focusing on high performance and cost-effectiveness across various hardware accelerators.

Key features of Distributed Inference with llm-d include:

- Efficiently handles large models using optimizations such as prefix-cache aware routing and disaggregated serving.
- Integrates into a standard Kubernetes environment, where it leverages specialized components like the Envoy proxy to handle networking and routing, and high-performance libraries such as vLLM and NVIDIA Inference Transfer Library (NIXL).
- Tested recipes and well-known presets reduce the complexity of deploying inference at scale, so users can focus on building applications rather than managing infrastructure.

1.1. ENABLE DISTRIBUTED INFERENCE WITH LLM-D

This procedure describes how to create a custom resource (CR) for an **LLMInferenceService** resource. You replace the default **InferenceService** with the **LLMInferenceService**.

Prerequisites

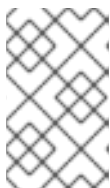
- You have enabled the model serving platform.
- You have access to an OpenShift cluster running version 4.19.9 or later.
- OpenShift Service Mesh v2 is not installed in the cluster.
- Verify that you have a **GatewayClass** and a **Gateway** named **openshift-ai-inference** in the **openshift-ingress** namespace as described in [Gateway API with OpenShift Container Platform networking](#).



IMPORTANT

Review the [Gateway API deployment topologies](#). Only use shared Gateways across trusted namespaces.

- If you are running OpenShift on a bare-metal cluster, your cluster administrator has an external entry point for the **openshift-ai-inference** Gateway service.



NOTE

By default, the Inference Gateway uses type: **LoadBalancer**. If the cluster does not already include support for **LoadBalancer** services, you can use the OpenShift option described in [Load balancing with MetalLB](#).

- **Optional:** Your cluster administrator has installed the **LeaderWorkerSet** Operator in OpenShift. This is an optional dependency because it is only a requirement for inference deployments

where a single server has any form of parallelism (tensor, pipeline, data) that is more than 8 accelerators and so is classified as **multi-node**. For more information, see the [Leader Worker Set Operator documentation](#).

- You have enabled authentication as described in *Configuring authentication for Distributed Inference with llm-d*.

Procedure

1. Log in to the OpenShift console as a developer.
2. Create the **LLMInferenceService** CR with the following information:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: sample-llm-inference-service
spec:
  replicas: 2
  model:
    uri: hf://RedHatAI/Qwen3-8B-FP8-dynamic
    name: RedHatAI/Qwen3-8B-FP8-dynamic
  router:
    route: {}
    gateway: {}
    scheduler: {}
  template:
    containers:
      - name: main
        resources:
          limits:
            cpu: '4'
            memory: 32Gi
            nvidia.com/gpu: "1"
          requests:
            cpu: '2'
            memory: 16Gi
            nvidia.com/gpu: "1"
```

Customize the following parameters in the **spec** section of the inference service:

- **replicas** - Specify the number of replicas.
- **model** - Specify the URI to the model based on how the model is stored (**uri**) and the model name to use in chat completion requests (**name**).
 - S3 bucket: **s3://<bucket-name>/<object-key>**
 - Persistent volume claim (PVC): **pvc://<claim-name>/<pvc-path>**
 - OCI container image: **oci://<registry_host>/<org_or_username>/<repository_name><tag_or_digest>**
 - HuggingFace: **hf://<model>/<optional-hash>**
- **router** - Provide an HTTPRoute and gateway, or leave blank to automatically create one.

3. Save the file.

CHAPTER 2. DEPLOY MODELS USING THE TOPOLOGY SELECTOR

Deploy large language models using the topology selector wizard in the OpenShift AI Dashboard. The topology selector enables you to choose from four validated deployment patterns based on your hardware and performance requirements: single-node, multi-node data-parallel, single-node prefill/decode disaggregate, and multi-node prefill/decode disaggregate. If you are a platform administrator, you can manage deployment topology and router configuration templates through the Dashboard admin settings.



IMPORTANT

LLM deployment topology selector is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

2.1. UNIFIED GENERATIVE MODEL SERVING

You can deploy models by using a UI wizard with a single **Deploy model** entry point for all generative AI models. When you select **Generative AI** or **Distributed inference with llm-d** from the model framework radio button options, the wizard guides you through the following:

- Selecting a deployment topology appropriate for your model size and hardware constraints
- Configuring hardware profiles and accelerator resources (for single-node deployments)
- Optionally applying advanced routing configurations for production workloads
- Deploying the model with validated configuration templates

This approach replaces manual YAML editing with template-based deployment, reducing errors and enabling self-service deployment for data scientists who do not have deep Kubernetes expertise.

Benefits of template-based deployment

The wizard-driven approach with topology selector provides several benefits over the YAML-only workflow:

Reduced deployment errors

Pre-validated configuration templates for each topology type eliminate common YAML syntax errors, missing required fields, and incompatible resource specifications.

Faster time to deployment

Users can deploy LLM models in minutes rather than hours spent constructing and debugging custom YAML configurations.

Self-service for data scientists

Data scientists and ML engineers can deploy models without requiring platform administrator assistance for each deployment, while administrators retain control over available topology and router configuration templates.

Consistent platform usage

All generative AI models follow the same deployment workflow regardless of the underlying runtime (KServe, vLLM, or llm-d), reducing training requirements and onboarding time.

Administrator control

While the wizard simplifies deployment for end users, platform administrators maintain control over available deployment options through **LLMInferenceServiceConfig** template management:

- Administrators create topology configuration templates that define resource limits, worker configurations, and scheduling policies for each deployment pattern
- Administrators create router configuration templates that provide advanced routing capabilities for production workloads
- Non-single-node topologies are disabled in the wizard until administrators create the required configuration templates, preventing users from attempting deployments that exceed cluster capacity

2.2. INFERENCE WORKLOAD DEPLOYMENT TOPOLOGY PATTERNS

You can deploy large language models by using validated topology patterns that balance resource allocation, latency, and throughput based on your model size and hardware constraints. Understanding these patterns helps you select the appropriate deployment strategy for your inference workloads.

Single-node topology

The single-node topology deploys all model components on a single GPU or set of GPUs within one Kubernetes pod. This pattern is ideal for development, testing, and production deployments of smaller models that fit within the memory and compute capacity of available accelerators.

Use single-node topology when:

- Your model fits within the memory of the available GPUs on a single node
- You are developing or testing model deployments before scaling
- Your inference workload requires simple resource management
- You need the fastest deployment path with minimal configuration

Single-node deployments support hardware profile selection, allowing you to specify the exact accelerator resources required for your model.

Multi-node data-parallel topology

The multi-node data-parallel topology distributes inference requests across many worker nodes, each running a complete copy of the model. This pattern provides horizontal scaling for high-throughput workloads by processing many requests in parallel.

Use multi-node data-parallel topology when:

- You need to handle high request volumes with parallel processing
- Your model fits on a single GPU but you require higher throughput than one instance can give

- You want to distribute inference load across many workers for reliability
- Your workload benefits from request-level parallelism

This topology requires administrator-created configuration templates to define the worker pool and resource allocation.

Single-node disaggregated prefill/decode topology

The single-node disaggregated prefill/decode topology separates the prefill phase, which is processing input prompts, from the decode phase, which is generating output tokens, on different GPUs within a single node. This pattern optimizes latency by specializing compute resources for each phase of LLM inference.

Use single-node disaggregated prefill/decode topology when:

- You have at least 2 GPUs and RDMA-capable networking available on a single node
- Your model has 30 billion or more parameters
- Your workload is suitable for disaggregated inference

The scheduler component coordinates request routing between prefill and decode pods, enabling KV cache-aware scheduling.

Multi-node disaggregated prefill/decode topology

The multi-node prefill/decode disaggregate topology distributes prefill and decode workloads across many nodes, combining disaggregation with horizontal scaling. This pattern supports the largest models and highest throughput requirements by separating phases and parallelizing within each phase. Use multi-node prefill/decode disaggregate topology when:

- You need to deploy models that require more GPUs than a single node provides
- You have RDMA-capable networking such as InfiniBand or RoCE v2 across nodes
- Your workload is suitable for disaggregated inference.

Wide expert parallelism (WideEP) topology

The WideEP topology distributes Mixture of Experts (MoE) model experts across multiple GPU nodes by using all-to-all communication backends such as DeepEP. This pattern enables cost-effective inference for very large MoE models such as DeepSeek-R1 by splitting expert layers across nodes rather than replicating the full model.



IMPORTANT

Wide expert parallelism (WideEP) is a Developer Preview feature only. Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to upcoming product features in advance of their possible inclusion in a Red Hat product offering, enabling customers to test functionality and provide feedback during the development process. These features might not have any documentation, are subject to change or removal at any time, and testing is limited. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA.

Use WideEP topology when:

- Your model uses a Mixture of Experts architecture
- The model size exceeds what multi-node data-parallel or prefill/decode disaggregate topologies can handle efficiently
- You have RDMA-capable networking such as RoCE or InfiniBand for expert parallelism communication
- You need all GPUs on each node allocated to a single pod for NVLink intranode communication

WideEP deployments use a one-pod-per-node topology where each pod requests all GPUs on the node. The LeaderWorkerSet Operator manages multi-node coordination across pods.

2.3. DEPLOYMENT TOPOLOGY CONFIGURATION LABELS

LLMInferenceServiceConfig resources use the **opendatahub.io/config-type** label to categorize configurations by topology type. The dashboard uses these labels to discover and filter configurations in the model serving wizard.

Table 2.1. Configuration type label values

Label value	Topology pattern
workload-single-node	Single-node deployment topology
workload-multi-node-data-parallel	Multi-node data-parallel deployment topology
workload-single-node-pd	Single-node disaggregated prefill/decode topology
workload-multi-node-data-parallel-pd	Multi-node disaggregated prefill/decode topology
router	Advanced routing configuration, filtered by supported-topologies annotation

Router configurations use the **opendatahub.io/supported-topologies** annotation to specify which deployment topologies they are compatible with. The annotation value is a JSON array of topology type strings. Valid topology type strings are:

- **workload-single-node**
- **workload-multi-node-data-parallel**
- **workload-single-node-pd**
- **workload-multi-node-data-parallel-pd**

2.4. HARDWARE REQUIREMENTS AND CONSIDERATIONS FOR LLM DEPLOYMENT TOPOLOGIES

Each LLM deployment topology has specific hardware requirements for GPU count, memory capacity, and network bandwidth. Use this reference to select the appropriate topology based on your available hardware resources.

The following table lists the minimum and recommended hardware requirements for each llm-d deployment topology pattern.

Table 2.2. Hardware requirements by topology type

Topology type	GPU count per node	Memory per GPU	Network bandwidth	Typical use cases
Single-node	1+	40-80 GB	Standard 10-25 Gbps cluster networking	Development, testing, and production deployments of smaller models up to 13B parameters
Multi-node data-parallel	8+	40-80 GB per worker	50-100 Gbps inter-node networking recommended	High-throughput inference workloads requiring request-level parallelism for mid-size models 7B-70B parameters
Single-node prefill/decode (P/D)	2-4	80 GB per GPU	200-400 Gbps RDMA networking (InfiniBand or RoCE v2) for KV cache transfer between prefill and decode pods	Latency-optimized deployments requiring phase separation for models 30B to 70B parameters
Multi-node prefill/decode (P/D)	8+	80 GB per GPU	200-400 Gbps RDMA networking (InfiniBand NDR or RoCE v2) for cross-node KV cache transfer	Large-scale inference for models exceeding 70B parameters requiring distributed disaggregated serving with high-bandwidth KV cache sharing

GPU architecture considerations

All llm-d deployment topologies require NVIDIA GPUs with the following characteristics:

- **Architecture:** Pascal or later (Volta, Turing, Ampere, Hopper, Blackwell architectures supported)
- **Compute capability:** 6.0 or higher

- **CUDA support:** CUDA 11.8 or later
- **Memory type:** HBM2 or HBM3 for optimal performance

Prefill/decode disaggregate topologies benefit from the following:

- **GPU-Direct RDMA:** Enables direct memory access between GPUs across nodes without CPU involvement
- **NVLink:** Provides high-bandwidth GPU-to-GPU communication within nodes for tensor parallelism. NVLink does not enable KV cache transfers between pods; RDMA networking is required for all prefill/decode disaggregated topologies, including single-node
- **High memory bandwidth:** 900 GB/s or higher for prefill phase optimization

Storage and network requirements

In addition to GPU requirements, consider these infrastructure requirements:

Model storage

S3-compatible object storage or OCI registry with enough capacity for model weights and checkpoints. Typical model sizes range from 10 GB (7B parameter models) to 500+ GB (405B parameter models).

Network bandwidth

Single-node topologies can operate on standard 10-25 Gbps cluster networking. Multi-node data-parallel topologies require 50-100 Gbps inter-node networking for efficient distributed inference. Multi-node prefill/decode disaggregate topologies require 200-400 Gbps RDMA networking such as InfiniBand NDR or RoCE v2 for efficient cross-node KV cache transfer.

Persistent storage

KV cache storage and intermediate state require fast persistent storage. NVMe-backed storage or Ceph with SSD pools recommended for production deployments.

Model size to topology mapping

Use the following guidance to map model parameter count to appropriate topology patterns:

Model size (parameters)	Recommended topology	Example models
Up to 30B	Single-node	Llama 3.3 70B (quantized), Mistral 7B, Qwen 2.5 7B, Phi-4
30B to 70B	Multi-node data-parallel or single-node P/D	Llama 3.1 70B, Mixtral 8x22B, Qwen 2.5 72B
70B to 175B	Multi-node P/D	Llama 3.3 70B (FP16), Mixtral 8x22B (FP16), Falcon 180B
175B and larger	Multi-node P/D with extended workers	Llama 3.1 405B, Qwen 2.5 Coder 32B Instruct, custom large models

2.5. ROUTER CONFIGURATIONS AND SUPPORTED TOPOLOGIES

Router configurations provide advanced routing capabilities for LLM inference workloads, including KV cache-aware scheduling and intelligent request gateway routing. The dashboard filters router configurations based on topology compatibility, ensuring users select only configurations appropriate for their chosen deployment pattern.

Router **LLMInferenceServiceConfig** resources enhance model deployments with intelligent request routing, load balancing, and resource optimization features. Unlike topology configurations that define the fundamental deployment structure, router configurations are optional enhancements that you apply when you need advanced routing capabilities.

Router configuration capabilities

Router configurations provide several advanced features for LLM inference:

KV cache-aware scheduling

Directs inference requests to model instances with relevant key-value cache entries, reducing redundant computation for similar prompts. This capability is particularly valuable for prefill/decode disaggregate topologies where the scheduler must route requests to appropriate prefill or decode workers.

Intelligent gateway routing

Balances inference requests across multiple model replicas based on current load, response time, or custom routing policies. This feature improves throughput and latency for high-volume production deployments.

Request batching

Aggregates multiple inference requests into batches to maximize GPU utilization and throughput. Router configurations define batching policies, timeout thresholds, and maximum batch sizes.

Circuit breaking and retry logic

Protects model deployments from cascading failures by implementing circuit breaker patterns and intelligent retry strategies when individual model instances become unavailable.

Topology-based filtering

The dashboard uses the **opendatahub.io/supported-topologies** annotation to filter router configurations based on the user's selected deployment topology. This prevents incompatible router configurations from appearing in the wizard and reduces configuration errors.

When you select a deployment topology in the model serving wizard and expand the advanced routing section, the dashboard:

1. Queries all **LLMInferenceServiceConfig** resources with **opendatahub.io/config-type: router** label
2. Filters the results client-side based on the **supported-topologies** annotation
3. Displays only router configurations that include the selected topology type in their annotation

For example, if you select the **Multi-node data-parallel** topology, the dashboard displays only router configs with:

```
metadata:
  annotations:
    opendatahub.io/supported-topologies: ["workload-multi-node-data-parallel"]
```

Or router configs that include **workload-multi-node-data-parallel** in a list of supported topologies:

```
metadata:
  annotations:
    opendatahub.io/supported-topologies: '["workload-single-node", "workload-multi-node-data-parallel"]'
```

When to use router configurations

Router configurations are optional for all deployment topologies. Consider applying a router configuration when:

High throughput requirements

Your workload requires processing hundreds or thousands of inference requests per minute, and you need intelligent load balancing across model replicas.

Latency optimization

You want to minimize request latency by routing requests to the least-loaded model instances or instances with relevant KV cache entries.

Production stability

You need circuit breaking, retry logic, and graceful degradation capabilities to maintain service availability when individual model instances fail.

Cost optimization

You want to maximize GPU utilization through request batching and efficient routing policies that reduce idle time.

For development, testing, or low-volume production workloads, you can deploy models without router configurations. The underlying KServe infrastructure provides basic load balancing and routing capabilities.

2.6. SELECTING AN LLM DEPLOYMENT TOPOLOGY

You can choose the appropriate deployment topology for your large language model based on model size, available hardware, and performance requirements.

Use this decision guide to select from the four validated deployment patterns: single-node, multi-node data-parallel, single-node prefill/decode disaggregate, or multi-node prefill/decode disaggregate.

Prerequisites

- You understand your model's parameter count and memory requirements.
- You know the GPU resources available in your cluster.
- You have determined your inference workload performance requirements for throughput and latency.

Procedure

The model deployment wizard displays topology options as radio buttons with descriptions. If a topology option is disabled (grayed out), it indicates that your administrator has not created any topology configurations for that deployment pattern. Single-node topology includes a built-in default configuration and is always available.

1. Determine your model size category:
 - a. If your model has up to 30 billion parameters, proceed to step 2.
 - b. If your model has 30–70 billion parameters, proceed to step 3.
 - c. If your model has more than 70 billion parameters, proceed to step 4.
2. For models up to 30B parameters:
 - a. Select **Single-node** topology.
 - b. This pattern deploys all model components within a single pod using the available GPUs on a single node.
 - c. Hardware profile selection is available for this topology, allowing you to specify the exact accelerator type.
 - d. Stop here—single-node topology is appropriate for your model size.
3. For models 30B–70B parameters:
 - a. If you have RDMA-capable networking and at least 2 GPUs, select **Single-node prefill/decode disaggregate** topology. This pattern optimizes latency by separating prefill and decode phases. RDMA networking is required for KV cache transfers between pods.
 - b. If you prioritize high throughput and can distribute across 4+ nodes, select **Multi-node data-parallel** topology. This pattern processes multiple requests in parallel across worker nodes.
 - c. Stop here—choose based on your latency vs. throughput priority.
4. For models 70B+ parameters:
 - a. If you have 8+ GPUs available across multiple nodes, RDMA-capable networking, and need both high throughput and optimized latency, select **Multi-node prefill/decode disaggregate** topology.
 - b. This pattern combines phase separation with horizontal scaling for the largest models.
 - c. Stop here—this is the recommended topology for very large models.

Verification

After selecting a topology in the model deployment wizard:

1. Verify that the topology option is enabled (not grayed out). If your selected topology is disabled, contact your administrator to create the required **LLMInferenceServiceConfig** template. The wizard validates topology selection and prevents you from proceeding to the next step without selecting a topology.
2. Check whether additional configuration sections appear based on your selection:
 - Single-node: Hardware profile section should be visible and enabled.
 - Multi-node or disaggregate topologies: Hardware profile section is hidden; resources are defined in the topology configuration.

3. Verify that the **Topology configuration** field is populated with available configurations for your selected topology type. This is a required field marked with an asterisk.
4. In the **Advanced routing** section, verify that **Default optimized routing** is pre-selected. If you need a specific routing pattern, you can select an administrator-created routing configuration from the list.

Decision matrix

Use this table for quick topology selection:

Model size	Available GPUs	Priority	Recommended topology	Rationale
Up to 30B	1+	Any	Single-node	Fits on minimal hardware, simplest deployment
30B-70B	2+, RDMA networking	Latency	Single-node P/D	Phase separation optimizes latency; requires RDMA for KV cache transfers
30B-70B	4+, multi-node	Throughput	Multi-node data-parallel	Horizontal scaling maximizes request throughput
70B+	8+, multi-node, RDMA networking	Latency + throughput	Multi-node P/D	Distributed disaggregation for large-scale inference; requires RDMA for KV cache transfers

Additional resources

- [LLM deployment topology patterns](#)
- [Hardware requirements for LLM deployment topologies](#)
- [Deploying models with the llm-d topology selector](#)

2.7. DEPLOYING MODELS WITH THE LLM-D TOPOLOGY SELECTOR

You can deploy large language models through the OpenShift AI dashboard using the llm-d topology selector to choose validated deployment patterns for single-node, multi-node data-parallel, or prefill/decode disaggregate topologies.

The model serving wizard guides you through selecting a deployment topology, configuring hardware resources, and optionally applying advanced routing configurations. This Technology Preview feature simplifies LLM deployment by providing pre-configured topology templates without requiring manual YAML editing.

Prerequisites

- You have installed OpenShift AI and enabled the model serving platform.
- An administrator has enabled the **llm-dTemplates** feature flag in the **OdhDashboardConfig** resource.

- You have logged in to OpenShift AI as a user with the appropriate role to deploy models, for example, a data scientist or administrator.
- You have created a data science project.
- You have stored your model in an S3-compatible object storage bucket, a persistent volume claim, or an OCI registry.
- You have obtained the S3 bucket URI, PVC name, or OCI image reference for your model.
- For non-single-node topologies: An administrator has created the required **LLMInferenceServiceConfig** resources for the topology you want to deploy. The single-node topology is available by default without administrator configuration.

Procedure

1. In the OpenShift AI dashboard, click **Data Science Projects**.
2. Click the name of the project where you want to deploy your model.
3. Click the **Models** tab.
4. Click **Deploy model**.
5. In the **Deploy model** dialog, configure the basic model details:
 - a. In the **Model name** field, enter a name for your deployed model.
 - b. From the **Model type** field, select **Generative AI**.
 - c. From the **Model server size** list, select a size based on your expected load.
 - d. Click **Deploy**.
6. On the **Model deployment** page, from the **Deployment method** field, select **Distributed inference with llm-d**.
7. Select your deployment topology from the radio button options:
 The **Topology type** determines how the inference workload is mapped to underlying GPUs, networking, and node resources. Each option is optimized for different model sizes and traffic patterns:
 - a. **Single-node**: Select this if your model and its Key-Value (KV) cache fit within the GPUs of a single machine. This provides standard, unified serving for basic workloads. This option includes a built-in default configuration and is always available.
 - b. **Multi-node data-parallel**: Select this to maximize concurrent request throughput (QPS). This replicates your model across multiple independent nodes to handle high volumes of user traffic. This option is disabled if no administrator-created configurations exist for this topology.
 - c. **Single-node prefill/decode disaggregate**: Select this to optimize long-context or prefill-heavy workloads such as RAG. This isolates heavy prompt processing and token generation onto separate GPUs within one machine, preventing massive inputs from stalling ongoing generation. This option is disabled if no administrator-created configurations exist for this topology.

- d. **Multi-node data-parallel prefill/decode disaggregate** Select this to maximize concurrent throughput for heavily prefill-skewed workloads. This replicates a disaggregated prefill/decode architecture across multiple nodes, combining the high-traffic capacity of data parallelism with the stutter-free performance of P/D isolation. Requires high-bandwidth RDMA networking such as InfiniBand or RoCE for efficient cross-node KV cache transfer. This option is disabled if no administrator-created configurations exist for this topology.



NOTE

If a topology option appears disabled with a tooltip indicating that it needs to be requested by an administrator, contact your OpenShift AI administrator to create the required **LLMInferenceServiceConfig** topology template.

8. From the **Topology configuration** list, select a configuration that matches your cluster resources and model requirements.
This is a required field marked with an asterisk. The list displays only configurations that match your selected topology type. For single-node topology, a built-in "single-node-default" option is always available.
9. If you selected **Single-node** topology, configure the hardware profile:
The **Hardware profile** section is only visible for single-node deployments. For multi-node or disaggregate topologies, resource allocation is handled by the topology configuration itself and this section is hidden.
 - a. Expand the **Hardware profile** section.
 - b. From the **Hardware profile** list, select an accelerator profile that matches your model requirements. For example, select **NVIDIA A100 80GB** for models requiring 40-80 GB GPU memory.
 - c. In the **Number of GPUs** field, specify how many GPUs to allocate to your model. The default is 1.
10. Configure the model location:
 - a. In the **Model location** section, select your storage type: **S3**, **PVC**, or **OCI image**.
 - b. Enter the connection details:
 - For S3: Enter the bucket path, region, and select your S3 data connection.
 - For PVC: Select the persistent volume claim containing your model files.
 - For OCI image: Enter the image reference, for example, **registry.example.com/myorg/mymodel:v1.0**.
11. Optional: Configure advanced routing:
 - a. Click **Advanced routing** to expand the routing options.
By default, **Default optimized routing** is pre-selected, which means the LLM-D controller uses built-in defaults.
 - b. If you need a specific routing pattern, select an administrator-created routing configuration from the **Router configuration** list.

The list displays only router configurations that are compatible with your selected deployment topology based on the **opendatahub.io/supported-topologies** annotation.

- c. If you want KV cache-aware scheduling or intelligent request gateway routing, select the appropriate router configuration.

12. Click **Create**.

Verification

1. On the **Models** tab of your project, verify that your deployed model appears in the list with a status of **Ready**.
2. Wait for the model server pods to be created and reach running state. This might take several minutes depending on the model size and cluster resources.
3. After the status changes to **Ready**, click the model name to view deployment details.
4. Copy the inference endpoint URL from the **Inference endpoint** field.
5. Test the inference endpoint by sending a request using the endpoint URL and your authentication token.

```
curl -k -H "Authorization: Bearer $TOKEN" \
  https://MODEL_ENDPOINT/v1/models/MODEL_NAME:predict \
  -d '{"inputs": [{"query": "What is the capital of France?"}]}'
```

Replace **MODEL_ENDPOINT** with your inference endpoint URL, **MODEL_NAME** with your model name, and **\$TOKEN** with your bearer token.

6. Verify that you receive a response with predictions from your model.

Troubleshooting

Model deployment stuck in "Pending" or "Loading" status

Check the model server pod logs for errors related to model loading or resource allocation. Verify that your S3 credentials are correct and the model files exist at the specified location. For non-single-node topologies, confirm that the **LLMInferenceServiceConfig** resource exists and has the correct **opendatahub.io/config-type** label.

Topology option is disabled in the wizard

Contact your administrator to create the required **LLMInferenceServiceConfig** template for the disabled topology type. Administrators can create topology configs from **Settings > llm-d topology configurations**.

Inference requests return 404 or 503 errors

Verify that the model status is **Ready** before sending inference requests. Check the model server logs for errors during model initialization. Confirm that your bearer token is valid and has not expired.

Hardware profile section does not appear

Hardware profile selection is only available for single-node topology deployments. If you selected a different topology, hardware allocation is defined in the **LLMInferenceServiceConfig** resource created by administrators.

Additional resources

- [LLM deployment topology patterns](#)
- [Hardware requirements for LLM deployment topologies](#)
- [Router configurations and supported topologies](#)
- [Managing topology configuration templates](#)

2.8. CONFIGURING ADVANCED ROUTING FOR DEPLOYED MODELS

You can apply router configurations to your deployed models to enable KV cache-aware scheduling, intelligent request gateway routing, and other advanced inference optimizations.

Router configurations are optional enhancements that improve inference performance for production workloads. The dashboard displays only router configurations that are compatible with your selected topology type.

Prerequisites

- You have started the model deployment workflow and selected a deployment topology.
- Your administrator has created one or more router **LLMInferenceServiceConfig** resources compatible with your selected topology.

Procedure

1. In the model deployment wizard, after selecting your deployment topology and configuring model storage, click **Advanced routing** to expand the routing options section.
By default, **Default optimized routing** is pre-selected, which means the LLM-D controller uses built-in defaults.
2. Review the list of available router configurations.
The dashboard displays only router configurations where the **opendatahub.io/supported-topologies** annotation includes your selected topology type.
3. To apply a specific router configuration, select a router configuration appropriate for your workload requirements:
 - a. If you need KV cache-aware scheduling to reduce redundant computation for similar prompts, select a router config with **kvCacheAware: true**.
 - b. If you need intelligent load balancing across model replicas, select a router config with a scheduling policy such as **least-loaded** or **round-robin**.
 - c. If you do not need advanced routing capabilities, keep the default **Default optimized routing** selection.
4. Continue with the model deployment workflow by clicking **Create**.

Verification

After deploying your model with a router configuration:

1. On the **Models** tab of your project, click your deployed model name to view details.

2. Verify that the model deployment includes the router configuration in its generated **LLMInferenceService** resource. You can view the generated resource by accessing the model details page and checking the YAML representation.
3. Confirm that the **spec.baseRefs** array includes your selected router configuration after the topology preset config:

```
spec:
  baseRefs:
    - name: config-llm-template      # Topology preset
      namespace: kserve-system
    - name: kv-cache-router-config  # Your selected router
      namespace: default
```

4. Send test inference requests to verify that the router is functioning correctly:

```
curl -k -H "Authorization: Bearer $TOKEN" \
  https://MODEL_ENDPOINT/v1/models/MODEL_NAME:predict \
  -d '{"inputs": [{"query": "What is machine learning?"}]}'
```

5. Monitor the model server logs to observe router behavior, such as request routing decisions or KV cache utilization.

Troubleshooting

No router configurations appear in the advanced routing section

Verify that your administrator has created router **LLMInferenceServiceConfig** resources with the **opendatahub.io/config-type: router** label. Check that at least one router config includes your selected topology type in its **supported-topologies** annotation. For example, if you selected **Multi-node data-parallel** topology, the router config must include **"multi-node-data-parallel"** in its annotation.

Router configuration appears in the list but does not apply to deployment

Verify that you selected the router configuration before clicking **Create**. If you expanded the advanced routing section but did not select a router from the list, no router is applied.

Model deployment fails after selecting router configuration

Check the model server pod logs for errors related to router initialization or configuration conflicts. Verify that the router configuration resource exists in the expected namespace. Ensure the router config does not conflict with required fields in the topology preset config.

Additional resources

- [Router configurations and supported topologies](#)
- [Managing router configurations](#)
- [LLMInferenceService baseRefs merge behavior](#)

2.9. MANAGING TOPOLOGY CONFIGURATION TEMPLATES

You can create and manage **LLMInferenceServiceConfig** topology templates that define deployment patterns for large language models. These configurations enable users to select validated topology options when deploying models through the dashboard wizard.

Topology configurations define resource allocation, scheduling behavior, and component distribution for four deployment patterns: single-node, multi-node data-parallel, single-node prefill/decode disaggregate, and multi-node prefill/decode disaggregate.

Prerequisites

- You have logged in to OpenShift AI with administrator privileges.
- You have installed the model serving platform.
- You have enabled the **llmdTemplates** feature flag in the **OdhdashboardConfig** resource.
- You understand the four LLM deployment topology patterns and their hardware requirements.

Procedure

1. In the OpenShift AI dashboard, click **Settings**.
2. Click **llm-d topology configurations**.
The **llm-d topology configurations** page displays a table with the following columns:
 - **Name:** The configuration name.
 - **Topology type:** The deployment pattern (single-node, multi-node data-parallel, single-node prefill/decode disaggregate, or multi-node prefill/decode disaggregate).
 - **Enabled:** Whether the configuration is available for users to select when deploying models.
 - **Pre-installed:** Whether the configuration is shipped by KServe as a built-in default.
 - **Actions:** Options to duplicate or delete the configuration.
3. To create a new topology configuration:
 - a. Click **Add**.
 - b. From the **Topology type** list, select the deployment pattern you want to configure:
 - **Single-node:** For models deployed on 1-2 GPUs within a single pod
 - **Multi-node data-parallel:** For distributed inference with request-level parallelism across workers
 - **Single-node prefill/decode:** For latency-optimized deployments with phase separation on a single node
 - **Multi-node prefill/decode:** For large-scale distributed disaggregated inference
 - c. Select a source for your configuration:
 - **Start from sample:** Use a pre-populated template with sensible defaults for the selected topology type.
 - **Upload file:** Upload an existing YAML configuration file from your local system.
 - **Start from scratch:** Create a new configuration using an empty YAML editor.

- d. Provide a name and optional description for the configuration.
 - e. Review or modify the YAML configuration as needed.
 - f. Click **Create**.
4. When editing the YAML configuration, verify the following settings:
 - a. Ensure that the **metadata.labels.opendatahub.io/config-type** field matches your selected topology type:
 - **workload-single-node**
 - **workload-multi-node-data-parallel**
 - **workload-single-node-pd**
 - **workload-multi-node-data-parallel-pd**
 - b. Adjust resource requests and limits in **spec.container.resources** to match your available GPU and memory capacity.
 - c. For multi-node topologies, configure the worker pool settings in **spec.workers** to define the number of worker replicas and their resource allocation.
 - d. For prefill/decode disaggregate topologies, ensure **spec.scheduler.replicas** is set to 1 or greater. The scheduler component is required for KV cache-aware request routing. If the configuration includes validation errors, the dashboard displays an error message from the KServe webhook. Common errors include:
 - **baseRefs field is forbidden in LLMInferenceServiceConfig**: Remove the **spec.baseRefs** field. Only **LLMInferenceService** resources can include baseRefs.
 - **scheduler replicas must be greater than 0**: Set **spec.scheduler.replicas** to at least 1 for prefill/decode topologies.

Verification

1. On the **llm-d topology configurations** page, verify that your new configuration appears in the list.
2. Navigate to **Data Science Projects**, select a project, and click **Deploy model**.
3. In the model deployment wizard, select **Generative AI** or **Distributed inference with llm-d** from the **Model framework** list.
4. On the **Model deployment** page, verify that the topology corresponding to your new configuration is now enabled and selectable.
5. If you created a multi-node data-parallel configuration, verify that the **Multi-node data-parallel** option is enabled.
6. If you created a prefill/decode disaggregate configuration, verify that the corresponding P/D topology option is enabled.

2.10. MANAGING EXISTING TOPOLOGY CONFIGURATIONS

You can edit, duplicate, or delete existing topology configurations from the **llm-d topology configurations** page.

To edit a topology configuration

1. Click the action menu (⋮) next to the configuration you want to modify.
2. Click **Edit**.
3. Modify the YAML configuration in the editor.
4. Click **Save**.

To duplicate a topology configuration

1. Click the action menu (⋮) next to the configuration you want to copy.
2. Click **Duplicate**.
3. Edit the **metadata.name** field to provide a unique name for the duplicate configuration.
4. Modify resource allocations or worker settings as needed.
5. Click **Save**.

To delete a topology configuration

1. Click the action menu (⋮) next to the configuration you want to remove.
2. Click **Delete**.
3. In the confirmation dialog, click **Delete** to confirm.



NOTE

Deleting a topology configuration does not affect existing deployed models that reference it. However, users will no longer be able to select this configuration when deploying new models.

Troubleshooting

Configuration does not appear in the wizard topology selector

Verify that the **opendatahub.io/config-type** label is set correctly for the topology type. Check that you are looking in the correct topology section. For example, a configuration with **workload-multi-node-data-parallel** should enable the **Multi-node data-parallel** option, not other topology options.

YAML editor displays validation errors when saving

Review the error message for specific field violations. Common issues include forbidden **baseRefs** fields, missing scheduler replicas for P/D topologies, or invalid resource specifications.

Cannot find the llm-d topology configurations page in Settings

Ensure you are logged in with administrator privileges. The **llm-d topology configurations** page is only visible to users with cluster administrator or OpenShift AI administrator roles. If you have administrator access but the page does not appear, verify that the model serving platform is installed and enabled.

Additional resources

- [LLM deployment topology patterns](#)

2.11. MANAGING ROUTER CONFIGURATIONS

You can create and manage router **LLMInferenceServiceConfig** templates that provide advanced routing capabilities for LLM deployments, including KV cache-aware scheduling and intelligent request gateway routing.

Router configurations are optional enhancements that users can select in the advanced routing section of the model deployment wizard. The dashboard filters router configurations based on topology compatibility defined by the **opendatahub.io/supported-topologies** annotation.

Prerequisites

- You have logged in to OpenShift AI with administrator privileges.
- You have installed the model serving platform.
- You have enabled the **llmdTemplates** feature flag in the **OdhDashboardConfig** resource.
- You understand LLM deployment topology patterns and how router configurations enhance inference performance.

Procedure

1. In the OpenShift AI dashboard, click **Settings**.
2. Click **llm-d routing configurations**.
The **llm-d routing configurations** page displays a table with the following columns:
 - **Name:** The router configuration name.
 - **Topology type:** The deployment patterns this router supports based on the **supported-topologies** annotation.
 - **Enabled:** Whether the configuration is available for users to select when deploying models.
 - **Actions:** Options to duplicate, enable, disable, or delete the configuration.
3. To create a new router configuration:
 - a. Click **Add**.
 - b. From the **Topology type** list, select the deployment pattern this router configuration should support.



NOTE

The **Topology type** form field only allows you to select one topology. If you need to configure a router that supports multiple topologies, you must edit the **opendatahub.io/supported-topologies** annotation directly in the YAML editor after creating the configuration.

- c. Select a source for your configuration:
 - **Start from sample:** Use a pre-populated template with sensible defaults for the selected topology type.
 - **Upload file:** Upload an existing YAML configuration file from your local system.
 - **Start from scratch:** Create a new configuration using an empty YAML editor.
 - d. Provide a name and optional description for the configuration.
 - e. Review or modify the YAML configuration as needed.
 - f. Click **Create**.
4. When editing the YAML configuration, verify the following settings:
- a. Ensure the **opendatahub.io/config-type** label is set to **router**:

```
metadata:
  name: kv-cache-router-single-node
  labels:
    opendatahub.io/config-type: router
```

- b. Add the **opendatahub.io/supported-topologies** annotation to specify which deployment topologies this router configuration supports. The annotation value must be a JSON array of topology type strings:

```
metadata:
  annotations:
    opendatahub.io/supported-topologies: '["workload-single-node", "workload-multi-node-data-parallel"]'
```

Valid topology type strings are:

- **workload-single-node**
- **workload-multi-node-data-parallel**
- **workload-single-node-pd**
- **workload-multi-node-data-parallel-pd**



NOTE

The form fields above the YAML editor will overwrite the corresponding values in the YAML when you save. If you need to configure multiple supported topologies, you must edit the YAML directly because the topology type form field only allows one selection.

- c. Define the router component configuration in the **spec** section:
For basic routing with intelligent load balancing:

```
spec:
  router:
```

```
scheduler: {} # Default intelligent routing
route: {}    # Default HTTPRoute
gateway: {}  # Default Gateway
```

For advanced routing with KV cache-aware scheduling:

```
spec:
  router:
    scheduler:
      replicas: 1
      config:
        inline: |
          plugins:
            - name: prefix-cache-scorer
              weight: 100
            - name: queue-scorer
              weight: 50
      route: {}
      gateway: {}
```

The router configuration is created and becomes available in the model deployment wizard's advanced routing section for users deploying models with compatible topologies.

Verification

1. On the **llm-d routing configurations** page, verify that your new router configuration appears in the list.
2. Navigate to **Data Science Projects**, select a project, and click **Deploy model**.
3. In the model deployment wizard, select a topology that matches one of the topology types in your router's **supported-topologies** annotation.
4. Expand the **Advanced routing** section.
5. Verify that your router configuration appears in the **Router configuration** list.
6. Select a different topology that is not included in the **supported-topologies** annotation.
7. Expand the **Advanced routing** section and verify that your router configuration does not appear in the list for incompatible topologies.

2.12. EDITING ROUTER CONFIGURATIONS

You can edit, duplicate, or delete existing router configurations from the **Router Configs** page.

Prerequisites

- You have logged in to OpenShift AI with administrator privileges.
- You have created at least one router configuration.

Procedure

To edit a router configuration

1. In the OpenShift AI dashboard, click **Settings**.
2. Click **llm-d routing configurations**.
3. Click the action menu (⋮) next to the configuration you want to modify.
4. Click **Edit**.
5. Modify the configuration:
You can update the form fields or edit the YAML configuration directly.



NOTE

The form fields above the YAML editor will overwrite the corresponding values in the YAML when you save. If you need to configure multiple supported topologies, you must edit the YAML directly because the topology type form field only allows one selection.

6. Click **Save**.

To duplicate a router configuration

1. In the OpenShift AI dashboard, click **Settings**.
2. Click **llm-d routing configurations**.
3. Click the action menu (⋮) next to the configuration you want to copy.
4. Click **Duplicate**.
5. Edit the **metadata.name** field to provide a unique name for the duplicate configuration.
6. Modify the **supported-topologies** annotation or router settings as needed.
7. Click **Save**.

To delete a router configuration

1. In the OpenShift AI dashboard, click **Settings**.
2. Click **llm-d routing configurations**.
3. Click the action menu (⋮) next to the configuration you want to remove.
4. Click **Delete**.
5. In the confirmation dialog, click **Delete** to confirm.



NOTE

Deleting a router configuration does not affect existing deployed models that reference it via baseRefs. However, users will no longer be able to select this configuration when deploying new models.

Verification

1. On the **llm-d routing configurations** page, verify that your changes appear correctly.
2. If you edited or created a router configuration, navigate to the model deployment wizard and verify that it appears in the advanced routing section for compatible topologies.

2.13. DEPLOY DISAGGREGATED PREFILL/DECODE TOPOLOGY

You can deploy large language models by using a disaggregated prefill/decode topology to optimize resource utilization and performance. In this topology, prefill operations (prompt processing) and decode operations (token generation) run on separate replica sets with different resource allocations and scheduler configurations optimized for each phase.

About this task

Disaggregated prefill/decode deployments separate the two phases of LLM inference into specialized replica sets:

Prefill phase

Processes the input prompt and generates the initial key-value (KV) cache. This phase is compute-intensive and benefits from high GPU compute capacity. Prefill replicas use a scheduler profile optimized for queue depth, KV cache utilization, and prefix cache reuse.

Decode phase

Generates output tokens one at a time by using the KV cache from the prefill phase. This phase is memory-intensive and benefits from high memory bandwidth. Decode replicas use a scheduler profile optimized for active request count and prefix cache affinity.

For guidance on when disaggregated serving is appropriate for your workload, see [Workload characteristics for disaggregated inference](#).

Prerequisites

- You have cluster administrator privileges or namespace administrator privileges for the namespace where you want to deploy the LLM inference service.
- You have installed OpenShift AI and configured the Distributed Inference with llm-d operator.
- RDMA-capable networking such as InfiniBand or RoCE v2 is available. NIXL KV cache transfers between prefill and decode pods require RDMA, even when both pods run on the same node. Kubernetes pods are separate processes and cannot use NVLink for inter-pod communication.
- You have sufficient GPU resources to support separate prefill and decode replica sets.
- You have network access to download the model from Hugging Face or have the model available in a private registry.
- For the full list of platform-specific prerequisites, see [Configure prefill/decode disaggregated serving](#).

Procedure

1. Create a YAML file for your disaggregated prefill/decode **LLMInferenceService** resource:

```
apiVersion: serving.kserve.io/v1alpha2
```

```

kind: LLMInferenceService
metadata:
  name: <llm_service_name>
  namespace: <namespace>
spec:
  model:
    uri: hf://<model_uri>
    name: <model_name>

  # Prefill replica configuration
  prefill:
    minReplicas: 2
    maxReplicas: 4
    resources:
      requests:
        nvidia.com/gpu: "1"
        cpu: "4"
        memory: "16Gi"
      limits:
        nvidia.com/gpu: "1"
        cpu: "8"
        memory: "32Gi"

  # Decode replica configuration
  minReplicas: 2
  maxReplicas: 8
  resources:
    requests:
      nvidia.com/gpu: "1"
      cpu: "2"
      memory: "32Gi"
    limits:
      nvidia.com/gpu: "1"
      cpu: "4"
      memory: "64Gi"

```

where:

<llm_service_name>

Specifies the name of your LLM inference service.

<namespace>

Specifies the namespace where you want to deploy the service.

<model_uri>

Specifies the Hugging Face model repository path, for example **meta-llama/Llama-3.1-8B-Instruct**.

<model_name>

Specifies the model name used in API requests.

spec.prefill

Configures the prefill replica set. When this field is present, the system automatically applies disaggregated prefill/decode topology. The Operator processes these values into preset templates and normalizes the stored **spec.prefill** to an empty object **{}**.

spec.prefill.resources

Specifies resource requests and limits for prefill pods. Allocate more CPU and compute-optimized GPUs for prompt processing.

spec.resources

Specifies resource requests and limits for decode pods (the default replica set). Allocate more memory and memory-optimized GPUs for token generation.

2. Apply the configuration to your cluster:

```
$ oc apply -f <llm_service_yaml>
```

3. Verify that both prefill and decode pods are running:

```
$ oc get pods -n <namespace> | grep <llm_service_name>
```

You should see separate pod sets for prefill and decode replicas.

Verification

- Verify that the disaggregated topology is active:

```
$ oc get llminferenceservice <llm_service_name> -n <namespace> -o jsonpath={.spec.prefill}
```

The output shows `{}` (an empty object), confirming disaggregated mode is enabled. The Operator normalizes **spec.prefill** to this empty presence marker after processing your configuration into workload templates.

- Check that the scheduler automatically configured separate profiles:

```
$ oc get pods -n <namespace> | grep router-scheduler
$ oc logs <epp_pod_name> -n <namespace> | grep -i "profile\|disagg"
```

The EPP pod name follows the pattern **<service_name>-kserve-router-scheduler-
<random_suffix>**. The EPP pod logs show initialization messages confirming disaggregated profile handler and separate prefill/decode scheduler profiles.

2.13.1. Understanding the automatic scheduler configuration

When you configure **spec.prefill**, the system automatically applies a disaggregated scheduler configuration optimized for the two-phase topology. You do not need to manually configure the scheduler for disaggregated deployments.

The automatic configuration includes:

Plugins

```
plugins:
- type: disagg-headers-handler    # Routes requests to prefill or decode phase
- type: prefill-filter           # Filters endpoints for prefill profile
- type: decode-filter            # Filters endpoints for decode profile
- type: queue-scorer
- type: kv-cache-utilization-scorer
```

- ```
- type: active-request-scorer # Used in decode profile
- type: prefix-cache-scorer
- type: max-score-picker
```

### Prefill profile (weight: 2:2:3)

Routes initial prompt processing requests to prefill replicas. Optimized for queue depth, KV cache utilization, and prefix cache reuse.

```
schedulingProfiles:
- name: prefill
 plugins:
 - pluginRef: prefill-filter
 - pluginRef: prefix-cache-scorer
 weight: 3
 - pluginRef: queue-scorer
 weight: 2
 - pluginRef: kv-cache-utilization-scorer
 weight: 2
 - pluginRef: max-score-picker
```

### Decode profile (weight: 2:3)

Routes token generation requests to decode replicas. Uses active-request-scorer instead of queue-scorer because active request count is a better signal for ongoing generation workloads.

```
- name: decode
 plugins:
 - pluginRef: decode-filter
 - pluginRef: active-request-scorer
 weight: 2
 - pluginRef: prefix-cache-scorer
 weight: 3
 - pluginRef: max-score-picker
```

The disaggregated scheduler configuration is created automatically when **spec.prefill** is present in your **LLMInferenceService** resource. You cannot override this configuration with inline or ConfigMap-based scheduler settings in disaggregated deployments.

### Additional resources

- For information about the default scheduler configuration for standard deployments, see [Section 10.1, “Scheduler configuration for LLM inference services”](#).
- For information about available scorer plugins, see [Section 10.6, “Available plugins for your inference workload”](#).



## CHAPTER 3. CONFIGURE GATEWAY API FOR DISTRIBUTED INFERENCE WITH LLM-D

Discover and select existing Kubernetes Gateway resources when deploying **LLMInferenceService** models. This Technology Preview feature builds on distributed inference capabilities by providing visibility into available Gateways.



### IMPORTANT

Gateway discovery for Distributed Inference with llm-d is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

### 3.1. GATEWAY DISCOVERY FOR DISTRIBUTED INFERENCE WITH LLM-D

When deploying **LLMInferenceService** models, you can discover and select an existing Kubernetes Gateway resource directly from the model serving UI. This eliminates the need to manually configure a Gateway reference and provides visibility into which Gateway you have permission to use.

Gateway discovery is disabled by default and must be enabled by a cluster administrator through the dashboard configuration resource.

#### Which Gateways appear in the dropdown?

A Gateway is available only when both of the following conditions are met:

- RBAC permissions: You must have permissions to create **LLMInferenceService** resources in the target namespace.
- Listener configuration: The Gateway must include a listener configured to accept routes from the target namespace.

To deploy models with specific Gateway configurations or to reference multiple Gateways, use the YAML-based deployment approach. For more information, see [Select Gateway using YAML](#).



### NOTE

The system does not validate against a specific **GatewayClass** resource or its controller. The default OpenShift Gateway controller provides all Distributed Inference with llm-d features. Third-party Gateway controllers might be incompatible with Distributed Inference with llm-d.

### 3.2. ENABLE GATEWAY DISCOVERY IN THE DASHBOARD

Enable Gateway discovery in the OpenShift AI dashboard so that you can discover and select Kubernetes Gateway resources during model deployment.

## Prerequisites

- You have cluster administrator access to OpenShift.
- You have enabled distributed inference with the Gateway API installed. For more information, see [Enabling distributed inference](#).

## Procedure

1. Enable Gateway discovery in the dashboard configuration:

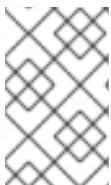
```
$ oc patch odhdashboardconfig odh-dashboard-config \
-n redhat-ods-applications \
--type merge \
-p '{"spec":{"dashboardConfig":{"llmGatewayField":true}}}'
```

## Verification

1. Log in to the OpenShift AI dashboard.
2. In the navigation menu, click **AI Hub**.
3. Click a project name, then click **Deploy model**.
4. Verify that the **Gateway** field is displayed in **Advanced Settings**.

## 3.3. SELECT A GATEWAY FROM THE MODEL DEPLOYMENT WIZARD

You can discover and select an existing Kubernetes Gateway resource during an **LLMInferenceService** deployment through the OpenShift AI model deployment wizard. The Gateway can be the shared **openshift-ai-inference** Gateway configured during distributed inference setup or a namespace-scoped Gateway.



### NOTE

If you edit a deployment with multiple Gateways through the model deployment wizard UI, the **Gateway Selection** field displays only the first Gateway. Gateway selection does not support MaaS Gateways.

## Prerequisites

- You have enabled distributed inference as described in [Enabling distributed inference](#).
- Gateway discovery is enabled in the dashboard. For more information, see [Enabling Gateway discovery in the dashboard](#).
- You have permission to create **LLMInferenceService** resources in your namespace.
- At least one Gateway exists with a listener configured to accept routes from your namespace.
- You have a model stored in S3-compatible storage, a persistent volume claim (PVC), or an OCI container registry or accessible through a URI.

## Procedure

1. Log in to the OpenShift AI dashboard.
2. In the navigation menu, click **AI Hub**.
3. Click the name of the project that you want to deploy a model in.
4. Click **Deploy model**.
5. Configure the model deployment properties as required for your model. For more information, see *Deploying models on the single-model serving platform* .
6. In **Advanced Settings**, locate the **Gateway Selection** section.
7. In the **Gateway** section, locate the **Select a gateway** dropdown.
8. Select the Gateway from the dropdown list.
9. Click **Next**.
10. Click **Deploy Model**.

### Verification

1. On the **Models** page, verify that the **LLMInferenceService** is listed with a checkmark in the **Status** column.
2. Click the deployed model name to view details.
3. Verify that the Gateway reference is configured in the deployed model resource:

```
$ oc get llminferenceservices <llmisvc_name> -n <namespace> -o yaml
```

where:

**<llmisvc\_name>**

Specifies the name of your **LLMInferenceService**.

**<namespace>**

Specifies the project namespace where you deployed the model.

The output includes the Gateway reference configuration. The Gateway reference appears under **spec.router.gateway**.

### Troubleshooting

#### The Gateway dropdown is empty or does not show any available Gateways

- **Check RBAC permissions:** You might not have permission to create **LLMInferenceService** resources in your namespace. Contact your cluster administrator to verify your RBAC permissions.
- **Check listener configuration:** You might not have Gateways that are configured to accept routes from your namespace. Verify that at least one Gateway has a listener configured with **allowedRoutes.namespaces** that includes your namespace.

### 3.4. SELECT GATEWAY USING YAML

Select Kubernetes Gateway resources for an **LLMInferenceService** deployment by creating the resource using YAML. This approach enables you to reference multiple Gateways or specify detailed Gateway configurations that are not available through the model deployment wizard.



#### NOTE

If you edit a deployment with multiple Gateways through the model deployment wizard UI, the **Gateway Selection** field displays only the first Gateway. Gateway selection does not support MaaS Gateways.

#### Prerequisites

- You have enabled distributed inference as described in [Enabling distributed inference](#).
- You have permission to create **LLMInferenceService** resources in your namespace.
- At least one Gateway exists with a listener configured to accept routes from your namespace.
- You have a model stored in S3-compatible storage, a persistent volume claim (PVC), an OCI container registry, or accessible through a URI.

#### Procedure

1. Create a YAML file for your **LLMInferenceService** with Gateway references:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
 name: <llmisvc_name>
 namespace: <namespace>
spec:
 modelSource:
 storageUri: <storage_uri>
 runtime:
 name: <runtime_name>
 router:
 gateway:
 refs:
 - name: <gateway_name>
 namespace: <gateway_namespace>
```

where:

#### <llmisvc\_name>

Specifies the name for your **LLMInferenceService**.

#### <namespace>

Specifies your project namespace.

#### <storage\_uri>

Specifies the location of your model, such as **s3://my-bucket/my-model/** or **pvc://my-pvc/my-model/**.

**<runtime\_name>**

Specifies the serving runtime to use, such as **vllm-runtime**.

**<gateway\_name>**

Specifies the name of the Gateway to use.

**<gateway\_namespace>**

Specifies the namespace where the Gateway is located.

- To reference multiple Gateways, add additional entries to the **refs** list:

```
router:
 gateway:
 refs:
 - name: <gateway_name_1>
 namespace: <gateway_namespace_1>
 - name: <gateway_name_2>
 namespace: <gateway_namespace_2>
```

- Apply the YAML file:

```
$ oc apply -f <filename>.yaml
```

## Verification

- Verify that the **LLMInferenceService** was created:

```
$ oc get llminferenceservices <llmisvc_name> -n <namespace>
```

where:

**<llmisvc\_name>**

Specifies the name of your **LLMInferenceService**.

**<namespace>**

Specifies your project namespace.

- Verify the Gateway references in the deployed resource:

```
$ oc get llminferenceservices <llmisvc_name> -n <namespace> -o yaml
```

The output includes the Gateway reference configuration under **spec.router.gateway.refs**.

- Verify that the **LLMInferenceService** is ready:

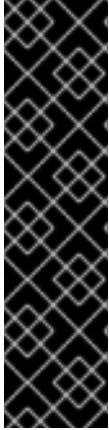
```
$ oc get llminferenceservices <llmisvc_name> -n <namespace> -o
jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
```

The output should be **True** when the service is ready.

## 3.5. CONFIGURE THE INFERENCE GATEWAY ON OPENSIFT CONTAINER PLATFORM

Verify the inference gateway that the Helm chart creates automatically on OpenShift Container Platform.

The Helm chart automatically creates a **GatewayClass** and **Gateway** named **openshift-ai-inference** in the **openshift-ingress** namespace. The KServe controller generates per-workload TLS certificates for secure communication between the gateway and backend inference services.



## IMPORTANT

The Helm chart requires you to set **allowedRoutes.namespaces.from** for the Gateway listener.

- Set it to **Selector** to restrict route attachment to namespaces that carry a specific label.
- Set it to **Same** to allow only the gateway namespace to attach routes.

To manage the Gateway independently of the Helm chart, disable the default Gateway and create one manually as described in the following alternative procedure.

## Prerequisites

- You have installed and configured the OpenShift CLI (**oc**) to access your cluster.
- You have logged in as a user with **cluster-admin** privileges.
- Distributed Inference with llm-d infrastructure components are deployed by the Helm chart (cert-manager, Istio, KServe).

## Procedure

1. Verify that the Helm chart created the **GatewayClass** and **Gateway**:

```
$ oc get gatewayclass openshift-ai-inference
```

```
$ oc get gateway openshift-ai-inference -n openshift-ingress -o wide
```

Wait until **PROGRAMMED** shows **True**, indicating the gateway is ready to accept traffic.

2. Verify that the gateway pod is running:

```
$ oc get pods -n openshift-ingress \
-l gateway.networking.k8s.io/gateway-name=openshift-ai-inference
```



## NOTE

If the gateway shows **Programmed: False**, check istiod logs: **oc logs deploy/istiod -n istio-system | grep gateway**. A common cause is a missing or misconfigured Gateway resource.

## Verification

Verify the gateway is configured correctly:

- The Gateway resource exists and is **PROGRAMMED**:

```
$ oc get gateway openshift-ai-inference -n openshift-ingress -o wide
```

- The Gateway pod is running:

```
$ oc get pods -n openshift-ingress -l gateway.networking.k8s.io/gateway-name=openshift-ai-inference
```

- After deploying an **LLMInferenceService**, on bare metal or disconnected clusters, verify that Route discovery has populated the **status.addresses** field:

```
$ oc get llm_svc -n <NAMESPACE> -o jsonpath='{.items[*].status.addresses}'
```

<**NAMESPACE**> specifies the namespace where your **LLMInferenceService** workloads are deployed.

Look for entries with **origin.kind: Route**, which indicate that the controller discovered OpenShift Routes and is using them to provide external URLs.



#### NOTE

Route discovery is automatic when the inference gateway does not have externally reachable IP addresses. Routes with path-based routing are not compatible with automatic discovery. For more information, see [OpenShift Route discovery for LLMInferenceService](#).

## 3.6. CREATE A SECURE INFERENCE GATEWAY MANUALLY ON OPENSIFT CONTAINER PLATFORM

To restrict which namespaces can attach routes to the inference gateway, disable the default Gateway that the Helm chart creates and create a Gateway manually with **allowedRoutes.namespaces.from: Selector**.

### Procedure

1. Re-run the Helm chart with the Gateway creation disabled:

```
$ helm upgrade --install rhoai \
 oci://registry.redhat.io/rhai/rhai-on-openshift-chart:v3.5 \
 -n rhoai-gitops --create-namespace \
 --set profile=rhaii \
 --set operator.type=rhoai \
 --set operator.rhoai.olm.channel=3.5-stable \
 --set components.kserve.gateway.create=false
```

2. Create the Gateway resource with restricted namespace access:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
 name: openshift-ai-inference
 namespace: openshift-ingress
```

```

labels:
 istio.io/rev: openshift-gateway
spec:
 gatewayClassName: openshift-ai-inference
 listeners:
 - name: https
 port: 443
 protocol: HTTPS
 allowedRoutes:
 namespaces:
 from: Selector
 selector:
 matchLabels:
 inference-gateway-access: "true"
 tls:
 mode: Terminate
 certificateRefs:
 - name: openshift-ai-inference-tls

```

The **allowedRoutes.namespaces.from: Selector** setting restricts route attachment to only namespaces labeled with **inference-gateway-access=true**, preventing unauthorized services from attaching routes to this gateway.

3. Label the namespaces that need to attach routes to the gateway:

```
$ oc label ns openshift-ingress inference-gateway-access=true
```

To label all namespaces that contain **LLMInferenceService** workloads, run the following command:

```

$ oc get llminferenceservice -A \
 -o jsonpath='{range .items[*]}{.metadata.namespace}"\n"}{end}' \
 | sort -u \
 | xargs -l{} oc label ns {} inference-gateway-access=true --overwrite

```

4. Apply the Gateway resource:

```
$ oc apply -f openshift-ai-inference.yaml
```

5. Verify the gateway is ready:

```
$ oc get gateway openshift-ai-inference -n openshift-ingress -o wide
```

Wait until **PROGRAMMED** shows **True**.

## Verification

1. Verify the gateway pod is running:

```

$ oc get pods -n openshift-ingress \
 -l gateway.networking.k8s.io/gateway-name=openshift-ai-inference

```



## CHAPTER 4. CONFIGURE AUTHENTICATION FOR DISTRIBUTED INFERENCE WITH LLM-D BY USING RED HAT CONNECTIVITY LINK

Red Hat Connectivity Link provides Kubernetes-native authentication and authorization capabilities for Distributed Inference with llm-d inference endpoints when platform authentication is enabled on an **LLMInferenceService**. Red Hat Connectivity Link works with the gateway to intercept incoming traffic before it reaches the vLLM inference service, validating the requests based on authentication tokens and authorization policies. For more information about Red Hat Connectivity Link concepts and capabilities, see [Introduction to Red Hat Connectivity Link](#).



### IMPORTANT

This procedure applies when you enable platform authentication on **LLMInferenceService** resources. Red Hat Connectivity Link is not required for every Distributed Inference with llm-d deployment. To deploy without platform authentication, set **security.opendatahub.io/enable-auth: "false"** on the **LLMInferenceService** as described in [Enabling authentication and authorization for an LLM inference service](#).

### Prerequisites

- You have installed Red Hat Connectivity Link version 1.1.1 or later. For more information, see [Installing Connectivity Link on OpenShift](#).
- You have access to the OpenShift CLI (**oc**).
- The ServiceAccount has permission to get the corresponding **LLMInferenceService** and you have generated a JSON web token (JWT).

### Procedure

1. Create the Kuadrant custom resource (CR) to set up required objects:

```
oc apply -f - <<EOF
apiVersion: kuadrant.io/v1beta1
kind: Kuadrant
metadata:
 name: kuadrant
 namespace: kuadrant-system
EOF
```

2. Wait for Kuadrant to become ready:

```
oc wait Kuadrant -n kuadrant-system kuadrant --for=condition=Ready --timeout=10m
```

3. Add the **ServingCert** annotation to the Authorino Service:

```
oc annotate svc/authorino-authorino-authorization service.beta.openshift.io/serving-cert-secret-name=authorino-server-cert -n kuadrant-system
```

4. Wait for the secret to be created:

```
sleep 2
```

5. Update Authorino to enable SSL:

```
oc apply -f - <<EOF
apiVersion: operator.authorino.kuadrant.io/v1beta1
kind: Authorino
metadata:
 name: authorino
 namespace: kuadrant-system
spec:
 replicas: 1
 clusterWide: true
 listener:
 tls:
 enabled: true
 certSecretRef:
 name: authorino-server-cert
 oidcServer:
 tls:
 enabled: false
EOF
```

6. Verify that the Authorino pods are ready:

```
oc wait --for=condition=ready pod -l authorino-resource=authorino -n kuadrant-system --
timeout 150s
```

7. If OpenShift AI was installed before installing Connectivity Link and Kuadrant, restart the controllers:

```
oc delete pod -n redhat-ods-applications -l app=odh-model-controller
oc delete pod -n redhat-ods-applications -l control-plane=kserve-controller-manager
```

## CHAPTER 5. ENABLE AUTHENTICATION AND AUTHORIZATION FOR AN LLM INFERENCE SERVICE

In OpenShift AI 3.0 and later, authentication and authorization are automatically enabled for **LLMInferenceService** resources when Red Hat Connectivity Link is configured. You can use the **security.opendatahub.io/enable-auth: "true"** annotation to explicitly enable authentication, such as re-enabling it after it was previously disabled.

### Prerequisites

- You have configured Red Hat Connectivity Link for Distributed Inference with *llm-d* as described in *Configuring authentication for Distributed Inference with llm-d using Red Hat Connectivity Link*.
- You have created an **LLMInferenceService** resource as described in *Enabling Distributed Inference with llm-d*.
- You have access to the OpenShift CLI (**oc**).

### Procedure

1. Disable platform authentication when Red Hat Connectivity Link is not installed or you do not want platform authentication on the resource:

```
$ oc annotate llminferenceservice <LLMISVC-NAME> -n <LLMISVC-NAMESPACE>
security.opendatahub.io/enable-auth=false --overwrite
```

Replace **<LLMISVC-NAME>** with your **LLMInferenceService** name and **<LLMISVC-NAMESPACE>** with your project namespace. The **--overwrite** flag is required if the annotation already exists on the resource.

Alternatively, include the annotation in the **LLMInferenceService** manifest before you deploy the resource:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
 name: <LLMISVC-NAME>
 annotations:
 security.opendatahub.io/enable-auth: "false"
spec:
 ...
```

2. To enable or re-enable platform authentication, complete the following prerequisites and annotate the resource:
  - You have configured Red Hat Connectivity Link for Distributed Inference with *llm-d* as described in [Configuring authentication for Distributed Inference with llm-d using Red Hat Connectivity Link](#).  
By default, platform authentication is enabled when Red Hat Connectivity Link is installed. To explicitly enable authentication or to re-enable it after disabling, annotate your **LLMInferenceService** resource:

```
apiVersion: serving.kserve.io/v1alpha1
```

```

kind: LLMInferenceService
metadata:
 name: sample-llm-inference-service
 annotations:
 security.opendatahub.io/enable-auth: "true"
spec:
 replicas: 2
 model:
 uri: hf://RedHatAI/Qwen3-8B-FP8-dynamic
 name: RedHatAI/Qwen3-8B-FP8-dynamic
 router:
 route: {}
 gateway: {}
 scheduler: {}
 template:
 containers:
 - name: main
 resources:
 limits:
 cpu: '4'
 memory: 32Gi
 nvidia.com/gpu: "1"
 requests:
 cpu: '2'
 memory: 16Gi
 nvidia.com/gpu: "1"

```

3. Apply the configuration:

```
$ oc apply -f <llm-inference-service-file>.yaml
```

## Verification

- Confirm that the **LLMInferenceService** resource has the annotation:

```
$ oc get llminferenceservice <LLMISVC-NAME> -n <LLMISVC-NAMESPACE> -o
jsonpath={.metadata.annotations.security\.opendatahub\.io/enable-auth}
```

The command returns **true**.

- Verify that the inference service is protected by attempting to access it without authentication:

```
$ curl -v https://<inference-endpoint-url>/v1/models
```

The request returns a **401 Unauthorized** response, confirming that unauthenticated requests are rejected.

## IMPORTANT

When using controlled deployment with routing groups, all members of a routing group must have the same authentication posture. The **security.opendatahub.io/enable-auth** annotation must have the same value, either **"true"** or **"false"**, on every **LLMInferenceService** in the group.

If the authentication posture does not match across members, the controller emits an **AuthPostureMismatch** warning event. A member annotated with **security.opendatahub.io/enable-auth: "false"** receives an anonymous **AuthPolicy** on its own **HTTPRoute**. Members with gateway-level authentication enabled have no per-route policy and are protected by the gateway-scoped policy. Because every member's route carries the same publisher-path rule, Gateway API conflict resolution selects the oldest matching route for the shared publisher path. If the oldest member, typically the production version, has authentication disabled, its anonymous policy applies to the publisher path for the entire group. Authentication is enabled by default. Only an explicit **security.opendatahub.io/enable-auth: "false"** annotation disables authentication.

To check for authentication posture mismatches in a routing group:

```
$ oc get events -n <namespace> --field-selector reason=AuthPostureMismatch
```

To resolve a mismatch, update the deviating member's annotation to match the other group members:

```
$ oc annotate llminferenceservice <deviating-llmisvc-name> -n <namespace> \
 security.opendatahub.io/enable-auth=true --overwrite
```

### Additional resources

- [Override your Gateway policies for auth and rate limiting](#)

## 5.1. MAKE AUTHENTICATED INFERENCE REQUESTS TO DISTRIBUTED INFERENCE WITH LLM-D

When you enable authentication for an **LLMInferenceService**, you must include a valid JSON web token (JWT) in your inference requests. You can generate a token from a ServiceAccount or use an OIDC token from an identity provider.

### Prerequisites

- You have enabled authentication for your **LLMInferenceService** as described in *Enabling authentication and authorization for an LLM inference service*.
- You have access to the OpenShift CLI (**oc**).
- You have the inference endpoint URL for your deployed model.

### Procedure

1. Create a ServiceAccount with permissions to access the **LLMInferenceService**:

```
oc create serviceaccount llm-user -n <namespace>
```

2. Create a Role that grants permission to get the **LLMInferenceService**:

```
oc apply -f - <<EOF
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: llm-inference-viewer
 namespace: <namespace>
rules:
- apiGroups: ["serving.kserve.io"]
 resources: ["llminferenceservices"]
 verbs: ["get"]
 resourceNames: ["<llm-inference-service-name>"]
EOF
```

3. Bind the Role to the ServiceAccount:

```
oc create rolebinding llm-user-binding \
 --role=llm-inference-viewer \
 --serviceaccount=<namespace>:llm-user \
 -n <namespace>
```

4. Generate a JWT token from the ServiceAccount:

```
TOKEN=$(oc create token llm-user -n <namespace> --duration=1h)
```

The **--duration** flag sets the token validity period. Adjust this value based on your workload requirements. For production use, consider using a shorter duration or integrating with an OIDC provider.

## Verification

- Verify that the authenticated request was successful:

```
curl -v https://<inference-endpoint-url>/v1/chat/completions \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer ${TOKEN}" \
 -d '{
 "model": "<model-name>",
 "messages": [{
 "role": "user",
 "content": "What is Red Hat OpenShift AI?"
 }]
 }'
```

A successful response indicates that authentication is working correctly.

- Verify that requests without authentication are rejected:

```
curl -v https://<inference-endpoint-url>/v1/chat/completions \
 -H "Content-Type: application/json" \
 -d '{
 "model": "<model-name>",
 "messages": [{
```

```
"role": "user",
"content": "What is Red Hat OpenShift AI?"
}]
'
```

The request returns a **401 Unauthorized** response with a message indicating that authentication is required.

- Verify that requests with an invalid or expired token are rejected:

```
curl -v https://<inference-endpoint-url>/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer invalid-token" \
-d '{
 "model": "<model-name>",
 "messages": [{
 "role": "user",
 "content": "What is Red Hat OpenShift AI?"
 }]
'
```

The request returns a **401 Unauthorized** response, confirming that invalid tokens are rejected.

### Additional resources

- For information about OpenShift authentication, see [Understanding authentication](#).
- For information about ServiceAccount tokens, see [Using bound service account tokens](#).

## CHAPTER 6. VALIDATE INFERENCE WORKLOAD CHANGES WITH CONTROLLED DEPLOYMENT

You can use controlled deployment with Distributed Inference with llm-d to validate engine upgrades, model version rotations, and configuration changes on a fraction of production traffic before committing to a full rollout. You deploy two or more versions of an inference workload side by side in the same routing group, control the traffic fraction each version receives using declarative weight-based traffic splitting, and compare per-version Prometheus metrics in real time.

### 6.1. CONTROLLED DEPLOYMENT FOR INFERENCE WORKLOADS

You can use controlled deployment with Distributed Inference with llm-d to validate engine upgrades, model version rotations, and configuration changes on a fraction of production traffic before committing to a full rollout. Controlled deployment replaces all-or-nothing updates with a declarative, reversible workflow that avoids the risk of degrading inference quality for all users when a new version performs worse than expected.

Controlled deployment uses a symmetric group-and-weight model. Each **LLMInferenceService** independently declares the routing group it belongs to and its proportional share of traffic. There is no distinction between a "baseline" and a "canary" version: all members of a routing group are peers that share weighted traffic distribution through a Gateway API **HTTPRoute**.

#### 6.1.1. How the group and weight model works

You assign a routing group by setting the **spec.router.route.group** field on each **LLMInferenceService** that participates in traffic splitting. You set the proportional traffic share by configuring the **spec.router.route.weight** field. The controller translates declared weights into Gateway API **HTTPRoute** weighted **backendRef** entries by using proportional semantics.

Weights are ratios, not percentages. There is no requirement that weights sum to 100. For example, if version A has weight 9 and version B has weight 1, the gateway distributes approximately 90% of traffic to version A and 10% to version B. You can use any integer values between 0 and 1,000,000 to express the required ratio. A weight of 0 removes the member from new request routing after gateway propagation completes. In-flight requests and requests arriving during the configuration propagation window can still reach the member. The propagation window is typically seconds, with no guaranteed upper bound. The **backendRef** remains in the **HTTPRoute** at weight 0 rather than being removed.

#### 6.1.2. Version isolation

Each version in a controlled deployment is a fully independent **LLMInferenceService** resource with its own runtime pods, **InferencePool**, and inference scheduler instance. This per-version isolation ensures that a failed version cannot affect the other version's scheduling, KV cache state, or resource allocation.

Running multiple versions requires provisioning separate compute resources, including GPU nodes, for each version.

#### 6.1.3. Per-member HTTPRoute lifecycle

Each **LLMInferenceService** in a routing group creates and owns its own **HTTPRoute**. Every member's **HTTPRoute** carries weighted **backendRef** entries for all active, non-stopped members of the group that serve the same model. Gateway API precedence rules select the oldest matching route as the active route for traffic decisions.



The controller uses finalizers to manage **HTTPRoute** lifecycle safely. When you delete a version, the controller removes the corresponding **backendRef** from peer members' **HTTPRoutes** before allowing the deletion to proceed. A member's own **HTTPRoute** is deleted when the member itself is removed.

### 6.1.4. Observability

Per-version metric collection requires creating **PodMonitor** and **ServiceMonitor** resources for each version. A **PodMonitor** with a relabeling rule injects an **llm\_isvc\_name** label into vLLM metrics, enabling cross-version comparison of inference health metrics such as time to first token, throughput, error rate, and queue depth. For more information about configuring per-version observability, see [Monitor per-version inference metrics during controlled deployment](#).

### 6.1.5. Controlled deployment lifecycle

A controlled deployment follows four phases:

1. **Deploy:** Create a second **LLMInferenceService** with the same routing group name as the production version. Start with a low weight, such as 1, so only a small fraction of traffic reaches the new version.
2. **Observe:** Compare per-version inference metrics, such as time to first token, throughput, error rate, and queue depth, using Prometheus queries filtered by **llm\_isvc\_name**. Use the metrics comparison to make an evidence-based promotion or rollback decision.
3. **Promote or roll back** Shift traffic by adjusting weights on both versions. To promote, set the old version weight to 0. To roll back, set the new version weight to 0 and restore the original version weight.
4. **Clean up:** After promotion, optionally force-stop the old version to reclaim GPU resources. Delete the old version only after verifying that all group members have reconciled.

Promotion by weight change and deletion by resource cleanup are separate operations. You can separate them by hours or days, keeping the old version available for rollback while it receives no traffic.

### 6.1.6. Constraints

The following constraints apply to controlled deployments:

- All members of a routing group must be in the same namespace.
- Controlled deployment requires Gateway API mode. Ingress mode is not supported.
- Controlled deployment requires controller-managed routes. Custom **HTTPRoute** refs, also known as bring-your-own (BYO) routes, are not supported with traffic splitting.
- All members of a routing group must have the same authentication posture: the **security.opendatahub.io/enable-auth** annotation must have the same value on every member. A mismatch produces an **AuthPostureMismatch** warning event.

### 6.1.7. Model access patterns

When you deploy multiple versions in a routing group, clients can access the model through different URL patterns:

#### Per-participant path

A direct URL to a specific **LLMInferenceService** instance, following the pattern `/_<namespace>/_<llmisvc-name>_`. This path is version-specific and changes when you rotate versions.

#### Publisher path

A stable, version-independent URL that follows the pattern `/publishers/_<namespace>/models/_<model-name>_`. This path survives version rotations because it references the model name rather than the **LLMInferenceService** instance name. Publisher paths require model-level RBAC authorization.

#### Model-routing header

A request-level header that routes traffic to a specific model through the gateway scheduler. Model-routing uses the model name for routing decisions and does not require a version-specific URL.

### 6.1.8. Scope limitations

The following capabilities are not supported in controlled deployment:

- Automated promotion or rollback based on metric thresholds. All promotion and rollback decisions are manual.
- A/B testing with user segmentation or cookie-based session persistence.
- KV cache migration between versions.
- Prefill-decode disaggregated controlled deployment. Although the routing group controller does not block disaggregated workloads, this combination has not been validated.

## 6.2. DEPLOY A CANARY VERSION OF AN INFERENCE WORKLOAD

You can deploy a canary version of an inference workload alongside an existing production version to test changes on a fraction of live traffic. By assigning both versions to the same routing group and configuring their weights, you control how much traffic each version receives while avoiding an all-or-nothing rollout.

#### Prerequisites

- You have a production **LLMInferenceService** deployed and serving traffic.
- Your **LLMInferenceService** uses Gateway API mode with controller-managed routes. Ingress mode and custom **HTTPRoute** refs, also known as bring-your-own (BYO) routes, are not supported with traffic splitting.
- You have sufficient compute resources, such as GPUs, to run two versions of the inference workload simultaneously.
- Prometheus metrics collection is enabled for your Distributed Inference with llm-d deployment.
- You have access to the OpenShift CLI (``oc``).

#### Procedure

1. Add the **group** and **weight** fields to your existing production **LLMInferenceService** if they are not already set.

Open your production **LLMInferenceService** manifest and add **spec.router.route.group** and **spec.router.route.weight**:

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
 name: <production-llmisvc-name>
 namespace: <namespace>
spec:
 model:
 name: <model-name>
 uri: <model-uri>
 router:
 route:
 group: <group-name>
 weight: 9
 gateway: {}
 scheduler: {}
 # ...existing spec fields...
```

where:

**\_\_<production-llmisvc-name>\_\_**

Specifies the name of your existing production **LLMInferenceService**.

**\_\_<namespace>\_\_**

Specifies the namespace where the production deployment runs.

**\_\_<model-name>\_\_**

Specifies the model name.

**\_\_<model-uri>\_\_**

Specifies the URI from where the model is downloaded.

**\_\_<group-name>\_\_**

Specifies the routing group name. Use 1 to 63 lowercase alphanumeric characters or hyphens, matching the pattern **^[a-z0-9]([a-z0-9-]\*[a-z0-9])?\$**.

Apply the updated manifest:

```
$ oc apply -f <production-llmisvc-file>.yaml
```

2. Create a manifest for the canary **LLMInferenceService** with the same routing group name as the production version, starting at weight 1:

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
 name: <canary-llmisvc-name>
 namespace: <namespace>
spec:
 model:
 name: <model-name>
 uri: <canary-model-uri>
 replicas: <replica-count>
```

```

router:
 route:
 group: <group-name>
 weight: 1
 gateway: {}
 scheduler: {}
 template:
 containers:
 - name: main
 resources:
 limits:
 nvidia.com/gpu: "<gpu-count>"
...resource configuration...

```

where:

**\_\_<canary-llmisvc-name>\_\_**

Specifies a name for the canary version that is distinct from the production version name.

**\_\_<canary-model-uri>\_\_**

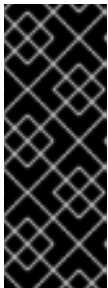
Specifies the model URI for the canary version, such as a new model revision or engine build.

**\_\_<group-name>\_\_**

Specifies the same routing group name used by the production version.

**\_\_<model-name>\_\_**

Specifies the model name for the canary version. In a typical canary deployment, use the same model name as the production version.



## IMPORTANT

Start the canary weight at a small nonzero value such as 1 rather than 0. Deploying at weight 0 and then increasing the weight later can cause transient connection failures while the backend warms up. A small nonzero weight ensures the backend cluster is warmed immediately with minimal traffic. With a production weight of 9 and a canary weight of 1, approximately 10% of traffic goes to the canary version.

3. Apply the canary manifest:

```
$ oc apply -f <canary-llmisvc-file>.yaml
```

4. Wait for the **GroupReady** condition to become **True**:

```
$ oc wait llminferenceservice <canary-llmisvc-name> -n <namespace> \
 --for=condition=GroupReady --timeout=300s
```

5. Verify the routing group membership by listing all **LLMInferenceService** resources in the namespace:

```
$ oc get llminferenceservice -n <namespace> \
 -o custom-
columns=NAME:.metadata.name,GROUP:.status.router.group.name,WEIGHT:.spec.router.rou
te.weight,READY:.status.conditions[?(@.type=="GroupReady")].status
```

The output lists all group members with their configured weights and **GroupReady** status:

| NAME                        | GROUP          | WEIGHT | READY |
|-----------------------------|----------------|--------|-------|
| _<production-llmisvc-name>_ | _<group-name>_ | 9      | True  |
| _<canary-llmisvc-name>_     | _<group-name>_ | 1      | True  |

Verify that both versions belong to the same routing group and that **GroupReady** is **True** on both.

- Adjust the traffic split to shift more traffic to the canary by updating the weight fields on both versions.

For example, to shift to a 50/50 split:

```
$ oc patch llminferenceservice <production-llmisvc-name> -n <namespace> \
 --type merge -p {"spec":{"router":{"route":{"weight":5}}}}
$ oc patch llminferenceservice <canary-llmisvc-name> -n <namespace> \
 --type merge -p {"spec":{"router":{"route":{"weight":5}}}}
```

## Verification

- Verify that the canary version is serving inference requests:

```
$ curl -s -H "Authorization: Bearer <token>" \
 "$(oc get llminferenceservice <canary-llmisvc-name> -n <namespace> \
 -o jsonpath={.status.url})/v1/models"
```

A successful response confirms that the canary version is ready to handle traffic.



## NOTE

All members of a routing group must have matching **security.opendatahub.io/enable-auth** annotation values. If the authentication posture does not match across members, the controller emits an **AuthPostureMismatch** warning event. Check for mismatch events by running:

```
$ oc get events -n <namespace> --field-selector reason=AuthPostureMismatch
```

## Next steps

- [Monitor per-version inference metrics during controlled deployment](#)
- [Promote or roll back a controlled deployment](#)

## 6.3. MONITOR PER-VERSION INFERENCE METRICS DURING CONTROLLED DEPLOYMENT

You can compare inference metrics across versions during a controlled deployment to make evidence-based promotion or rollback decisions. Per-version filtering requires a **PodMonitor** that injects an **llm\_isvc\_name** label into vLLM metrics through Prometheus relabeling.

## Prerequisites

- Prometheus metrics collection is enabled for your Distributed Inference with llm-d deployment.
- You have an active controlled deployment with two or more **LLMInferenceService** versions in the same routing group.
- You have access to the OpenShift web console or a Prometheus-compatible query interface.
- You have access to the OpenShift CLI (``oc``).

## Procedure

1. Create a **PodMonitor** to scrape vLLM metrics with per-version labeling.  
vLLM exposes metrics on port 8000, but its **model\_name** label is the same across versions that serve the same model. To distinguish versions, the **PodMonitor** copies the pod's **app.kubernetes.io/name** label into an **llm\_isvc\_name** metric label:

```
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
 name: llmisvc-vllm
 namespace: <namespace>
spec:
 selector:
 matchLabels:
 kserve.io/component: workload
 podMetricsEndpoints:
 - portNumber: 8000
 path: /metrics
 interval: 10s
 params:
 format:
 - prometheus
 relabelings:
 - sourceLabels: [__meta_kubernetes_pod_label_app_kubernetes_io_name]
 targetLabel: llm_isvc_name
```

Apply the **PodMonitor**:

```
$ oc apply -f <podmonitor-file>.yaml
```

2. Verify that per-version metrics are available in Prometheus by checking for distinct **llm\_isvc\_name** values:

```
vllm:request_success_total{llm_isvc_name=~"<v1-name>|<v2-name>"}
```

Replace **<v1-name>** and **<v2-name>** with the **LLMInferenceService** names for each version. If the **llm\_isvc\_name** label is missing, verify that the **PodMonitor** is active and that the relabeling rule is correct.

3. Verify that the actual traffic distribution matches your configured weights by comparing request rates across versions:

```
sum by(llm_isvc_name) (
 rate(vllm:request_success_total{llm_isvc_name=~"<v1-name>|<v2-name>"}[5m])
)
```

The ratio of request rates between versions should approximate the ratio of configured weights. For example, a 9:1 weight split should produce approximately 90% and 10% of total requests per version.

4. Compare time to first token across versions by running the following PromQL query in the OpenShift web console or your Prometheus interface:

```
histogram_quantile(0.95,
 sum by(le, llm_isvc_name) (
 rate(vllm:time_to_first_token_seconds_bucket{llm_isvc_name=~"<v1-name>|<v2-name>"}
 [5m])
)
)
```

5. Compare additional key metrics using PromQL queries filtered by **llm\_isvc\_name**. The following table lists recommended metrics for promotion decisions.

**Table 6.1. Key metrics for cross-version comparison**

| Metric                   | PromQL query pattern                                                                                                                                     |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Generation throughput    | <b>sum by(llm_isvc_name)</b><br>(rate(vllm:generation_tokens_total{llm_isvc_name=~"_<v1-name>_ <v2-name>_"[5m]))                                         |
| Request success rate     | <b>sum by(llm_isvc_name)</b><br>(rate(vllm:request_success_total{llm_isvc_name=~"_<v1-name>_ <v2-name>_"[5m]))                                           |
| Queue depth              | <b>sum by(llm_isvc_name)</b><br>(vllm:num_requests_waiting{llm_isvc_name=~"_<v1-name>_ <v2-name>_"})                                                     |
| End-to-end latency (P95) | <b>histogram_quantile(0.95, sum by(le, llm_isvc_name)</b><br>(rate(vllm:e2e_request_latency_seconds_bucket{llm_isvc_name=~"_<v1-name>_ <v2-name>_"[5m])) |
| KV cache utilization     | <b>avg by(llm_isvc_name)</b><br>(vllm:kv_cache_usage_perc{llm_isvc_name=~"_<v1-name>_ <v2-name>_"})                                                      |

6. If end-to-end latency is higher than expected, isolate whether the cause is a capacity issue or an engine bottleneck by comparing queue time against prefill and decode time:

**Table 6.2. Latency breakdown queries**

| Component          | PromQL query pattern                                                                                                                                   |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Queue time (P95)   | <b>histogram_quantile(0.95, sum by(le, llm_isvc_name) (rate(vllm:request_queue_time_seconds_bucket{llm_isvc_name=~"_{v1-name}_{v2-name}"}[5m])))</b>   |
| Prefill time (P95) | <b>histogram_quantile(0.95, sum by(le, llm_isvc_name) (rate(vllm:request_prefill_time_seconds_bucket{llm_isvc_name=~"_{v1-name}_{v2-name}"}[5m])))</b> |
| Decode time (P95)  | <b>histogram_quantile(0.95, sum by(le, llm_isvc_name) (rate(vllm:request_decode_time_seconds_bucket{llm_isvc_name=~"_{v1-name}_{v2-name}"}[5m])))</b>  |

If queue time is significantly higher than prefill and decode time combined, the version needs more replicas. If prefill or decode time is high but queue time is low, the latency is an engine-level bottleneck rather than a capacity issue.

- Based on the metric comparison, decide whether to promote or roll back.

**Table 6.3. Decision criteria for controlled deployment**

| Signal               | Promote                                 | Roll back                                                       |
|----------------------|-----------------------------------------|-----------------------------------------------------------------|
| Error rate           | Equal to or lower than baseline         | Higher than baseline                                            |
| Time to first token  | Within acceptable threshold of baseline | Significantly higher than baseline                              |
| End-to-end latency   | Within acceptable threshold of baseline | Significantly higher than baseline                              |
| Queue depth          | Stable and not growing                  | Growing, indicating the version cannot handle its traffic share |
| KV cache utilization | Within expected bounds                  | Approaching capacity or significantly higher than baseline      |
| Pod health           | All pods running and ready              | Pod restarts, readiness failures, or reduced pod count          |



## NOTE

Automated promotion or rollback based on metric thresholds is not supported. All promotion and rollback decisions during controlled deployment are manual. Use the metrics comparison to inform your decision, then follow the promotion or rollback procedures.

## Additional resources



- [Promote or roll back a controlled deployment](#)
- [Controlled deployment metrics for promotion decisions](#)
- [PromQL queries for llm-d monitoring](#)
- [Configure distributed tracing for Distributed Inference with llm-d deployments](#)

## 6.4. PROMOTE OR ROLL BACK A CONTROLLED DEPLOYMENT

After you deploy a canary version and compare per-version inference metrics, you can promote the canary to full traffic or roll back to the previous version. You can also force-stop a version to reclaim GPU resources, and delete old versions after successful promotion.

### Prerequisites

- You have deployed a canary version as described in [Deploy a canary version of an inference workload](#).
- The **GroupReady** condition is **True** on both versions.
- You have access to the OpenShift CLI (`oc`).

### Procedure

1. To promote the canary version, set the production version weight to 0 so that all traffic goes to the canary:

```
$ oc patch llminferenceservice <production-llmisvc-name> -n <namespace> \
 --type merge -p {"spec":{"router":{"route":{"weight":0}}}}
```

The production version pods remain active but idle. Because the production version still exists in the routing group at weight 0, you can roll back by restoring its weight.

2. To roll back during a controlled deployment, restore the original version weight and set the canary version weight to 0:

```
$ oc patch llminferenceservice <production-llmisvc-name> -n <namespace> \
 --type merge -p {"spec":{"router":{"route":{"weight":9}}}}
$ oc patch llminferenceservice <canary-llmisvc-name> -n <namespace> \
 --type merge -p {"spec":{"router":{"route":{"weight":0}}}}
```



### NOTE

Rollback options depend on the state of the original version:

- If the original version still exists with weight 0, adjust weights back to restore traffic.
- If the original version was stopped with the **serving.kserve.io/stop** annotation, remove the annotation and restore the weight.
- If the original version was deleted, you must redeploy it from the original manifest.

- Optional: To reclaim GPU resources from a version that is no longer receiving traffic, force-stop it by applying the **serving.kserve.io/stop** annotation:

```
$ oc annotate llminferenceservice <llmisvc-name> -n <namespace> \
 serving.kserve.io/stop=true
```

A stopped member remains in the routing group but is omitted from **backendRef** entries in peer members' **HTTPRoutes**, regardless of the **spec.router.route.weight** value. The member **stopped** field in **status.router.group.members** is set to **true**.

To resume a stopped version, remove the annotation:

```
$ oc annotate llminferenceservice <llmisvc-name> -n <namespace> \
 serving.kserve.io/stop-
```

- To delete an old version after successful promotion, delete the **LLMInferenceService** resource:

```
$ oc delete llminferenceservice <old-llmisvc-name> -n <namespace>
```



#### WARNING

Do not delete a version that has a weight greater than 0. Deleting a member with active traffic causes **500** errors for in-flight requests until the remaining group members reconcile. Always set the weight to 0 and verify that the **GroupReady** condition reflects the updated group membership before deleting.

The control plane holds the deletion until peer members remove the corresponding **backendRef** from their **HTTPRoutes**, so in-flight requests to other members are not disrupted.

### Verification

- After promotion or rollback, verify the routing group state:

```
$ oc get llminferenceservice -n <namespace> \
 -o custom-
columns=NAME:.metadata.name,GROUP:.status.router.group.name,WEIGHT:.spec.router.ro
ute.weight,READY:.status.conditions[?(@.type=="GroupReady")].status
```

- After force-stopping a version, verify that the member shows as stopped by querying a peer member that is still running in the same routing group:

```
$ oc get llminferenceservice <peer-llmisvc-name> -n <namespace> \
 -o jsonpath={.status.router.group.members} | jq .
```

The peer's **status.router.group.members** lists the stopped member with **"stopped": true**. The **weight** field reports the member's declared **spec.router.route.weight** value.

**NOTE**

Query a peer member, not the stopped service itself. The stopped service's own **status.router** is cleared when the stop annotation is applied.

The following table summarizes each lifecycle operation, its precondition, and expected outcome.

**Table 6.4. Controlled deployment lifecycle operations**

| Operation      | Command                                                        | Precondition                                   | Expected outcome                                                                                                             |
|----------------|----------------------------------------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Promote canary | <b>oc patch</b> old version weight to 0                        | Both versions at <b>GroupReady=True</b>        | All traffic goes to canary. Old version pods remain active but idle.                                                         |
| Roll back      | <b>oc patch</b> canary weight to 0, restore old version weight | Old version still exists in group              | All traffic returns to old version. Canary pods remain active but idle.                                                      |
| Force-stop     | <b>oc annotate</b> with <b>serving.kserve.io/stop=true</b>     | Version weight is 0                            | GPU resources are reclaimed. Member remains in group with <b>stopped=true</b> and is omitted from <b>backendRef</b> entries. |
| Delete         | <b>oc delete llminferenceservice</b>                           | Version weight is 0 and <b>GroupReady=True</b> | Resource removed. Control plane holds deletion until peers reconcile.                                                        |

## 6.5. AUTHORIZATION MODEL FOR CONTROLLED DEPLOYMENT

When you deploy multiple versions of an inference workload in a routing group, you can choose between two RBAC authorization patterns: instance-level access that targets a specific **LLMInferenceService** by name, or model-level access that targets a model name shared across all routing group members. Model-level access provides a stable authorization identity that does not change when you rotate versions.

### 6.5.1. Instance-level and model-level access patterns

Instance-level RBAC uses **SubjectAccessReview** (SAR) checks against the **serving.kserve.io/llminferenceservices** resource with the **get** verb. The resource name is the **LLMInferenceService** name extracted from the per-participant path URL. This pattern works for standalone deployments, but breaks during version rotations because the **LLMInferenceService** name changes when you create a new version.

Model-level RBAC uses SAR checks against the **serving.opendatahub.io/models** virtual resource with the **post** verb. The model name is extracted from the publisher-path URL pattern **/publishers/\_<namespace>\_/models/\_<model-name>\_**. Because the model name is defined by **spec.model.name**, which is the same across all routing group members, access grants survive version rotations without requiring per-instance RBAC updates for every new version.

Table 6.5. Two parallel RBAC domains

| Access pattern                   | URL pattern                                            | SAR resource                                   | SAR verb    |
|----------------------------------|--------------------------------------------------------|------------------------------------------------|-------------|
| Per-participant (instance-level) | /_<namespace>/_<llmisvc-name>/_v1/...                  | <b>serving.kserve.io/llm inferenceservices</b> | <b>get</b>  |
| Publisher path (model-level)     | /publishers/_<namespace>/_models/_<model-name>/_v1/... | <b>serving.opendatahub.io/models</b>           | <b>post</b> |

### 6.5.2. The models virtual resource

The **models** resource in the **serving.opendatahub.io** API group is a virtual resource. It has no backing Custom Resource Definition (CRD), and **oc get models.serving.opendatahub.io** returns a "not found" error. This is by design: the resource exists only as a SAR authorization contract that the controller-managed **AuthPolicy** evaluates at request time.

The **resourceNames** values used in RBAC roles for model-level access follow the publisher-path format: **publishers/\_<namespace>/\_models/\_<model-name>\_**. These are path-shaped strings, not standard Kubernetes resource names.

### 6.5.3. Cross-tenant deny rule

The controller creates an **AuthPolicy** rule named **deny-misrouted-model-header** that blocks requests meeting all of the following conditions:

- The path is not a publisher path: the path does not start with **/publishers/**.
- The path is not a batch path: the path does not start with **/v1/files** or **/v1/batches**.
- A valid model routing header is present in the request.

This rule prevents a request from being routed by one tenant's model header while being authorized by a different tenant's per-participant path identity. The deny rule fires at priority 0, before any SAR-based authorization rules, and returns an immediate **403 Forbidden** response without an API server round-trip.

### 6.5.4. Backward compatibility

Model-level RBAC is additive. Existing per-participant path RBAC continues to work unchanged. You can use instance-level access for standalone deployments and model-level access for routing groups, or both patterns together in the same cluster.

Namespace viewers, editors, and administrators automatically receive model-level access through **ClusterRole** aggregation. The **kserve-models-view** ClusterRole aggregates to **view**, **kserve-models-edit** aggregates to **edit**, and **kserve-models-admin** aggregates to **admin**. No additional RBAC configuration is needed for users who already have namespace-level access.

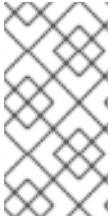
### 6.5.5. Known limitations

- Model names containing a literal **/v1/** segment are not supported because the authorization rule extracts the model name from the URL path using **/v1/** as a delimiter.

- Direct **/v1/** inference endpoints with a model routing header are denied by the **deny-misrouted-model-header** rule. This is a planned follow-up for backend-based routing (BBR) integration.

### 6.5.6. Grant model-level access for inference workloads

You can grant model-level RBAC access so that inference consumers can use publisher-path URLs to access models without needing per-instance RBAC updates on every version rotation. Model-level access uses the **serving.opendatahub.io/models** virtual resource for authorization, providing a stable identity that survives version changes in routing groups.



#### NOTE

Namespace viewers, editors, and administrators automatically receive model-level access through ClusterRole aggregation without additional configuration. You only need to create explicit Role and RoleBinding resources when you want to grant model-level access to users or service accounts that do not have namespace-level roles.

#### Prerequisites

- Authentication is enabled on your **LLMInferenceService** resources. To verify, run **oc get llminferenceservice \_<name>\_ -n \_<namespace>\_ -o jsonpath='{.metadata.annotations.security\.opendatahub\.io/enable-auth}'** and confirm the result is **true** or empty, which indicates the default enabled state. For more information, see [Enable authentication and authorization for an LLM inference service](#).
- You have access to the OpenShift CLI (``oc``).
- You have **admin** or **cluster-admin** permissions on the namespace where the **LLMInferenceService** resources are deployed.

#### Procedure

1. Determine whether to use model-level or instance-level RBAC for your access pattern.

**Table 6.6. When to use each RBAC pattern**

| Access pattern                               | Use model-level RBAC                                                                | Use instance-level RBAC                                                                                                   |
|----------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Controlled deployment with version rotations | Yes. Publisher-path URLs reference the model name, which is stable across versions. | Not recommended. Per-participant URLs reference the <b>LLMInferenceService</b> name, which changes with each new version. |
| Standalone deployment with a single version  | Optional. Model-level access works for standalone deployments.                      | Yes. Per-participant URLs are stable when there is no version rotation.                                                   |
| Batch processing with delegation             | Yes. Use the <b>delegate</b> sub-resource for batch processor service accounts.     | Not recommended. The batch processor should use model-level access for controlled deployments.                            |

2. Create a **Role** that grants **post** access to the **models** resource in the target namespace:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: model-access
 namespace: <namespace>
rules:
- apiGroups:
 - serving.opendatahub.io
 resources:
 - models
 verbs:
 - post
```

This grants access to all models in the namespace.



#### NOTE

The **post** verb and the **models** resource in the **serving.opendatahub.io** API group are virtual constructs enforced by the gateway authorization policy, not standard Kubernetes API resources. Running **oc auth can-i post models.serving.opendatahub.io** produces warnings about an unknown verb and resource type. These warnings are expected. To verify that model-level RBAC is configured correctly, send an authenticated inference request as shown in the Verification section.

To restrict access to specific models, add a **resourceNames** field with publisher-qualified model names:

```
rules:
- apiGroups:
 - serving.opendatahub.io
 resources:
 - models
 verbs:
 - post
 resourceNames:
 - "publishers/<namespace>/models/<model-name>"
```

3. Create a **RoleBinding** to bind the role to the target user or service account:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: model-access-binding
 namespace: <namespace>
subjects:
- kind: User
 name: <username>
 apiGroup: rbac.authorization.k8s.io
roleRef:
```

```
kind: Role
name: model-access
apiGroup: rbac.authorization.k8s.io
```

4. Apply the Role and RoleBinding:

```
$ oc apply -f <role-file>.yaml
$ oc apply -f <rolebinding-file>.yaml
```

5. Optional: To grant delegation access for batch processing, create an additional Role that includes the **delegate** sub-resource:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: model-access-delegate
 namespace: <namespace>
rules:
- apiGroups:
 - serving.opendatahub.io
 resources:
 - models/delegate
 verbs:
 - post-delegate
```

Bind this role to the batch processor service account. Regular users who lack the **post-delegate** permission cannot forward requests on behalf of other users.

## Verification

- Send an authenticated request to a publisher-path URL to verify access:

```
$ curl -H "Authorization: Bearer <token>" \
https://<gateway-url>/publishers/<namespace>/models/<model-name>/v1/models
```

A **200 OK** response confirms that model-level RBAC is configured correctly. A **403 Forbidden** response indicates that the user or service account lacks the required **post** permission on the **models** resource. A **500 Internal Server Error** response might indicate that the **LLMInferenceService** backend is not ready rather than an RBAC misconfiguration. Before troubleshooting RBAC, verify that the **LLMInferenceService** shows **Ready=True** by running **oc get llminferenceservice \_<name>\_ -n \_<namespace>\_**.

## 6.6. CONTROLLED DEPLOYMENT API FIELDS

You can configure controlled deployment by using the **spec.router.route.group** and **spec.router.route.weight** fields on **LLMInferenceService** resources. The controller uses these fields to manage Gateway API **HTTPRoute** weighted **backendRef** entries for proportional traffic splitting across routing group members.

### 6.6.1. Spec fields for controlled deployment

The following fields are available on the **LLMInferenceService** resource under **spec.router.route**.

Table 6.7. Controlled deployment spec fields

| Field                           | Type   | Required                          | Description                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------|--------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>spec.router.route.group</b>  | string | Yes, when using traffic splitting | The routing group name that this <b>LLMInferenceService</b> belongs to. All members with the same group name share weighted traffic distribution. The value must be 1 to 63 lowercase alphanumeric characters or hyphens, matching the pattern <b>^[a-z0-9]([a-z0-9-]*[a-z0-9])?\$</b> . Must be set together with <b>weight</b> .                              |
| <b>spec.router.route.weight</b> | int32  | Yes, when using traffic splitting | The proportional traffic share for this member. Follows Gateway API <b>backendRef</b> weight semantics where values are proportional ratios, not percentages. The valid range is 0 to 1,000,000. A weight of 0 removes the member from new request routing but the <b>backendRef</b> remains in the <b>HTTPRoute</b> . Must be set together with <b>group</b> . |

### 6.6.2. Admission validation rules

The admission webhook validates the following cross-field constraints when you create or update an **LLMInferenceService** with traffic splitting fields.

Table 6.8. Traffic splitting validation rules

| Configuration                                                                              | Validation result                                                                                        |
|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Neither <b>group</b> nor <b>weight</b> is set                                              | Valid. The <b>LLMInferenceService</b> operates as a standalone deployment.                               |
| <b>weight</b> is set without <b>group</b>                                                  | Rejected. The error message states that <b>weight</b> requires <b>group</b> .                            |
| <b>group</b> is set without <b>weight</b>                                                  | Rejected. The error message states that <b>group</b> requires <b>weight</b> .                            |
| Both <b>group</b> and <b>weight</b> are set                                                | Valid, if no conflicting route configuration exists.                                                     |
| <b>group</b> and <b>weight</b> are set with <b>route.http.refs</b> (custom HTTPRoute refs) | Rejected. Traffic splitting requires controller-managed routes. Custom HTTPRoute refs are not supported. |

### 6.6.3. Status fields for routing groups

The controller reports the observed routing group state in **status.router.group**.

Table 6.9. Routing group status fields



| Field                                           | Type    | Description                                                                                                                                                                                                   |
|-------------------------------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>status.router.group.name</b>                 | string  | The name of the routing group this member belongs to.                                                                                                                                                         |
| <b>status.router.group.members</b>              | array   | A list of all observed group members, including their names, weights, stopped status, and resolved backend references.                                                                                        |
| <b>status.router.group.members[].name</b>       | string  | The name of the group member <b>LLMInferenceService</b> .                                                                                                                                                     |
| <b>status.router.group.members[].weight</b>     | int32   | The declared traffic weight for this member, as set in <b>spec.router.route.weight</b> .                                                                                                                      |
| <b>status.router.group.members[].stopped</b>    | boolean | Set to <b>true</b> when the member has the <b>serving.kserve.io/stop</b> annotation. A stopped member remains in the group but is omitted from <b>backendRef</b> entries in peer members' <b>HTTPRoutes</b> . |
| <b>status.router.group.members[].backendRef</b> | object  | The resolved <b>backendRef</b> for this member, pointing to the member's <b>InferencePool</b> or Service.                                                                                                     |

#### 6.6.4. Status conditions for routing groups

The controller sets the following conditions to report routing group health.

Table 6.10. Routing group conditions

| Condition            | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>GroupReady</b>    | Set to <b>True</b> when this member's <b>HTTPRoute</b> has been admitted with the group's weighted <b>backendRef</b> entries. This condition reflects only this member's route; peers reconcile independently and may not have reached <b>GroupReady</b> at the same time.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>GroupDegraded</b> | Set to <b>True</b> with reason <b>MemberDivergence</b> when group members serve different models or LoRA adapter sets. The controller partitions the group so that each member's <b>HTTPRoute</b> carries weighted <b>backendRef</b> entries only for members serving the same model, and <b>status.router.group.members</b> lists only same-model peers. Members in a different partition continue to serve their own model at their own publisher path. This condition does not block readiness: <b>GroupReady</b> and <b>Ready</b> can both be <b>True</b> while <b>GroupDegraded</b> is <b>True</b> . The controller emits a <b>Warning</b> event with reason <b>MemberDivergence</b> . Members that are not yet ready or not routable are silently excluded from the traffic split without setting this condition. |

#### 6.6.5. Weight semantics

Weights follow Gateway API proportional semantics as defined in Gateway Enhancement Proposal (GEP) 718:

- Weights are ratios, not percentages. A weight of 9 and a weight of 1 produce a 90/10 traffic split, identical to a weight of 90 and a weight of 10.
- There is no constraint that weights must sum to 100 or any other value.
- A weight of 0 removes the member from new request routing after gateway propagation completes, but in-flight requests can still reach the member during the propagation window. The **backendRef** remains in the **HTTPRoute** at weight 0 rather than being removed. This is useful for completing the promotion lifecycle before deleting a version.
- The valid range is 0 to 1,000,000.

### 6.6.6. Status addresses and URL patterns

The **status.addresses** field reports the network endpoints where the service is reachable. Each address entry includes a URL and can include an origin reference identifying the networking resource that produced it.

When an **LLMInferenceService** participates in a routing group, the following URL patterns are relevant:

**Table 6.11. URL patterns for controlled deployment**

| URL pattern                                                                     | Description                                                                     | Stable across version rotations                                                             |
|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>/_&lt;namespace&gt;/_&lt;llm_svc_name&gt;/_v1/...</code>                  | Per-participant path. Routes to a specific <b>LLMInferenceService</b> instance. | No. The URL changes when the instance name changes.                                         |
| <code>/publishers/_&lt;namespace&gt;/_models/_&lt;model-name&gt;/_v1/...</code> | Publisher path. Routes to the model by name through weighted traffic splitting. | Yes. The URL references the model name, which is the same across all routing group members. |

Publisher-path URLs require model-level RBAC authorization. For more information, see [Authorization model for controlled deployment](#).

## 6.7. CONTROLLED DEPLOYMENT METRICS FOR PROMOTION DECISIONS

You can use the following vLLM Prometheus metrics to compare inference performance across versions during a controlled deployment. Per-version filtering uses the **llm\_svc\_name** label, which is injected by the **PodMonitor** relabeling rule described in [Monitor per-version inference metrics during controlled deployment](#).

### 6.7.1. Key metrics for cross-version comparison

The following table lists the vLLM metrics most relevant to promotion and rollback decisions during controlled deployment. Filter each query by **llm\_svc\_name** to isolate per-version data.

**Table 6.12. vLLM metrics for controlled deployment**

| Metric name                             | Type      | Description                                                                                                                                                                               | PromQL pattern for cross-version comparison                                                                                                                         |
|-----------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>vllm:time_to_first_token_seconds</b> | histogram | Time from request receipt to the first generated token.                                                                                                                                   | <b>histogram_quantile(0.95, sum by(le, llm_isvc_name) (rate(vllm:time_to_first_token_seconds_bucket{llm_isvc_name=~"&lt;v1-name&gt;_ &lt;v2-name&gt;_"} [5m])))</b> |
| <b>vllm:generation_tokens_total</b>     | counter   | Total number of generated tokens.                                                                                                                                                         | <b>sum by(llm_isvc_name) (rate(vllm:generation_tokens_total{llm_isvc_name=~"&lt;v1-name&gt;_ &lt;v2-name&gt;_"} [5m]))</b>                                          |
| <b>vllm:request_success_total</b>       | counter   | Total number of successful inference requests. Use this metric to compare request success throughput across versions.                                                                     | <b>sum by(llm_isvc_name) (rate(vllm:request_success_total{llm_isvc_name=~"&lt;v1-name&gt;_ &lt;v2-name&gt;_"} [5m]))</b>                                            |
| <b>vllm:num_requests_waiting</b>        | gauge     | Number of requests waiting in the inference queue.                                                                                                                                        | <b>sum by(llm_isvc_name) (vllm:num_requests_waiting{llm_isvc_name=~"&lt;v1-name&gt;_ &lt;v2-name&gt;_"}))</b>                                                       |
| <b>vllm:e2e_request_latency_seconds</b> | histogram | End-to-end inference request latency, from request receipt to final response.                                                                                                             | <b>histogram_quantile(0.95, sum by(le, llm_isvc_name) (rate(vllm:e2e_request_latency_seconds_bucket{llm_isvc_name=~"&lt;v1-name&gt;_ &lt;v2-name&gt;_"} [5m])))</b> |
| <b>vllm:kv_cache_usage_perc</b>         | gauge     | Percentage of KV cache currently in use.                                                                                                                                                  | <b>avg by(llm_isvc_name) (vllm:kv_cache_usage_perc{llm_isvc_name=~"&lt;v1-name&gt;_ &lt;v2-name&gt;_"}))</b>                                                        |
| <b>vllm:request_queue_time_seconds</b>  | histogram | Time a request spent waiting in the queue before processing began. Compare against prefill and decode time to determine whether high latency is a capacity issue or an engine bottleneck. | <b>histogram_quantile(0.95, sum by(le, llm_isvc_name) (rate(vllm:request_queue_time_seconds_bucket{llm_isvc_name=~"&lt;v1-name&gt;_ &lt;v2-name&gt;_"} [5m])))</b>  |

| Metric name                              | Type      | Description                                   | PromQL pattern for cross-version comparison                                                                                                                           |
|------------------------------------------|-----------|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>vllm:request_prefill_time_seconds</b> | histogram | Time spent on the prefill phase of a request. | <b>histogram_quantile(0.95, sum by(le, llm_isvc_name) (rate(vllm:request_prefill_time_seconds_bucket{llm_isvc_name=~"_&lt;v1-name&gt;_ _&lt;v2-name&gt;_"})[5m]))</b> |
| <b>vllm:request_decode_time_seconds</b>  | histogram | Time spent on the decode phase of a request.  | <b>histogram_quantile(0.95, sum by(le, llm_isvc_name) (rate(vllm:request_decode_time_seconds_bucket{llm_isvc_name=~"_&lt;v1-name&gt;_ _&lt;v2-name&gt;_"})[5m]))</b>  |

Replace `_<v1-name>_` and `_<v2-name>_` with the **LLMInferenceService** names for each version.

#### Additional resources

- [vLLM metrics reference](#)
- [EPP metrics reference](#)
- [PromQL queries for llm-d monitoring](#)

## 6.8. MODEL-LEVEL RBAC ROLES FOR INFERENCE ACCESS

OpenShift AI provides aggregate **ClusterRoles** that grant model-level access for publisher-path URLs. These **ClusterRoles** aggregate into the standard Kubernetes **admin**, **edit**, and **view** **ClusterRoles** so that namespace-level users automatically receive model-level access.

### 6.8.1. Aggregate ClusterRoles

The following aggregate **ClusterRoles** are available for model-level inference access.

Table 6.13. Model-level aggregate **ClusterRoles**

| ClusterRole                | Verbs granted                           | Aggregates to | Description                                                                                         |
|----------------------------|-----------------------------------------|---------------|-----------------------------------------------------------------------------------------------------|
| <b>kserve-models-admin</b> | Inherits from <b>kserve-models-edit</b> | <b>admin</b>  | Grants full model access. Aggregation role that inherits all rules from <b>kserve-models-edit</b> . |

| ClusterRole               | Verbs granted | Aggregates to                    | Description                                                                                                                                                                                              |
|---------------------------|---------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>kserve-models-edit</b> | <b>post</b>   | <b>edit, kserve-models-admin</b> | Grants model inference access via publisher-path URLs. Users can send inference requests to models.                                                                                                      |
| <b>kserve-models-view</b> | <b>post</b>   | <b>view</b>                      | Grants model inference access via publisher-path URLs. The <b>post</b> verb is used because the virtual resource has no read semantics: model-level access is an inference action, not a read operation. |



#### NOTE

Both **kserve-models-edit** and **kserve-models-view** grant the **post** verb. The **models** virtual resource has no distinct read semantics because accessing a model through a publisher-path URL is an inference action.

### 6.8.2. Virtual resource details

The authorization rules evaluate against the following virtual resource.

**Table 6.14. Virtual resource for model-level RBAC**

| Property                                 | Value                                                                    |
|------------------------------------------|--------------------------------------------------------------------------|
| API group                                | <b>serving.opendatahub.io</b>                                            |
| Resource                                 | <b>models</b>                                                            |
| Sub-resource                             | <b>delegate</b> (for batch processing delegation)                        |
| Verbs                                    | <b>post</b> (inference access), <b>post-delegate</b> (delegation access) |
| Backing Custom Resource Definition (CRD) | None. This is a SubjectAccessReview (SAR)-only authorization contract.   |

### 6.8.3. AuthPolicy rules for model access

The controller generates the following **AuthPolicy** rules for **LLMInferenceService** resources with authentication enabled. When authentication is not set to anonymous, the policy applies at the Gateway level for all **LLMInferenceService** resources as a global rule.

Table 6.15. Controller-generated AuthPolicy rules

| Rule name                          | Fires when                                                                                                                                                                                                                                                                                               | SAR resource                                      | SAR verb             | Priority |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|----------------------|----------|
| <b>model-access-path</b>           | Path starts with <b>/publishers/</b>                                                                                                                                                                                                                                                                     | <b>serving.opendatahub.io/models</b>              | <b>post</b>          | 1        |
| <b>model-access-path-delegate</b>  | Path starts with <b>/publishers/</b> and <b>x-maas-user</b> header is present                                                                                                                                                                                                                            | <b>serving.opendatahub.io/models/delegate</b>     | <b>post-delegate</b> | 1        |
| <b>deny-misrouted-model-header</b> | All of the following conditions are true: the path does not match the publisher-path pattern <b>/publishers/_&lt;namespace&gt;/models/</b> ; the path does not match <b>/v1/files</b> , <b>/v1/files/*</b> , <b>/v1/batches</b> , or <b>/v1/batches/*</b> ; and a valid model routing header is present  | N/A (immediate deny)                              | N/A                  | 0        |
| <b>inference-access</b>            | Path has at least 2 segments, does not start with <b>/v1/</b> , and does not match the publisher-path pattern <b>/publishers/_&lt;namespace&gt;/models/</b> . A path under <b>/publishers/</b> without the <b>/models/</b> segment is not recognized as a publisher path and falls through to this rule. | <b>serving.kserve.io/Inferencervices</b>          | <b>get</b>           | 1        |
| <b>inference-access-delegate</b>   | Same conditions as <b>inference-access</b> plus <b>x-maas-user</b> header                                                                                                                                                                                                                                | <b>serving.kserve.io/Inferencervices/delegate</b> | <b>post-delegate</b> | 1        |

The model routing header is **x-gateway-model-name** by default. You can override the header name per-cluster by setting **ingress.modelBasedRoutingHeaderName** in the **inferenceservice-config** ConfigMap. A valid header value matches the pattern **^publishers/[^\s]+/models/.+\$**.

### 6.8.4. Publisher-path URL format

Publisher-path URLs follow this pattern:

```
/publishers/<namespace>/models/<model-name>/v1/<api-endpoint>
```

For example, the chat completions endpoint for a model named **RedHatAI/Qwen3-8B-FP8-dynamic** in the **my-project** namespace has the following publisher-path URL:

```
/publishers/my-project/models/RedHatAI/Qwen3-8B-FP8-dynamic/v1/chat/completions
```

The model name is extracted from the URL path segment between **/models/** and **/v1/**. Multi-segment model names such as **RedHatAI/Qwen3-8B-FP8-dynamic** are supported.

#### Additional resources

- [Aggregated \*\*ClusterRoles\*\* in the Kubernetes RBAC documentation](#)

## 6.9. TROUBLESHOOTING CONTROLLED DEPLOYMENT

If your controlled deployment is not working as expected, use the following troubleshooting steps to diagnose and resolve common issues.

- Traffic is not splitting between versions:  
Verify that the **HTTPRoute** has weighted backend references for both versions:

```
$ oc get httproute -n <namespace> -o yaml
```

If only one backend reference is displayed, check that both **LLMInferenceService** resources declare the same **spec.router.route.group** value and that both have **spec.router.route.weight** set.

Verify that both versions are ready:

```
$ oc get llminferenceservice -n <namespace>
```

A version that is not ready does not receive traffic even if its weight is nonzero.

- The **GroupReady** condition is **False**:  
Check the condition details for the reason:

```
$ oc get llminferenceservice <llmsvc-name> -n <namespace> \
 -o jsonpath={.status.conditions[?(@.type=="GroupReady")]}
```

Common reasons:

- **FinalizationPending**: A group member is being deleted and finalization has not completed. Wait for the deletion to finish.
- If the condition is absent, verify that the **group** and **weight** fields are set on the **LLMInferenceService**.
- The **GroupDegraded** condition shows **MemberDivergence**:

Group members declare different **spec.model.name** values. The controller excludes divergent members from the traffic split and continues routing with matching members.

Verify the model names on all group members:

```
$ oc get llminferenceservice -n <namespace> \
 -o custom-
 columns=NAME::metadata.name,MODEL::spec.model.name,GROUP::spec.router.route.group
```

Update the divergent member's **spec.model.name** to match the other group members.

- The canary version is not receiving traffic:  
Check that the canary weight is greater than 0:

```
$ oc get llminferenceservice <canary-llmisvc-name> -n <namespace> \
 -o jsonpath={.spec.router.route.weight}
```

Verify that the canary version has ready pods. Check the **LLMInferenceService** status:

```
$ oc get llminferenceservice <canary-llmisvc-name> -n <namespace> \
 -o jsonpath={.status.workloads.primary.readyReplicas}
```

Or list the pods directly:

```
$ oc get pods -n <namespace> -l app.kubernetes.io/name=<canary-llmisvc-name>
```

If the version has zero ready pods, the gateway does not route traffic to it. Check the pod status for errors:

```
$ oc get pods -n <namespace> -l app.kubernetes.io/name=<canary-llmisvc-name>,app.kubernetes.io/part-of=llminferenceservice
```

- The **HTTPRoute** was not created:  
Controlled deployment requires Gateway API routing. Verify that you are not using Ingress-based routing or custom **HTTPRoute** references:

```
$ oc get llminferenceservice <llmisvc-name> -n <namespace> -o yaml | grep -A5 router
```

The **spec.router** section should include **route.group** and **route.weight** and should not include **ingress** or **route.http.refs**.

Check the KServe controller logs for errors:

```
$ oc logs -n redhat-ods-applications deployment/kserve-controller-manager --tail=50
```

- The old version still receives traffic after setting its weight to 0:  
Active requests on the old version complete before traffic fully shifts. Wait for in-flight requests to finish.

Verify that the **HTTPRoute** reflects the updated weights:



```
$ oc get httproute -n <namespace> -o yaml
```

If weights are not updated, check for errors in the KServe controller logs:

```
$ oc logs -n redhat-ods-applications deployment/kserve-controller-manager --tail=50
```

- A stopped version is unreachable after removing the stop annotation:  
After removing the **serving.kserve.io/stop** annotation, the version must become ready before it receives traffic. Wait for the version to become ready:

```
$ oc wait llminferenceservice <llmisvc-name> -n <namespace> \
 --for=condition=Ready --timeout=300s
```

If the version does not become ready, check the pod status for errors:

```
$ oc get pods -n <namespace> -l app.kubernetes.io/name=<llmisvc-name>,app.kubernetes.io/part-of=llminferenceservice
```

- Validation error when creating an **LLMInferenceService** with **group** or **weight**:  
Common validation failures:
  - **weight requires group** or **group requires weight**: Both fields must be specified together.
  - **spec.router.route.group in body should match**: The value must be a DNS-compatible label: lowercase alphanumeric characters or hyphens, 1 to 63 characters, starting and ending with an alphanumeric character.
  - **spec.router.route.weight in body should be less than or equal to 1000000**: The weight must be in the range 0 to 1,000,000.
  - **traffic splitting requires Gateway API routing**: Controlled deployment requires controller-managed Gateway API routes, not Ingress routing.
  - **traffic splitting cannot be used with custom HTTPRoute refs**: Use controller-managed routes instead of custom **HTTPRoute** references.

## CHAPTER 7. VLLM ARGUMENTS REFERENCE

When you deploy models using Distributed Inference with llm-d, the vLLM runtime handles inference requests. Red Hat AI Inference Server provides comprehensive documentation for vLLM runtime arguments and advanced configuration that you can apply to your llm-d deployments.

For the complete list of vLLM runtime arguments and detailed configuration options, see the following resources:

### Argument reference

- [vLLM Server Arguments](#)

### Advanced features

- [Extending Red Hat AI Inference Server with Tool Calling Capabilities](#)
- [Using custom chat templates with models](#)

## 7.1. COMMON USE CASES FOR VLLM ARGUMENTS

Configure vLLM runtime arguments to achieve the following goals:

### Optimize resource usage

- Reduce GPU memory consumption to fit larger models or enable more concurrent requests. For example, using **--gpu-memory-utilization=0.85** leaves headroom for system operations and prevents out-of-memory errors.
- Adjust context length limits to match your application's requirements. For example, use **--max-model-len=8192** for smaller context length to reduce memory requirements.

### Improve performance

- Minimize latency in production by controlling log verbosity. For example, using **--disable-access-log-for-endpoints=/health,/metrics,/ping** reduces I/O overhead from logging high-frequency health checks while maintaining visibility into inference requests.
- Enable faster text generation using speculative decoding. For example, **--speculative-model=<draft-model>** and **--use-v2-block-manager** use a smaller draft model to predict tokens, which is verified by the main model.

### Support specific model requirements

- Enable tool calling with custom models. For example, using **--enable-auto-tool-choice** and **-tool-call-parser=hermes** to enable the model to select tools automatically and parse tool calls using model-specific formats.
- Support custom chat templates for models that require specific formatting.
- Enable prefix caching to speed up requests with repeated prompts using **--enable-prefix-caching**.

### Meet operational needs

- Configure trust settings for custom or private model sources using **--trust-remote-code**.
- Control log volume for different operational scenarios by using endpoint-specific filtering or blanket suppression.

## 7.2. CONFIGURE VLLM ARGUMENTS FOR LLM-D DEPLOYMENTS

You can configure vLLM runtime arguments to optimize inference performance by using the **VLLM\_ADDITIONAL\_ARGS** environment variable to control memory allocation, request handling, and model behavior without redeploying or rebuilding containers.

### Prerequisites

- You have OpenShift AI 3.4 EA2 or later installed.
- You have deployed a model using Distributed Inference with llm-d for distributed inference or single-GPU deployments.
- You have stored a model in S3, a persistent volume claim (PVC), an OCI container registry, downloaded to hostPath storage in your Kubernetes cluster, or Hugging Face.
- You have access to the OpenShift CLI (**oc**) or the OpenShift web console.

### Procedure

1. Identify the vLLM arguments you want to configure. For the complete argument reference, see [Red Hat AI Inference Server vLLM Server Arguments](#).
2. Create or edit the **LLMInferenceService** custom resource to include the **VLLM\_ADDITIONAL\_ARGS** environment variable under **spec.template.containers** for the container named **main**.

Example vLLM argument configuration:

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
 name: my-vllm-service
 namespace: <my_namespace>
spec:
 replicas: 2
 model:
 uri: hf://RedHatAI/Qwen3-8B-FP8-dynamic
 name: RedHatAI/Qwen3-8B-FP8-dynamic
 router:
 route: {}
 gateway: {}
 scheduler: {}
 template:
 containers:
 - name: main
 env:
 - name: VLLM_ADDITIONAL_ARGS
 value: "--max-model-len=10000"
 resources:
```

```
limits:
 cpu: '4'
 memory: 32Gi
 nvidia.com/gpu: "1"
requests:
 cpu: '2'
 memory: 16Gi
 nvidia.com/gpu: "1"
```

where:

### **--max-model-len=10000**

Limits the combined prompt and output length to 10,000 tokens. If you do not specify this option, vLLM derives the maximum context length from the model configuration. Set a lower value when the model's full context length is not required or cannot be accommodated by the available KV cache capacity. This value also affects memory and runtime structures that vLLM configures when initializing the model.



### **NOTE**

The arguments that you specify in **VLLM\_ADDITIONAL\_ARGS** are merged with the default arguments provided by the system. If you specify the same argument as a default, your value takes precedence. You can override specific defaults without affecting other default arguments. To specify multiple arguments, use the YAML folded block scalar **>** with each argument on its own line.

You can also use **VLLM\_ADDITIONAL\_ARGS** to enable tool calling for models that support it. Add **--enable-auto-tool-choice** and **--tool-call-parser=<parser>** to the environment variable value to configure tool calling on the vLLM runtime. For the complete tool calling configuration procedure, see [Configure tool calling for Distributed Inference with llm-d deployments](#).

3. Apply the custom resource.

```
$ oc apply -f llminferenceservice.yaml -n <my_namespace>
```

### **Example output**

```
llminferenceservice.serving.kserve.io/my-vllm-service configured
```

If you are creating a new resource, the output shows **created** instead of **configured**.

### **Verification**

1. Wait for the **LLMInferenceService** to become ready. This might take several minutes while the model downloads and loads.

```
$ oc get llminferenceservice my-vllm-service -n <my_namespace> -w
```

Wait until the **READY** column shows **True** before proceeding.

## 2. Verify that the arguments are applied correctly.

- a. Check that the **LLMInferenceService** is running:

```
$ oc get llminferenceservice my-vllm-service -n <my_namespace>
```

### Example output

```
NAME READY AGE
my-vllm-service True 5m
```

The READY column shows **True**.

- b. Verify your custom arguments appear in the pod specification:  
Get the pod name:

```
$ oc get pods -n <my_namespace> -l app.kubernetes.io/name=my-vllm-service
```

Inspect the environment variables:

```
$ oc describe pod <pod_name> -n <my_namespace> | grep -A 1
"VLLM_ADDITIONAL_ARGS"
```

### Example output

```
VLLM_ADDITIONAL_ARGS: --max-model-len=10000
```

Your custom arguments should appear in the **VLLM\_ADDITIONAL\_ARGS** value.

- c. Test that inference requests work with your configured arguments:  
Get the route URL:

```
$ ROUTE_URL=$(oc get llmisvc -n <my_namespace> my-vllm-service -o
jsonpath='{.status.url}')
```

Send a test request:

```
$ curl -X POST "${ROUTE_URL}/v1/completions" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $(oc whoami -t)" \
-d { "model": "RedHatAI/Qwen3-8B-FP8-dynamic", "prompt": "Explain what Red Hat
OpenShift AI is in one sentence.", "max_tokens": 100 }
```

A successful response confirms that your vLLM service is running with the configured arguments.

## Additional resources

- [Red Hat AI Inference Server vLLM Server Arguments](#)
- [Using custom chat templates with models](#)

## CHAPTER 8. ABOUT VLLM UVICORN ACCESS LOGS IN DISTRIBUTED INFERENCE

You can control vLLM uvicorn access log behavior to balance operational visibility with infrastructure capacity. When you enable access logs for all endpoints, the Endpoint Picker (EPP) scheduler generates a high volume of logs. This log volume can overwhelm the OpenShift web console log viewer and consume significant storage in log aggregation infrastructure.

In Red Hat OpenShift AI 3.5, the default **LLMInferenceServiceConfig** template uses the **--disable-access-log-for-endpoints** flag to filter specific endpoint paths. By default, the following endpoints are filtered from access logs:

- **/health** - Kubernetes liveness and readiness probes
- **/metrics** - Prometheus metrics scraping
- **/ping** - Basic connectivity checks

All other endpoints remain visible in access logs, including:

- **/v1/completions** - Text completion requests
- **/v1/chat/completions** - Chat completion requests
- Custom inference endpoints specific to your deployment

This targeted filtering approach restores visibility into inference request traffic while maintaining suppression of noisy health check and metrics endpoints.

### Version-specific behavior

The logging behavior differs based on the vLLM version deployed:

#### vLLM 0.16 or later

Uses targeted endpoint filtering with **--disable-access-log-for-endpoints /health,/metrics,/ping**. By default, Uvicorn shows inference request logs and suppresses health check logs.

#### vLLM versions below 0.16

For vLLM versions below 0.16, the platform template reverts to the blanket **--disable-uvicorn-access-log** flag. This behavior disables all uvicorn access logs, including inference requests.

The version detection and fallback mechanism operates automatically without requiring manual configuration changes. Existing deployments that use pre-0.16 vLLM images continue to function with their original blanket suppression behavior after upgrading to Red Hat OpenShift AI 3.5.

### When to customize endpoint filtering

The default filtered endpoint list balances operational visibility with log volume for most deployments. You might need to customize the filtered endpoint list in the following scenarios:

- Your deployment includes custom health check endpoints beyond the standard **/health**, **/metrics**, and **/ping** paths
- You want to filter additional endpoints that generate high-frequency traffic specific to your application

- You need to temporarily see health check logs for debugging infrastructure issues
- You want to reduce the filtered list to see logs for specific endpoints that are currently suppressed

### When to enable all access logs

You might need to enable access logs for all vLLM endpoints, including health checks and metrics, in the following scenarios:

- Debugging Kubernetes probe failures that require visibility into health check request patterns
- Troubleshooting metrics scraping issues that prevent monitoring from functioning correctly
- Investigating unexpected pod restart behavior that might be related to probe configuration
- Analyzing complete request patterns during active troubleshooting sessions

## 8.1. CONFIGURING TARGETED VLLM ENDPOINT LOG FILTERING

You can customize the list of endpoints filtered from vLLM access logs to balance log volume with observability needs. By default, Red Hat OpenShift AI 3.5 filters **/health**, **/metrics**, and **/ping** endpoints while logging all other requests. You can override this default by specifying your own endpoint list in the **LLMInferenceService** custom resource. Your custom list replaces the default entirely, so include all endpoints you want filtered.

The **--disable-access-log-for-endpoints** flag uses exact path matching: **/metrics** matches **/metrics** but not **/metrics/prometheus**. Use a comma-separated list for multiple endpoints with no spaces (for example, **/health,/metrics,/ping**).

### Prerequisites

- You have vLLM version 0.16 or later installed
- You have installed Red Hat OpenShift AI 3.5 or later
- You have a running **LLMInferenceService** deployment
- You have access to the OpenShift CLI (``oc``) or the OpenShift web console

### Procedure

1. Edit your **LLMInferenceService** custom resource:

```
$ oc edit llminferenceservice <service_name> -n <namespace>
```

Replace **<service\_name>** with the name of your **LLMInferenceService** and **<namespace>** with the namespace where the **LLMInferenceService** is deployed.

2. In the editor, navigate to the **spec.template.containers** section.
3. Locate the container named **main**.
4. Add the **--disable-access-log-for-endpoints** flag to the **args** list with your custom endpoint list:

**NOTE**

The default filtering behavior is applied automatically by the platform template and does not appear in the **LLMInferenceService** custom resource. When you add the flag to your **args** list, your custom list overrides the platform default.

To use the same endpoint list as the platform default (filtering health checks and metrics):

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
 name: my-vllm-service
 namespace: <namespace>
spec:
 replicas: 2
 model:
 uri: hf://RedHatAI/Qwen3-8B-FP8-dynamic
 name: RedHatAI/Qwen3-8B-FP8-dynamic
 router:
 route: {}
 gateway: {}
 scheduler: {}
 template:
 containers:
 - name: main
 args:
 - --disable-access-log-for-endpoints=/health,/metrics,/ping
 - --max-model-len=10000
 - --gpu-memory-utilization=0.9
```

To filter additional endpoints such as **/readyz** and **/livez**, include them in your custom list along with the default endpoints:

```
...
 containers:
 - name: main
 args:
 - --disable-access-log-for-endpoints=/health,/metrics,/ping,/readyz,/livez
 - --max-model-len=10000
 - --gpu-memory-utilization=0.9
 ...
```

To reduce filtering and see logs for the **/ping** endpoint, specify only the endpoints you want filtered:

```
...
 containers:
 - name: main
 args:
 - --disable-access-log-for-endpoints=/health,/metrics
 - --max-model-len=10000
 - --gpu-memory-utilization=0.9
 ...
```



5. Save and close the editor.

The **LLMInferenceService** controller automatically applies the updated configuration and restarts the affected pods.

## Verification

1. Verify that the **LLMInferenceService** is running:

```
$ oc get llminferenceservice <service_name> -n <namespace>
```

The output shows the service status:

```
NAME READY AGE
my-vllm-service True 5m
```

2. View the pod logs to confirm that health check and metrics requests are not logged:

```
$ oc logs <pod_name> -n <namespace> | grep "/metrics"
```

Replace **<pod\_name>** with the name of one of the vLLM pods.

The output should be empty, confirming that **/metrics** requests are filtered.

3. Make a test inference request to the **/v1/completions** endpoint.
4. View the pod logs again to verify that the inference request was logged:

```
$ oc logs <pod-name> -n <namespace> | grep "/v1/completions"
```

The output shows the inference request:

```
INFO: 172.30.45.2:54325 - "POST /v1/completions HTTP/1.1" 200 OK
```

The IP addresses shown are internal pod network addresses and will vary by deployment.

5. Confirm that only actual API usage is logged while health checks are filtered.

## Additional resources

- [Understanding vLLM uvicorn access logs](#)
- [Configuring vLLM arguments for llm-d](#)
- [vLLM serve CLI reference](#)
- [vLLM logging configuration](#)

## 8.2. ENABLE VLLM UVICORN ACCESS LOGS

You can enable vLLM uvicorn access logs for all endpoints, including health checks and metrics, to debug infrastructure issues and troubleshoot Kubernetes probe behavior. Inference request logs are visible by default through targeted endpoint filtering. This procedure demonstrates how you can enable the additional health check and metrics logs that are suppressed by default.



## NOTE

For vLLM 0.16 or later, the platform template applies targeted filtering with the **--disable-access-log-for-endpoints** flag. For vLLM versions below 0.16, the platform template reverts to blanket suppression with the **--disable-uvicorn-access-log** flag.

## Prerequisites

- You have installed the OpenShift CLI (``oc``).
- You have logged in as a user with **cluster-admin** privileges.
- You have deployed an **LLMInferenceService** custom resource.
- You know which vLLM version your deployment uses.

## Procedure

1. Determine which vLLM version your deployment uses.

### TIP

The vLLM version is displayed in the pod startup logs as follows:

```
[access-log-detect] vllm version='<version>'
[access-log-detect] selected ACCESS_LOG_ARGS='<flags>'
```

The version is also shown as a label in the OpenShift AI under the **Deployment resource**.

2. For vLLM 0.16 or later, add the **--disable-access-log-for-endpoints** flag with an empty value:
  - a. Edit the **LLMInferenceService** custom resource:

```
$ oc edit llminferenceservice <service_name> -n <namespace>
```

- b. In the editor, navigate to **spec.template.containers** and find the container named **main**.
- c. Add the **--disable-access-log-for-endpoints=** flag with an empty value to the **args** list:

```
spec:
 template:
 containers:
 - name: main
 args:
 - --disable-access-log-for-endpoints=
 - --max-model-len=10000
 - --gpu-memory-utilization=0.9
```

The empty value disables endpoint filtering entirely, causing all endpoints to be logged.

- d. Save and close the editor.
- The **LLMInferenceService** controller automatically applies the updated configuration and restarts the affected pods.

- For vLLM versions below 0.16, add the **--no-disable-uvicorn-access-log** flag:



## NOTE

This applies only to deployments using custom vLLM images below version 0.16. All shipped Red Hat OpenShift AI 3.5 images use vLLM 0.19.1 or later.

- Edit the **LLMInferenceService** custom resource:

```
$ oc edit llminferenceservice <service_name> -n <namespace>
```

- In the editor, navigate to **spec.template.containers** and find the container named **main**.
- Add the **--no-disable-uvicorn-access-log** flag to the **args** list:

```
spec:
 template:
 containers:
 - name: main
 args:
 - --no-disable-uvicorn-access-log
 - --max-model-len=10000
 - --gpu-memory-utilization=0.9
```

- Save and close the editor.

The **LLMInferenceService** controller automatically applies the updated configuration and restarts the affected pods.

## Verification

- Verify that the **LLMInferenceService** is running:

```
$ oc get llminferenceservice <service_name> -n <namespace>
```

The output shows the service status:

```
NAME READY AGE
my-vllm-service True 5m
```

- View the pod logs to confirm that health check and metrics access log entries appear:

```
$ oc logs <pod_name> -n <namespace> | grep -E "/(health|metrics|ping)"
```

Replace **<pod\_name>** with the name of one of the vLLM pods.

The output shows HTTP access log entries for health checks and metrics:

```
INFO: 172.30.45.2:54321 - "GET /metrics HTTP/1.1" 200 OK
INFO: 172.30.45.2:54322 - "GET /health HTTP/1.1" 200 OK
INFO: 172.30.45.2:54323 - "GET /ping HTTP/1.1" 200 OK
```

The IP addresses shown are internal pod network addresses and will vary by deployment.

3. Verify that inference request logs also appear:

```
$ oc logs <pod_name> -n <namespace> | grep "/v1/completions"
```

The output shows inference request access log entries:

```
INFO: 172.30.45.2:54324 - "POST /v1/completions HTTP/1.1" 200 OK
```

4. Confirm that all endpoint types are now logged.

## 8.3. EXAMPLE USAGE FOR DISTRIBUTED INFERENCE WITH LLM-D

These examples show how to use Distributed Inference with llm-d in common scenarios.

### 8.3.1. Single-node GPU deployment

Use single-GPU-per-replica deployment patterns for development, testing, or production deployments of smaller models, such as 7-billion-parameter models.

For examples using single-node GPU deployments, see [Single-Node GPU Deployment Examples](#).

### 8.3.2. Multi-node deployment

For examples using multi-node deployments, see [DeepSeek-R1 Multi-Node Deployment Examples](#).

### 8.3.3. Intelligent inference scheduler with KV cache routing

You can configure the scheduler to track key-value (KV) cache blocks across inference endpoints and route requests to the endpoint with the highest cache hit rate. This configuration improves throughput and reduces latency by maximizing cache reuse.

For an example, see [Precise Prefix KV Cache Routing](#).

## CHAPTER 9. TOOL CALLING THROUGH DISTRIBUTED INFERENCE WITH LLM-D

Tool calling enables a large language model to request the execution of external functions during an inference conversation. Agent frameworks such as LangChain, CrewAI, and AutoGen rely on tool calling to build multi-step workflows where the model selects tools, generates structured arguments, and processes tool results across multiple turns.

The Distributed Inference with llm-d serving stack preserves tool calling parameters transparently. When a request includes tool definitions, the stack passes them through every layer, from the Gateway to the Endpoint Picker (EPP) to vLLM, without requiring any llm-d-specific configuration. The following Chat Completions API parameters pass through the stack unaltered:

- **tools:** Function definitions including name, description, and JSON Schema parameters
- **tool\_choice:** Tool selection mode, such as **auto**, **required**, **none**, or a named function
- **parallel\_tool\_calls:** Boolean that controls whether the model can generate multiple tool calls in a single response
- **response\_format:** Structured output schema for JSON mode or JSON Schema constrained decoding

Tool call response objects, including **tool\_calls** arrays with **function.name** and **function.arguments**, return to the client with the same fidelity. Streaming tool call deltas also pass through every stack layer without modification. This passthrough behavior works in both non-disaggregated and disaggregated prefill-decode deployments. Non-tool-calling requests are not affected in latency or throughput.

To use tool calling through Distributed Inference with llm-d, set the appropriate vLLM tool calling arguments on the **LLMInferenceService** custom resource.

The **tool\_choice** request parameter controls how the model handles tool definitions. When a request includes **tools** but omits **tool\_choice**, vLLM defaults to **auto**. The **auto** mode and the **required** and named function modes use different enforcement mechanisms. In **auto** mode, the model generates free-form output and the configured tool-call parser extracts tool calls from the result. In **required** and named function modes, vLLM uses guided decoding to constrain generation to the tool schemas, which produces structurally valid tool call arguments without relying on the parser. Use **auto** for agent workflows where the model needs to decide whether to call a tool. Use **required** or a named function when you need guaranteed tool call output with schema-valid arguments. For the full list of **tool\_choice** values and their runtime requirements, see [Supported tool calling parameters for Distributed Inference with llm-d](#).



### IMPORTANT

Tool calling has not been validated with speculative decoding or Multi-Token Prediction (MTP). These features can interfere with the token generation process that vLLM uses to construct tool call objects. Known failure modes include corrupted **function.arguments** JSON in streaming responses and incomplete tool call parsing when multiple tool calls span a single streaming chunk. Non-streaming requests might succeed while streaming responses exhibit corruption. If you experience issues with tool calling while using speculative decoding or MTP, disable them for workloads that require tool calling.

### 9.1. TOOL CALLING COMPONENT RESPONSIBILITIES

The Distributed Inference with llm-d serving stack is responsible for preserving tool calling parameters in transit. The vLLM runtime, not llm-d, handles the following concerns:

- Model-specific tool calling support and generation quality
- Chat template processing of tool definitions and tool-call history
- Tool call parser selection and model-specific output parsing
- **tool\_choice** behavior
- Parallel tool call response handling
- Structured output enforcement for required, named, and strict automatic tool calls

For model-specific tool calling configuration, see the vLLM tool calling documentation and the Red Hat AI Inference tool calling guide.

### Additional resources

- [Configure tool calling for Distributed Inference with llm-d deployments](#)
- [Supported tool calling parameters for Distributed Inference with llm-d](#)
- [vLLM tool calling documentation](#)
- [Extending Red Hat AI Inference with Tool Calling Capabilities](#)

## 9.2. CONFIGURE TOOL CALLING FOR DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENTS

You can enable tool calling for models served through Distributed Inference with llm-d by adding vLLM tool calling arguments to the **LLMInferenceService** custom resource. The Distributed Inference with llm-d serving stack forwards tool calling parameters to vLLM without modification, so no Distributed Inference with llm-d-specific configuration is required.

### Prerequisites

- You have deployed a model using Distributed Inference with llm-d.
- The model you deployed supports tool calling. For supported models and parser values, see [vLLM tool calling documentation](#).
- You have access to the OpenShift CLI (**oc**).

### Procedure

1. Identify the correct **--tool-call-parser** value for your model.  
The parser must match the chat template format used by the model. For the list of supported parsers by model, see [vLLM tool calling documentation](#). Most supported models provide a compatible chat template, but some models also require an explicit **--chat-template**.
2. Create or edit the **LLMInferenceService** custom resource to include the tool calling arguments.

Add **--enable-auto-tool-choice** and **--tool-call-parser** to the **VLLM\_ADDITIONAL\_ARGS** environment variable under **spec.template.containers** for the container named **main**. Save the following YAML to a file named **llminferenceservice.yaml**:

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
 name: my-tool-calling-service
 namespace: <my_namespace>
spec:
 replicas: 1
 model:
 uri: hf://RedHatAI/Qwen3.5-9B-FP8-dynamic
 name: RedHatAI/Qwen3.5-9B-FP8-dynamic
 router:
 route: {}
 gateway: {}
 scheduler: {}
 template:
 containers:
 - name: main
 env:
 - name: VLLM_ADDITIONAL_ARGS
 value: >-
 --enable-auto-tool-choice
 --tool-call-parser=qwen3_coder
 --reasoning-parser=qwen3
 resources:
 limits:
 cpu: '4'
 memory: 32Gi
 nvidia.com/gpu: "1"
 requests:
 cpu: '2'
 memory: 16Gi
 nvidia.com/gpu: "1"
```

where:

#### **--enable-auto-tool-choice**

Enables automatic tool choice for supported models. When a request uses **tool\_choice="auto"**, the model can decide whether to return a text response or generate one or more tool calls. Automatic tool choice requires a compatible **--tool-call-parser** for the model. This flag defaults to **false** in vLLM, so it must be set explicitly.

#### **--tool-call-parser**

Specifies the parser that vLLM uses to extract tool calls from the model's output. This value must match the chat template format of the model.

#### **--reasoning-parser**

Specifies the parser that vLLM uses to separate the model's internal reasoning from its generated content. This flag does not enable reasoning in the model but tells the engine to treat reasoning and output separately. The reasoning parser is unrelated to tool calling but is recommended for this model.

3. Apply the custom resource:

```
$ oc apply -f llminferenceservice.yaml -n <my_namespace>
```

- Wait for the deployment to become ready:

```
$ oc get llminferenceservice my-tool-calling-service -n <my_namespace> -w
```

### Example output

```
NAME READY AGE
my-tool-calling-service True 3m
```

The READY column shows **True** when the deployment is ready to serve requests.

### Verification

- Get the route URL for your model endpoint:

```
$ ROUTE_URL=$(oc get llmsvc -n <my_namespace> my-tool-calling-service -o
jsonpath='{.status.url}')
```

- Send a tool calling request to verify the configuration:

```
$ curl -s -X POST "${ROUTE_URL}/v1/chat/completions" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $(oc whoami -t)" \
-d { "model": "RedHatAI/Qwen3.5-9B-FP8-dynamic", "temperature": 0, "messages": [{
 "role": "system", "content": "You are a helpful assistant with access to tools. Use the
 get_weather tool when asked about weather." }, { "role": "user", "content": "What is the
 weather in San Francisco?" }], "tools": [{ "type": "function", "function": { "name":
 "get_weather", "description": "Get the current weather in a given location.", "parameters": {
 "type": "object", "properties": { "location": { "type": "string", "description": "City and state." },
 "unit": { "type": "string", "enum": ["celsius", "fahrenheit"] } }, "required": ["location"] } } },
 "tool_choice": "auto" } | python3 -m json.tool
```

The `| python3 -m json.tool` part of the command is optional and formats the JSON output for readability.

Setting **"tool\_choice": "auto"** in the request lets the model automatically decide whether to call a tool based on the request content. For the full list of **tool\_choice** values, see [Supported tool calling parameters for Distributed Inference with llm-d](#).

- Verify the response contains a **tool\_calls** array with **function.name** set to **get\_weather** and **function.arguments** containing a **location** value.

### Example output

```
{
 "id": "chatcmpl-abc123",
 "object": "chat.completion",
 "model": "RedHatAI/Qwen3.5-9B-FP8-dynamic",
 "choices": [
 {
 "index": 0,
```



```

 "message": {
 "role": "assistant",
 "content": null,
 "tool_calls": [
 {
 "id": "call_abc123",
 "type": "function",
 "function": {
 "name": "get_weather",
 "arguments": "{\"location\": \"San Francisco, CA\"}"
 }
 }
]
 },
 "finish_reason": "tool_calls"
 }
]
}

```

If the response contains plain text in the **content** field instead of a **tool\_calls** array, see [Tool calling troubleshooting for Distributed Inference with llm-d](#).

To complete a multi-turn tool calling loop, send the tool execution result back to the model in a follow-up message with **role** set to **tool**. The Distributed Inference with llm-d stack preserves tool result messages in the same way as tool call requests. For the complete multi-turn pattern, see [Extending Red Hat AI Inference with Tool Calling Capabilities](#).

#### Additional resources

- [Tool calling through Distributed Inference with llm-d](#)
- [Supported tool calling parameters for Distributed Inference with llm-d](#)
- [Tool calling troubleshooting for Distributed Inference with llm-d](#)
- [Using custom chat templates with models](#)

## 9.3. SUPPORTED TOOL CALLING PARAMETERS FOR DISTRIBUTED INFERENCE WITH LLM-D

You can use the following Chat Completions API tool calling parameters with models served through Distributed Inference with llm-d. The Distributed Inference with llm-d serving stack preserves these parameters in both request and response paths, passing them through the Gateway, Endpoint Picker (EPP), and vLLM layers without modification.

### 9.3.1. Tool calling request parameters

The following request parameters pass through the Distributed Inference with llm-d stack to the vLLM runtime. The **response\_format** parameter is a general Chat Completions parameter, not specific to tool calling, but is included here because it is commonly used alongside tool definitions in agentic workflows.

Table 9.1. Tool calling request parameters

| Parameter                      | Type             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>tools</b>                   | Array of objects | Function definitions that the model can call. Each tool object contains a <b>type</b> field set to <b>function</b> and a <b>function</b> object with <b>name</b> , <b>description</b> , and <b>parameters</b> fields. The <b>parameters</b> field uses JSON Schema to define the function's input arguments.                                                                                                                                                                                      |
| <b>tools[].function.strict</b> | Boolean          | Controls whether vLLM enforces the function's argument schema when generating a tool call. For <b>tool_choice="auto"</b> , set <b>strict: true</b> to require generated tool arguments to match the schema defined for the function. Strict enforcement for automatic tool calling is enabled by default and can be disabled globally on the vLLM runtime by setting <b>VLLM_ENFORCE_STRICT_TOOL_CALLING=false</b> .                                                                              |
| <b>tool_choice</b>             | String or object | Controls whether the model can generate tool calls and, when applicable, which tools it can select. Supported values are <b>auto</b> , <b>required</b> , and <b>none</b> . You can also specify a particular function by using <b>{"type": "function", "function": {"name": "&lt;function_name&gt;"}}</b> . If <b>tools</b> is specified and <b>tool_choice</b> is omitted, vLLM defaults to <b>auto</b> .                                                                                        |
| <b>parallel_tool_calls</b>     | Boolean          | Controls whether a response can contain multiple tool calls. The default is <b>true</b> , which allows vLLM to return more than one tool call when the model and tool-calling format support it. Setting the parameter to <b>false</b> limits the response to zero or one tool call.                                                                                                                                                                                                              |
| <b>response_format</b>         | Object           | Specifies the output format for the model response. You can use <b>{"type": "json_object"}</b> for JSON mode or <b>{"type": "json_schema", "json_schema": {...}}</b> for JSON Schema constrained decoding. This parameter applies to all responses, including responses that contain tool calls. This parameter is not the mechanism used to validate tool arguments. Tool argument constraints are derived from the JSON Schema in the tool definition and the selected <b>tool_choice</b> mode. |

### 9.3.2. Tool choice behavior in vLLM

The **tool\_choice** request parameter controls how the model handles tool definitions in a request. The behavior varies by mode, and some modes require specific vLLM runtime flags. When a request includes **tools** but omits **tool\_choice**, vLLM defaults to **auto**. This means that any deployment serving tool

calling requests requires **--enable-auto-tool-choice** and a compatible **--tool-call-parser** unless every request explicitly sets **tool\_choice** to **required**, a named function, or **none**. For an overview of how these modes differ, see [Tool calling through Distributed Inference with llm-d](#).

Table 9.2. Tool choice behavior

| tool_choice     | Tool selection                                                                                                                                    | Schema enforcement                                                                                                                                                                                                                                                                   | Runtime requirements                                                                                                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>auto</b>     | The model decides whether to return normal content or generate one or more calls to the available tools.                                          | By default, generation is unconstrained and the configured tool-call parser extracts tool calls from the generated output. If a tool specifies <b>strict: true</b> , vLLM can constrain tool call generation to the tool schema when the configured parser supports structural tags. | Requires the vLLM runtime to be started with <b>--enable-auto-tool-choice</b> and a compatible <b>--tool-call-parser</b> .                                                                                 |
| <b>required</b> | The model must generate one or more tool calls and selects from the tools supplied in the request.                                                | vLLM uses structured output constraints derived from the supplied tool schemas.                                                                                                                                                                                                      | Supported by default. <b>--enable-auto-tool-choice</b> is not required.                                                                                                                                    |
| Named function  | The request specifies the function that the model must call by using <b>{"type": "function", "function": {"name": "&lt;function_name&gt;"}}</b> . | vLLM uses structured output constraints derived from the selected function's parameter schema.                                                                                                                                                                                       | Supported by default. <b>--enable-auto-tool-choice</b> is not required.                                                                                                                                    |
| <b>none</b>     | The model does not generate tool calls and returns normal response content.                                                                       | Not applicable.                                                                                                                                                                                                                                                                      | Supported by default. By default, vLLM still provides the tool definitions to the chat template. You can start vLLM with <b>--exclude-tools-when-tool-choice-none</b> to omit them when this mode is used. |

### 9.3.3. Tool calling response objects

The following response objects pass through the Distributed Inference with llm-d stack from the vLLM runtime to the client.

Table 9.3. Tool calling response objects

| Object | Type | Description |
|--------|------|-------------|
|--------|------|-------------|

| Object                             | Type             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>tool_calls</b>                  | Array of objects | An array of tool call objects in the response <b>choices[].message.tool_calls</b> field. Each object contains <b>id</b> with a unique identifier, <b>type</b> set to <b>function</b> , and a <b>function</b> object with <b>name</b> and <b>arguments</b> fields. The <b>arguments</b> field contains a JSON string of the function arguments generated by the model.                                                                                                                                                      |
| <b>tool_calls</b> streaming deltas | Array of objects | In streaming responses, tool call data arrives incrementally in <b>choices[].delta.tool_calls</b> . Each delta contains an <b>index</b> field that identifies which tool call the chunk belongs to and a <b>function</b> object with partial <b>arguments</b> data. Each tool call includes an index that identifies the call to which the delta belongs. Function arguments can be split across multiple deltas, so clients must concatenate the arguments values for each index before parsing the completed JSON value. |



## NOTE

When a request includes multiple tools, the **tool\_calls** array in the response might not contain all of them. The number of tool calls returned depends on the model, the tool-call parser, and the prompt. Some models generate tool calls one at a time across multiple conversation turns rather than returning all calls in a single response. If you expect multiple tool calls but receive fewer, verify that the model and **--tool-call-parser** support parallel tool calling for your use case.

These parameters are specific to what the Distributed Inference with llm-d serving stack preserves in transit. The vLLM runtime handles the following aspects of tool calling:

- Model-specific tool calling support and generation quality
- Chat template processing of tool definitions and tool-call history
- Tool call parser selection and model-specific output parsing
- **tool\_choice** behavior
- Parallel tool call response handling
- Structured output enforcement for required, named, and strict automatic tool calls

For information about which models support tool calling and which **--tool-call-parser** values to use, see [vLLM tool calling documentation](#).

## Additional resources

- [Tool calling through Distributed Inference with llm-d](#)

- [Configure tool calling for Distributed Inference with llm-d deployments](#)
- [vLLM tool calling documentation](#)

## 9.4. TOOL CALLING TROUBLESHOOTING FOR DISTRIBUTED INFERENCE WITH LLM-D

Diagnose and resolve issues with tool calling through the Distributed Inference with llm-d serving stack. The Distributed Inference with llm-d stack passes tool calling parameters through transparently, so tool calling failures are almost always caused by the vLLM runtime configuration, not the stack itself.

### Model returns text instead of tool calls

#### Symptoms:

- Agent frameworks receive plain text where **tool\_calls** objects are expected
- The response **content** field contains text and **tool\_calls** is absent or empty
- Application workflows stall because the agent framework cannot determine a next action
- The HTTP response status is **200 OK** with no error

#### Diagnosis:

1. Verify the **LLMInferenceService** has vLLM tool calling arguments configured:

```
$ oc get llm_svc <service_name> -n <my_namespace> -o yaml | grep -A 2
"VLLM_ADDITIONAL_ARGS"
```

The **VLLM\_ADDITIONAL\_ARGS** value must include both **--enable-auto-tool-choice** and **--tool-call-parser=<parser>**. Missing flags are the most common cause: the model does not process tool definitions without them.

2. Verify the **--tool-call-parser** value matches the model.  
The parser must match the chat template format of the deployed model. An incorrect parser causes the model to generate tool calls that vLLM cannot extract, resulting in plain text responses. For the correct parser value, see [vLLM tool calling documentation](#).
3. Verify the request includes a **tools** array with valid function definitions.  
Inspect the request body sent by the agent framework or test client. The request must include a **tools** array with at least one tool definition containing **name**, **description**, and **parameters**.

#### Resolution:

- If vLLM tool calling arguments are missing or incorrect, update the **LLMInferenceService** custom resource. See [Configure tool calling for Distributed Inference with llm-d deployments](#).
- If the request is missing the **tools** array, update the client or agent framework configuration.
- If the vLLM configuration is correct and the request includes tools, the issue is likely model-specific. For model-level troubleshooting, see [Extending Red Hat AI Inference with Tool Calling Capabilities](#).

## Tool calls are corrupted or incomplete in streaming responses

### Symptoms:

- The **function.arguments** field in **tool\_calls** contains malformed or truncated JSON
- Multi-tool streaming responses are missing one or more expected **tool\_calls** entries
- The issue occurs only in streaming mode; non-streaming requests return correct tool calls
- Agent frameworks fail to parse tool call arguments or receive fewer tool calls than expected

### Diagnosis:

1. Verify whether speculative decoding or Multi-Token Prediction (MTP) is enabled on the deployment.

Check the **LLMInferenceService** for arguments such as **--speculative-model**, **--num-speculative-tokens**, or MTP-related flags:

```
$ oc get llm_svc <service_name> -n <my_namespace> -o yaml | grep -A 2
"VLLM_ADDITIONAL_ARGS"
```

2. Test the same request with speculative decoding or MTP disabled.  
Remove the speculative decoding or MTP arguments from the **VLLM\_ADDITIONAL\_ARGS** environment variable in the **LLMInferenceService** custom resource and reapply it. If tool calling works correctly after removing these arguments, the issue is caused by the interaction between speculative decoding or MTP and tool call token generation.
3. Test with non-streaming mode to confirm the issue is specific to streaming.  
Set **"stream": false** in the request body and verify that the response contains correct **tool\_calls** objects.

### Resolution:

- Disable speculative decoding or MTP for workloads that require tool calling. These features have not been validated with tool calling and can interfere with the token generation process that vLLM uses to construct streaming tool call deltas.  
For more information, see [Tool calling through Distributed Inference with llm-d](#).

### Additional resources

- [Configure tool calling for Distributed Inference with llm-d deployments](#)
- [Tool calling through Distributed Inference with llm-d](#)
- [vLLM tool calling documentation](#)

## CHAPTER 10. CONFIGURE REQUEST ROUTING FOR {LLM-D}

You can configure custom scheduler settings for large language model (LLM) inference services to optimize request routing and improve performance characteristics such as cache efficiency and load distribution.

### 10.1. SCHEDULER CONFIGURATION FOR LLM INFERENCE SERVICES

You can configure custom scheduler settings for large language model (LLM) inference services to optimize request routing and improve performance characteristics, such as prefix cache hit rates and request distribution patterns.

The EndpointPicker (EPP) routes incoming requests to available model endpoints in your LLM inference service deployment. When multiple replicas of a model are running, the scheduler decides which specific replica handles each request.

#### Default scheduler configuration

If you do not specify custom scheduler configuration, the system uses an optimized default profile that combines four scorer plugins to balance request routing across replicas. The default configuration includes the following:

- **queue-scorer** (weight: 2): Scores replicas based on queue depth to distribute load evenly
- **kv-cache-utilization-scorer** (weight: 2): Scores replicas based on available KV cache capacity
- **prefix-cache-scorer** (weight: 3): Routes requests with similar prompt prefixes to the same replica for cache reuse
- **no-hit-lru-scorer** (weight: 2): Provides a Least Recently Used (LRU) tiebreaker when prefix cache misses occur

This 4-scorer configuration with weights 2:2:3:2 prioritizes KV cache reuse while maintaining balanced load distribution and efficient memory use. The prefix cache scorer uses hash-based cache hit detection, making it compatible with disconnected environments.



#### NOTE

This 4-scorer configuration applies to standard LLM inference deployments. Disaggregated prefill/decode deployments use a different configuration with separate scorer profiles optimized for each phase.

#### Disconnected environment compatibility

The default scheduler configuration functions in disconnected environments. The hash-based **prefix-cache-scorer** in the default configuration does not require network access. If your deployment has network access and you need improved prefix cache hit accuracy, you can optionally upgrade to the **precise-prefix-cache-scorer**, which uses token-level matching but requires a Unix Domain Socket (UDS) Tokenizer sidecar that downloads tokenizer models during initialization.

#### Deployment patterns

Standard LLM inference deployments work best with the default configuration. Specialized workloads automatically apply configuration adjustments or might benefit from custom scheduler settings.

#### Disaggregated prefill/decode deployments

When you configure **spec.prefill** in the **LLMInferenceService** resource, the system automatically applies separate scorer profiles optimized for each phase. For information about deploying disaggregated prefill/decode topologies, including resource requirements and when to use this topology, see [Section 2.13, “Deploy disaggregated prefill/decode topology”](#).

### Hardware restart scenarios

When replicas are starting or recovering, the active-request-scorer can operate in binary mode, for example 0.0 or 1.0 scores, to spread initial load across newly available replicas. This prevents thundering herd problems during cluster scale-up or recovery events.

### Cache-heavy workloads

For workloads with high prompt similarity where cache hit accuracy is critical, you can upgrade to the **precise-prefix-cache-scorer** plugin in connected environments. The precise prefix cache scorer uses token-level matching instead of hash-based detection, improving cache hit accuracy for workloads with subtle prompt variations.

### Latency-sensitive workloads

For deployments where response latency is the primary optimization goal, you can add the optional **latency-scorer** plugin to the configuration. The latency-scorer routes requests to replicas with the lowest observed response times.

You can configure scheduler settings by using two methods:

- **Inline configuration:** Inline configuration embeds the scheduler settings directly in the **LLMInferenceService** resource specification. This method is suitable for simple configurations or when you have unique scheduler settings for a specific service. It keeps all settings for a service in one place for easy viewing.
  - **ConfigMap-based configuration:** ConfigMap-based configuration stores scheduler settings in a Kubernetes ConfigMap and references it from the **LLMInferenceService** resource. This method is ideal when you want to reuse the same scheduler configuration across multiple LLM inference services, or when you prefer to manage configurations separately from service definitions. It also enables centralized configuration management and easier updates to multiple services.
- The two configuration methods are mutually exclusive. You cannot use both inline and ConfigMap-based configuration in the same **LLMInferenceService** resource.

## 10.2. ENDPOINT PICKER ARCHITECTURE

The Endpoint Picker (EPP) is the request orchestration component in Distributed Inference with llm-d that sits between the Inference Gateway and backend model servers. The Endpoint Picker includes two distinct layers that control how requests flow through the system: the flow control layer and the Scheduling layer.

### 10.2.1. Request flow through the Endpoint Picker

When an inference request arrives at the Inference Gateway, it passes through the Endpoint Picker before reaching a backend model server. The Endpoint Picker processes each request through two sequential layers:

#### Flow Control layer

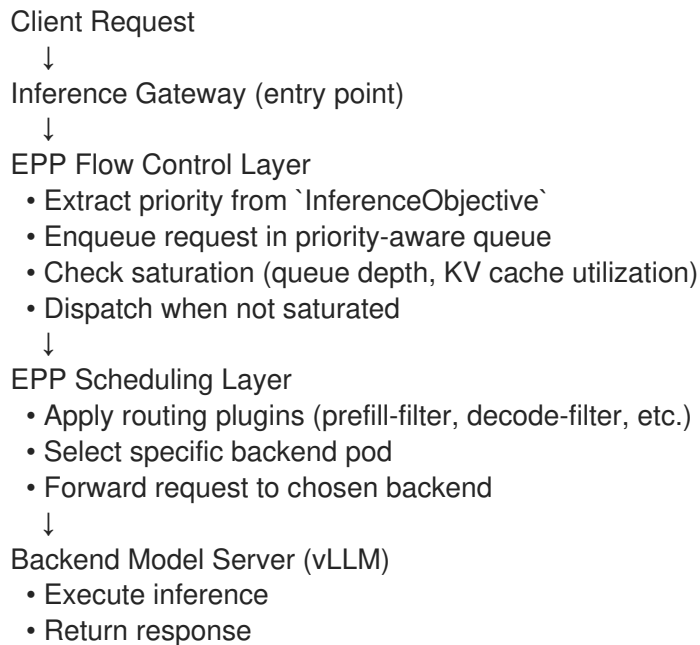
Controls **when** and **in what order** requests are dispatched to backends. This layer manages priority-based queuing, saturation detection, and load shedding to enforce service-level objectives and prevent pool overload.

#### Scheduling layer



Controls **where** each request is routed among available backends. This layer uses routing plugins to select the optimal backend pod based on factors such as prefill versus decode capacity, prefix cache affinity, and load distribution. In WideEP deployments that use external multi-port DP mode, the Scheduling layer can route requests to individual DP ranks rather than whole pods, which enables prefix-cache-aware scheduling at rank-level granularity.

### 10.2.2. Complete request path



The Scheduling layer is always enabled. Flow Control is optional and can be enabled through the **flowControl** feature gate for priority-based queuing and saturation detection.



#### NOTE

In WideEP deployments that use external multi-port DP mode, the "Select specific backend pod" step in the request path routes to individual DP ranks within a pod. Each DP rank exposes a separate serving port, which gives the EPP per-rank visibility for prefix-cache-aware scheduling decisions.

## 10.3. CONFIGURE SCHEDULER INLINE IN LLMINFERENCESERVICE

You can configure scheduler settings directly within your **LLMInferenceService** resource specification by using inline configuration. This approach embeds the scheduler configuration in the same YAML file as your service definition, making it suitable for service-specific settings.

### Prerequisites

- You have cluster administrator privileges or namespace administrator privileges for the namespace where you want to deploy the LLM inference service.
- You have installed OpenShift AI and configured model serving.
- You have a model that you want to deploy as an LLM inference service, or you have an existing **LLMInferenceService** resource that you want to update.

### Procedure

1. Create a YAML file for your **LLMInferenceService** resource or edit an existing service definition for editing.
2. Add the **router.scheduler.config.inline** section to the service specification:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
 name: <llm_service_with_scheduler>
 namespace: <your_namespace>
spec:
 model:
 uri: hf://<your_model_uri>
 name: <your_model_name>
 router:
 scheduler:
 config:
 inline:
 apiVersion: llm-d.ai/v1alpha1
 kind: EndpointPickerConfig
 plugins:
 - type: queue-scorer
 - type: kv-cache-utilization-scorer
 - type: prefix-cache-scorer
 - type: no-hit-lru-scorer
 schedulingProfiles:
 - name: default
 plugins:
 - pluginRef: queue-scorer
 weight: 2
 - pluginRef: kv-cache-utilization-scorer
 weight: 2
 - pluginRef: prefix-cache-scorer
 weight: 3
 - pluginRef: no-hit-lru-scorer
 weight: 2
```

where:

- **router.scheduler.config.inline.plugins.type:** This example shows the default configuration with four scorer plugins: queue-scorer, kv-cache-utilization-scorer, prefix-cache-scorer, and no-hit-lru-scorer with weights 2:2:3:2. You can configure the scheduler plugins and weights according to your requirements.

3. Apply the configuration to your cluster:

```
$ oc apply -f llm-service.yaml
```

4. Verify that the LLM inference service is running with the custom scheduler configuration:

```
$ oc get llminferenceservice <llm_service_with_scheduler> -n <your_namespace>
```

The output shows the service status. Wait until the service reaches the **Ready** state.

## Verification

- Send test requests to your LLM inference service to verify that it is handling requests correctly with the configured scheduler settings.

## 10.4. CONFIGURE SCHEDULER USING CONFIGMAP REFERENCES

You can configure scheduler settings by storing the configuration in a Kubernetes ConfigMap and referencing it from your LLMInferenceService resource. This approach is ideal for sharing configurations across multiple services or managing configurations separately from service definitions.

### Prerequisites

- You have cluster administrator privileges or namespace administrator privileges for the namespace where you want to deploy the LLM inference service.
- You have installed OpenShift AI and configured model serving.
- You have a model that you want to deploy as an LLM inference service, or you have an existing **LLMInferenceService** resource that you want to update.
- You are familiar with Kubernetes ConfigMaps.

### Procedure

1. Create a ConfigMap that contains your scheduler configuration. Create a YAML file named **scheduler-config.yaml** with the following content:

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: <llm_scheduler_config>
 namespace: <your_namespace>
data:
 default-scheduler: |
 apiVersion: llm-d.ai/v1alpha1
 kind: EndpointPickerConfig
 plugins:
 - type: precise-prefix-cache-producer
 - type: prefix-cache-scorer
 parameters:
 prefixMatchInfoProducerName: precise-prefix-cache-producer
 - type: queue-scorer
 schedulingProfiles:
 - name: default
 plugins:
 - pluginRef: precise-prefix-cache-producer
 weight: 2
 - pluginRef: prefix-cache-scorer
 weight: 2
 - pluginRef: queue-scorer
 weight: 3
```

where:

- **data:** This section contains a key, **default-scheduler**, with the scheduler configuration as its value.

- **data.default-scheduler:** The scheduler configuration. You can include multiple configurations by adding more keys to the **data** section.

2. Apply the ConfigMap to your cluster:

```
$ oc apply -f scheduler-config.yaml
```

3. Verify that the ConfigMap was created successfully:

```
$ oc get configmap llm-scheduler-config -n <your_namespace>
```

4. Create or edit your **LLMInferenceService** resource to reference the ConfigMap. For example, create a YAML file named **llm-service.yaml**:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
 name: <llm_service_with_configmap>
 namespace: <your_namespace>
spec:
 model:
 uri: hf://<your_model_uri>
 name: <your_model_name>
 router:
 scheduler:
 config:
 ref:
 name: <llm_scheduler_config>
 key: <default_scheduler>
```

where:

- **ref.name:** Specifies the ConfigMap name
- **ref.key:** Specifies which key within the ConfigMap contains the scheduler configuration.

5. Apply the LLMInferenceService configuration:

```
$ oc apply -f llm-service.yaml
```

6. Verify that the LLM inference service is running with the scheduler configuration from the ConfigMap:

```
$ oc get llminferenceservice llm-service-with-configmap -n <your_namespace>
```

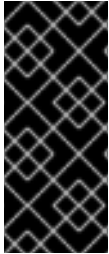
Wait until the service reaches the **Ready** state.

## Verification

- Send test requests to your LLM inference service to verify that it handles requests correctly with the configured scheduler settings.

## 10.5. IMPROVE CACHE HIT ACCURACY WITH TOKEN-LEVEL PREFIX MATCHING

You can upgrade from the hash-based **prefix-cache-scorer** to the **precise-prefix-cache-scorer** plugin to improve prefix cache hit accuracy. The precise prefix cache scorer uses token-level matching instead of hash-based detection, resulting in more exact identification of cached prompt prefixes.



### IMPORTANT

The **precise-prefix-cache-scorer** plugin requires a Unix Domain Socket (UDS) Tokenizer sidecar that downloads tokenizer models from the internet during EndpointPicker (EPP) pod initialization. This scorer is only compatible with connected environments where network access is available. Do not enable precise prefix caching in air-gap (disconnected) environments.

### Prerequisites

- You have cluster administrator privileges or namespace administrator privileges for the namespace where the LLM inference service is deployed.
- Your deployment has internet access for downloading tokenizer models during EPP pod startup.
- You have an existing **LLMInferenceService** resource that you want to upgrade to precise prefix caching.

### Procedure

1. Verify that your environment has network access for tokenizer downloads:

```
$ curl -I https://huggingface.co
```

If the command returns HTTP 200 or 301/302 redirect responses, network access is available. If the command times out or fails, your environment does not have the required network access.

2. Update your **LLMInferenceService** resource to use the **precise-prefix-cache-scorer** plugin. For inline configuration:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
 name: <llm_service_name>
 namespace: <namespace>
spec:
 model:
 uri: hf://<model_uri>
 name: <model_name>
 router:
 scheduler:
 config:
 inline:
 apiVersion: llm-d.ai/v1alpha1
 kind: EndpointPickerConfig
 plugins:
 - type: queue-scorer
```

```

- type: kv-cache-utilization-scorer
- type: precise-prefix-cache-scorer # Changed from prefix-cache-scorer
- type: no-hit-lru-scorer
schedulingProfiles:
- name: default
 plugins:
 - pluginRef: queue-scorer
 weight: 2
 - pluginRef: kv-cache-utilization-scorer
 weight: 2
 - pluginRef: precise-prefix-cache-scorer # Changed from prefix-cache-scorer
 weight: 3
 - pluginRef: no-hit-lru-scorer
 weight: 2

```

3. Apply the updated configuration:

```
$ oc apply -f <llm_service_yaml>
```

4. Wait for the EPP pod to restart with the new configuration:

```
$ oc get pods -n <namespace> -w | grep epp
```

Wait until the EPP pod reaches the **Running** state with all containers ready. The container count will vary based on your configuration.

5. Verify that the tokenizer sidecar started successfully:

```
$ oc logs <epp_pod_name> -c tokenizer -n <namespace>
```

Look for log entries indicating successful tokenizer model download and initialization.

## Verification

- Verify the **precise-prefix-cache-scorer** is active by checking the EPP pod logs:

```
$ oc logs <epp_pod_name> -n <namespace> | grep -i "precise-prefix-cache-scorer"
```

You should see log entries indicating the precise prefix cache scorer plugin was loaded during EPP initialization.

## Troubleshooting

If the EPP pod fails to start or the tokenizer sidecar shows download errors:

- Verify network access is available from the pod:

```
$ oc debug pod/<epp_pod_name> -n <namespace> -- curl -I https://huggingface.co
```

- Check for network policy restrictions that might block egress traffic.
- If your environment is air-gapped or network access cannot be provided, revert to the hash-based **prefix-cache-scorer** plugin.

## 10.6. AVAILABLE PLUGINS FOR YOUR INFERENCE WORKLOAD

Scheduler plugins implement specific routing logic that determines how the EndpointPicker (EPP) distributes requests across model replicas. You can configure these plugins in the **EndpointPickerConfig** to customize request routing behavior.

When you configure more than one plugin, the scheduler combines their scores to make the final routing decision. The scheduler evaluates each plugin for every request and selects the endpoint with the highest combined score.

### queue-scorer

Scores replicas based on queue depth. Replicas with shorter queues score higher, helping to distribute requests evenly across available replicas and prevent overloading individual instances.

- **Default weight:** 2
- **Disconnected compatible:** Yes
- **Network dependencies:** None
- **Use cases:** Load balancing, preventing queue buildup on individual replicas

### kv-cache-utilization-scorer

Scores replicas based on KV cache utilization. Replicas with lower KV cache utilization (more available cache capacity) score higher, ensuring requests are routed to replicas that have space for new KV cache entries. This scorer helps maximize GPU memory efficiency across the inference pool and prevents cache eviction.

- **Default weight:** 2
- **Disconnected compatible:** Yes
- **Network dependencies:** None
- **Use cases:** Optimizing KV cache usage, preventing memory exhaustion

### prefix-cache-scorer

Optimizes request routing for improved prefix cache hit rates by using hash-based cache hit detection. Routes requests with similar prompt prefixes to the same model replica, increasing the likelihood that the replica can reuse previously computed results for common prompt prefixes.

- **Default weight:** 3
- **Disconnected compatible:** Yes (hash-based variant)
- **Network dependencies:** None
- **Use cases:** Maximizing prefix cache reuse for workloads with common prompt patterns

### no-hit-lru-scorer

Provides a Least Recently Used (LRU) tiebreaker for cache miss scenarios. When prefix cache scorers do not identify a clear cache hit, this scorer routes requests to the least recently used replica, distributing new prompt patterns across available capacity.

- **Default weight:** 2

- **Disconnected compatible:** Yes
- **Network dependencies:** None
- **Use cases:** Balanced distribution of cache misses, preventing replica starvation

## Optional scorer plugins

The following scorer plugins are available for custom configurations but are not included in the default configuration.

### precise-prefix-cache-scorer

Provides token-level prefix cache matching with improved accuracy compared to the hash-based **prefix-cache-scorer**. Requires a UDS Tokenizer sidecar that downloads tokenizer models during pod initialization. Tokenizer download occurs once per pod startup and typically completes in 30-60 seconds depending on network speed and tokenizer size. If the download fails, the pod will not start successfully. This scorer is only compatible with connected environments where network access is available.

- **Recommended weight:** 3
- **Disconnected compatible:** No (requires network access for tokenizer download)
- **Network dependencies:** Requires internet access during EPP pod startup
- **Use cases:** Connected environments where maximum prefix cache hit accuracy is required

### active-request-scorer

Scores replicas based on the number of active requests. Replicas with fewer active requests score higher. This scorer is used in disaggregated prefill/decode topologies as part of the decode profile.

- **Recommended weight:** 2-3
- **Disconnected compatible:** Yes
- **Network dependencies:** None
- **Use cases:** Disaggregated prefill/decode deployments, minimizing decode-phase latency

### latency-scorer

Scores replicas based on predicted response latency. Replicas with lower predicted latency score higher. This scorer requires the **predicted-latency-producer** plugin to generate latency predictions and uses **weighted-random-picker** for final endpoint selection instead of **max-score-picker**. The latency predictor learns from historical request patterns to estimate response times.

- **Recommended weight:** 3
- **Disconnected compatible:** Yes
- **Network dependencies:** None
- **Use cases:** Latency-sensitive workloads where response time is the primary optimization goal
- **Required plugins:** predicted-latency-producer, weighted-random-picker



**predicted-latency-producer**

A data producer plugin that calls the latency predictor service and annotates each candidate endpoint with predicted TTFT (Time To First Token) and TPOT (Time Per Output Token) values. This plugin is required by **latency-scorer** to produce the per-endpoint latency estimates it scores against. The predictor learns from historical request patterns to estimate response times based on workload characteristics.

- **Plugin type:** Data producer
- **Disconnected compatible:** Yes
- **Network dependencies:** None
- **Required by:** **latency-scorer**

**weighted-random-picker**

A picker plugin that selects endpoints with probability that is proportional to their score, using reservoir sampling. This approach avoids the hot-spotting behavior of a pure max-score picker while still preferring higher-scored endpoints. When used with **latency-scorer**, this picker distributes requests across replicas based on predicted latency while maintaining some randomization to prevent overloading the fastest replica.

- **Plugin type:** Picker
- **Disconnected compatible:** Yes
- **Network dependencies:** None
- **Required by:** **latency-scorer**

**Optimizing request routing for your workload**

The default configuration balances competing optimization goals based on the upstream llm-d project baseline.

**Queue-scorer vs Prefix-cache-scorer**

The queue-scorer optimizes for even load distribution across replicas by routing requests to replicas with shorter queues. The prefix-cache-scorer optimizes for request affinity by routing requests with similar prompt prefixes to the same replica, increasing cache reuse. The greater prefix-cache weight, for example 3 compared with 2, prioritizes cache reuse over perfect load balance, improving throughput for workloads with common prompt patterns.

**KV-cache-utilization-scorer**

Routes requests to replicas with lower KV cache use to maximize GPU memory efficiency. This scorer prevents individual replicas from exhausting their KV cache capacity, which would force cache eviction and degrade performance. The equal weight with queue-scorer (2:2) balances memory efficiency with load distribution.

**No-hit-lru-scorer**

Provides an LRU tiebreaker when prefix cache scorers do not identify a clear cache hit. Routes new prompt patterns to the least recently used replica, distributing cold-start workloads across available capacity. This scorer prevents replica starvation and ensures balanced use when cache hits are not available.

**Active-request-scorer for decode**

In disaggregated prefill/decode deployments, the decode phase uses active-request-scorer instead of queue-scorer. Active request count is a better signal than queue depth for ongoing generation workloads because it reflects the actual concurrency load during the decode phase.

## 10.7. EPP AND INFERENCE SCHEDULER METRICS FOR LLM-D

The Endpoint Picker (EPP), or the inference scheduler, in Distributed Inference with llm-d deployments expose Prometheus metrics at the **/metrics** endpoint on the **metrics** service port. You can use these metrics to monitor request routing decisions, scheduling latency, inference performance, and prefix cache indexing performance.



### NOTE

Metrics in Distributed Inference with llm-d use the **llm\_d\_epp\_** prefix. The **inference\_objective\_**, **inference\_extension\_**, and **inference\_pool\_** metric prefixes are deprecated. These prefixes remain available for backward compatibility.

### Inference objective metrics

The service-level request metrics track end-to-end inference performance at the Endpoint Picker (EPP). These metrics measure user-perceived latency, which includes scheduler queue wait time and network latency between the scheduler and the model server. These metrics use the **llm\_d\_epp\_request\_** prefix.

Table 10.1. Service-level request metrics

| Metric name                                     | Type      | Labels                                                                                               | Description                                                                                                                                                                       |
|-------------------------------------------------|-----------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_request_total</b>                  | counter   | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b>                       | The total number of inference requests processed by the scheduler.                                                                                                                |
| <b>llm_d_epp_request_error_total</b>            | counter   | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority,</b><br><b>error_code</b> | The total number of inference request errors, broken down by error code.                                                                                                          |
| <b>llm_d_epp_request_duration_seconds</b>       | histogram | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b>                       | The end-to-end inference request duration distribution in seconds.                                                                                                                |
| <b>llm_d_epp_request_ttft_seconds</b>           | histogram | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b>                       | The time to first token (TTFT) distribution in seconds, measured from request receipt to first response byte. For non-streaming requests, this equals the total request duration. |
| <b>llm_d_epp_request_streaming_tpot_seconds</b> | histogram | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b>                       | The average time per output token (TPOT) distribution in seconds for streaming requests, computed as (end-to-end latency - TTFT) / (output tokens - 1).                           |

| Metric name                                    | Type      | Labels                                                      | Description                                                                                                                                                                                                                                                                 |
|------------------------------------------------|-----------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_request_streaming_itl_seconds</b> | histogram | <b>model_name, target_model_name, fairness_id, priority</b> | The inter-token latency (ITL) distribution in seconds for streaming requests, measured as the time between consecutive response body chunks. Use ITL to detect decode-phase lag spikes that TPOT averaging can mask. The EPP emits this metric only for streaming requests. |
| <b>llm_d_epp_request_input_tokens</b>          | histogram | <b>model_name, target_model_name, fairness_id, priority</b> | The input token count distribution per request.                                                                                                                                                                                                                             |
| <b>llm_d_epp_request_output_tokens</b>         | histogram | <b>model_name, target_model_name, fairness_id, priority</b> | The output token count distribution per request.                                                                                                                                                                                                                            |
| <b>llm_d_epp_request_running</b>               | gauge     | <b>model_name, target_model_name, fairness_id, priority</b> | The current number of active inference requests.                                                                                                                                                                                                                            |

### Scheduling and routing metrics

The scheduling and routing metrics track the internal performance of the Endpoint Picker. These metrics use the **llm\_d\_epp\_** prefix.

Table 10.2. Scheduling and routing metrics

| Metric name                                     | Type      | Labels                                                           | Description                                                                                         |
|-------------------------------------------------|-----------|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_scheduler_e2e_duration_seconds</b> | histogram | None                                                             | The end-to-end scheduling duration in seconds, measured from request receipt to endpoint selection. |
| <b>llm_d_epp_scheduler_attempts_total</b>       | counter   | <b>status, target_model_name, endpoint_name, namespace, port</b> | The total number of scheduling attempts, broken down by status: <b>success</b> or <b>failure</b> .  |

| Metric name                                    | Type      | Labels                                              | Description                                                                                                                                                                       |
|------------------------------------------------|-----------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_plugin_duration_seconds</b>       | histogram | <b>extension_point, plugin_type, plugin_name</b>    | The plugin processing duration in seconds, broken down by extension point, plugin type, and plugin name. Use this metric to identify slow plugins that impact scheduling latency. |
| <b>llm_d_epp_model_rewrite_decisions_total</b> | counter   | <b>model_rewrite_name, model_name, target_model</b> | The total number of model rewrite decisions made by the scheduler.                                                                                                                |

### Prefix cache indexer metrics

The prefix cache indexer metrics track KV cache prefix matching performance in the scheduler. These metrics use the **llm\_d\_epp\_** prefix.

Table 10.3. Prefix cache indexer metrics

| Metric name                               | Type      | Labels | Description                                                                                                                                           |
|-------------------------------------------|-----------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_prefix_indexer_size</b>      | gauge     | None   | The current size of the prefix cache index maintained by the scheduler.                                                                               |
| <b>llm_d_epp_prefix_indexer_hit_ratio</b> | histogram | None   | The prefix cache hit ratio distribution. A higher hit ratio indicates that requests frequently match cached prefixes, reducing redundant computation. |
| <b>llm_d_epp_prefix_indexer_hit_bytes</b> | histogram | None   | The prefix cache hit size distribution in bytes. Larger hit sizes indicate more effective prefix reuse.                                               |

### Inference pool metrics

The inference pool metrics track aggregate statistics across all endpoints in a pool. These metrics use the **llm\_d\_epp\_** prefix.

Table 10.4. Inference pool metrics

| Metric name                                   | Type  | Labels      | Description                                                                  |
|-----------------------------------------------|-------|-------------|------------------------------------------------------------------------------|
| <b>llm_d_epp_average_kv_cache_utilization</b> | gauge | <b>name</b> | The average KV cache utilization across all endpoints in the inference pool. |
| <b>llm_d_epp_average_queue_size</b>           | gauge | <b>name</b> | The average request queue size across all endpoints in the inference pool.   |

| Metric name                               | Type  | Labels                             | Description                                                                        |
|-------------------------------------------|-------|------------------------------------|------------------------------------------------------------------------------------|
| <b>llm_d_epp_average_running_requests</b> | gauge | <b>name</b>                        | The average number of running requests across all endpoints in the inference pool. |
| <b>llm_d_epp_ready_endpoints</b>          | gauge | <b>name</b>                        | The number of ready endpoints in the inference pool.                               |
| <b>llm_d_epp_per_endpoint_queue_size</b>  | gauge | <b>name, model_server_endpoint</b> | The request queue size for a specific endpoint in the inference pool.              |

### Flow control metrics

The flow control metrics track priority-based queuing, request dispatching, and pool saturation when flow control is enabled. These metrics use the **llm\_d\_epp\_flow\_control\_** prefix.

Table 10.5. Flow control metrics

| Metric name                                   | Type    | Labels                                                                      | Description                                                                                                                                                                                                                 |
|-----------------------------------------------|---------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_flow_control_queue_size</b>      | gauge   | <b>fairness_id, priority, inference_pool, model_name, target_model_name</b> | The current number of requests actively held in the flow control queue, broken down by fairness group and priority band.                                                                                                    |
| <b>llm_d_epp_flow_control_queue_bytes</b>     | gauge   | <b>fairness_id, priority, inference_pool, model_name, target_model_name</b> | The current total size in bytes of requests actively held in the flow control queue, broken down by fairness group and priority band.                                                                                       |
| <b>llm_d_epp_flow_control_pool_saturation</b> | gauge   | <b>inference_pool</b>                                                       | The current saturation level of the inference pool, ranging from 0.0 (empty) to 1.0 (fully saturated). When saturation reaches the configured threshold, dispatch halts for all priority bands.                             |
| <b>llm_d_epp_flow_control_requests_total</b>  | counter | <b>outcome, priority, inference_pool</b>                                    | The total number of requests processed by the flow control layer, broken down by outcome:<br><b>Dispatched, RejectedCapacity, RejectedNoEndpoints, RejectedOther, EvictedTTL, EvictedContextCancelled, or EvictedOther.</b> |

| Metric name                                                    | Type      | Labels                                                                               | Description                                                                                                                                             |
|----------------------------------------------------------------|-----------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_flow_control_request_queue_duration_seconds</b>   | histogram | <b>fairness_id, priority, outcome, inference_pool, model_name, target_model_name</b> | The distribution of total time that requests wait in the flow control layer, measured from enqueue to final outcome.                                    |
| <b>llm_d_epp_flow_control_dispatch_cycle_duration_seconds</b>  | histogram | None                                                                                 | The distribution of time taken for each internal dispatch cycle in the flow control layer. Use this metric to identify dispatch processing bottlenecks. |
| <b>llm_d_epp_flow_control_request_enqueue_duration_seconds</b> | histogram | <b>fairness_id, priority, outcome</b>                                                | The distribution of time taken to enqueue requests into the flow control layer.                                                                         |

## 10.8. MANAGE EPP SCHEDULER CONFIGURATION DURING RHOAI 3.4 TO 3.5 UPGRADES

When upgrading from RHOAI 3.4 to RHOAI 3.5, the system handles EPP scheduler configuration changes automatically based on whether you have custom scheduler configuration. You can choose to adopt the new RHOAI 3.5 optimized defaults or retain your existing RHOAI 3.4 configuration.

### Prerequisites

- You are upgrading from RHOAI 3.4 to RHOAI 3.5.
- You have cluster administrator privileges or namespace administrator privileges for the namespace where LLM inference services are deployed.
- You have existing **LLMInferenceService** resources deployed in RHOAI 3.4.

### About this task

During the RHOAI 3.4 to 3.5 upgrade, the system applies the following automatic behavior:

- **LLMInferenceService resources with no custom scheduler configuration** These services automatically adopt the RHOAI 3.5 default configuration (4 scorers with weights 2:2:3:2) during the next reconciliation cycle after upgrade.
- **LLMInferenceService resources with custom scheduler configuration** These services preserve their existing custom configuration. The system does not automatically change custom configurations during upgrade.

The RHOAI 3.4 default used two scorers (**queue-scorer** and **prefix-cache-scorer** with weights 2:3). The RHOAI 3.5 default adds two additional scorers (**kv-cache-utilization-scorer** and **no-hit-lru-scorer**) for improved performance.

### Procedure

1. Before upgrading, identify which LLM inference services have custom scheduler configuration:

```
$ oc get llmsvc -A -o json | jq -r '.items[] | select(.spec.router.scheduler.config != null) |
.metadata.name'
```

This command lists all LLM inference services with custom scheduler configuration. Services not listed will automatically adopt the RHOAI 3.5 defaults after upgrade.

2. Upgrade RHOAI from 3.4 to 3.5 following the standard upgrade procedure.
3. After the upgrade completes, verify which scheduler configuration is active for your services by following the verification procedure in [Verify active EPP scheduler configuration](#).
4. **Optional:** To adopt the RHOAI 3.5 optimized defaults for services with custom configuration:

- a. Remove the custom scheduler configuration from the **LLMInferenceService** spec:

```
$ oc patch llmsvc <service_name> -n <namespace> --type=json \
-p [{"op": "remove", "path": "/spec/router/scheduler/config"}]
```

- b. Wait for reconciliation to occur (typically within 5 minutes), or restart the EPP pod to apply changes immediately:

```
$ oc delete pod <epp_pod_name> -n <namespace>
```

- c. Verify the new default configuration is active by querying EPP metrics.

5. **Optional:** To restore RHOAI 3.4 default configuration for services that auto-adopted RHOAI 3.5 defaults:

- a. Create a YAML file with the RHOAI 3.4 scheduler configuration:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
 name: <service_name>
 namespace: <namespace>
spec:
 # ...
 model:
 uri: hf://<model_uri>
 name: <model_name>
 # ...
 router:
 scheduler:
 config:
 inline:
 apiVersion: llm-d.ai/v1alpha1
 kind: EndpointPickerConfig
 plugins:
 - type: queue-scorer
 - type: prefix-cache-scorer
 schedulingProfiles:
 - name: default
 plugins:
 - pluginRef: queue-scorer
```

```
weight: 2
- pluginRef: prefix-cache-scorer
weight: 3
```

b. Apply the configuration:

```
$ oc apply -f <llm_service_yaml>
```

## Verification

- Verify the active scheduler configuration by following the procedure in [Verify active EPP scheduler configuration](#).
- Monitor inference service performance metrics to confirm the scheduler configuration is behaving as expected.

## Additional resources

- For more information about the RHOAI 3.5 default configuration, see [Scheduler configuration for LLM inference services](#).
- For more information about customizing scheduler configuration, see [Configure scheduler inline in LLMInferenceService](#).

### 10.8.1. Verify active EndpointPicker scheduler configuration

You can verify which EndpointPicker (EPP) scheduler configuration is active by inspecting your **LLMInferenceService** resource specification. This verification is useful after deploying a new LLM inference service or upgrading from RHOAI 3.4 to 3.5 to confirm the expected scheduler configuration is in effect.

## Prerequisites

- You have cluster administrator privileges or namespace administrator privileges for the namespace where the LLM inference service is deployed.
- You have at least one **LLMInferenceService** resource deployed and running.
- You have the **oc** CLI installed and configured to access your cluster.

## Procedure

1. Check if your LLM inference service has custom scheduler configuration:

```
$ oc get llminferenceservice <llm_service_name> -n <namespace> -o
jsonpath={.spec.router.scheduler.config}
```

where:

**<llm\_service\_name>**

Specifies the name of your LLM inference service.

**<namespace>**

Specifies the namespace where your LLM inference service is deployed.



## 2. Interpret the output:

- **Empty output or no scheduler config** The service uses the default RHOAI 3.5 scheduler configuration with four scorers (queue-scorer, kv-cache-utilization-scorer, prefix-cache-scorer, no-hit-lru-scorer) with weights 2:2:3:2.
- **Inline configuration present:** View the custom inline scheduler configuration:

```
$ oc get llminferenceservice <llm_service_name> -n <namespace> -o
jsonpath={.spec.router.scheduler.config.inline} | jq .
```

- **ConfigMap reference present** View the ConfigMap-based scheduler configuration:

```
$ CONFIG_MAP=$(oc get llminferenceservice <llm_service_name> -n <namespace> -o
jsonpath={.spec.router.scheduler.config.ref.name})
$ CONFIG_KEY=$(oc get llminferenceservice <llm_service_name> -n <namespace> -o
jsonpath={.spec.router.scheduler.config.ref.key})
$ oc get configmap $CONFIG_MAP -n <namespace> -o jsonpath="
{.data.$CONFIG_KEY}"
```

## 3. Optional: Verify the EPP pod is running with the expected configuration:

```
$ oc get pods -n <namespace> | grep router-scheduler
$ oc logs <epp_pod_name> -n <namespace> | grep -i "scheduler\|config"
```

The EPP pod name follows the pattern **<service\_name>-kserve-router-scheduler-  
<random\_suffix>**. The EPP pod logs show scheduler initialization messages that confirm which plugins and profiles are loaded.

## Verification

- For services with no custom scheduler configuration, confirm that no inline or ConfigMap configuration is present in the resource spec. The default four-scorer configuration applies automatically.
- For services with custom configuration, verify that the inline YAML or ConfigMap content matches your intended scheduler settings.
- If the configuration does not match expectations, check for typos in the **LLMInferenceService** resource spec or verify that the referenced ConfigMap exists and contains valid **EndpointPickerConfig** YAML.

## Additional resources

- For information about the default scheduler configuration, see [Section 10.1, "Scheduler configuration for LLM inference services"](#).
- For information about customizing scheduler settings, see [Section 10.3, "Configure scheduler inline in LLMInferenceService"](#).

## CHAPTER 11. MANAGE MIXED WORKLOADS BY USING PRIORITY QUEUING

With flow control, you can offer different quality of service for latency-sensitive and throughput-sensitive workloads, or clients, on the same model servers without violating service-level agreements (SLAs), reducing capital expenses and maximizing accelerators return on investment.

### 11.1. FLOW CONTROL AND PRIORITY-BASED QUEUING

Flow control is a pool defense mechanism in the Endpoint Picker (EPP) component of Distributed Inference with llm-d. It manages request queuing, prioritization, and fairness in a multitenant inference serving environment, which helps with consolidating latency-sensitive and throughput-sensitive workloads on a single cluster, reducing infrastructure costs while protecting critical service level objectives (SLOs).

Use flow control in the following scenarios:

- You run many tenant applications with different service level agreement (SLA) requirements on shared GPU infrastructure
- You need to guarantee latency for interactive workloads while maximizing GPU usage with batch jobs
- You want to merge separate Distributed Inference with llm-d deployments for different service tiers onto a single cluster
- You need to protect critical workloads from noisy neighbor effects
- You want to implement service tiering for multitenant LLM inference

If you only have a single workload type with uniform SLA requirements, flow control adds unnecessary complexity.

#### 11.1.1. How flow control works

Flow control manages request queuing, prioritization, and fairness between the Inference Gateway and InferencePool backends. Flow control is work-conserving: when the pool has available capacity, requests dispatch immediately with no added latency. Queuing and priority enforcement activate only when the pool is saturated.

When a request arrives:

1. The Gateway automatically injects two headers based on authentication:
  - **x-gateway-inference-fairness-id** - Groups requests from the same authentication source for fair queuing
  - **x-gateway-inference-objective** - Determines request priority by matching an **InferenceObjective** resource
2. The request enters a priority-aware queue in the Endpoint Picker and waits for dispatch.
3. The dispatch logic applies strict priority ordering:
  - Higher-priority requests dispatch before lower-priority requests

- Requests at the same priority level use intra-band fairness policies. The default **global-strict-fairness-policy** is a greedy strategy that ignores tenant boundaries and offers no isolation. A single tenant in a priority band can starve other tenants of the same priority. For equitable service distribution, configure **round-robin-fairness-policy**
4. A saturation check monitors queue depth and KV cache utilization on model servers:
    - When pool saturation is detected, dispatch is halted
    - All requests wait in their priority queues until capacity becomes available
  5. When capacity is available, the request proceeds to the Scheduling layer. The endpoint is selected at dispatch time based on current capacity and prefix cache state, not when the request first enters the queue. This late-binding approach prevents requests from being locked to a server that might be saturated by the time the request is processed.

Flow control queues requests when the **InferencePool** has no ready backends, hiding startup time from clients during scale-from-zero scenarios.



#### NOTE

Flow control depends on the Endpoint Picker (EPP) to enforce priority and fairness policies. If the EPP becomes unavailable, the default Distributed Inference with llm-d configuration fails open: the gateway routes requests directly to model servers without flow control protection until the EPP recovers. During this time, priority ordering, fairness policies, and saturation gating are not enforced.

### 11.1.2. Key flow control concepts

#### Priority levels

Platform administrators create **InferenceObjective** resources to define priority tiers. Higher integer values represent higher priority. Negative values designate sheddable requests that are queued and dispatched last.

Example tier structure:

| Tier      | Priority Value | Use Case                 |
|-----------|----------------|--------------------------|
| Critical  | 100            | Interactive applications |
| Standard  | 0 (default)    | Standard API calls       |
| Sheddable | -1             | Batch workloads          |

#### Authentication-based prioritization

The Gateway **AuthPolicy** automatically sets the **x-gateway-inference-objective** header:

- ServiceAccount tokens: Header is set to the ServiceAccount's namespace
- User tokens: Header is set to **authenticated**
- Anonymous requests: Header is set to **unauthenticated**

Create **InferenceObjective** resources with names matching these header values to assign priorities.

### Starvation protection

Under strict priority ordering, sustained high-priority load can permanently defer lower-priority traffic. Flow control provides inter-priority mechanisms that help ensure lower-priority workloads process when capacity is available, even under sustained contention. This differs from intra-priority fairness, which distributes requests equitably within a single priority band. You can configure usage limit policies and per-band capacity limits to protect lower-priority workloads.

### Sheddable requests

Requests designated as sheddable with negative priority values receive controlled load shedding behavior. When flow control is enabled, sheddable requests are queued rather than immediately dropped, but operators can bound queue capacity per priority band. When capacity limits are exceeded or requests time out, flow control rejects or evicts requests with appropriate HTTP error codes. You can configure per-band capacity limits and request timeouts to control load shedding behavior.

### Request refusal behavior

When flow control rejects or evicts a request, the response includes an **x-llm-d-request-dropped-reason** header with a specific reason value:

| HTTP Status | Reason                            | Cause                                                                  |
|-------------|-----------------------------------|------------------------------------------------------------------------|
| 429         | <b>rejected-saturated</b>         | The priority band's queue capacity is exhausted                        |
| 503         | <b>rejected-ttl-expired</b>       | The request waited longer than the configured <b>defaultRequestTTL</b> |
| 503         | <b>rejected-context-cancelled</b> | The client disconnected while the request was queued                   |
| 503         | <b>rejected-no-endpoints</b>      | The pool has zero ready endpoints                                      |
| 503         | <b>rejected-shutting-down</b>     | The Endpoint Picker is shutting down gracefully                        |

## 11.1.3. Configuration strategies

Choose a configuration strategy based on your operational goal. Each strategy combines fairness policies, band limits, request TTL, and saturation thresholds to match a specific workload pattern.

### Multitenant isolation

Use when multiple tenants share GPU infrastructure and you need equitable service distribution.

- Fairness: **round-robin-fairness-policy** on all priority bands
- Saturation thresholds: **queueDepthThreshold: 4, kvCacheUtilThreshold: 0.8**

- Queue limits: Set per-band **maxBytes** to approximately 1 GB per band
- Request TTL: **defaultRequestTTL: 60s**
- Result: Each tenant receives equal dispatch turns within a priority band. Traffic floods are absorbed in the flooding tenant's own queue. Rejections are bounded by TTL and per-band capacity limits. Tradeoff: no latency advantage among tenants in the same priority band.

### Batch-heavy consolidation

Use when batch workloads dominate request volume and you need to prevent batch traffic from consuming shared queue capacity.

- Set per-band **maxBytes** or **maxRequests** on the sheddable priority band to bound batch queue size
- Set a longer **defaultRequestTTL** on the batch band to allow deferred processing
- Saturation thresholds: Use utilization detector defaults
- Result: Batch traffic queues within bounded capacity. When limits are exceeded, batch requests are rejected rather than crowding out higher-priority work. Tradeoff: the capped band rejects requests earlier by design.

### Maximum throughput

Use when you have a single workload type or uniform SLA requirements and want the lowest additional cost.

- Fairness: By default, the **global-strict-fairness-policy** is set
- Do not configure **priorityBands** or **flowControl.usageLimitPolicyPluginRef**
- Result: Strict arrival order with minimal scheduling cost. Tradeoff: no tenant isolation. A high-volume tenant wins dispatch share proportionally to its request volume.

### Additional resources

- [Configure flow control for Distributed Inference with llm-d](#)

## 11.2. CONFIGURE FLOW CONTROL FOR DISTRIBUTED INFERENCE WITH LLM-D

You can enable flow control in Distributed Inference with llm-d and create **InferenceObjective** resources to implement priority-based queuing for multitenant workloads. This procedure configures the flow control layer, defines priority tiers, and validates that priority queuing is working correctly.

### Prerequisites

- Distributed Inference with llm-d is deployed with Inference Gateway and Endpoint Picker (EPP).
- You have enabled authentication and authorization for your LLM Inference Service. To do this, follow the steps for [Enabling authentication and authorization for an LLM inference service](#)
- You have cluster administrator access.

## Procedure

1. Configure the Endpoint Picker to enable flow control and configure saturation detection. The **EndpointPickerConfig** is embedded within the **LLMInferenceService** custom resource under **spec.router.scheduler**. Edit your **LLMInferenceService** resource:

```
$ oc edit llminferenceservice <llm-service-name> -n <namespace>
```

2. In the **spec.router.scheduler.config.inline** section, add the **flowControl** feature gate and configure the saturation detector. The following YAML shows the content to add under **spec.router.scheduler.config.inline** in your **LLMInferenceService** resource. For advanced settings including global capacity limits, priority bands, and usage limit policies, see step 6.

```
apiVersion: llm-d.ai/v1alpha1
kind: EndpointPickerConfig
featureGates:
 - "flowControl"
flowControl:
 saturationDetector:
 pluginRef: utilization-detector
plugins:
 - name: utilization-detector
 type: utilization-detector
 parameters:
 queueDepthThreshold: 5
 kvCacheUtilThreshold: 0.8
 metricsStalenessThreshold: 200ms
```

where:

- **flowControl.saturationDetector.pluginRef**: Specifies the name of the saturation detector plugin to use. If omitted, defaults to **utilization-detector**. You can also use **concurrency-detector** for concurrency-based saturation detection.
- **plugins[].parameters.queueDepthThreshold**: Specifies the queue depth threshold on model servers.  
A value of 5 (default) balances latency and throughput. Set to 1 for maximum fairness, which forces all queuing into EPP priority-aware queues. Set to 1-2x max batch size for maximum throughput, which allows model servers to form fuller batches. To decide your model server's batch size for tuning **queueDepthThreshold**, check the vLLM deployment's **--max-batch-total-tokens** parameter or refer to your **LLMInferenceService** configuration. If unsure, start with the default value of 5 and adjust based on observed latency and throughput metrics.
- **plugins[].parameters.kvCacheUtilThreshold** - Specifies the KV cache utilization threshold from 0.0 to 1.0. Default is 0.8.
- **plugins[].parameters.metricsStalenessThreshold** - Specifies how long metrics can be stale before being considered invalid. Default is 200ms.  
As an alternative to the utilization-based saturation detector, you can use the concurrency-based saturation detector. The concurrency detector supports three modes: request-based, token-based, and hybrid.

### Request mode

```

plugins:
 - name: concurrency-detector
 type: concurrency-detector
 parameters:
 concurrencyMode: "requests"
 maxConcurrency: 100
 headroom: 0.2
flowControl:
 saturationDetector:
 pluginRef: concurrency-detector

```

### Token mode

```

plugins:
 - name: concurrency-detector
 type: concurrency-detector
 parameters:
 concurrencyMode: "tokens"
 maxTokenConcurrency: 500000
 headroom: 0.2
 - name: inflight-load-producer
 type: inflight-load-producer
 parameters:
 addEstimatedOutputTokens: true
flowControl:
 saturationDetector:
 pluginRef: concurrency-detector

```

### Hybrid mode

```

plugins:
 - name: concurrency-detector
 type: concurrency-detector
 parameters:
 concurrencyMode: "hybrid"
 maxConcurrency: 100
 maxTokenConcurrency: 500000
flowControl:
 saturationDetector:
 pluginRef: concurrency-detector

```

Hybrid mode uses the more constraining of request and token ratios per endpoint.

3. Save and exit the editor.
4. Create an **InferenceObjective** resource to define a priority tier:

```

apiVersion: llm-d.ai/v1alpha2
kind: InferenceObjective
metadata:
 name: <objective-name>
 namespace: <namespace>
spec:
 priority: <priority-value>

```

```
poolRef:
 group: llm-d.ai
 kind: InferencePool
 name: <inference-pool-name>
```

where:

- **metadata.name** - Specifies the **InferenceObjective** name.
- **metadata.namespace** - Specifies the namespace where the **InferencePool** is deployed.
- **priority** - Specifies the priority value. Higher integer values represent higher priority. Negative values designate lower-priority requests that are queued and dispatched last. Requests can be rejected when:
  - The priority band's capacity is exhausted (HTTP 429)
  - They time out (HTTP 503)
  - The client disconnects abruptly (HTTP 503)
- **poolRef.group** - Specifies the API group for the pool reference. Use **llm-d.ai**.
- **poolRef.name** - Specifies the name of the **InferencePool** resource this objective applies to.

5. Apply the **InferenceObjective** resource:

```
$ oc apply -f <inference-objective-file>.yaml
```

6. Optional: Configure advanced flow control settings for priority bands.

After creating **InferenceObjective** resources, you can customize flow control behavior by configuring priority band capacity, request timeouts, fairness policies, and usage limit policies. Edit your **LLMInferenceService** resource and update the **spec.router.scheduler.config.inline** section with the following configuration:

```
apiVersion: llm-d.ai/v1alpha1
kind: EndpointPickerConfig
featureGates:
 - "flowControl"
flowControl:
 maxBytes: 1073741824
 maxRequests: 5000
 defaultRequestTTL: 1m
 usageLimitPolicyPluginRef: my-holdback-policy
 saturationDetector:
 pluginRef: utilization-detector
 priorityBands:
 - priority: 100
 maxBytes: 536870912
 orderingPolicyRef: fcfs-ordering-policy
 fairnessPolicyRef: round-robin-fairness-policy
 - priority: 0
 orderingPolicyRef: fcfs-ordering-policy
 fairnessPolicyRef: round-robin-fairness-policy
 - priority: -1
```



```

maxRequests: 1000
orderingPolicyRef: fcfs-ordering-policy
fairnessPolicyRef: global-strict-fairness-policy
plugins:
- name: utilization-detector
 type: utilization-detector
 parameters:
 queueDepthThreshold: 5
 kvCacheUtilThreshold: 0.8
 metricsStalenessThreshold: 200ms
- name: my-holdback-policy
 type: priority-holdback-policy
 parameters:
 domain: rank
 shape: linear
 minCeiling: 0.3
 maxCeiling: 1.0

```

where:

- **flowControl.maxBytes** – Specifies an optional global queue capacity in bytes aggregated across all priority bands. When this limit is exceeded, new requests receive HTTP 429 errors. If set to 0 or omitted, only per-band limits apply.
- **flowControl.maxRequests** – Specifies an optional global queue capacity by request count aggregated across all priority bands. When this limit is exceeded, new requests receive HTTP 429 errors. If set to 0 or omitted, only per-band limits apply.
- **flowControl.defaultRequestTTL** – Specifies the request timeout. Requests waiting longer than this duration receive HTTP 503 errors. Default is 1 minute.
- **flowControl.usageLimitPolicyPluginRef** – Specifies a usage limit policy plugin that applies globally across all priority bands. Use **priority-holdback-policy** to gate lower-priority bands at lower saturation thresholds.
- **priorityBands[].priority** – Specifies the priority value for this band. Must match the priority values defined in your **InferenceObjective** resources.
- **priorityBands[].maxBytes** – Specifies per-band capacity in bytes. Controls load shedding for individual priority tiers.
- **priorityBands[].maxRequests** – Specifies per-band capacity by request count. Use for lower-priority bands to limit how many low-priority requests can queue.
- **priorityBands[].orderingPolicyRef** – Specifies the ordering policy. Default is **fcfs-ordering-policy** (first-come, first-served). Other supported policies are **edf-ordering-policy** (earliest deadline first, orders by absolute deadline) and **slo-deadline-ordering-policy** (orders by SLO-derived deadline).
- **priorityBands[].fairnessPolicyRef** – Specifies the fairness policy. The default **global-strict-fairness-policy** offers no tenant isolation and can result in starvation within a priority band. For equitable service distribution across tenants, use **round-robin-fairness-policy**.
- **plugins[].type: priority-holdback-policy** – Distributes saturation ceilings across priority bands so that lower-priority traffic is gated before higher-priority traffic under sustained load. The **domain** field controls how priority levels are mapped to positions in the ceiling

range: **rank** maps by ordinal position, **value** maps proportionally to numerical values. The **shape** field selects the interpolation curve. Currently only **linear** is supported, which distributes ceilings evenly across the range. The **minCeiling** field is required and sets the admission ceiling for the lowest-priority traffic. The **maxCeiling** field defaults to 1.0 and sets the ceiling for the highest-priority traffic.

## Verification

1. Verify that flow control is routing requests by priority.  
Check the Endpoint Picker **/metrics** endpoint for the **llm\_d\_epp\_flow\_control\_request\_queue\_duration\_seconds** metric with a **priority** label matching your configured priority values. If this metric exists, flow control is classifying and queuing requests by priority.
2. Verify that flow control queues are active under load.  
Send sustained traffic with varying request priorities by using load testing tools. Priority differentiation is active whenever there are multiple requests waiting in the queue. Check that **vllm:num\_requests\_running** is at or near the **--max-num-seqs** cap and **vllm:num\_requests\_waiting** is above zero on model server pods. If both conditions are met, requests are queuing at the Endpoint Picker and being dispatched by policy.
3. Monitor flow control metrics in Prometheus:
  - **llm\_d\_epp\_flow\_control\_queue\_size** - Queue depth for requests waiting to dispatch, broken down by fairness group and priority band
  - **llm\_d\_epp\_flow\_control\_queue\_bytes** - Queue size in bytes
  - **llm\_d\_epp\_flow\_control\_request\_queue\_duration\_seconds** - Time requests sit in the flow control queue
  - **llm\_d\_epp\_flow\_control\_pool\_saturation** - Current pool saturation level. A value of 1.0 is the gating set point for the default **static-usage-limit-policy**, where dispatch halts. Values above 1.0 indicate the magnitude of oversubscription past the gating threshold. When a **priority-holdback-policy** is configured, lower-priority bands are gated at lower saturation thresholds
  - **llm\_d\_epp\_flow\_control\_requests\_total** - Total requests processed by flow control, broken out by outcome label
4. If queue depth remains at 0, increase concurrency to create saturation conditions where priority differentiation occurs.

## Additional resources

- [Configure scheduler for LLM inference services](#)
- [EPP metrics reference](#)

## 11.3. INFERENCEOBJECTIVE CUSTOM RESOURCE REFERENCE

The **InferenceObjective** Custom Resource defines priority values for inference requests in flow control. You create **InferenceObjective** resources to establish priority tiers for different workload classes.

```

apiVersion: llm-d.ai/v1alpha2
kind: InferenceObjective
metadata:
 name: <service-account-namespace>
 namespace: <llmisvc-namespace>
spec:
 priority: 100 # Higher = higher priority
 poolRef:
 group: llm-d.ai
 kind: InferencePool
 name: <llmisvc-name>-inference-pool

```

Table 11.1. InferenceObjective specifications

| Field                | Type    | Required | Description                                                                                                                                             | Valid Values                                                                                                                              |
|----------------------|---------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>priority</b>      | integer | No       | Priority value for requests matching this <b>InferenceObjective</b> . Higher integer values represent higher priority. If not specified, defaults to 0. | Positive values (higher priority, example: 100), Zero (default priority), Negative values (lower priority, dispatched last, example: -10) |
| <b>poolRef.group</b> | string  | No       | API group for the InferencePool resource. If not specified, defaults to <b>llm-d.ai</b> .                                                               | <b>llm-d.ai</b>                                                                                                                           |
| <b>poolRef.kind</b>  | string  | No       | Resource kind. If not specified, defaults to <b>InferencePool</b> .                                                                                     | <b>InferencePool</b>                                                                                                                      |
| <b>poolRef.name</b>  | string  | Yes      | Name of the InferencePool resource this <b>InferenceObjective</b> applies to.                                                                           | Name of your InferencePool resource                                                                                                       |

## 11.4. VLLM PARAMETER CONFIGURATION FOR FLOW CONTROL

You configure vLLM parameters to control when flow control triggers request queuing.

vLLM parameters affect the saturation metrics that the Endpoint Picker monitors, allowing you to balance latency and throughput for your workload requirements. Flow control monitors two vLLM metrics to determine when model servers are saturated:

- **vllm:num\_requests\_waiting** - controlled by the **--max-num-seqs** parameter

- **vllm:kv\_cache\_usage\_perc** - controlled by the **--gpu-memory-utilization** and **--max-model-len** parameters

#### **--max-num-seqs**

Affects the **num\_requests\_waiting** metric. Controls how many requests can be in-flight simultaneously. Lower values trigger queue saturation sooner.

- Latency-sensitive workloads: Decrease to 1-2 to force queuing into Endpoint Picker priority queues.
- Throughput-optimized workloads: Increase to 8-16 to allow larger batches.

#### **--gpu-memory-utilization**

Affects the **kv\_cache\_usage\_perc** metric. Controls GPU memory allocated for KV cache. Lower values trigger cache saturation sooner. \* Latency-sensitive workloads: Decrease to 0.6-0.7 to trigger queuing when cache fills. \* Throughput-optimized workloads: Increase to 0.85-0.9 to maximize cache usage before queuing.

#### **--max-model-len**

Affects the **kv\_cache\_usage\_perc** metric. Controls maximum sequence length. Lower values reduce KV cache memory per request.

- Long-context workloads: Increase to support longer sequences.
- Short-context workloads: Decrease to allow more concurrent requests.

Pool saturation is calculated as:

```
saturation = Average across pods of Max(
 num_requests_waiting / queueDepthThreshold,
 kv_cache_usage_perc / kvCacheUtilThreshold
)
```

When saturation reaches or exceeds the gating threshold, flow control queues new requests until capacity becomes available. With the default **static-usage-limit-policy**, the gating threshold is 1.0 for all priority bands. When a **priority-holdback-policy** is configured, lower-priority bands are gated at lower saturation thresholds.

## 11.4.1. Workload tuning profiles

### Latency-sensitive workloads

For interactive chat and real-time applications where minimizing time-per-output-token (TPOT) variance is critical:

- vLLM parameters: **--max-num-seqs=1** or **--max-num-seqs=2** to force single or double batching. **--gpu-memory-utilization=0.7** to trigger queuing when cache fills.
- Saturation thresholds: **queueDepthThreshold: 1** and **kvCacheUtilThreshold: 0.7**.
- Tradeoff: Lower throughput but consistent low latency.

### Throughput-optimized workloads

For batch inference and offline processing where maximizing requests processed per second is the priority:

- vLLM parameters: **--max-num-seqs=8** to **--max-num-seqs=16** to allow larger batch sizes. **-gpu-memory-utilization=0.85** to maximize cache usage.
- Saturation thresholds: **queueDepthThreshold: 8** and **kvCacheUtilThreshold: 0.85**.
- Tradeoff: Higher throughput but increased tail latency.

### Mixed workloads with multiple priority tiers

For environments running multiple workload types with different service level objectives: \* vLLM parameters: **--max-num-seqs=4** for moderate batch sizes. **--gpu-memory-utilization=0.75** for balanced cache usage. \* Saturation thresholds: **queueDepthThreshold: 3** and **kvCacheUtilThreshold: 0.8**. \* Use **round-robin-fairness-policy** for high-priority bands to prevent starvation, and set larger **maxBytes** capacity for low-priority bands to absorb batch workloads.

## 11.4.2. Common issues and remedies

Table 11.2. Troubleshooting flow control tuning

| Symptom                                                                            | Likely cause                                                                 | Remedy                                                                                                |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Premature queuing with low GPU utilization and high queue depth at Endpoint Picker | <b>queueDepthThreshold</b> or <b>kvCacheUtilThreshold</b> too low            | Increase thresholds or increase vLLM <b>--max-num-seqs</b>                                            |
| Excessive tail latency with high TPOT variance                                     | <b>queueDepthThreshold</b> or <b>kvCacheUtilThreshold</b> too high           | Decrease thresholds to trigger Endpoint Picker queuing sooner, or decrease vLLM <b>--max-num-seqs</b> |
| KV cache OOM errors                                                                | <b>--gpu-memory-utilization</b> too high or <b>--max-model-len</b> too large | Decrease GPU memory utilization or reduce max sequence length                                         |
| Low throughput despite available GPU                                               | <b>--max-num-seqs</b> too low                                                | Increase max concurrent sequences to allow larger batches                                             |
| <b>HTTP 429</b> errors with <b>rejected-saturated</b> reason                       | Per-band <b>maxBytes</b> or <b>maxRequests</b> limit exceeded                | Increase per-band capacity limits or reduce request volume for that priority tier                     |
| <b>HTTP 503</b> errors with <b>rejected-ttl-expired</b> reason                     | Requests waiting longer than <b>defaultRequestTTL</b>                        | Increase request TTL, add capacity, or reduce request volume                                          |



### NOTE

Check the **x-llm-d-request-dropped-reason** response header for the specific reason when flow control rejects requests.

### Additional resources

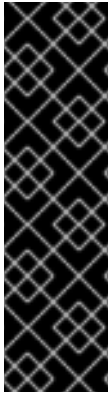
- [vLLM metrics server arguments](#)
- [vLLM metrics documentation](#)
- [Practical strategies for vLLM performance tuning](#)
- [5 steps to triage vLLM performance](#)

### Additional resources

- [Configure scheduler for LLM inference services](#)

## CHAPTER 12. BATCH INFERENCE WITH DISTRIBUTED INFERENCE WITH LLM-D

You can use batch inference with Distributed Inference with llm-d to process large volumes of inference requests asynchronously. Batch inference enables fire-and-forget job submission through the OpenAI-compatible `/v1/batches` and `/v1/files` API, with durable job state and priority-aware scheduling that protects real-time inference service level objectives (SLOs).



### IMPORTANT

Batch inference for Distributed Inference with llm-d is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

### 12.1. BATCH INFERENCE FOR DISTRIBUTED INFERENCE WITH LLM-D

You can use batch inference with Distributed Inference with llm-d to submit large volumes of inference requests as asynchronous jobs and retrieve results later, without maintaining active connections. The batch inference subsystem persists jobs across gateway pod restarts and uses priority-aware scheduling to run batch workloads during low-utilization periods, protecting real-time inference service level objectives (SLOs) on shared GPU infrastructure.

#### 12.1.1. When to use batch inference

Use batch inference for workloads where immediate responses are not required and you need to maximize throughput across a large volume of requests while meeting defined completion time targets.

Batch inference enables use cases including the following:

- Autonomous background agents performing multi-step reasoning and deep research
- Offline evaluations
- Dataset processing
- Embedding generation
- Large-scale model evaluation

All of these use cases follow the same workflow: upload a JSONL input file, create a batch job, monitor progress, and download results. The specific use case is determined by the content and structure of the JSONL input file you provide.

Batch inference provides the following benefits:

- Increases GPU infrastructure use by filling capacity during periods of lower interactive traffic
- Protects real-time inference SLOs by running batch workloads at lower priority without interfering with interactive traffic

- Enables cost-optimized processing by taking advantage of differential billing between batch and interactive workloads

### 12.1.2. Inference modes in Distributed Inference with llm-d

Distributed Inference with llm-d supports two inference modes that address different latency and throughput requirements:

#### Real-time inference

Synchronous request-response with latency on the order of seconds to minutes. Model servers process requests immediately. Use real-time inference for interactive applications such as chatbots, code completion, and live search.

#### Batch inference

Asynchronous fire-and-forget job submission with latency on the order of hours. You can submit requests as batch jobs through the OpenAI-compatible **/v1/batches** API. Requests are processed in the background. Use batch inference for high-volume workloads that do not require immediate responses, such as embedding generation and model evaluation.

### 12.1.3. Batch inference architecture

The batch inference subsystem consists of the following components:

#### Batch gateway API server

Exposes the OpenAI-compatible **/v1/batches** and **/v1/files** endpoints. Receives batch job submissions, validates input files, and stores job metadata. The API server is deployed and managed by the Batch Gateway Operator.

#### Batch processor

Reads pending batch jobs from a queue, concurrently dispatches inference requests from each job to model servers through the internal ClusterIP gateway, and writes results to S3-compatible storage. The processor uses AIMD (additive-increase / multiplicative-decrease) adaptive concurrency control to dynamically adjust concurrent request limits based on success and failure signals, automatically backing off under load to protect interactive traffic. Retry logic with exponential backoff handles transient failures.

To maximize throughput, the processor sorts requests by system-prompt hash before dispatching, optimizing prefix cache reuse across the serving pool by keeping identical-prefix KV-cache entries hot. The processor integrates with Distributed Inference with llm-d's intelligent request routing and flow control mechanisms, allowing batch workloads to benefit from prefix-cache-aware routing and automatic load balancing.

#### Garbage collector

Periodically cleans up completed, failed, and expired batch jobs based on configurable retention periods. Removes associated input and output files from storage.

The batch inference subsystem uses a pluggable storage architecture with two layers:

- A database layer for persisting batch metadata, including job state, request counts, and token usage
- An exchange layer for job queuing, priority queues, event channels, and in-flight job tracking

The available storage plugins are PostgreSQL for the database layer and Redis or Valkey for the exchange layer. The subsystem also requires S3-compatible storage or filesystem storage for batch input and output files.



### 12.1.4. OpenAI Batch API compatibility

The batch inference API is compatible with the OpenAI Batch API specification. You submit batch jobs by uploading a JSONL input file through the **/v1/files** endpoint and creating a batch job through the **/v1/batches** endpoint. The batch inference subsystem supports the **/v1/chat/completions**, **/v1/completions**, **/v1/embeddings**, **/v1/responses**, and **/v1/moderations** inference endpoints within batch requests.

Each line in the JSONL input file has one inference request with a unique **custom\_id** that correlates the request to its result in the output file. The API server validates the input file format before scheduling the job.

By default, batches are limited to 50,000 requests and input files are limited to 200 MB. These limits are configurable. Optionally, you can apply a **RateLimitPolicy** on the external gateway to enforce per-user rate limiting for batch API requests.

### 12.1.5. Priority-aware dispatching for batch workloads

The batch gateway maintains an internal priority queue of jobs sorted by SLO deadline. Each job SLO is computed from its **completion\_window** at creation time, so jobs with the nearest deadline are processed first.

You can use shared GPU infrastructure for both real-time and batch workloads without dedicated GPU pools for each workload type. To achieve this, configure the batch gateway to work with the flow control mechanism in Distributed Inference with Distributed Inference with llm-d. In this setup, the SLOs of real-time workloads are protected while batch workloads make progress toward completion by utilizing free GPU capacity. When dispatching inference requests, the batch processor sets flow control headers that communicate the job SLO deadline and priority band to the router. The priority band is determined by a configurable **InferenceObjective** CRD that you specify in the model gateway configuration. Batch workloads are typically assigned a lower priority band than interactive workloads, so that they are dispatched only when capacity is available.

### 12.1.6. Multi-tenancy

The batch inference subsystem supports multi-tenancy through the **X-MaaS-Username** header. Each user's batch jobs and files are isolated. Authentication is delegated to the external gateway's **AuthPolicy**, which validates bearer tokens in the **Authorization** header (Kubernetes ServiceAccount tokens or user tokens) and injects the authenticated username into the request headers for the batch gateway to use for tenant identification.

#### Additional resources

- [Flow Control Setup for Batch and Interactive Inference](#)

## 12.2. CONFIGURE BATCH INFERENCE FOR DISTRIBUTED INFERENCE WITH LLM-D

You can deploy and configure the batch inference subsystem for Distributed Inference with llm-d to enable asynchronous batch job submission on shared GPU infrastructure. The batch gateway is deployed through the AI Gateway component in the **DataScienceCluster** CR and configured with backing services, internal routing for model authorization, and external API routes for batch job submission.

## Prerequisites

- Distributed Inference with llm-d is deployed and you have configured the inference gateway.
- The batch gateway component is enabled in the **DataScienceCluster** resource.
- You have cluster administrator access.
- You have installed the OpenShift CLI (`oc`). For more information, see [Installing the OpenShift CLI](#).
- A Redis or Valkey instance is available for job queuing.  
In disconnected environments, deploy Redis or Valkey cluster-locally.
- A PostgreSQL instance is available for batch metadata storage.  
In disconnected environments, deploy PostgreSQL cluster-locally.
- S3-compatible storage, such as AWS S3, MinIO, or Ceph, is available for batch input and output files.  
In disconnected environments, deploy MinIO or another S3-compatible storage service cluster-locally.

## Procedure

1. Verify that the batch gateway component is enabled in the **DataScienceCluster** resource:

```
$ oc get datasciencecluster default-dsc -o yaml
```

Expected output:

```
spec:
 components:
 aigateway:
 managementState: Managed
 batchGateway:
 managementState: Managed
```

If the batch gateway component is not enabled, edit the **DataScienceCluster** resource:

```
$ oc edit datasciencecluster default-dsc
```

Add or update the following configuration:

```
spec:
 components:
 aigateway:
 managementState: Managed
 batchGateway:
 managementState: Managed
```

Save the changes and wait for the **DataScienceCluster** to reconcile. The operator creates the batch gateway namespace and deploys the Batch Gateway Operator.

2. Create an internal ClusterIP gateway for batch processor inference routing.

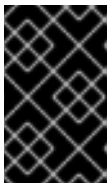
The batch processor routes inference requests through this internal gateway to enforce model-level authorization while bypassing per-user token rate limits on the external gateway.

Save the following YAML to a file named **batch-internal-gateway.yaml**:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
 name: batch-internal-gateway
 namespace: openshift-ingress
 annotations:
 networking.istio.io/service-type: ClusterIP
spec:
 gatewayClassName: openshift-default
 listeners:
 - name: http
 port: 80
 protocol: HTTP
 allowedRoutes:
 namespaces:
 from: Selector
 selector:
 matchLabels:
 llm-d.ai/gateway-route: "true"
```

Apply the Gateway resource:

```
$ oc apply -f batch-internal-gateway.yaml
```



### IMPORTANT

The **networking.istio.io/service-type: ClusterIP** annotation ensures the gateway is not externally accessible. The internal gateway uses HTTP only because traffic is cluster-internal.

3. Create an **HTTPRoute** to route batch processor inference requests to the model server through the internal gateway.

The batch processor rewrites the request path to match the model server's API and includes the user's original authentication token.

Save the following YAML to a file named **batch-llm-route.yaml**:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
 name: batch-llm-route
 namespace: <llm_svc_namespace>
spec:
 parentRefs:
 - name: batch-internal-gateway
 namespace: openshift-ingress
 rules:
 - matches:
 - path:
```

```

 type: PathPrefix
 value: /<llmsvc_namespace>/<llmsvc_name>/v1/chat/completions
 filters:
 - type: URLRewrite
 urlRewrite:
 path:
 type: ReplacePrefixMatch
 replacePrefixMatch: /v1/chat/completions
 backendRefs:
 - group: inference.networking.k8s.io
 kind: InferencePool
 name: <inference_pool_name>
 port: 80
- matches:
- path:
 type: PathPrefix
 value: /<llmsvc_namespace>/<llmsvc_name>/v1/completions
 filters:
 - type: URLRewrite
 urlRewrite:
 path:
 type: ReplacePrefixMatch
 replacePrefixMatch: /v1/completions
 backendRefs:
 - group: inference.networking.k8s.io
 kind: InferencePool
 name: <inference_pool_name>
 port: 80
- matches:
- path:
 type: PathPrefix
 value: /<llmsvc_namespace>/<llmsvc_name>/v1/embeddings
 filters:
 - type: URLRewrite
 urlRewrite:
 path:
 type: ReplacePrefixMatch
 replacePrefixMatch: /v1/embeddings
 backendRefs:
 - group: inference.networking.k8s.io
 kind: InferencePool
 name: <inference_pool_name>
 port: 80
- matches:
- path:
 type: PathPrefix
 value: /<llmsvc_namespace>/<llmsvc_name>/v1/responses
 filters:
 - type: URLRewrite
 urlRewrite:
 path:
 type: ReplacePrefixMatch
 replacePrefixMatch: /v1/responses
 backendRefs:
 - group: inference.networking.k8s.io
 kind: InferencePool

```

```

 name: <inference_pool_name>
 port: 80
 - matches:
 - path:
 type: PathPrefix
 value: /<llmisvc_namespace>/<llmisvc_name>/v1/moderations
 filters:
 - type: URLRewrite
 urlRewrite:
 path:
 type: ReplacePrefixMatch
 replacePrefixMatch: /v1/moderations
 backendRefs:
 - group: inference.networking.k8s.io
 kind: InferencePool
 name: <inference_pool_name>
 port: 80

```

where:

- **<llmisvc\_namespace>**: Specifies the namespace where the **LLMInferenceService** is deployed.
- **<llmisvc\_name>**: Specifies the name of the **LLMInferenceService** resource.
- **<inference\_pool\_name>**: Specifies the name of the **InferencePool** resource created by the **LLMInferenceService**.

4. Apply the HTTPRoute resource:

```
$ oc apply -f batch-llm-route.yaml
```

5. Create an **AuthPolicy** for the batch-llm-route to enforce model-level authorization:  
Save the following YAML to a file named **batch-llm-authpolicy.yaml**:

```

apiVersion: kuadrant.io/v1
kind: AuthPolicy
metadata:
 name: batch-llm-authpolicy
 namespace: <llmisvc_namespace>
spec:
 targetRef:
 group: gateway.networking.k8s.io
 kind: HTTPRoute
 name: batch-llm-route
 rules:
 authentication:
 "kubernetes-token":
 kubernetesTokenReview:
 audiences:
 - "https://kubernetes.default.svc"
 authorization:
 "llmisvc-access":
 kubernetesSubjectAccessReview:
 user:

```

```

 selector: auth.identity.user.username
 resourceAttributes:
 group: serving.kserve.io
 resource: llminferenceservices
 name: <llmisvc_name>
 namespace: <llmisvc_namespace>
 verb: get
 response:
 success:
 headers:
 "X-MaaS-Username":
 plain:
 selector: auth.identity.user.username

```

Apply the AuthPolicy resource:

```
$ oc apply -f batch-llm-authpolicy.yaml
```

The **AuthPolicy** validates the user's Kubernetes token and checks whether the user has permission to **get** the specific **LLMInferenceService** resource by using **SubjectAccessReview**.

6. Create a Kubernetes Secret with connection details for Redis, PostgreSQL, and S3-compatible storage.

Save the following YAML to a file named **batch-gateway-secrets.yaml**:

```

apiVersion: v1
kind: Secret
metadata:
 name: batch-gateway-secrets
 namespace: <batch_gateway_namespace>
type: Opaque
stringData:
 redis-url: "redis://<redis_host>:<redis_port>/0"
 postgresql-url: "postgresql://<db_user>:<db_password>@<db_host>:
<db_port>/<db_name>?sslmode=disable"
 s3-secret-access-key: "<s3_secret_key>"

```

where:

- **<batch\_gateway\_namespace>**: Specifies the namespace for the batch gateway deployment.
- **<redis\_host>**: Specifies the hostname or IP address of the Redis or Valkey instance.
- **<redis\_port>**: Specifies the port number of the Redis or Valkey instance: default is 6379.
- **<db\_user>**: Specifies the PostgreSQL username.
- **<db\_password>**: Specifies the PostgreSQL password.
- **<db\_host>**: Specifies the hostname or IP address of the PostgreSQL instance.
- **<db\_port>**: Specifies the port number of the PostgreSQL instance: default is 5432.
- **<db\_name>**: Specifies the name of the PostgreSQL database for batch metadata.

- **<s3\_secret\_key>**: Specifies the secret access key for S3-compatible storage.



#### NOTE

S3 configuration settings (region, endpoint, access key ID, and bucket name) are non-sensitive and are configured in the **LLMBatchGateway** custom resource, not in this secret. Only the S3 secret access key belongs in the secret.

Apply the Secret:

```
$ oc apply -f batch-gateway-secrets.yaml
```

7. Deploy infrastructure dependencies for the batch gateway.

The batch gateway requires Redis or Valkey for job queuing, PostgreSQL for metadata storage, and S3-compatible storage for batch files. Deploy these services before creating the **LLMBatchGateway** custom resource.

For production deployments, use managed services or deploy these components with appropriate persistence, backups, and high availability. For development or testing environments, you can deploy minimal instances by using Helm charts or Kubernetes manifests.



#### NOTE

For example deployment manifests for Redis, PostgreSQL, and MinIO, see the [batch gateway RHOAI deployment guide](#).

8. Create an **LLMBatchGateway** custom resource to deploy the batch gateway components.  
Save the following YAML to a file named **llm-batch-gateway.yaml**:

```
apiVersion: llm-d.ai/v1alpha1
kind: LLMBatchGateway
metadata:
 name: batch-gateway
 namespace: <batch_gateway_namespace>
spec:
 secretRef:
 name: batch-gateway-secrets
 dbBackend: postgresql
 fileStorage:
 s3:
 region: <s3_region>
 endpoint: <s3_endpoint>
 accessKeyId: <s3_access_key_id>
 prefix: <s3_bucket_name>
 usePathStyle: true
 autoCreateBucket: true
 apiServer:
 replicas: 1
 processor:
 replicas: 1
 globalInferenceGateway:
 url: http://batch-internal-gateway.openshift-
```

```

ingress.svc.cluster.local/<llmisvc_namespace>/<llmisvc_name>/v1
 requestTimeout: 5m
 maxRetries: 3
 initialBackoff: 1s
 maxBackoff: 60s
 config:
 inferenceObjective: batch-workload
 gc:
 interval: 30m
 tls:
 enabled: true
 certManager:
 issuerName: selfsigned-issuer
 issuerKind: ClusterIssuer
 dnsNames:
 - batch-gateway-apiserver
 - batch-gateway-apiserver.<batch_gateway_namespace>.svc.cluster.local
 - localhost

```

where:

- **<batch\_gateway\_namespace>**: Specifies the namespace where the batch gateway is deployed.
- **<s3\_region>**: Specifies the S3 region, for example, **us-east-1**.
- **<s3\_endpoint>**: Specifies the S3 endpoint URL. For MinIO deployed in-cluster, use **http://minio.<batch\_gateway\_namespace>.svc.cluster.local:9000**.
- **<s3\_access\_key\_id>**: Specifies the S3 access key ID.
- **<s3\_bucket\_name>**: Specifies the S3 bucket name for batch input and output files.
- **<llmisvc\_namespace>**: Specifies the namespace of the **LLMInferenceService**.
- **<llmisvc\_name>**: Specifies the name of the **LLMInferenceService** resource.  
Apply the **LLMBatchGateway** resource:

```
$ oc apply -f llm-batch-gateway.yaml
```

The Batch Gateway Operator deploys the batch-gateway-apiserver, batch-processor, and garbage-collector components. Component images are pinned by the operator from its deployment configuration.



## NOTE

To use filesystem storage instead of S3, replace the **fileStorage.s3** section with:

```

fileStorage:
 fs:
 basePath: /tmp/batch-gateway
 claimName: <your-pvc-name>

```

The PersistentVolumeClaim must have **ReadWriteMany** access mode.



9. Create an **HTTPRoute** to route external batch API traffic to the batch-gateway-apiserver. Save the following YAML to a file named **batch-api-route.yaml**:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
 name: batch-api-route
 namespace: <batch_gateway_namespace>
spec:
 parentRefs:
 - name: <external_gateway_name>
 namespace: <gateway_namespace>
 rules:
 - matches:
 - path:
 type: PathPrefix
 value: /v1/batches
 - path:
 type: PathPrefix
 value: /v1/files
 backendRefs:
 - name: batch-gateway-apiserver
 port: 8000
```

where:

- **<external\_gateway\_name>**: Specifies the name of the external Kubernetes Gateway resource that receives incoming batch API requests.
- **<gateway\_namespace>**: Specifies the namespace of the external gateway, typically **openshift-ingress**.

Apply the HTTPRoute resource:

```
$ oc apply -f batch-api-route.yaml
```

Apply an **AuthPolicy** for the batch API routes.

Save the following YAML to a file named **batch-api-authpolicy.yaml**:

```
apiVersion: kuadrant.io/v1
kind: AuthPolicy
metadata:
 name: batch-api-authpolicy
 namespace: <batch_gateway_namespace>
spec:
 targetRef:
 group: gateway.networking.k8s.io
 kind: HTTPRoute
 name: batch-api-route
 rules:
 authentication:
 "kubernetes-token":
 kubernetesTokenReview:
 audiences:
 - "https://kubernetes.default.svc"
```

```

response:
 success:
 headers:
 "X-MaaS-Username":
 plain:
 selector: auth.identity.user.username

```

Apply the AuthPolicy resource:

```
$ oc apply -f batch-api-authpolicy.yaml
```

The batch API **AuthPolicy** performs authentication only. Model-level authorization is enforced by the **batch-llm-authpolicy** on the internal gateway when the processor forwards inference requests.

10. Apply a **RateLimitPolicy** to enforce per-user rate limits on the batch API.  
Save the following YAML to a file named **batch-rate-limit-policy.yaml**:

```

apiVersion: kuadrant.io/v1
kind: RateLimitPolicy
metadata:
 name: batch-rate-limit
 namespace: <batch_gateway_namespace>
spec:
 targetRef:
 group: gateway.networking.k8s.io
 kind: HTTPRoute
 name: batch-api-route
 limits:
 "per-user":
 rates:
 - limit: 20
 window: 1m
 counters:
 - auth.identity.user.username

```

Apply the RateLimitPolicy resource:

```
$ oc apply -f batch-rate-limit-policy.yaml
```

11. Create an **InferenceObjective** at priority -1 for batch workloads.  
Save the following YAML to a file named **batch-inference-objective.yaml**:

```

apiVersion: llm-d.ai/v1alpha2
kind: InferenceObjective
metadata:
 name: batch-workload
 namespace: <llmisvc_namespace>
spec:
 priority: -1
 poolRef:
 group: llm-d.ai
 kind: InferencePool
 name: <inference_pool_name>

```

where:

- **<llmisvc\_namespace>**: Specifies the namespace where the **InferencePool** is deployed.
- **<inference\_pool\_name>**: Specifies the name of the **InferencePool** resource for the target model.

Apply the InferenceObjective resource:

```
$ oc apply -f batch-inference-objective.yaml
```

The negative priority value (**priority: -1**) means batch requests run at lower priority than interactive workloads and are the first to be dropped when the system reaches saturation, protecting real-time inference SLOs.

## Verification

1. Verify that the batch gateway pods are running:

```
$ oc get pods -n <batch_gateway_namespace>
```

The output shows pods for the batch-gateway-apiserver, batch-gateway-processor, and batch-gateway-gc components in **Running** status.

2. Submit a test batch job to verify the subsystem is functioning:

- a. Create a test JSONL input file:

```
$ cat > /tmp/test-batch.jsonl << 'EOF'
{"custom_id": "test-1", "method": "POST", "url": "/v1/chat/completions", "body": {"model": "
<model_name>", "messages": [{"role": "user", "content": "Hello"}], "max_tokens": 10}}
EOF
```

- b. Upload the file:

```
$ curl -s -X POST "https://<batch_gateway_url>/v1/files" \
-H "Authorization: Bearer <auth_token>" \
-F "file=@/tmp/test-batch.jsonl" \
-F "purpose=batch"
```

- c. Create a batch job by using the returned **file\_id**:

```
$ curl -s -X POST "https://<batch_gateway_url>/v1/batches" \
-H "Authorization: Bearer <auth_token>" \
-H "Content-Type: application/json" \
-d '{
 "input_file_id": "<file_id>",
 "endpoint": "/v1/chat/completions",
 "completion_window": "24h"
}'
```

- d. Poll the batch status until it reaches **completed**:

```
$ curl -s "https://<batch_gateway_url>/v1/batches/<batch_id>" \
-H "Authorization: Bearer <auth_token>"
```

■

A successful test batch job transitions through **validating**, **in\_progress**, **finalizing**, and **completed** states.

#### Additional resources

- [Deploy distributed inference with llm-d on OpenShift Container Platform](#)
- [Deploy distributed inference with llm-d on Azure or CoreWeave Kubernetes Service](#)

## 12.3. SUBMIT A BATCH INFERENCE JOB

You can submit a batch inference job to process large volumes of inference requests asynchronously through the OpenAI-compatible **/v1/batches** and **/v1/files** API endpoints. Batch jobs persist across gateway pod restarts and do not require an active client connection after submission.

#### Prerequisites

- The batch inference subsystem is configured by a cluster administrator.
- You have an authentication token for the batch-gateway endpoint: a Kubernetes ServiceAccount token or a user token.
- A deployed Distributed Inference with llm-d model is available for inference.
- You have the batch-gateway API endpoint URL.

#### Procedure

1. Prepare a JSONL input file with one inference request per line.  
Each line must be a valid JSON object with the following fields:

```
{ "custom_id": "request-1", "method": "POST", "url": "/v1/chat/completions", "body": { "model": "<model_name>", "messages": [{ "role": "user", "content": "Summarize the benefits of container orchestration." }], "max_tokens": 256 } }
{ "custom_id": "request-2", "method": "POST", "url": "/v1/chat/completions", "body": { "model": "<model_name>", "messages": [{ "role": "user", "content": "Explain Kubernetes networking." }], "max_tokens": 256 } }
```

where:

##### **custom\_id**

Specifies a unique identifier for correlating this request with its result in the output file.

##### **method**

Specifies the HTTP method. Must be **POST**.

##### **url**

Specifies the inference endpoint path. Supported values are **/v1/chat/completions**, **/v1/completions**, **/v1/embeddings**, **/v1/responses**, and **/v1/moderations**.

##### **body**

Specifies the request body, following the same schema as the corresponding synchronous inference endpoint.



## NOTE

This example shows chat completion requests. You can use this same workflow for different use cases such as embedding generation, offline evaluations, or dataset processing by adjusting the content and structure of your JSONL file. For example, use **/v1/embeddings** for embedding generation or construct evaluation-specific prompts for offline model evaluation.

Save this file locally, for example as **batch-input.jsonl**.

2. Upload the input file to the batch gateway:

```
$ curl -s -X POST "https://<batch_gateway_url>/v1/files" \
-H "Authorization: Bearer <auth_token>" \
-F "file=@batch-input.jsonl" \
-F "purpose=batch"
```

where:

### <batch\_gateway\_url>

Specifies the URL of the batch-gateway-apiserver endpoint.

### <auth\_token>

Specifies your Kubernetes ServiceAccount token or user token.

The response includes a **file\_id** that you use to create the batch job:

```
{
 "id": "file-abc123",
 "object": "file",
 "purpose": "batch",
 "filename": "batch-input.jsonl",
 "bytes": 512,
 "created_at": 1719878400
}
```



## NOTE

For additional file operations such as listing files, retrieving file metadata, or deleting files, see [Section 12.4, "Batch inference API reference"](#).

3. Create a batch job by using the uploaded file:

```
$ curl -s -X POST "https://<batch_gateway_url>/v1/batches" \
-H "Authorization: Bearer <auth_token>" \
-H "Content-Type: application/json" \
-d '{
 "input_file_id": "<file_id>",
 "endpoint": "/v1/chat/completions",
 "completion_window": "24h"
}'
```

where:

#### <file\_id>

Specifies the **id** value from the file upload response.

The response includes a **batch\_id** and shows the initial status as **validating**:

```
{
 "id": "batch-xyz789",
 "object": "batch",
 "endpoint": "/v1/chat/completions",
 "input_file_id": "file-abc123",
 "status": "validating",
 "created_at": 1719878400
}
```



#### NOTE

This example shows required parameters only. For optional parameters such as custom metadata or batch listing and filtering options, see [Section 12.4, “Batch inference API reference”](#).

4. Poll the batch job status until it reaches **completed**:

```
$ curl -s "https://<batch_gateway_url>/v1/batches/<batch_id>" \
-H "Authorization: Bearer <auth_token>"
```

where:

#### <batch\_id>

Specifies the **id** value from the batch creation response.

The batch job transitions through the following states: **validating** → **in\_progress** → **finalizing** → **completed**. A completed batch response includes **output\_file\_id** and request counts:

```
{
 "id": "batch-xyz789",
 "object": "batch",
 "status": "completed",
 "input_file_id": "file-abc123",
 "output_file_id": "file-out456",
 "request_counts": {
 "total": 2,
 "completed": 2,
 "failed": 0
 }
}
```

5. Retrieve the batch results by downloading the output file:

```
$ curl -s "https://<batch_gateway_url>/v1/files/<output_file_id>/content" \
-H "Authorization: Bearer <auth_token>"
```

-

where:

### <output\_file\_id>

Specifies the **output\_file\_id** value from the completed batch response.

The output is a JSONL file with one result per line, correlated by **custom\_id**:

```
{
 "id": "response-1",
 "custom_id": "request-1",
 "response": {
 "status_code": 200,
 "body": {
 "choices": [
 {
 "message": {
 "role": "assistant",
 "content": "Container orchestration provides..."
 }
 }
]
 }
 }
},
{
 "id": "response-2",
 "custom_id": "request-2",
 "response": {
 "status_code": 200,
 "body": {
 "choices": [
 {
 "message": {
 "role": "assistant",
 "content": "Kubernetes networking..."
 }
 }
]
 }
 }
}
```

#### 6. Optional: Cancel a batch job:

```
$ curl -s -X POST "https://<batch_gateway_url>/v1/batches/<batch_id>/cancel" \
-H "Authorization: Bearer <auth_token>"
```

You can cancel a batch job in the **validating** or **in\_progress** state. Jobs still queued in the **validating** state are cancelled immediately and transition directly to **cancelled**. Jobs already being processed in the **in\_progress** state transition through **cancelling** to **cancelled** as the processor winds down. Requests that were already completed before cancellation are included in the output file.

### Verification

- Verify that the batch job completed successfully by checking the **status** field in the batch response. A successful batch shows **status: completed** with **request\_counts.failed: 0**.
- Verify that the output file contains results for all input requests by comparing the number of lines in the output file with the **request\_counts.total** value.

### Additional resources

- [Batch inference API reference](#)

## 12.4. BATCH INFERENCE API REFERENCE

The batch inference subsystem for Distributed Inference with Llm-d exposes OpenAI-compatible REST API endpoints for submitting and managing asynchronous batch inference jobs. This reference documents all available endpoints, parameters, response schemas, and error formats.

Consult this reference when you need to the following:

- Use optional parameters not covered in the basic workflow, such as metadata or filtering options
- Understand the complete Batch object schema and lifecycle states
- Troubleshoot API errors by reviewing status codes and error response formats
- Integrate batch inference into automated workflows or scripts

All requests require a bearer token in the **Authorization** header. Use a Kubernetes ServiceAccount token or a user token validated by the gateway **AuthPolicy**.

## /v1/batches endpoints

The **/v1/batches** endpoints manage batch inference jobs.

Table 12.1. POST /v1/batches – Create a batch job

| Field                                  | Description                                                                                                                                                                                                   |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Request body                           |                                                                                                                                                                                                               |
| <b>input_file_id</b><br>(required)     | The ID of the uploaded JSONL input file. The file <b>purpose</b> field must be set to <b>batch</b> .                                                                                                          |
| <b>endpoint</b> (required)             | The inference endpoint to use for all requests in the batch. Supported values: <b>/v1/chat/completions</b> , <b>/v1/completions</b> , <b>/v1/embeddings</b> , <b>/v1/responses</b> , <b>/v1/moderations</b> . |
| <b>completion_window</b><br>(required) | The time window for batch completion. Accepts any valid Go duration string, such as <b>"1h"</b> , <b>"30m"</b> , <b>"24h"</b> , or <b>"48h"</b> .                                                             |
| <b>metadata</b> (optional)             | A map of key-value pairs for custom metadata. Maximum 16 pairs; keys up to 64 characters, values up to 512 characters.                                                                                        |
| Response                               | A Batch object with <b>status: validating</b> .                                                                                                                                                               |

Table 12.2. GET /v1/batches/{batch\_id} – Retrieve batch status

| Field                      | Description                                                       |
|----------------------------|-------------------------------------------------------------------|
| Path parameter             |                                                                   |
| <b>batch_id</b> (required) | The ID of the batch to retrieve.                                  |
| Response                   | A Batch object with current status, request counts, and file IDs. |

Table 12.3. GET /v1/batches – List batches

| Field                   | Description                                                                                      |
|-------------------------|--------------------------------------------------------------------------------------------------|
| Query parameters        |                                                                                                  |
| <b>limit</b> (optional) | Maximum number of batches to return. Default: 20, maximum: 100.                                  |
| <b>after</b> (optional) | An integer offset for pagination. Returns batches starting from this position in the result set. |



| Field    | Description                                         |
|----------|-----------------------------------------------------|
| Response | A list of Batch objects for the authenticated user. |

Table 12.4. POST /v1/batches/{batch\_id}/cancel – Cancel a batch

| Field                      | Description                                                                                                                           |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Path parameter             |                                                                                                                                       |
| <b>batch_id</b> (required) | The ID of the batch to cancel. The batch must be in <b>validating</b> , <b>in_progress</b> , or <b>cancelling</b> status.             |
| Response                   | A Batch object with <b>status: cancelling</b> (if the job was in progress) or <b>status: cancelled</b> (if the job was still queued). |

### /v1/files endpoints for batch inference

The **/v1/files** endpoints in the batch inference context manage JSONL input and output files.

Table 12.5. POST /v1/files – Upload a file

| Field                     | Description                                                                                              |
|---------------------------|----------------------------------------------------------------------------------------------------------|
| Form data                 |                                                                                                          |
| <b>file</b> (required)    | The JSONL file to upload. Maximum size: 200 MB.                                                          |
| <b>purpose</b> (required) | Must be <b>batch</b> .                                                                                   |
| Response                  | A File object with <b>id</b> , <b>filename</b> , <b>bytes</b> , <b>purpose</b> , and <b>created_at</b> . |

Table 12.6. GET /v1/files – List files

| Field                   | Description                                                                                                                |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Query parameters        |                                                                                                                            |
| <b>limit</b> (optional) | Maximum number of files to return. Default: 20, maximum: 10,000.                                                           |
| <b>after</b> (optional) | An integer offset for pagination. Returns files starting from this position in the result set.                             |
| Response                | A list wrapper with <b>data</b> (array of File objects), <b>has_more</b> (boolean), <b>first_id</b> , and <b>last_id</b> . |

Table 12.7. GET /v1/files/{file\_id} – Retrieve file metadata

| Field                     | Description                                                                                                                 |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Path parameter            |                                                                                                                             |
| <b>file_id</b> (required) | The ID of the file to retrieve metadata for.                                                                                |
| Response                  | A File object with metadata including <b>id</b> , <b>filename</b> , <b>bytes</b> , <b>purpose</b> , and <b>created_at</b> . |

Table 12.8. GET /v1/files/{file\_id}/content – Download file content

| Field                     | Description                                                                            |
|---------------------------|----------------------------------------------------------------------------------------|
| Path parameter            |                                                                                        |
| <b>file_id</b> (required) | The ID of the file whose content to download. Use this to retrieve batch output files. |
| Response                  | The raw JSONL file content.                                                            |

Table 12.9. DELETE /v1/files/{file\_id} – Delete a file

| Field                     | Description                                                         |
|---------------------------|---------------------------------------------------------------------|
| Path parameter            |                                                                     |
| <b>file_id</b> (required) | The ID of the file to delete.                                       |
| Response                  | A deletion confirmation with the file ID and <b>deleted: true</b> . |

## JSONL input format

Each line in the input JSONL file must be a valid JSON object with the following fields:

Table 12.10. JSONL input line schema

| Field            | Type   | Required | Description                                                                                             |
|------------------|--------|----------|---------------------------------------------------------------------------------------------------------|
| <b>custom_id</b> | string | Yes      | A unique identifier for this request. Used to correlate the request with its result in the output file. |
| <b>method</b>    | string | Yes      | The HTTP method. Must be <b>POST</b> .                                                                  |

| Field       | Type   | Required | Description                                                                                                                                                                   |
|-------------|--------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>url</b>  | string | Yes      | The inference endpoint path. Supported values: <b>/v1/chat/completions</b> , <b>/v1/completions</b> , <b>/v1/embeddings</b> , <b>/v1/responses</b> , <b>/v1/moderations</b> . |
| <b>body</b> | object | Yes      | The request body, following the same schema as the corresponding synchronous inference endpoint. Must include the <b>model</b> field.                                         |

Example input line:

```
{"custom_id": "req-001", "method": "POST", "url": "/v1/chat/completions", "body": {"model": "<model_name>", "messages": [{"role": "user", "content": "What is Kubernetes?"}], "max_tokens": 100}}
```

### JSONL output format

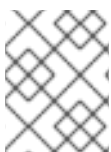
Each line in the output JSONL file contains the result for one input request:

Table 12.11. JSONL output line schema

| Field            | Type           | Description                                                                                                                                                           |
|------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>id</b>        | string         | A unique identifier for this response.                                                                                                                                |
| <b>custom_id</b> | string         | The <b>custom_id</b> from the corresponding input request, used for correlation.                                                                                      |
| <b>response</b>  | object         | Contains <b>status_code</b> (integer) and <b>body</b> (object). The <b>body</b> follows the same schema as the corresponding synchronous inference endpoint response. |
| <b>error</b>     | object or null | If the request failed, contains <b>code</b> (string) and <b>message</b> (string). Null for successful requests.                                                       |

### Batch object schema

The Batch object represents a batch inference job and is returned by all **/v1/batches** endpoints.



#### NOTE

This table shows principal Batch object fields. For the complete schema including all timestamp fields, usage statistics, and error details, see the [OpenAI Batch API reference](#).

Table 12.12. Batch object fields

| Field                 | Type            | Description                                                                                          |
|-----------------------|-----------------|------------------------------------------------------------------------------------------------------|
| <b>id</b>             | string          | The unique identifier for the batch.                                                                 |
| <b>object</b>         | string          | Always <b>batch</b> .                                                                                |
| <b>endpoint</b>       | string          | The inference endpoint used for this batch.                                                          |
| <b>input_file_id</b>  | string          | The ID of the input JSONL file.                                                                      |
| <b>output_file_id</b> | string or null  | The ID of the output JSONL file. Available when the batch reaches <b>completed</b> status.           |
| <b>error_file_id</b>  | string or null  | The ID of the error file containing failed requests. Available when the batch completes with errors. |
| <b>status</b>         | string          | The current lifecycle state of the batch.                                                            |
| <b>request_counts</b> | object          | Contains <b>total</b> , <b>completed</b> , and <b>failed</b> counts for requests in the batch.       |
| <b>metadata</b>       | object or null  | Custom key-value pairs set when creating the batch.                                                  |
| <b>created_at</b>     | integer         | Unix timestamp of batch creation.                                                                    |
| <b>in_progress_at</b> | integer or null | Unix timestamp when the batch started processing.                                                    |
| <b>completed_at</b>   | integer or null | Unix timestamp when the batch completed.                                                             |
| <b>failed_at</b>      | integer or null | Unix timestamp when the batch failed.                                                                |
| <b>cancelled_at</b>   | integer or null | Unix timestamp when the batch was cancelled.                                                         |
| <b>expired_at</b>     | integer or null | Unix timestamp when the batch expired.                                                               |

### Batch lifecycle states

A batch job transitions through the following states:

Table 12.13. Batch lifecycle state machine

| State             | Description                                                                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>validating</b> | The input file is being validated. The batch transitions to <b>in_progress</b> if validation succeeds, or to <b>failed</b> if the input file is malformed. |

| State              | Description                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>in_progress</b> | Individual inference requests are being processed by the batch processor. The batch remains in this state until all requests are completed or the batch is cancelled. |
| <b>finalizing</b>  | All requests have been processed. The output file is being assembled and uploaded to storage.                                                                         |
| <b>completed</b>   | The batch finished successfully. The <b>output_file_id</b> is set and results are available for download.                                                             |
| <b>failed</b>      | The batch failed due to validation errors or unrecoverable processing errors. Check the <b>error_file_id</b> for details.                                             |
| <b>expired</b>     | The batch did not complete within the <b>completion_window</b> .                                                                                                      |
| <b>cancelling</b>  | A cancellation request has been received. The processor stops dispatching new requests.                                                                               |
| <b>cancelled</b>   | The batch was cancelled. Requests completed before cancellation are included in the output file.                                                                      |

### Rate limits and scale limits

Table 12.14. Batch inference limits

| Limit                      | Default value | Description                                                                                                     |
|----------------------------|---------------|-----------------------------------------------------------------------------------------------------------------|
| Maximum requests per batch | 50,000        | The default maximum number of inference request lines in a single JSONL input file. This limit is configurable. |
| Maximum input file size    | 200 MB        | The default maximum size of an uploaded JSONL input file. This limit is configurable.                           |

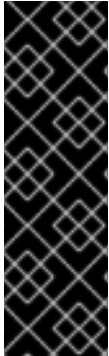
Optionally, you can apply a **RateLimitPolicy** on the external gateway to enforce per-user rate limiting for batch API requests. The rate limit value is defined in the policy configuration.

### Additional resources

- [Monitor batch inference workloads](#)
- [Managing mixed workloads with priority queuing](#)

## CHAPTER 13. AUTOSCALE DISTRIBUTED INFERENCE WITH LLM-D MODEL DEPLOYMENTS

You can configure intelligent, inference-aware automatic scaling for Distributed Inference with llm-d model deployments by using the workload variant autoscaler (WVA). WVA automatically adjusts replica counts and AI accelerator allocation based on real-time traffic signals and hardware capacity, helping you optimize infrastructure usage while maintaining inference performance.



### IMPORTANT

Workload variant autoscaler is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

### 13.1. WORKLOAD VARIANT AUTOSCALER

The workload variant autoscaler (WVA) is a Kubernetes controller that provides intelligent, inference-aware autoscaling for Distributed Inference with llm-d model deployments. You can use the WVA to automatically adjust replica counts and AI accelerator allocation based on real-time traffic signals, hardware capacity, and workload characteristics instead of relying on generic metrics such as CPU or memory utilization.

Traditional Kubernetes autoscaling approaches rely on metrics such as CPU utilization, memory consumption, or queries per second (QPS). These metrics do not accurately represent the resource demands of large language model (LLM) inference workloads because LLM inference is GPU-bound, not CPU-bound. CPU utilization does not reflect the actual load on the inference server.

The WVA addresses these limitations by deriving scaling decisions from inference-specific signals that directly reflect the state of the model server.

The WVA operates as part of the Distributed Inference with llm-d inference stack alongside the following components:

#### vLLM model servers

The inference serving backends that the WVA scales.

#### Inference Gateway

Routes incoming requests to available model server replicas.

#### Inference scheduler

Selects optimal endpoints for each request based on load and locality.

#### KV cache management

Manages key-value cache storage and transfer between replicas. This component is embedded in the inference scheduler and does not run as a separate container.

#### KEDA

Provides the autoscaling actuator that the WVA uses to manage HPA resources. KEDA reads the **wva\_desired\_replicas** metric from Prometheus and manages the HPA resource that scales the target deployment.

## 13.2. KEY METRICS FOR AUTOSCALING DECISIONS

The workload variant autoscaler (WVA) uses the following inference-specific metrics from vLLM model servers to drive scaling decisions:

- **KV cache utilization (`vllm:kv_cache_usage_perc`):** The percentage of key-value cache in use on each replica. The WVA queries the maximum value over a 1-minute window. A replica is considered saturated when its KV cache utilization reaches the configured **kvCacheThreshold**.
- **Queue length (`vllm:num_requests_waiting`):** The number of requests waiting to be processed on each replica. The WVA queries the maximum value over a 1-minute window. A replica is considered saturated when its queue length reaches the configured **queueLengthThreshold**.

The WVA evaluates these metrics across all replicas of a deployment to calculate spare capacity and determine whether to scale up or scale down.

## 13.3. HOW THE WORKLOAD VARIANT AUTOSCALER WORKS

The workload variant autoscaler (WVA) is deployed as a controller that watches your Distributed Inference with llm-d model server deployments and their associated scaling resources, and adjusts replica counts based on inference load. The WVA discovers the deployments to manage from the autoscaling configuration in the **LLMInferenceService** custom resource and emits the target replica count for KEDA or a Horizontal Pod Autoscaler to act on. To deploy the WVA controller, you must enable it in the **DataScienceCluster** custom resource.

The WVA uses a saturation-based spare capacity model to make scaling decisions. It evaluates three main dimensions:

### Token-level demand estimation

By measuring KV cache utilization as a proxy for tokens actively being processed, and the number of requests waiting in queue, the WVA computes token-level demand rather than relying on generic throughput metrics such as CPU or memory utilization.

### Saturation-aware spare capacity

The WVA compares token-level demand against each replica's effective capacity: the minimum of its memory-bound KV cache size limit and compute-bound limit, or batch saturation. Scaling decisions are triggered when spare capacity across replicas falls below a threshold, rather than waiting for replicas to become fully saturated.

### Hardware-aware optimization

At a per-variant level, the WVA accounts for AI accelerator saturation levels. In the **LLMISVC** CRD, different accelerators can be selected for the requirements of each variant, for example:

- Prefill optimized
- Generic inference workers capable of both workloads
- Other configurations that can change the model server's behavior

### 13.3.1. Saturation scaling algorithm

The workload variant autoscaler (WVA) uses a saturation-based spare capacity model to make scaling decisions. The algorithm evaluates whether the current replica set has sufficient spare capacity to absorb traffic increases.

The scaling algorithm works as follows:

1. **Identify non-saturated replicas** A replica is non-saturated if its KV cache utilization is below **kvCacheThreshold** and its queue length is below **queueLengthThreshold**.
2. **Calculate spare capacity**: For each non-saturated replica, the WVA calculates the spare KV capacity and spare queue capacity.
3. **Average spare capacity**: The WVA averages the spare capacity across all non-saturated replicas.
4. **Scale-up decision**: The WVA signals a scale-up if the average spare KV capacity is below **kvSpareTrigger** or the average spare queue capacity is below **queueSpareTrigger**.
5. **Scale-down safety check**: The WVA simulates removing one replica and redistributing load. Scale-down is safe only if remaining spare capacity still exceeds the triggers and at least 2 non-saturated replicas exist.

A variant is eligible for scaling if it does not already have a pending scaling decision from a previous reconciliation cycle. If a variant's desired replica count differs from its current count, the WVA preserves the existing decision and skips that variant when selecting candidates for new scaling actions.

To prevent over-provisioning during the model loading period, the WVA skips variants that have pending replicas when selecting a variant to scale up. A replica is considered pending when the pod exists but is not yet reporting metrics. This cascade scaling prevention ensures that only variants with all replicas ready are eligible for additional scale-up.

### 13.3.2. LLMInferenceService parallelism and WVA scaling

The **LLMInferenceService** custom resource (CR) includes a **spec.parallelism** section that controls how inference workloads are distributed across GPUs and nodes. The workload variant autoscaler (WVA) supports scaling distributed inference workloads.

The WVA monitors KV cache utilization and queue depth across all replicas in a parallel deployment. When spare capacity falls below the configured thresholds, the WVA signals a scale-up by increasing the **Deployment** replica count. Each new replica joins the data-parallel group, increasing the aggregate inference throughput.

For parallel deployments, the WVA accounts for the following factors:

- The accelerator type specified in **spec.labels.inference.optimization/acceleratorName** to apply hardware-specific performance data.
- The per-replica KV cache capacity, which is affected by **dataLocal** and the GPU memory available on each node.
- The communication usage introduced by expert parallelism, which can affect the effective throughput per replica.

The following example shows a minimal **spec.parallelism** configuration for a Mixture of Experts (MoE) model:

#### Example MoE parallelism configuration

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
```



```

metadata:
 name: deepseek-r1
spec:
 model:
 uri: pvc://model-storage
 name: deepseek-ai/DeepSeek-R1-0528
 replicas: 1
 parallelism:
 data: 32
 dataLocal: 8
 expert: true
 tensor: 1
 # ...

```

### spec.parallelism

The **spec.parallelism** fields control the following dimensions of distributed inference:

- **data**: Specifies the total degree of data parallelism across the cluster. Combined with **dataLocal**, this value determines how many pod replicas are created. The replica count is derived from the formula **data** / **dataLocal**. For example, **data: 32** with **dataLocal: 8** produces 4 replicas.
- **dataLocal**: Specifies the degree of data parallelism within a single pod. Each pod handles this many data-parallel shards locally. This value typically matches the number of GPUs per node.
- **expert**: Enables expert parallelism for MoE models such as DeepSeek-R1 or Mixtral. When set to **true**, model experts are distributed across pods by using all-to-all communication backends. Set this to **true** only for MoE model architectures.
- **tensor**: Specifies the degree of tensor parallelism within a pod. Controls how model layers are split across GPUs within a single pod. Set to **1** when using expert parallelism as the primary distribution strategy.

### Replica count and resource requirements

The number of pods created by an **LLMInferenceService** deployment is determined by the parallelism configuration, not the **replicas** field alone. The effective replica count is calculated as **data** / **dataLocal**. Each pod requires the number of GPUs specified by the **dataLocal** value. Ensure that your cluster has sufficient GPU capacity to support the total number of GPUs across all replicas.

### Communication backends

The communication backend determines how pods exchange data during distributed inference. You configure the backend by adding the **--all2all-backend** CLI argument to the container **args** in the **LLMInferenceService** CR template, for example:

```

template:
 containers:
 - name: main
 args:
 # ...
 - --all2all-backend=deepen_high_throughput

```

The following backends are available:

- **deepep\_high\_throughput**: Optimized for batch processing throughput. Requires RDMA-capable networking such as RoCE or InfiniBand. Use this backend for prefill-heavy workloads or high-throughput serving.
- **deepep\_low\_latency**: Optimized for low-latency decode operations. Requires RDMA-capable networking. Use this backend for latency-sensitive decode workloads.
- **allgather\_reducescatter**: The default backend. Uses NVIDIA Collective Communications Library (NCCL) **allgather** and **reducescatter** collective operations.

### Disaggregated serving and parallelism

When using prefill and decode separation, each stage can have its own parallelism settings. The prefill stage and decode stage run as independent deployment groups, each with its own replica count derived from the **data** / **dataLocal** calculation.

In a disaggregated configuration, the WVA can scale each stage independently based on its workload characteristics. For example, the prefill stage might scale based on request queue depth while the decode stage scales based on KV cache utilization.

KV cache transfer between stages is handled by the **NixlConnector**, a vLLM plugin that manages high-speed GPU memory transfers over RDMA-capable networking. The **NixlConnector** generates a compatibility hash for each pod based on the model configuration and vLLM version. Tensor parallelism degree and block size are intentionally excluded from the hash, which allows heterogeneous configurations between prefill and decode pods. Two pods can exchange KV cache data only when their hashes match. Hash mismatches commonly occur during rolling upgrades when new pods run a different vLLM version or configuration than existing pods. During a mixed-version window, KV cache transfers between pods with different hashes fail, causing either client errors or local recomputation depending on the configured **kv\_load\_failure\_policy**.

### DP mode selection for WideEP deployments

When you deploy a WideEP model with expert parallelism, you must also select a data-parallel load balancing mode. The **.spec.parallelism.data** and **.spec.parallelism.dataLocal** fields, combined with the DP mode selection, determine whether the Endpoint Picker can route requests to individual DP ranks for prefix-cache-aware scheduling.

For WideEP deployments that require per-rank routing granularity, use the external multi-port DP mode by adding the **--data-parallel-multi-port-external-lb** flag to the vLLM serve arguments. This mode launches the DP Supervisor to coordinate DP rank lifecycle and aggregate health probes.

### Additional resources

- [DeepSeek-R1 multi-node deployment examples](#)

## 13.4. DISTRIBUTED INFERENCE WITH LLM-D INFERENCE STACK COMPONENTS

You can create an inference stack with an **LLMInferenceService** custom resource (CR). You can create multiple **LLMInferenceService** CRs within a single stack to serve the same model on different accelerator types.

There is a single inference stack per namespace, you cannot deploy multiple inference stacks in the same namespace. An inference stack consists of the following per-namespace resources:

- **HTTPRoute**

- **InferencePool**
- Scheduler deployment that reconciles the **InferencePool**
- Decode workload variant, deployed as a **Deployment** or **LeaderWorkerSet**
- For prefill-decode disaggregation: one prefill workload variant, deployed as a **Deployment** or **LeaderWorkerSet**
- For monitoring: **ServiceMonitor** resources for prefill and decode

The following resources are *not* part of the per-namespace inference stack:

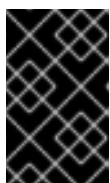
- **Gateway**: Either shared across multiple inference stacks or omitted in a gatewayless deployment
- WVA controller: A single controller instance operates per cluster
- OpenTelemetry Collector: Used only for distributed tracing and not part of the inference stack

## 13.5. ENABLE THE WORKLOAD VARIANT AUTOSCALER FOR DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENTS

You can enable intelligent autoscaling for your Distributed Inference with llm-d model deployments by configuring the workload variant autoscaler (WVA). The WVA controller is automatically deployed when the OpenShift AI Operator is installed. After you configure autoscaling in the **LLMInferenceService** custom resource, the WVA automatically adjusts the replica count of your model server based on real-time inference traffic and AI accelerator capacity.

### Prerequisites

- You have an OpenShift cluster on version **4.20** or later.
- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- You have installed Red Hat OpenShift AI 3.5 with the Distributed Inference with llm-d stack enabled.



### IMPORTANT

Dependencies for installing and using the Distributed Inference with llm-d stack require configuration of the underlying Red Hat OpenShift AI deployment. You cannot use WVA without first configuring OpenShift AI.

- Optional: You have installed **jq**.
- You have installed compatible AI accelerators in the cluster.
- You have enabled OpenShift User Workload Monitoring (UWM) in the cluster for Prometheus metrics collection. See [Configuring user workload monitoring](#).

- You have installed the **custom-metrics-autoscaler** Operator from OperatorHub. See [Automatically scaling pods with the Custom Metrics Autoscaler Operator](#).
- You have installed the **Red Hat Connectivity Link** Operator from OperatorHub. See [Installing Red Hat Connectivity Link](#).
- You have installed the **Red Hat OpenShift Service Mesh 3** Operator in your cluster.



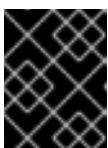
#### NOTE

The Operator is installed by default on any OpenShift cluster version 4.20 or later.

- Optional: If you plan to use **LeaderWorkerSet**, you must install the Red Hat build of LeaderWorkerSet Operator in your cluster and create a **LeaderWorkerSetOperator** custom resource (CR). For more information, see [LeaderWorkerSet Operator for Red Hat OpenShift](#).
- You must have **DataScienceClusterInitialization (DSCI)** and **DataScienceCluster (DSC)** CRDs in your cluster, enabling the **workload-variant-autoscaler-controller-manager**, **llmisvc-controller-manager** and **kserve-controller-manager** controllers. The **DataScienceClusterInitialization** controller is automatically created by the OpenShift AI Operator. This example excerpt from the **DataScienceCluster** manifest enables the WVA controller:

```
apiVersion: datasciencecluster.opendatahub.io/v2
kind: DataScienceCluster
metadata:
 name: default-dsc
 labels:
 app.kubernetes.io/name: datasciencecluster
spec:
 components:
 kserve:
 # Create the KServe and LLMISVC controller managers
 managementState: "Managed"
 nim:
 managementState: "Managed"
 rawDeploymentServiceConfig: "Headed"
 wva:
 # Create the workload-variant-autoscaler controller manager
 managementState: "Managed"
 # ...Other DSC components as desired
```

- You have confirmed that no other **LLMInferenceService** CR exists in the namespace you intend to deploy the WVA in.



#### IMPORTANT

Each namespace should contain a single Distributed Inference with llm-d inference stack only.

### Procedure

1. Verify that the required controllers exist in the cluster. Run the following command:

```
$ oc get pods -n redhat-ods-applications \
 -l 'app.kubernetes.io/name in (kserve-controller-manager, llmsvc-controller-manager,
workload-variant-autoscaler)'
```

All controllers should report as ready:

| NAME                                           | READY | STATUS  | RESTARTS | AGE |
|------------------------------------------------|-------|---------|----------|-----|
| kserve-controller-manager                      | 1/1   | Running | 0        | 38m |
| llmsvc-controller-manager                      | 1/1   | Running | 0        | 38m |
| workload-variant-autoscaler-controller-manager | 1/1   | Running | 0        | 38m |

2. Verify the default scaling configuration.

Run the following command to inspect the **workload-variant-autoscaler-saturation-scaling-config** ConfigMap:

```
$ oc get cm workload-variant-autoscaler-saturation-scaling-config -n redhat-ods-applications
-o jsonpath='{.data.default}'
```

You should see the following default values:

```
kvCacheThreshold: 0.80
queueLengthThreshold: 5
kvSpareTrigger: 0.1
queueSpareTrigger: 3
enableLimiter: false
```

These values represent the default scaling thresholds that the WVA uses to determine when to scale your inference workloads up or down.

3. Grant the KEDA controller permissions to read from OpenShift monitoring resources.



#### NOTE

The Custom Metrics Autoscaler Operator does not automatically have permission to read metrics from the **openshift-monitoring** or **openshift-user-workload-monitoring** stack because OpenShift restricts access to cluster monitoring APIs. Access to Prometheus and Thanos endpoints requires explicit RBAC permissions such as the **cluster-monitoring-view** role.

- a. Authorize KEDA to read OpenShift user workload monitoring (UWM) metrics. Create the following CRs:

```
apiVersion: v1
kind: ServiceAccount
metadata:
 name: keda-metrics-reader
 namespace: openshift-keda

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 name: keda-metrics-reader-monitoring
roleRef:
```

```

 apiGroup: rbac.authorization.k8s.io
 kind: ClusterRole
 name: cluster-monitoring-view
 subjects:
 - kind: ServiceAccount
 name: keda-metrics-reader
 namespace: openshift-keda

 apiVersion: v1
 kind: Secret
 metadata:
 name: keda-metrics-reader-token
 namespace: openshift-keda
 annotations:
 kubernetes.io/service-account.name: keda-metrics-reader
 type: kubernetes.io/service-account-token

```

- b. Allow KEDA to authenticate against OpenShift user workload monitoring. To give KEDA the proper permissions, create a **ClusterTriggerAuthentication** CR, which references the **keda-metrics-reader-token** and **Secret** CRs acting as a Token for the **keda-metrics-reader** and **ServiceAccount** CRs created in the previous step. Create the **ClusterTriggerAuthentication** CR:

```

 apiVersion: keda.sh/v1alpha1
 kind: ClusterTriggerAuthentication
 metadata:
 name: ai-inference-keda-thanos
 spec:
 secretTargetRef:
 - parameter: bearerToken
 name: keda-metrics-reader-token
 key: token
 - parameter: ca
 name: keda-metrics-reader-token
 key: ca.crt

```

KEDA can now pull metrics from the user workload monitoring stack.

4. Create an inference stack with autoscaling enabled:

- a. Create the namespace.

Only one inference stack per namespace is supported. Create a dedicated namespace for the deployment. You can use any namespace, provided you adjust the manifests accordingly.

```
$ oc create ns autoscaling-example && oc project autoscaling-example
```

- b. Create the gateway. You can create the gateway by using Service Mesh with TLS, or by configuring the ingress gateway with HTTP.
  - Optional: Create a gateway that uses TLS with OpenShift Service Mesh.

**NOTE**

OpenShift Service Mesh is installed by default on OpenShift version 4.20 and later.

1. Apply the following **ConfigMap** manifest:

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: autoscaling-example-gateway-config
 namespace: autoscaling-example
data:
 service: |
 metadata:
 annotations:
 service.beta.openshift.io/serving-cert-secret-name: "autoscaling-example-
gateway-tls"
 spec:
 type: ClusterIP
 deployment: |
 spec:
 template:
 spec:
 containers:
 - name: istio-proxy
 resources:
 limits:
 cpu: "16"
 memory: 16Gi
 requests:
 cpu: "4"
 memory: 4Gi
```

**NOTE**

This example uses a **ClusterIP** service type for compatibility across OpenShift environments that do not have **LoadBalancer** service type integration. The istio-proxy resource limits are increased to handle the significant load generation required to trigger a scaling event during verification. Adjust these values according to your expected traffic patterns.

2. Apply the following **Gateway** manifest:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
 name: autoscaling-example-gateway
 namespace: autoscaling-example
spec:
 gatewayClassName: data-science-gateway-class
 infrastructure:
 parametersRef:
```

```

group: ""
kind: ConfigMap
name: autoscaling-example-gateway-config
listeners:
- allowedRoutes:
 namespaces:
 from: Same
 name: https
 port: 443
 protocol: HTTPS
 tls:
 certificateRefs:
 - group: ""
 kind: Secret
 name: autoscaling-example-gateway-tls
 mode: Terminate

```

- Optional: Create a gateway that uses ingress gateway with HTTP.

1. Apply the following **GatewayClass** and **Gateway** manifests:

```

apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
 name: openshift-default
spec:
 controllerName: openshift.io/gateway-controller/v1

apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
 name: openshift-ai-inference
 namespace: openshift-ingress
spec:
 gatewayClassName: openshift-default
 listeners:
 - name: http
 port: 80
 protocol: HTTP
 allowedRoutes:
 namespaces:
 from: Selector
 selector:
 matchLabels:
 kubernetes.io/metadata.name: autoscaling-example

```

2. Set the **gateway** field in the **LLMInferenceService** CR to **gateway: {}** instead of providing explicit gateway references.
- c. Create the **LLMInferenceService** CR with autoscaling configurations enabled.
- The following example creates the **LLMInferenceService** CR with the TLS gateway created by using OpenShift Service Mesh. If you created the gateway by using the ingress gateway with HTTP, set **gateway: {}** in the **spec.router** section instead of providing explicit gateway references.



```

apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
 name: autoscaling-example-qwen
 namespace: autoscaling-example
 annotations:
 prometheus.io/scrape: "true"
 prometheus.io/port: "8000"
 prometheus.io/path: "/metrics"
 security.opendatahub.io/enable-auth: 'false'
spec:
 router:
 scheduler: {}
 route: {}
 gateway:
 refs:
 - name: autoscaling-example-gateway
 namespace: autoscaling-example
 model:
 uri: hf://Qwen/Qwen2.5-7B-Instruct
 name: Qwen/Qwen2.5-7B-Instruct
 labels:
 inference.optimization/acceleratorName: H100
 scaling:
 minReplicas: 1
 maxReplicas: 5
 wva:
 keda:
 pollingInterval: 5
 cooldownPeriod: 30
 template:
 containers:
 - name: main
 image: registry.redhat.io/rhail/vllm-cuda-rhel9:3.5.0
 resources:
 limits:
 cpu: '4'
 memory: 32Gi
 nvidia.com/gpu: 1
 requests:
 cpu: '2'
 memory: 16Gi
 nvidia.com/gpu: 1
 startupProbe:
 httpGet:
 path: /health
 port: 8000
 scheme: HTTPS
 readinessProbe:
 httpGet:
 path: /health
 port: 8000
 scheme: HTTPS
 livenessProbe:
 httpGet:

```

```
path: /health
port: 8000
scheme: HTTPS
```

The following fields are important to understand when you customize the **LLMInferenceService** CR for your use case:

- **.metadata.annotations.security.opendatahub.io/enable-auth:** Disables authentication and rate limiting for this example. In OpenShift AI, user authentication and rate limiting are handled by the Red Hat Connectivity Link Operator. This example disables authentication so that the load test can generate sufficient traffic to trigger a scaling event without being rate-limited. Production deployments must not set this annotation.
- **.spec.router.gateway.refs:** Specifies the reference to the pre-created **Gateway** that the **LLMInferenceService** uses.
- **.spec.labels.inference.optimization/acceleratorName:** Specifies the type of accelerator this workload uses, such as **A100**, **H100**, or **cpu**. The WVA uses this value to group variants by hardware type and look up accelerator-specific performance data when making scaling decisions. There is no fixed list of valid values: use whatever matches your hardware for semantic identification. If omitted, it defaults to **unknown**.
- **.spec.scaling:** Specifies the user-facing scaling configurations at a per-**LLMInferenceService** level, which represents a per-inference-stack level. Do not confuse this with the configurations from the **workload-variant-autoscaler-saturation-scaling-config** ConfigMap, which represents the default scaling configurations that the **workload-variant-autoscaler** uses to determine the threshold at which it scales your inference workloads up or down.
- **.spec.template.containers[0].image:** Specifies the inference image. Different inference images are used for different accelerators. If you are using accelerators, use the corresponding inference image.
- **.spec.template.containers[0].resources:** Specifies the resource requests and limits. Adjust the resources as needed for more GPUs and add the tensor parallel flags to the **VLLM\_ADDITIONAL\_ARGS** environment variable, for example:

```
env:
- name: VLLM_ADDITIONAL_ARGS
 value: "--tensor-parallel 2"
```



#### NOTE

The same requirement applies for accelerated networking devices such as RoCE or InfiniBand.

- **.spec.template.containers[0].startupProbe,**  
**.spec.template.containers[0].readinessProbe,**  
**.spec.template.containers[0].livenessProbe:** Specifies the probe configurations. Because this example uses TLS end to end, ensure that your probes use the HTTPS scheme. If the probes do not use the TLS scheme, they fail to reach the endpoints and declare the pods as not ready or not healthy.

- d. Verify the **LLMInferenceService** and **ScaledObject** CRs were created successfully.

```
$ oc get llmismvc -n autoscaling-example
```

```
NAME URL READY REASON AGE
autoscaling-example-qwen True 102s
```

```
$ oc get scaledobject -n autoscaling-example
```

```
NAME SCALETARGETKIND SCALETARGETNAME
MIN MAX READY ACTIVE FALLBACK PAUSED TRIGGERS
AUTHENTICATIONS AGE
autoscaling-example-qwen-kserve-keda apps/v1.Deployment autoscaling-example-
qwen-kserve 1 5 True True False False prometheus ai-inference-keda-
thanos 119s
```

## Verification

Verify autoscaling behavior by confirming that inference server metrics appear in Prometheus, that the WVA emits scaling metrics, and that an end-to-end autoscaling event occurs.

1. Verify that inference server component metrics appear in Prometheus.  
Ensure that the inputs to the WVA appear in Prometheus, specifically the inference server metrics:

```
$ TOKEN=$(oc whoami -t)
```

```
$ THANOS=$(oc get route thanos-querier -n openshift-monitoring -o jsonpath='{.spec.host}')
for m in llm_d_epp_flow_control_queue_size llm_d_epp_flow_control_queue_size; do
 curl -sk -G -H "Authorization: Bearer $TOKEN" "https://$THANOS/api/v1/query" \
 --data-urlencode "query=${m}{exported_namespace=\"autoscaling-example\"}" \
 | jq '.data.result[0]'
done
```

You should see three separate JSON payloads, one for each metric. The following example shows the expected output format:

```
{
 "metric": {
 "__name__": "vllm:num_requests_running",
 "container": "main",
 "endpoint": "8000",
 "engine": "0",
 "instance": "10.129.0.66:8000",
 "job": "autoscaling-example/kserve-llm-isvc-vllm-engine",
 "llm_isvc_component": "workload",
 "llm_isvc_name": "autoscaling-example-qwen",
 "llm_isvc_role": "both",
 "model_name": "Qwen/Qwen2.5-7B-Instruct",
 "namespace": "autoscaling-example",
 "pod": "autoscaling-example-qwen-kserve-649dc95869-ntlwq",
 "prometheus": "openshift-user-workload-monitoring/user-workload"
 },
}
```

```

"value": [
 1774817937.592,
 "0"
]
}

```



## NOTE

When using the default percentage-based analyzer, the WVA does not use metrics from the scheduler. When you enable the token-based capacity analyzer, the WVA uses EPP scheduler metrics such as **llm\_d\_epp\_flow\_control\_queue\_size** to account for upstream queue demand in scaling decisions.

2. Verify that the WVA is emitting metrics back to Prometheus.  
The metrics to look for are **wva\_current\_replicas**, **wva\_desired\_replicas**, and **wva\_desired\_ratio**. Run the following commands to query these metrics:

```
$ TOKEN=$(oc whoami -t)
```

```

$ THANOS=$(oc get route thanos-querier -n openshift-monitoring -o jsonpath='{.spec.host}')
for m in wva_desired_replicas wva_current_replicas wva_desired_ratio; do
 curl -sk -G -H "Authorization: Bearer $TOKEN" "https://$THANOS/api/v1/query" \
 --data-urlencode "query=${m}{exported_namespace=\"autoscaling-example\"}" \
 | jq '.data.result[0]'
done

```

You should see JSON payloads for each metric. The following example shows the expected output format:

```

{
 "metric": {
 "__name__": "wva_desired_replicas",
 "accelerator_type": "H100",
 "endpoint": "https",
 "exported_namespace": "autoscaling-example",
 "instance": "10.128.0.64:8443",
 "job": "workload-variant-autoscaler-controller-manager-metrics-service",
 "namespace": "redhat-ods-applications",
 "pod": "workload-variant-autoscaler-controller-manager-85b8d895cd-x6gnr",
 "prometheus": "openshift-user-workload-monitoring/user-workload",
 "service": "workload-variant-autoscaler-controller-manager-metrics-service",
 "variant_name": "autoscaling-example-qwen-kserve-va"
 },
 "value": [
 1774819127.020,
 "1"
]
}

```

At this point, verify the **ScaledObject** resource that was created from your **LLMInferenceService**:

```
$ oc get scaledObject -n autoscaling-example
```

| NAME                                 | SCALETARGETKIND    | SCALETARGETNAME                 | MIN | MAX | READY | ACTIVE | FALLBACK | PAUSED | TRIGGERS   | AUTHENTICATIONS          | AGE |
|--------------------------------------|--------------------|---------------------------------|-----|-----|-------|--------|----------|--------|------------|--------------------------|-----|
| autoscaling-example-qwen-kserve-keda | apps/v1.Deployment | autoscaling-example-qwen-kserve | 1   | 5   | True  | True   | Unknown  | False  | prometheus | ai-inference-keda-thanos | 83m |

The **ScaledObject** should show **READY=true**. Once the WVA controller starts reconciling the **ScaledObject**, it should also become **ACTIVE=True**. If you do not see these values, run **oc describe scaledobject <name> -n <namespace>** and check the **Status** section.

3. Verify that the inference worker gets autoscaled.

To make scaling events easier to trigger during testing, set the **kvCacheThreshold** key in the **workload-variant-autoscaler-saturation-scaling-config** ConfigMap in the **redhat-ods-applications** namespace to **0.10**:

```
$ oc edit cm workload-variant-autoscaler-saturation-scaling-config -n redhat-ods-applications
```

Change **kvCacheThreshold** from **0.80** to **0.10**.

The following script deploys a load-generator pod inside the cluster and watches for scaling events. The script resolves the gateway service, sends thousands of concurrent requests with long token generation to saturate the vLLM KV cache past the WVA scaling threshold, and monitors the replica count every 5 seconds:



#### NOTE

The scaling behavior and load values in this script are dependent on the accelerators in use. This example was tested with NVIDIA H100 AI accelerators.

```
#!/bin/bash
set -euo pipefail

NS="{1:-autoscaling-example}"
ISVC="{2:-autoscaling-example-qwen}"
CONCURRENCY="{3:-200}"
REQUESTS="{4:-5000}"
POD_NAME="load-test-$(date +%s)"
CM_NAME="script-${POD_NAME}"

GATEWAY_SVC=$(oc get svc -n "$NS" -l gateway.networking.k8s.io/gateway-name \
-o jsonpath='{.items[0].metadata.name}' 2>/dev/null)
if [-z "$GATEWAY_SVC"]; then
 echo "ERROR: No gateway service found in namespace $NS"
 exit 1
fi
URL="https://${GATEWAY_SVC}.${NS}.svc.cluster.local/${NS}/${ISVC}/v1/chat/completions"

echo "=== WVA Autoscaling Test ==="
echo "URL: $URL"
echo "Concurrency: $CONCURRENCY"
```

```

echo "Requests: $REQUESTS"
echo ""
oc get deployment "${ISVC}-kserve" -n "$NS" \
 -o jsonpath='Initial state: {.spec.replicas}/{.status.readyReplicas} ready' && echo ""
echo ""

cleanup() {
 echo ""
 echo "Cleaning up..."
 oc delete pod "$POD_NAME" -n "$NS" --ignore-not-found --wait=false 2>/dev/null
 oc delete configmap "$CM_NAME" -n "$NS" --ignore-not-found 2>/dev/null
}
trap cleanup EXIT INT TERM

read -r -d " LOAD_SCRIPT << 'EOF' || true
#!/bin/sh
set -e
URL="$1"; CONCURRENCY="$2"; REQUESTS="$3"

cat > /tmp/req.sh << 'REQEOF'
#!/bin/sh
curl -sk --max-time 600 "$1" \
 -H "Content-Type: application/json" \
 -d '{"model":"Qwen/Qwen2.5-7B-Instruct","messages":
[{"role":"user","content":"Request $2. Write a detailed essay about topic $2 covering
history, analysis, and predictions."}], "max_tokens":2048}' \
 -o /dev/null -w "req=$2 status=%{http_code} time=%{time_total}s\n"
REQEOF
chmod +x /tmp/req.sh

echo "Smoke test..."
STATUS=$(curl -sk --max-time 30 "$URL" \
 -H "Content-Type: application/json" \
 -d '{"model":"Qwen/Qwen2.5-7B-Instruct","messages":
[{"role":"user","content":"Hi"}], "max_tokens":5}' \
 -o /dev/null -w "%{http_code}")
echo "Status: $STATUS"
if ["$STATUS" != "200"]; then
 echo "ERROR: Smoke test failed (HTTP $STATUS)"
 exit 1
fi

echo "Sending $REQUESTS requests ($CONCURRENCY concurrent)..."
seq 1 "$REQUESTS" | xargs -P "$CONCURRENCY" -I{} /tmp/req.sh "$URL" {}
echo "Done."
EOF

oc create configmap "$CM_NAME" -n "$NS" --from-literal=load.sh="$LOAD_SCRIPT"

cat <<MANIFEST | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
 name: $POD_NAME
 namespace: $NS
spec:

```

```

restartPolicy: Never
containers:
- name: load
 image: curlimages/curl
 command: ["sh", "/scripts/load.sh"]
 args: ["$URL", "$CONCURRENCY", "$REQUESTS"]
 resources:
 requests: { cpu: "4", memory: "4Gi" }
 limits: { cpu: "8", memory: "8Gi" }
 volumeMounts:
 - { name: script, mountPath: /scripts }
volumes:
- name: script
 configMap: { name: $CM_NAME, defaultMode: 0755 }
MANIFEST

echo "Waiting for pod..."
oc wait --for=condition=Ready pod/"$POD_NAME" -n "$NS" --timeout=120s 2>/dev/null ||
true
sleep 2

POD_PHASE=$(oc get pod "$POD_NAME" -n "$NS" -o jsonpath='{.status.phase}'
2>/dev/null)
if ["$POD_PHASE" = "Failed"]; then
 echo "ERROR: Pod failed:"
 oc logs "$POD_NAME" -n "$NS" 2>/dev/null
 exit 1
fi

oc logs -f "$POD_NAME" -n "$NS" 2>/dev/null &
LOGS_PID=$!

echo ""
echo "--- Watching replicas (Ctrl+C to stop) ---"
for _ in $(seq 1 120); do
 replicas=$(oc get deployment "${ISVC}-kserve" -n "$NS" -o jsonpath='{.spec.replicas}'
2>/dev/null)
 ready=$(oc get deployment "${ISVC}-kserve" -n "$NS" -o jsonpath='{.status.readyReplicas}'
2>/dev/null)
 echo " [$(date +%H:%M:%S)] Replicas: ${replicas:-?} (${ready:-0} ready)"
 ["${replicas:-1}" -gt 1] && echo " ** Scale-up detected! **"
 oc get pod "$POD_NAME" -n "$NS" -o jsonpath='{.status.phase}' 2>/dev/null \
 | grep -q "Succeeded\|Failed" && echo " Load pod finished." && break
 sleep 5
done

kill "$LOGS_PID" 2>/dev/null || true
wait "$LOGS_PID" 2>/dev/null || true

```

You should see the replica count increase as load is applied:

```

[17:30:03] Replicas: 1 (1 ready)
[17:30:58] Replicas: 2 (1 ready)
** Scale-up detected! **

```

During the load, you should see additional inference worker pods:

```
$ oc get pods -n autoscaling-example
```

| NAME                                                            | READY | STATUS  | RESTARTS | AGE  |
|-----------------------------------------------------------------|-------|---------|----------|------|
| autoscaling-example-gateway-data-science-gateway-class-66c4qvpb | 1/1   | Running | 0        | 81m  |
| autoscaling-example-qwen-kserve-56b64d69d-b297x                 | 1/1   | Running | 0        | 106m |
| autoscaling-example-qwen-kserve-56b64d69d-kvvmc                 | 1/1   | Running | 0        | 10m  |
| autoscaling-example-qwen-kserve-56b64d69d-m78nz                 | 1/1   | Running | 0        | 7m1s |
| autoscaling-example-qwen-kserve-router-scheduler-77cd5549p4w95  | 2/2   | Running | 0        | 106m |

After the load ceases, the deployment scales back down to 1 replica:

| NAME                                                            | READY | STATUS  | RESTARTS | AGE  |
|-----------------------------------------------------------------|-------|---------|----------|------|
| autoscaling-example-gateway-data-science-gateway-class-66c4qvpb | 1/1   | Running | 0        | 81m  |
| autoscaling-example-qwen-kserve-56b64d69d-b297x                 | 1/1   | Running | 0        | 106m |
| autoscaling-example-qwen-kserve-router-scheduler-77cd5549p4w95  | 2/2   | Running | 0        | 106m |



#### NOTE

Scaling behavior depends on the analyzer in use. The default percentage-based analyzer scales up one replica at a time, imposing a stabilization window on scaling events so that a traffic burst does not greedily attempt to occupy all your GPUs. The token-based capacity analyzer scales up immediately and can add multiple replicas at once to meet demand.

## 13.6. REFERENCE WVA CONFIGURATION FOR A MULTI-NODE MODEL DEPLOYMENT

The following reference **LLMInferenceService** custom resource (CR) describes a solution for deploying and autoscaling a DeepSeek-Coder-V2-Lite MoE model with expert parallelism, LeaderWorkerSet, and the workload variant autoscaler (WVA) across multiple NVIDIA GPU nodes on Google Cloud Platform (GCP).



#### NOTE

The NVIDIA Collective Communications Library (NCCL) environment variables in this configuration are tuned for environments that use TCP-based networking and do not have Remote Direct Memory Access (RDMA) available. If your cluster nodes use RDMA-capable networking such as InfiniBand or RoCE, you must adjust the NCCL settings accordingly.

### Reference DeepSeek-Coder-V2-Lite model deployment with workload variant autoscaler on GCP

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
```



```

metadata:
 name: deepseek-coder-v2
 namespace: autoscaling-example
 annotations:
 prometheus.io/scrape: "true"
 prometheus.io/port: "8000"
 prometheus.io/path: "/metrics"
 security.opendatahub.io/enable-auth: 'false'
spec:
 model:
 # Alternatively, use pvc:// URI for pre-initialized PVC storage for the model
 uri: hf://deepseek-ai/DeepSeek-Coder-V2-Lite-Instruct
 name: deepseek-ai/DeepSeek-Coder-V2-Lite-Instruct
 labels:
 inference.optimization/acceleratorName: H100 # Match AI accelerator profile name
 parallelism:
 data: 2
 dataLocal: 1
 expert: true
 tensor: 1
 router:
 scheduler: { }
 route: { }
 gateway:
 refs:
 - name: autoscaling-example-gateway
 namespace: autoscaling-example
 scaling:
 minReplicas: 1
 maxReplicas: 5
 wva:
 keda:
 pollingInterval: 5
 cooldownPeriod: 30
 template:
 containers:
 - name: main
 env:
 - name: VLLM_API_SERVER_COUNT
 value: "1"
 - name: VLLM_LOGGING_LEVEL
 value: INFO
 # GCP gVNIC: Optimized for TCP-based high-performance networking
 # No RDMA devices available, using gVNIC with NCCL TCP optimization
 - name: CUDA_DEVICE_ORDER
 value: "PCI_BUS_ID"
 # vLLM memory and performance settings
 - name: VLLM_ADDITIONAL_ARGS
 value: "--gpu-memory-utilization 0.95 --max-model-len 8192 --enforce-eager"
 - name: VLLM_ALL2ALL_BACKEND
 value: naive
 - name: PYTORCH_CUDA_ALLOC_CONF
 value: "expandable_segments:True"
 # NCCL settings for TCP over gVNIC
 - name: NCCL_IB_DISABLE
 value: "1" # No InfiniBand available

```

```

- name: NCCL_NET_GDR_LEVEL
 value: "0" # No GPUDirect RDMA
- name: NCCL_P2P_LEVEL
 value: "NVL" # NVLink for intra-node P2P
- name: NCCL_SOCKET_IFNAME
 value: eth0
- name: NCCL_NSOCKS_PERTHREAD
 value: "2" # Lower count reduces memory pressure
- name: NCCL_SOCKET_NTHREADS
 value: "2"
- name: NCCL_BUFFSIZE
 value: "2097152" # 2 MB
- name: NCCL_DEBUG
 value: WARN
NVSHMEM over UCX/TCP
- name: NVSHMEM_REMOTE_TRANSPORT
 value: "ucx"
- name: NVSHMEM_DISABLE_CUDA_VMM
 value: "0"
- name: NVSHMEM_BOOTSTRAP_TWO_STAGE
 value: "1"
- name: NVSHMEM_BOOTSTRAP_TIMEOUT
 value: "300"
- name: NVSHMEM_BOOTSTRAP_UID_SOCK_IFNAME
 value: eth0
- name: NVSHMEM_DEBUG
 value: INFO
UCX transport layers
- name: UCX_TLS
 value: "tcp,sm,self,cuda_copy,cuda_ipc"
- name: UCX_NET_DEVICES
 value: eth0
Intra-node GPU-to-GPU transfers
- name: NVIDIA_GDRCOPY
 value: enabled
resources:
 limits:
 cpu: 128
 ephemeral-storage: 100Gi
 memory: 512Gi
 nvidia.com/gpu: "2"
 # No rdma/roce_gdr needed for TCP over gVNIC
 requests:
 cpu: 64
 ephemeral-storage: 100Gi
 memory: 256Gi
 nvidia.com/gpu: "2"
livenessProbe:
 httpGet:
 path: /health
 port: 8000
 scheme: HTTPS
 initialDelaySeconds: 4800
 periodSeconds: 10
 timeoutSeconds: 10
 failureThreshold: 3

```

```

worker:
containers:
- name: main
env:
- name: VLLM_API_SERVER_COUNT
 value: "1"
- name: VLLM_LOGGING_LEVEL
 value: INFO
- name: CUDA_DEVICE_ORDER
 value: "PCI_BUS_ID"
vLLM memory and performance settings
- name: VLLM_ADDITIONAL_ARGS
 value: "--gpu-memory-utilization 0.95 --max-model-len 8192 --enforce-eager"
- name: VLLM_ALL2ALL_BACKEND
 value: naive
- name: PYTORCH_CUDA_ALLOC_CONF
 value: "expandable_segments:True"
NCCL settings for TCP over gVNIC
- name: NCCL_IB_DISABLE
 value: "1" # No InfiniBand available
- name: NCCL_NET_GDR_LEVEL
 value: "0" # No GPUDirect RDMA
- name: NCCL_P2P_LEVEL
 value: "NVL" # NVLink for intra-node P2P
- name: NCCL_SOCKET_IFNAME
 value: eth0
- name: NCCL_NSOCKS_PERTHREAD
 value: "2" # Reduce sockets to avoid allocation errors
- name: NCCL_SOCKET_NTHREADS
 value: "2"
- name: NCCL_BUFFSIZE
 value: "2097152" # 2MB buffer to reduce memory usage
- name: NCCL_DEBUG
 value: WARN
NVSHMEM over UCX/TCP
- name: NVSHMEM_REMOTE_TRANSPORT
 value: "ucx"
- name: NVSHMEM_DISABLE_CUDA_VMM
 value: "0"
- name: NVSHMEM_BOOTSTRAP_TWO_STAGE
 value: "1"
- name: NVSHMEM_BOOTSTRAP_TIMEOUT
 value: "300"
- name: NVSHMEM_BOOTSTRAP_UID_SOCK_IFNAME
 value: eth0
- name: NVSHMEM_DEBUG
 value: INFO
UCX transport layers
- name: UCX_TLS
 value: "tcp,sm,self,cuda_copy,cuda_ipc"
- name: UCX_NET_DEVICES
 value: eth0
Intra-node GPU-to-GPU transfers
- name: NVIDIA_GDRCOPY
 value: enabled
resources:

```

```

limits:
 cpu: 128
 ephemeral-storage: 100Gi
 memory: 512Gi
 nvidia.com/gpu: "2"
 # No rdma/roce_gdr needed for TCP over gVNIC
requests:
 cpu: 64
 ephemeral-storage: 100Gi
 memory: 256Gi
 nvidia.com/gpu: "2"

```

## 13.7. SATURATION SCALING CONFIGMAP REFERENCE

The workload variant autoscaler (WVA) uses a **ConfigMap** named **workload-variant-autoscaler-saturation-scaling-config** for saturation-based scaling thresholds.

Each **ConfigMap** contains two types of entries:

- A **default** entry that contains global parameters that apply to all models in the scope (cluster-wide or namespace-scoped)
- Optional override entries that contain model-specific customizations with arbitrary names, such as **granite-override** or **llama-override**

The following example shows a **ConfigMap** CR with cluster-level defaults and a model-specific override:

The **default** entry contains global parameters that apply to all models unless a matching override exists.

```

apiVersion: v1
kind: ConfigMap
metadata:
 name: workload-variant-autoscaler-saturation-scaling-config
 namespace: redhat-ods-applications
data:
 default: |
 kvCacheThreshold: <kv_cache_threshold>
 queueLengthThreshold: <queue_length_threshold>
 kvSpareTrigger: <kv_spare_trigger>
 queueSpareTrigger: <queue_spare_trigger>
 enableLimiter: <enable_limiter>
 analyzers: <analyzers>
 <override_name>: |
 model_id: "<model_id>"
 namespace: "<namespace>"
 kvCacheThreshold: <kv_cache_threshold>
 kvSpareTrigger: <kv_spare_trigger>

```

- **kvCacheThreshold**: Specifies the KV cache utilization threshold as a percentage. A replica is considered saturated if its KV cache utilization is greater than or equal to this value. Valid range: **0.0** to **1.0**. Lower values trigger scale-up sooner. Default: **0.80**.

- **queueLengthThreshold**: Specifies the queue length threshold. A replica is considered saturated if its queue length is greater than or equal to this value. Lower values trigger scale-up sooner. Default: **5**.
- **kvSpareTrigger**: Specifies the KV cache spare capacity trigger. The WVA signals a scale-up if the average spare KV capacity across non-saturated replicas falls below this value. Valid range: **0.0** to **1.0**. Default: **0.10**.
- **queueSpareTrigger**: Specifies the queue spare capacity trigger. The WVA signals a scale-up if the average spare queue capacity across non-saturated replicas falls below this value. Default: **3**.
- **enableLimiter**: Enables the GPU limiter to constrain scaling based on available cluster GPU capacity. When **true**, the WVA limits desired replicas so that deployments do not request more GPUs than are available in the cluster, preventing new replicas from entering a pending state. Default: **false**.
- **analyzers**: Specifies an optional list of analyzer configurations. To enable the token-based capacity analyzer, set the value to - **name: saturation**. The token-based analyzer measures demand and supply in KV cache tokens instead of percentages, providing more granular scaling decisions. When this field is omitted, the WVA uses the default percentage-based analyzer.
- **<override\_name>**: Override entry name, such as **llm-d** or **granite-override**. The WVA matches overrides by using the **model\_id** and **namespace** fields, not the entry name.
- **model\_id**: Specifies the model identifier that this override applies to, such as **Qwen/Qwen3-0.6B** or **ibm/granite-13b**. The WVA uses this value together with **namespace** to match the override to a specific model deployment.
- **namespace**: Specifies the namespace of the model deployment that this override applies to. The WVA uses this value together with **model\_id** to match the override to a specific model deployment.



#### NOTE

The WVA resolves configuration by first looking for an override that matches both **model\_id** and **namespace**. If no matching override is found, the WVA uses the **default** entry. Any parameters that are not specified in the matched entry receive hardcoded default values.

## 13.8. ENABLE THE TOKEN-BASED CAPACITY ANALYZER

By default, the workload variant autoscaler (WVA) uses a percentage-based capacity analyzer that measures KV cache utilization as a fraction of total capacity. You can enable an alternative token-based capacity analyzer that measures demand and supply in KV cache tokens. The token-based analyzer provides more granular scaling decisions by computing total token-level demand and supply from multiple sources: requests waiting in Endpoint Picker (EPP) flow control, requests waiting in the replicas, and requests being served in the replicas.

### Prerequisites

- You have enabled the WVA for your Distributed Inference with llm-d deployment.
- You have verified that the WVA controller is running and emitting metrics.

## Procedure

1. Edit the **workload-variant-autoscaler-saturation-scaling-config** ConfigMap to enable the token-based analyzer:

```
$ oc edit cm workload-variant-autoscaler-saturation-scaling-config -n redhat-ods-applications
```

2. Append the **analyzers** section to the **default** entry in the ConfigMap data:

```
data:
 default: |
 # ...
 analyzers:
 - name: saturation
```



### NOTE

You do not need to restart the WVA controller manager pod. The WVA controller automatically detects and applies the updated ConfigMap.

## Verification

- Search the WVA controller manager logs for **V2** to confirm that the token-based analyzer is active:

```
$ oc logs -n redhat-ods-applications -l app.kubernetes.io/name=workload-variant-autoscaler | grep V2
```

You should see log entries similar to the following:

```
2026-07-06T14:00:55Z INFO saturation/engine.go:816 Processing model (V2)
{"modelID": "unsloth/Meta-Llama-3.1-8B", "namespace": "autoscaling-example",
"variantCount": 1, "groupKey": "unsloth/Meta-Llama-3.1-8B|autoscaling-example"}
2026-07-06T14:00:55Z INFO saturation/engine_v2.go:67 V2 saturation analysis
completed {"modelID": "unsloth/Meta-Llama-3.1-8B", "totalSupply": 13107, "totalDemand":
0, "utilization": 0, "requiredCapacity": 0, "spareCapacity": 0}
```

The **V2 saturation analysis completed** entry confirms that the token-based analyzer is processing your model, with **totalSupply** and **totalDemand** measured in KV cache tokens.

## 13.9. AUTOSCALING METRICS

The workload variant autoscaler (WVA) exposes the following autoscaling-specific metrics through Prometheus. These metrics reflect the scaling decisions and state of the WVA controller.

Table 13.1. WVA autoscaling metrics

| Metric name                 | Type  | Labels                                                               | Description                                                         |
|-----------------------------|-------|----------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>wva_current_replicas</b> | gauge | <b>variant_name,</b><br><b>namespace,</b><br><b>accelerator_type</b> | The current number of replicas managed by the WVA for each variant. |

| Metric name                              | Type      | Labels                                                                                     | Description                                                                                                                                                                                                                                                     |
|------------------------------------------|-----------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>wva_desired_replicas</b>              | gauge     | <b>variant_name,</b><br><b>namespace,</b><br><b>accelerator_type</b>                       | The target number of replicas calculated by the WVA. This is the metric read by KEDA or HPA for actuation.                                                                                                                                                      |
| <b>wva_replica_scaling_total</b>         | counter   | <b>variant_name,</b><br><b>namespace,</b><br><b>direction,</b> <b>reason</b>               | The total number of replica scaling events triggered by the WVA. The <b>direction</b> label is <b>up</b> or <b>down</b> . The <b>reason</b> label indicates the cause of the scaling event.                                                                     |
| <b>wva_desired_ratio</b>                 | gauge     | <b>variant_name,</b><br><b>namespace,</b><br><b>accelerator_type</b>                       | The ratio of required replicas to current replicas, indicating the scaling pressure on the deployment.                                                                                                                                                          |
| <b>wva_models_processed</b>              | gauge     | <b>namespace</b>                                                                           | The number of models processed in the last optimization cycle by the WVA controller.                                                                                                                                                                            |
| <b>wva_optimization_duration_seconds</b> | histogram | <b>namespace,</b> <b>status</b>                                                            | The duration in seconds that the WVA controller spent in the last optimization cycle. The <b>status</b> label is <b>success</b> if the cycle completed without error, or <b>error</b> if it failed.                                                             |
| <b>wva_saturation_utilization</b>        | gauge     | <b>variant_name,</b><br><b>namespace,</b><br><b>model_name,</b><br><b>accelerator_type</b> | The per-variant utilization ratio from saturation analysis, as a value of <b>0.0</b> to <b>1.0</b> . The percentage-based analyzer uses the mean of per-replica KV cache usage fractions. The token-based analyzer uses total demand divided by total capacity. |
| <b>wva_spare_capacity</b>                | gauge     | <b>variant_name,</b><br><b>namespace,</b><br><b>model_name,</b><br><b>accelerator_type</b> | The per-variant spare KV cache capacity, as a value of <b>0.0</b> to <b>1.0</b> . The percentage-based analyzer uses threshold-relative spare capacity. The token-based analyzer uses <b>1.0</b> minus utilization.                                             |
| <b>wva_required_capacity</b>             | gauge     | <b>variant_name,</b><br><b>namespace,</b><br><b>model_name,</b> <b>unit</b>                | The model-level required capacity. A value greater than <b>0</b> indicates that a scale-up is needed. The <b>unit</b> label differentiates between <b>binary</b> for the percentage-based analyzer and <b>continuous</b> for the token-based analyzer.          |

| Metric name                     | Type    | Labels                                                                           | Description                                                                                                                                                                    |
|---------------------------------|---------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>wva_kv_cache_tokens_used</b> | gauge   | <b>variant_name,</b><br><b>namespace,</b><br><b>model_name</b>                   | The total KV cache tokens currently in use across all replicas of a variant.                                                                                                   |
| <b>wva_errors_total</b>         | counter | <b>component,</b><br><b>error_type</b>                                           | The total number of errors by component. Components include <b>collector</b> , <b>analyzer</b> , <b>optimizer</b> , <b>limiter</b> , <b>enforcer</b> , and <b>controller</b> . |
| <b>wva_config_info</b>          | gauge   | <b>analyzer_name,</b><br><b>limiter_enabled,</b><br><b>scale_to_zero_enabled</b> | WVA configuration information. The value is always <b>1</b> . Use this metric to verify the active WVA configuration.                                                          |

For a complete list of WVA metrics, see [WVA metrics](#) in the upstream project documentation.

The WVA uses the following vLLM inference metrics as input signals for scaling decisions. These metrics are exposed by the vLLM model server, not by the WVA controller.

**Table 13.2. vLLM metrics used by the percentage-based analyzer**

| Metric name                      | Description                                                                                                                                                   |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>vllm:kv_cache_usage_perc</b>  | KV cache utilization as a percentage. A value of <b>0.0</b> to <b>1.0</b> . The WVA uses <b>max_over_time</b> over a 1-minute window to determine saturation. |
| <b>vllm:num_requests_waiting</b> | Number of requests waiting to be processed. The WVA uses <b>max_over_time</b> over a 1-minute window to determine queue depth.                                |

The token-based capacity analyzer uses the following additional vLLM and EPP metrics.

**Table 13.3. Additional metrics used by the token-based capacity analyzer**

| Metric name                       | Description                                                                                                                                                                                       |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>vllm:cache_config_info</b>     | Cache information including <b>num_gpu_blocks</b> and <b>block_size</b> details. The analyzer reads these values to calculate the total KV token capacity per replica.                            |
| <b>vllm:request_prompt_tokens</b> | Number of prefill tokens processed, exposed as a histogram. The analyzer uses the <b>_sum</b> and <b>_count</b> suffixes to calculate the average input tokens per request for demand estimation. |



| Metric name                               | Description                                                                                                                                                                                            |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>vllm:prefix_cache_hits</b>             | Prefix cache hits, in terms of number of cached tokens. Used with <b>vllm:prefix_cache_queries</b> to calculate the prefix cache hit rate, which discounts demand estimation for cached prompt tokens. |
| <b>vllm:prefix_cache_queries</b>          | Prefix cache queries, in terms of number of queried tokens. Used with <b>vllm:prefix_cache_hits</b> to calculate the prefix cache hit rate.                                                            |
| <b>llm_d_epp_flow_control_queue_size</b>  | The current number of requests being actively managed by the EPP flow-control layer. The analyzer uses this metric to account for demand from requests that have not yet reached any pod.              |
| <b>llm_d_epp_flow_control_queue_bytes</b> | Total bytes queued in the EPP flow-control layer. Used as an alternative to <b>llm_d_epp_flow_control_queue_size</b> when queue size is derived from byte-level measurements.                          |



#### NOTE

The vLLM metrics must include **pod**, **model\_name**, and **namespace** labels for the WVA to correlate metrics with the correct variant. The EPP metrics must include **namespace** labels.

## 13.10. INSTALL WVA PROMETHEUS ALERT RULES

The workload variant autoscaler (WVA) provides a set of Prometheus alert rules that notify you when the controller encounters conditions such as GPU exhaustion, elevated error rates, or scaling thrashing. Install the rules by applying the **controller-manager-alerts PrometheusRule** resource, then verify that the **wva.rules** group is active in your monitoring stack.

### Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- You have **jq** installed for parsing JSON responses.
- OpenShift User Workload Monitoring is enabled in the cluster.

### Procedure

1. Install the WVA Prometheus alert rules by applying the **controller-manager-alerts PrometheusRule** resource:

```
$ cat <<'EOF' | oc apply -n openshift-monitoring -f -
apiVersion: monitoring.coreos.com/v1
```

```

kind: PrometheusRule
metadata:
 name: controller-manager-alerts
 labels:
 control-plane: controller-manager
 app.kubernetes.io/name: workload-variant-autoscaler
 release: kube-prometheus-stack
spec:
 groups:
 - name: wva.rules
 rules:
 - alert: WVAHighErrorRate
 expr: sum by (component, error_type) (rate(wva_errors_total[5m])) > 0.1
 for: 5m
 labels:
 severity: warning
 annotations:
 summary: "WVA error rate elevated in {{ $labels.component }}"
 description: "WVA component '{{ $labels.component }}' error_type '{{ $labels.error_type }}' rate is {{ $value | printf \"%.2f\" }}/sec (>6/min threshold) sustained for 5+ minutes. Check controller logs for error patterns."
 - alert: WVAOptimizationLoopStalled
 expr: max_over_time(wva_models_processed[10m]) == 0 or
absent_over_time(wva_models_processed[10m])
 for: 15m
 labels:
 severity: info
 annotations:
 summary: "WVA has processed zero models for 15+ minutes"
 description: "wva_models_processed has been 0 (or absent) over a 10-minute window, sustained for 15+ minutes. This is EXPECTED when no VariantAutoscaling resources are being managed (an idle controller). If you do have managed workloads, it can indicate the optimization loop is stalled/crashed or metric collection is failing — check controller pod status and logs, and Prometheus connectivity."
 - alert: WVAMetricsCollectionFailing
 expr: sum by (query_type, reason) (rate(wva_metrics_collection_errors_total[5m])) >
0.0833
 for: 5m
 labels:
 severity: warning
 annotations:
 summary: "WVA metrics collection failing for {{ $labels.query_type }}"
 description: "Metrics collection query_type '{{ $labels.query_type }}' failing with reason '{{ $labels.reason }}' at {{ $value | printf \"%.2f\" }}/sec (>5/min threshold) sustained for 5+ minutes. Scaling decisions may be based on stale data. Verify Prometheus/metrics endpoint connectivity."
 - alert: WVAGPUResourceExhausted
 expr: wva_available_gpus == 0
 for: 5m
 labels:
 severity: warning
 annotations:
 summary: "GPU-labeled {{ $labels.accelerator_type }} nodes report zero allocatable accelerators"
 description: "WVA sees zero allocatable {{ $labels.accelerator_vendor }} {{ $labels.accelerator_model }} ({{ $labels.accelerator_type }}) GPUs for 5+ minutes. Note: this

```

reflects node `*allocatable*` capacity (not free/schedulable GPUs), and `wva_available_gpus` is only emitted when the GPU limiter is enabled (`enableLimiter=true`). A GPU-labeled node advertising zero allocatable accelerators usually means it is `NotReady` or its device plugin/driver is not installed — check GPU node status and device-plugin pods."

```
- alert: WVAReplicaScalingThrashing
 expr: sum by (namespace, variant_name) (rate(wva_replica_scaling_total[10m])) > 0.033
 for: 10m
 labels:
 severity: warning
 annotations:
 summary: "WVA scaling thrashing detected for {{ $labels.namespace }}/{{ $labels.variant_name }}"
 description: "Variant {{ $labels.namespace }}/{{ $labels.variant_name }} scaling at {{ $value | printf \"%.3f\" }}/sec (>2/min threshold) sustained for 10+ minutes, indicating possible thrashing. Review scaling thresholds and consider adjusting stabilization window or thresholds."
 EOF
```

2. Verify that the **controller-manager-alerts PrometheusRule** resource was created:

```
$ oc get prometheusrules controller-manager-alerts -n openshift-monitoring
```

### Example output

```
NAME AGE
controller-manager-alerts 65m
```

3. Verify that the **wva.rules** group is loaded by querying the Thanos rules endpoint:

```
$ TOKEN=$(oc whoami -t)
```

```
$ THANOS=$(oc get route thanos-querier -n openshift-monitoring -o jsonpath='{.spec.host}')
curl -sk -G -H "Authorization: Bearer $TOKEN" "https://$THANOS/api/v1/rules" | jq
'.data.groups[] | select(.name == "wva.rules")'
```

The following shortened output shows the **WVAGPUResourceExhausted** alert, which fires when **wva\_available\_gpus** equals **0** for five minutes or more:

```
{
 "name": "wva.rules",
 "file": "/etc/prometheus/rules/prometheus-k8s-rulefiles-0/openshift-monitoring-controller-manager-alerts.yaml",
 "rules": [
 {
 "state": "inactive",
 "name": "WVAGPUResourceExhausted",
 "query": "wva_available_gpus == 0",
 "duration": 300,
 "labels": {
 "prometheus": "openshift-monitoring/k8s",
 "severity": "warning"
 },
 "annotations": {
 "summary": "GPU-labeled nodes report zero allocatable accelerators",

```

```

 "description": "WVA sees zero allocatable GPUs for 5+ minutes."
 },
 "health": "ok",
 "type": "alerting"
}
]
}

```

The **wva.rules** group contains the following alerts.

**Table 13.4. WVA Prometheus alert rules**

| Alert name                         | Severity | Condition                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>WVAGPUResourceExhausted</b>     | warning  | GPU-labeled nodes report zero allocatable AI accelerators for 5+ minutes. This metric reflects node allocatable capacity, not free or schedulable GPUs, and is only emitted when the GPU limiter is enabled with <b>enableLimiter=true</b> . A GPU-labeled node advertising zero allocatable AI accelerators usually means the node is <b>NotReady</b> or its device plugin or driver is not installed. |
| <b>WVAHighErrorRate</b>            | warning  | A WVA component error rate exceeds six errors per minute, sustained for 5+ minutes. Check the controller logs for error patterns.                                                                                                                                                                                                                                                                       |
| <b>WVAMetricsCollectionFailing</b> | warning  | Metrics collection queries are failing at more than five per minute, sustained for 5+ minutes. Scaling decisions might be based on stale data. Verify Prometheus and metrics endpoint connectivity.                                                                                                                                                                                                     |
| <b>WVAReplicaScalingThrashing</b>  | warning  | A variant is scaling more than twice per minute, sustained for 10+ minutes, which indicates possible thrashing. Review the scaling thresholds and consider adjusting the stabilization window.                                                                                                                                                                                                          |
| <b>WVAOptimizationLoopStalled</b>  | info     | The controller has processed zero models for 15+ minutes. This is expected when no variants are managed. If you have managed workloads, it can indicate that the optimization loop is stalled or that metric collection is failing.                                                                                                                                                                     |

## 13.11. HPA BEHAVIOR DEFAULTS

The workload variant autoscaler (WVA) uses KEDA to manage HPA resources. KEDA configures the following default HPA scaling behavior. These values control the stabilization window and scaling velocity.

**Table 13.5. HPA scaling behavior defaults**

| Parameter                                   | Default     | Description                                                                                                                                |
|---------------------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>scaleUp.stabilizationWindowSeconds</b>   | <b>240</b>  | The stabilization window for scale-up decisions, in seconds. The HPA waits this long after the last scaling event before scaling up again. |
| <b>scaleDown.stabilizationWindowSeconds</b> | <b>240</b>  | The stabilization window for scale-down decisions, in seconds.                                                                             |
| <b>selectPolicy</b>                         | <b>Max</b>  | The policy selection strategy. <b>Max</b> selects the policy that results in the largest change.                                           |
| <b>policies[0].type</b>                     | <b>Pods</b> | The scaling policy type.                                                                                                                   |
| <b>policies[0].value</b>                    | <b>10</b>   | The maximum number of pods to add or remove per scaling period.                                                                            |
| <b>policies[0].periodSeconds</b>            | <b>150</b>  | The duration of the scaling period, in seconds.                                                                                            |

## 13.12. UPGRADE THE WVA CONFIGMAP FROM VERSION 3.4 TO 3.5

When you upgrade from Red Hat OpenShift AI 3.4 to 3.5, the workload variant autoscaler (WVA) renames one of its **ConfigMap** resources. If you customized any WVA **ConfigMap** values in your 3.4 deployment, you must manually reapply those customizations after the upgrade. The upgrade process does not migrate custom **ConfigMap** values automatically.



### NOTE

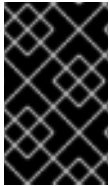
In OpenShift AI 3.4, the following WVA **ConfigMap** resources exist in the **redhat-ods-applications** namespace:

- **workload-variant-autoscaler-saturation-scaling-config**
- **workload-variant-autoscaler-wva-variantautoscaling-config**

After the upgrade to 3.5, the **workload-variant-autoscaler-wva-variantautoscaling-config** resource is renamed to **workload-variant-autoscaler-manager-config**. The **workload-variant-autoscaler-saturation-scaling-config** resource name is unchanged.

### Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- You have customized WVA **ConfigMap** values in your 3.4 deployment.



## IMPORTANT

You must record your customized **ConfigMap** values before upgrading. After the upgrade, the **workload-variant-autoscaler-wva-variantautoscaling-config** ConfigMap is renamed and your previous values are not preserved.

- Before the 3.4 to 3.5 upgrade, record your customized **ConfigMap** values by running the following commands:

```
$ oc get configmap workload-variant-autoscaler-saturation-scaling-config -n redhat-ods-applications -o yaml
```

```
$ oc get configmap workload-variant-autoscaler-wva-variantautoscaling-config -n redhat-ods-applications -o yaml
```

Save the output for reference.

## Procedure

- Upgrade OpenShift AI from version 3.4 to 3.5.
- After the upgrade, reapply your customizations to the **workload-variant-autoscaler-saturation-scaling-config** ConfigMap:

```
$ oc edit configmap workload-variant-autoscaler-saturation-scaling-config -n redhat-ods-applications
```

- Reapply your customizations to the renamed **workload-variant-autoscaler-manager-config** ConfigMap:

```
$ oc edit configmap workload-variant-autoscaler-manager-config -n redhat-ods-applications
```

## Verification

- Verify that your customizations are applied to the updated ConfigMap resources:

```
$ oc get configmap workload-variant-autoscaler-saturation-scaling-config -n redhat-ods-applications -o yaml
```

```
$ oc get configmap workload-variant-autoscaler-manager-config -n redhat-ods-applications -o yaml
```

## 13.13. TROUBLESHOOT WORKLOAD VARIANT AUTOSCALER

If your workload variant autoscaler (WVA) deployment is not scaling as expected, use the following troubleshooting steps to diagnose and resolve common issues.

- Check the WVA controller status:  
View WVA controller logs to diagnose issues:

```
$ oc logs -l app.kubernetes.io/name=workload-variant-autoscaler -n redhat-ods-applications
```

- Verify that KEDA resources are functioning correctly:  
Check the status of the KEDA **ScaledObject** resources in your namespace:

```
$ oc get scaledobjects -n <namespace>
```

Check the KEDA operator logs for errors:

```
$ oc logs -l app=keda-operator -n openshift-keda
```

- Verify Prometheus metrics are being collected:  
Confirm that vLLM metrics are available in Prometheus:

```
$ oc port-forward -n openshift-monitoring prometheus-k8s-0 9090:9090
```

Then query Prometheus at **http://localhost:9090** for **vllm:kv\_cache\_usage\_perc** and **vllm:num\_requests\_waiting**.

- Check the WVA scaling metrics in Prometheus:  
Query the **wva\_desired\_replicas** metric to verify the WVA is calculating scaling targets:

```
$ oc exec -n openshift-monitoring prometheus-k8s-0 -- promtool query instant
http://localhost:9090 'wva_desired_replicas'
```

- Enable user workload monitoring:  
If metrics are not being collected, verify that OpenShift User Workload Monitoring is enabled:

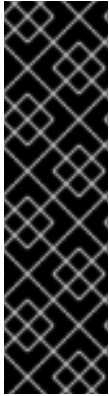
```
$ oc get configmap cluster-monitoring-config -n openshift-monitoring
```

If the ConfigMap does not exist, create it:

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: cluster-monitoring-config
 namespace: openshift-monitoring
data:
 config.yaml: |
 enableUserWorkload: true
```

## CHAPTER 14. WIDE EP DEPLOYMENT TOPOLOGY AND DP LOAD BALANCING MODES

You can deploy large Mixture of Experts (MoE) models such as DeepSeek-R1 across multiple GPU nodes by using wide expert parallelism (WideEP). WideEP deployments use a one-pod-per-node topology that requires all GPUs on a node to reside in a single pod. Understanding this topology and the available data-parallel (DP) load balancing modes helps you select the correct configuration for prefix-cache-aware inference scheduling.



### IMPORTANT

Wide expert parallelism (WideEP) is a Developer Preview feature only. Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to upcoming product features in advance of their possible inclusion in a Red Hat product offering, enabling customers to test functionality and provide feedback during the development process. These features might not have any documentation, are subject to change or removal at any time, and testing is limited. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA.



### IMPORTANT

DP-aware load balancing is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

### 14.1. WIDE EP ONE-POD-PER-NODE TOPOLOGY

WideEP distributes MoE model experts across multiple GPU nodes by using all-to-all communication backends such as DeepEP. DeepEP uses NVLink buffers for intranode sparse dispatch and combine operations, which cannot cross pod boundaries. As a result, all GPUs on a node must reside in a single pod.

This topology constraint has the following implications:

- Each pod requests all GPUs on the node, which means a single pod occupies the entire node.
- The **.spec.parallelism.dataLocal** field in the **LLMInferenceService** CR must equal the number of GPUs per node.
- The LeaderWorkerSet Operator manages multi-node coordination across pods.
- Data-parallel ranks run as separate processes within each pod rather than as separate pods.

When multiple data-parallel ranks run inside a single pod, you need two kinds of endpoints:

- **Serving ports:** One port per DP rank so the Endpoint Picker can route inference requests to a specific DP rank.



- **Admin port:** One aggregated health endpoint for the pod so Kubernetes probes can decide whether the pod is ready to receive traffic.

A single **vllm serve** command in external multi-port DP mode provides both: it starts the local DP ranks on sequential serving ports and runs the DP Supervisor on a separate admin port.

## 14.2. DP LOAD BALANCING MODES

vLLM supports four data-parallel load balancing modes. The following table compares the three modes most relevant to WideEP deployments. A fourth mode, External LB (**--data-parallel-external-lb**), provides single-endpoint external load balancing without the multi-port supervisor and is not covered here.

**Table 14.1. DP load balancing modes**

| Mode                   | Endpoint granularity     | WideEP suitability      | Endpoint Picker (EPP) routing capability                                 |
|------------------------|--------------------------|-------------------------|--------------------------------------------------------------------------|
| Internal DP            | Single endpoint per pod  | Not suitable for WideEP | EPP routes to pods only; no per-rank visibility                          |
| Hybrid DP              | One endpoint per node    | Limited                 | EPP routes to nodes; node-level but not rank-level granularity           |
| External multi-port DP | One endpoint per DP rank | Required for WideEP     | EPP routes to individual DP ranks; enables prefix-cache-aware scheduling |

For WideEP deployments, use the external multi-port DP mode. This mode is the only option that exposes individual DP rank endpoints on separate ports, which enables the EPP to make prefix-cache-aware scheduling decisions at rank-level granularity.

## 14.3. WHY EXTERNAL MULTI-PORT MODE ENABLES PREFIX-CACHE-AWARE ROUTING

The EPP evaluates prefix cache residency when selecting a backend for each request. To make accurate scheduling decisions, the EPP needs visibility into each DP rank as a separate endpoint.

In internal and hybrid DP modes, the EPP sees only pod-level or node-level endpoints. All DP ranks within a pod share a single endpoint, which prevents the EPP from directing a request to the specific DP rank that has the relevant KV cache entries populated.

In external multi-port DP mode, each DP rank exposes its own serving port. The EPP can query each DP rank independently and route requests to the DP rank most likely to have a warm prefix cache, which reduces redundant prompt processing and improves both throughput and time-to-first-token.

## 14.4. SUPPORTED LAUNCH PATH: SERVING PORTS AND ADMIN HEALTH

For WideEP, enable external multi-port DP mode with a single **vllm serve** command. That launch path gives you:

- One serving port per local DP rank for Endpoint Picker routing
- The DP Supervisor on a dedicated admin port for Kubernetes probes and coordinated shutdown

The DP Supervisor is a non-inference process. It starts the local DP rank processes, aggregates their health, and does not handle inference requests.

The DP Supervisor uses all-or-nothing health semantics:

- The supervisor returns **HTTP 200** on **/health** and **/readyz** only when every local DP rank is healthy.
- If any single DP rank fails or becomes unhealthy, the supervisor returns **HTTP 503** on all health endpoints.
- Kubernetes probes target the supervisor admin port rather than per-rank serving ports.

This all-or-nothing model is intentional. When any DP rank on a node fails, the entire pod becomes unusable for prefix-cache-aware routing because the Endpoint Picker cannot distribute requests across an incomplete set of DP ranks.

The supervisor follows a state machine with three states:

1. **Not ready:** The supervisor has started, but not all DP ranks have passed their initial health checks. The supervisor does not serve health endpoints during this state.
2. **Ready:** All DP ranks are healthy. The supervisor serves **HTTP 200** on **/health**, **/ready**, and **/readyz**.
3. **Shutdown:** The supervisor received a SIGTERM signal or detected a rank failure. The supervisor immediately returns **HTTP 503** on all health endpoints, forwards SIGTERM to all child rank processes, and waits for graceful drain before force-killing remaining processes.

## 14.5. PORT ALLOCATION MODEL

In external multi-port DP mode, ports are allocated as follows:

- **Serving ports:** Each DP rank listens on a sequential port starting from the base **--port**. For example, with **--port 8000** and **--data-parallel-size-local 4**, DP ranks listen on ports 8000, 8001, 8002, and 8003.
- **Supervisor admin port:** Aggregated health endpoints listen on a separate port, which defaults to 9256. You can configure this port by using the **--data-parallel-supervisor-port** flag.

Ensure that the supervisor admin port does not overlap with any serving port in the range.

## 14.6. PARALLELISM CONFIGURATION AND DP MODE SELECTION

The **.spec.parallelism** fields in the **LLMInferenceService** CR determine the deployment topology. The **.spec.parallelism.data** and **.spec.parallelism.dataLocal** fields, combined with the DP mode selection, determine whether the EPP can route to individual DP ranks.

When you configure a WideEP deployment:

- Set **.spec.parallelism.data** to the total number of DP ranks across the cluster.
- Set **.spec.parallelism.dataLocal** to the number of DP ranks per node, which must match the number of GPUs per node.
- Set **.spec.parallelism.expert** to **true** to enable expert parallelism.
- Use the **--data-parallel-multi-port-external-lb** flag in the vLLM serve command to enable external multi-port DP mode.

## 14.7. KNOWN LIMITATIONS

DP-aware load balancing for WideEP has the following limitations in this Technology Preview:

- **All-or-nothing health model** If any single DP rank fails, the entire pod shuts down. A single GPU error or DP rank crash affects all DP ranks on the node, requiring a full pod restart.
- **No partial-rank degradation**: The supervisor cannot continue serving with fewer DP ranks if one fails. Plan availability around the blast radius of a single DP rank failure across all GPUs on the node.
- **No elastic EP scaling**: You cannot dynamically add or remove DP ranks at runtime. The DP Supervisor launches a fixed number of DP ranks at startup. To scale, deploy new pods rather than adding DP ranks to existing ones.
- **gRPC incompatibility**: The external multi-port mode does not support gRPC transport. Only HTTP and HTTPS transport are supported.
- **UNIX domain socket incompatibility**: UNIX domain socket (UDS) configuration is not supported with external multi-port mode.

### Additional resources

- [Configuring RoCE networking for distributed LLM deployments](#)

## CHAPTER 15. LAUNCH A WIDEEP INFERENCE SERVICE WITH DP-AWARE ROUTING

You can deploy large MoE models across multiple GPU nodes with external multi-port data-parallel (DP) mode so the Endpoint Picker can route requests to individual DP ranks for prefix-cache-aware scheduling. A single **vllm serve** command starts all local DP ranks, provides one serving port per DP rank, and uses the DP Supervisor admin port for Kubernetes probes. This path replaces custom shell wrappers for multi-rank WideEP deployments.



### IMPORTANT

Wide expert parallelism (WideEP) is a Developer Preview feature only. Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to upcoming product features in advance of their possible inclusion in a Red Hat product offering, enabling customers to test functionality and provide feedback during the development process. These features might not have any documentation, are subject to change or removal at any time, and testing is limited. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA.



### IMPORTANT

DP-aware load balancing is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

### Prerequisites

- Distributed Inference with llm-d infrastructure is deployed on OpenShift Container Platform.
- The Red Hat build of LeaderWorkerSet Operator is installed and a **LeaderWorkerSetOperator** CR is created. For more information, see [LeaderWorkerSet Operator for Red Hat OpenShift](#).
- A WideEP-capable MoE model is downloaded to a PVC or accessible through a model URL.
- Remote Direct Memory Access (RDMA)-capable networking such as RoCE or InfiniBand is configured for expert parallelism communication.
- Your cluster has sufficient GPU nodes for the WideEP topology. Each pod requests all GPUs on its node.

### Procedure

1. Determine the parallelism configuration for your model.  
Calculate the values for the **.spec.parallelism** fields based on your cluster topology:

- **data**: The total number of DP ranks across the cluster, calculated as **(nodes \* GPUs\_per\_node) / tensor**. For example, 4 nodes with 8 GPUs each and **tensor: 1** gives **data: 32**.
- **dataLocal**: The number of DP ranks per node, which must match the number of GPUs per node. For example, **dataLocal: 8** for nodes with 8 GPUs.
- **expert**: Set to **true** to enable expert parallelism.
- **tensor**: Set to **1** when using expert parallelism as the primary distribution strategy.

The formula **data / dataLocal** determines the number of pods. For example, **data: 32** with **dataLocal: 8** produces 4 pods.

2. Create an **LLMInferenceService** CR that uses external multi-port DP mode.  
Create a file called **wideep-inference-service.yaml** with the following content:

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
 name: deepseek-r1-wideep
 namespace: llm-inference
spec:
 model:
 uri: pvc://model-storage
 name: deepseek-ai/DeepSeek-R1-0528
 replicas: 1
 parallelism:
 data: 32
 dataLocal: 8
 expert: true
 tensor: 1
 router: # Uses default routing settings
 scheduler: {}
 route: {}
 gateway: {}
 template:
 containers:
 - name: main
 args:
 - --data-parallel-multi-port-external-lb
 - --data-parallel-size=32
 - --data-parallel-size-local=8
 - --data-parallel-supervisor-port=9256
 - --port=8000
 - --all2all-backend=deepep_high_throughput
 resources:
 limits:
 cpu: "32"
 memory: 256Gi
 nvidia.com/gpu: "8"
 requests:
 cpu: "16"
 memory: 128Gi
 nvidia.com/gpu: "8"
 startupProbe:
```

```

 httpGet:
 path: /readyz
 port: 9256
 scheme: HTTP
 periodSeconds: 10
 failureThreshold: 60
 timeoutSeconds: 5
 readinessProbe:
 httpGet:
 path: /readyz
 port: 9256
 scheme: HTTP
 periodSeconds: 5
 failureThreshold: 3
 timeoutSeconds: 5
 livenessProbe:
 httpGet:
 path: /health
 port: 9256
 scheme: HTTP
 periodSeconds: 15
 failureThreshold: 5
 timeoutSeconds: 5
 terminationGracePeriodSeconds: 120

```

where:

- **--data-parallel-multi-port-external-lb** - Enables external multi-port DP mode. Each local DP rank gets its own serving port so the Endpoint Picker can route to individual ranks. The DP Supervisor starts the local rank processes and aggregates their health on the admin port.
- **--data-parallel-size=32** - Specifies the total number of DP ranks across the cluster. Must match the **spec.parallelism.data** value.
- **--data-parallel-size-local=8** - Specifies the number of DP ranks per pod. Must match the **spec.parallelism.dataLocal** value and the number of GPUs per node.
- **--data-parallel-supervisor-port=9256** - Specifies the admin port for aggregated **/health** and **/readyz**. Point all Kubernetes probes at this port, not at per-rank serving ports.
- **--port=8000** - Specifies the base serving port for inference traffic. Each DP rank listens on a sequential port: 8000, 8001, 8002, through 8007 for 8 local ranks. The Endpoint Picker targets these ports when scheduling requests.
- **--all2all-backend=deepep\_high\_throughput** - Specifies the all-to-all communication backend for expert parallelism. The **deepep\_high\_throughput** backend is optimized for batch processing throughput and requires RDMA-capable networking such as RoCE or InfiniBand. For disaggregated deployments that separate prefill and decode stages, use **deepep\_low\_latency** for the decode stage.

#### port: 9256 in probe configurations

Targets the DP Supervisor admin port for all probes. Do not target per-rank serving ports.

- **failureThreshold: 60** in **startupProbe** – Allows up to 600 seconds for model loading. Calculate this value as **ceiling(max\_startup\_seconds / periodSeconds)**.
- **terminationGracePeriodSeconds: 120** – Allows the DP Supervisor to forward SIGTERM to all child processes and wait for graceful drain.

3. Deploy the inference service:

```
$ oc apply -f wideep-inference-service.yaml
```

4. Monitor the startup progress.

Large MoE models can take 7-10 minutes or more to load. Check the pod status during startup:

```
$ oc get pods -n llm-inference -w
```

The pod transitions through the following states:

- **Pending**: Waiting for GPU node scheduling.
- **ContainerCreating**: Pulling the container image.
- **Running** with startup probe active: Model is loading. DP ranks initialize and the supervisor waits for all DP ranks to become healthy.
- **Running** with all probes passing: All DP ranks are ready and the supervisor is serving health on the admin port.

5. Verify that **terminationGracePeriodSeconds** is set for coordinated shutdown.

The DP Supervisor requires enough time to forward SIGTERM to all child DP rank processes and wait for graceful drain. The recommended range is 60-120 seconds. The supervisor waits for the configured **shutdown\_timeout** plus 5 seconds before force-killing remaining processes.



#### NOTE

If you created the **LLMInferenceService** CR without **terminationGracePeriodSeconds**, patch the CR to add the specification:

```
$ oc patch llm_svc deepseek-r1-wideep -n llm-inference --type=merge \
-p '{"spec":{"template":{"terminationGracePeriodSeconds":120}}}'
```

#### Verification

- Verify that all DP ranks are ready by querying the supervisor health endpoint on one of the pods from the **oc get pods** output:

```
$ oc exec -n llm-inference <pod-name> -- \
curl -s http://localhost:9256/health -w "%{http_code}" -o /dev/null
```

A response of **200** confirms that all DP ranks on that pod are healthy. For a WideEP deployment with multiple pods, repeat this check for each pod to confirm that all nodes are ready.

- Verify that the **LLMInferenceService** is ready:

```
$ oc get llmismvc -n llm-inference
```

Expected output:

```
NAME READY AGE
deepseek-r1-wideep True 15m
```

- Verify that the service endpoints include the deployment pods:

```
$ oc get endpoints -n llm-inference
```

The output lists **Endpoints** objects by service name. Verify that the endpoints for the workload service include pod addresses, confirming that the service is routing traffic to the deployment pods. Per-rank serving ports are exposed inside each pod and are discovered by the Endpoint Picker through the **InferencePool**, not through Kubernetes Endpoints.

## 15.1. DP SUPERVISOR HEALTH ENDPOINTS AND PROBE CONFIGURATION

You can configure Kubernetes probes to target the DP Supervisor admin port for WideEP deployments that use external multi-port DP mode. Inference traffic still uses the per-rank serving ports; probes use the admin port so Kubernetes can treat the multi-rank pod as a single readiness unit. The supervisor aggregates health status from all local DP rank processes and provides **/health**, **/ready**, and **/readyz** endpoints on a dedicated port. Configuring probes correctly prevents repeated pod restarts during model loading and enables graceful shutdown.

### IMPORTANT

Wide expert parallelism (WideEP) is a Developer Preview feature only. Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to upcoming product features in advance of their possible inclusion in a Red Hat product offering, enabling customers to test functionality and provide feedback during the development process. These features might not have any documentation, are subject to change or removal at any time, and testing is limited. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA.

### IMPORTANT

DP-aware load balancing is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

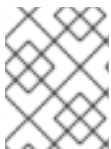
### 15.1.1. Health endpoint reference

The DP Supervisor serves the following health endpoints on the supervisor admin port, which defaults to port 9256:



Table 15.1. DP Supervisor health endpoints

| Endpoint             | HTTP status codes | Description                                                                                                                                                                                              |
|----------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/health</code> | 200 or 503        | Returns <b>HTTP 200</b> when all local DP ranks are healthy and the supervisor is not shutting down. Returns <b>HTTP 503</b> when any rank is unhealthy or the supervisor has received a SIGTERM signal. |
| <code>/ready</code>  | 200 or 503        | Functionally identical to <code>/health</code> . Returns <b>HTTP 200</b> when all local DP ranks are healthy.                                                                                            |
| <code>/readyz</code> | 200 or 503        | Functionally identical to <code>/health</code> . Returns <b>HTTP 200</b> when all local DP ranks are healthy.                                                                                            |

**NOTE**

All three endpoints share the same internal readiness state. This is an intentional simplification for Technology Preview.

### 15.1.2. Supervisor internal probe parameters

The DP Supervisor continuously probes each child DP rank process to determine health status. You can tune these parameters with vLLM CLI flags:

Table 15.2. Supervisor internal probe parameters

| CLI flag                                             | Default     | Description                                                                                                             |
|------------------------------------------------------|-------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>--data-parallel-supervisor-port</code>         | <b>9256</b> | Specifies the port where health endpoints are served. Kubernetes probes must target this port.                          |
| <code>--dp-supervisor-probe-interval-s</code>        | <b>5.0</b>  | Specifies how often the supervisor polls each child DP rank for health status, in seconds.                              |
| <code>--dp-supervisor-probe-timeout-s</code>         | <b>5.0</b>  | Specifies the timeout for each child health check attempt, in seconds.                                                  |
| <code>--dp-supervisor-probe-failure-threshold</code> | <b>3</b>    | Specifies the number of consecutive failures before the supervisor marks a DP rank as unhealthy and initiates shutdown. |

### 15.1.3. Recommended Kubernetes probe configuration

Configure Kubernetes probes to target the DP Supervisor admin port rather than per-rank serving ports. The following recommendations apply to WideEP deployments with external multi-port DP mode:

#### **startupProbe**

Target `/readyz` on the supervisor port. Use a generous **failureThreshold** to accommodate model

loading times, which can exceed 10 minutes for large MoE models. Calculate the threshold with the following formula: **failureThreshold = ceiling(max\_startup\_seconds / periodSeconds)**. For example, a 10-minute startup with **periodSeconds: 10** requires **failureThreshold: 60**.

### readinessProbe

Target **/readyz** on the supervisor port. Use shorter intervals after startup succeeds, because the model is already loaded and health checks respond quickly.

### livenessProbe

Target **/health** on the supervisor port. Use a longer interval and higher failure threshold than the readiness probe to avoid unnecessary pod restarts from transient issues.

### terminationGracePeriodSeconds

Set to 60-120 seconds to allow the DP Supervisor to forward SIGTERM to all child DP rank processes and wait for graceful drain.

The following example shows the probe configuration within the **spec.template** section of an **LLMInferenceService** CR for a DeepSeek-R1 deployment on 8 GPUs per node with a 10-minute maximum startup time:

```
template:
 containers:
 - name: main
 # ...
 startupProbe:
 httpGet:
 path: /readyz
 port: 9256
 scheme: HTTP
 periodSeconds: 10
 failureThreshold: 60
 timeoutSeconds: 5
 readinessProbe:
 httpGet:
 path: /readyz
 port: 9256
 scheme: HTTP
 periodSeconds: 5
 failureThreshold: 3
 timeoutSeconds: 5
 livenessProbe:
 httpGet:
 path: /health
 port: 9256
 scheme: HTTP
 periodSeconds: 15
 failureThreshold: 5
 timeoutSeconds: 5
 terminationGracePeriodSeconds: 120
```

where:

- **port: 9256:** Specifies the DP Supervisor admin port. This must match the value of **--data-parallel-supervisor-port**.

- **failureThreshold: 60**: Specifies the number of startup probe failures to tolerate. With **periodSeconds: 10**, this allows up to 600 seconds for model loading.
- **terminationGracePeriodSeconds: 120**: Specifies the time Kubernetes waits for the pod to shut down gracefully. This must be long enough for the DP Supervisor to forward SIGTERM to all child processes and wait for drain.



### IMPORTANT

Do not configure probes to target per-rank serving ports such as 8000, 8001, or 8002. Kubernetes probes target a single port per container, and only the DP Supervisor admin port provides aggregated health status for all DP ranks.

#### 15.1.4. SIGTERM shutdown behavior

When the DP Supervisor receives a SIGTERM signal from Kubernetes:

1. The supervisor immediately sets its health status to not ready and returns **HTTP 503** on all health endpoints.
2. The supervisor forwards SIGTERM to all child DP rank processes.
3. The supervisor waits for the configured **shutdown\_timeout** plus 5 seconds for child processes to exit gracefully.
4. Any child processes that have not exited after the grace period are force-killed.

This coordinated shutdown prevents orphaned GPU processes and ensures that inference requests in flight can drain before the pod terminates. Set **terminationGracePeriodSeconds** in the pod spec to a value that accommodates the shutdown timeout plus 5 seconds.

#### 15.1.5. Health model trade-offs

The all-or-nothing health model means that any single DP rank failure marks the entire pod as not ready. The Endpoint Picker (EPP) and Inference Gateway automatically retry requests to healthy pods.

This design has the following implications:

- A transient GPU error on one DP rank causes all DP ranks on the node to restart, even if the other DP ranks are healthy.
- Availability planning must account for the blast radius of a single DP rank failure across all GPUs on the node.
- The readiness probe removes the pod from the Kubernetes Service endpoints immediately, which allows the EPP to route new requests to other pods while the failed pod restarts.

## 15.2. DP SUPERVISOR CLI FLAGS AND CONFIGURATION OPTIONS

You can configure external multi-port DP mode and the DP Supervisor for WideEP deployments by using CLI flags with the **vllm serve** command. The following tables list the required and optional flags, validation rules, and incompatible options.



## IMPORTANT

Wide expert parallelism (WideEP) is a Developer Preview feature only. Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to upcoming product features in advance of their possible inclusion in a Red Hat product offering, enabling customers to test functionality and provide feedback during the development process. These features might not have any documentation, are subject to change or removal at any time, and testing is limited. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA.



## IMPORTANT

DP-aware load balancing is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

### 15.2.1. Required flags

The following flags are required to enable external multi-port DP mode with the DP Supervisor:

Table 15.3. Required DP Supervisor CLI flags

| Flag                                          | Default      | Description                                                                                                                                                                                                     |
|-----------------------------------------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>--data-parallel-multi-port-external-lb</b> | <b>false</b> | Enables external multi-port DP mode. When set, each local DP rank receives its own serving port, and the DP Supervisor starts those ranks and exposes aggregated health endpoints on the supervisor admin port. |
| <b>--data-parallel-size</b>                   | None         | Specifies the total number of DP ranks across the cluster. Must be greater than 1 and divisible by <b>--data-parallel-size-local</b> .                                                                          |
| <b>--data-parallel-size-local</b>             | None         | Specifies the number of DP ranks per pod. Must be 2 or greater. This value typically matches the number of GPUs per node.                                                                                       |
| <b>--port</b>                                 | <b>8000</b>  | Specifies the base serving port. Each DP rank listens on a sequential port starting from this value: port, port+1, port+2, and so on.                                                                           |

### 15.2.2. Optional flags

The following flags configure the DP Supervisor behavior and internal health probing:

Table 15.4. Optional DP Supervisor CLI flags

| Flag                                           | Default     | Description                                                                                                                                                                                                         |
|------------------------------------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>--data-parallel-supervisor-port</b>         | <b>9256</b> | Specifies the HTTP port where the DP Supervisor serves aggregated health endpoints: <b>/health</b> , <b>/ready</b> , and <b>/readyz</b> . Kubernetes probes must target this port.                                  |
| <b>--node-rank</b>                             | <b>0</b>    | Specifies this node's rank in the multi-node deployment. Used to infer the <b>--data-parallel-start-rank</b> value when it is not set explicitly.                                                                   |
| <b>--data-parallel-start-rank</b>              | Inferred    | Specifies the starting rank offset for DP ranks in this pod. If not set, the value is inferred from <b>--node-rank</b> multiplied by <b>--data-parallel-size-local</b> .                                            |
| <b>--dp-supervisor-probe-interval-s</b>        | <b>5.0</b>  | Specifies the interval in seconds between the supervisor's internal health probes to each child DP rank process.                                                                                                    |
| <b>--dp-supervisor-probe-timeout-s</b>         | <b>5.0</b>  | Specifies the timeout in seconds for each child health probe attempt. If a probe does not respond within this duration, the attempt is treated as a connection error and retried.                                   |
| <b>--dp-supervisor-probe-failure-threshold</b> | <b>3</b>    | Specifies the number of consecutive connection error retries before a child health probe is declared failed. After this threshold is reached, the supervisor marks the DP rank as unhealthy and initiates shutdown. |
| <b>--ssl-keyfile</b>                           | None        | Specifies the file path to the SSL key file. Must be provided together with <b>--ssl-certfile</b> .                                                                                                                 |
| <b>--ssl-certfile</b>                          | None        | Specifies the file path to the SSL certificate file. Must be provided together with <b>--ssl-keyfile</b> .                                                                                                          |

### 15.2.3. Validation rules

The following constraints apply when **--data-parallel-multi-port-external-lb** is enabled. The DP Supervisor validates these constraints at startup. If any validation fails, the process exits with an error:

- **--data-parallel-multi-port-external-lb** is mutually exclusive with **--data-parallel-external-lb** and **--data-parallel-hybrid-lb**. You cannot combine multiple DP modes.
- **--data-parallel-size-local** must be 2 or greater.
- **--data-parallel-size** must be greater than 1.
- **--data-parallel-size** must be divisible by **--data-parallel-size-local**.

- **--data-parallel-size-local** cannot exceed **--data-parallel-size**.
- The supervisor port must not overlap with the range of child DP rank serving ports, which starts at **--port** and spans **--data-parallel-size-local** consecutive ports.
- **--ssl-keyfile** and **--ssl-certfile** must be provided together. Specifying only one results in a validation error.
- **--data-parallel-rank** must not be set manually. The DP Supervisor manages child DP rank values internally.
- **--api-server-count** must be 1 or unset.

#### 15.2.4. Incompatible options

The following configurations are not supported with external multi-port DP mode:

- **gRPC transport:** The **--grpc** flag is not compatible with **--data-parallel-multi-port-external-lb**.
- **UNIX domain sockets:** The **--uds** flag is not compatible with **--data-parallel-multi-port-external-lb**.

### 15.3. KNOWN LIMITATIONS FOR DP-AWARE LOAD BALANCING

DP-aware load balancing for WideEP deployments has the following known limitations in this Technology Preview. Review these limitations before deploying WideEP inference services with the DP Supervisor to plan for availability, scaling, and transport constraints.



#### IMPORTANT

Wide expert parallelism (WideEP) is a Developer Preview feature only. Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to upcoming product features in advance of their possible inclusion in a Red Hat product offering, enabling customers to test functionality and provide feedback during the development process. These features might not have any documentation, are subject to change or removal at any time, and testing is limited. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA.



#### IMPORTANT

DP-aware load balancing is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Table 15.5. Known limitations for DP-aware load balancing

| Limitation                           | Description                                                                                                                                                                                                                                                                                    |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| All-or-nothing health model          | If any single DP rank fails or becomes unhealthy, the DP Supervisor shuts down the entire pod. A single GPU error or DP rank crash takes down all DP ranks on the node, requiring a full pod restart. The supervisor cannot mark individual DP ranks as unhealthy while keeping others active. |
| No partial-rank degradation          | The DP Supervisor cannot continue serving inference requests with fewer DP ranks if one fails. All DP ranks must be healthy for the pod to accept requests. Plan availability around the blast radius of a single DP rank failure across all GPUs on the node.                                 |
| No elastic EP scaling                | You cannot dynamically add or remove DP ranks at runtime. The DP Supervisor launches a fixed number of DP ranks at startup based on the <b>--data-parallel-size-local</b> flag. To scale capacity, deploy new pods rather than adding DP ranks to existing pods.                               |
| Synchronous forward pass requirement | All DP ranks must synchronize forward passes for all models in DP mode, including dummy passes on idle DP ranks. MoE models have additional synchronization requirements for expert-parallel collectives. Uneven request distribution across DP ranks does not reduce per-GPU compute cost.    |
| gRPC transport incompatibility       | The external multi-port DP mode does not support gRPC transport. Only HTTP and HTTPS transport are supported for DP Supervisor deployments. The <b>--grpc</b> flag cannot be combined with <b>--data-parallel-multi-port-external-lb</b> .                                                     |
| UNIX domain socket incompatibility   | UNIX domain socket (UDS) configuration is not supported with external multi-port DP mode. The <b>--uds</b> flag cannot be combined with <b>--data-parallel-multi-port-external-lb</b> .                                                                                                        |
| Node-local supervisor scope          | Each DP Supervisor manages only the DP ranks within its own pod. Cross-node coordination is handled by the <b>LLMInferenceService</b> controller and the LeaderWorkerSet Operator, not by the supervisor.                                                                                      |

## CHAPTER 16. CONFIGURE DISAGGREGATED PREFILL-DECODE SERVING

You can configure disaggregated prefill-decode serving to separate the compute-intensive prefill phase from the memory-intensive decode phase into independent pod pools. Disaggregated prefill-decode enables independent scaling, hardware optimization, and improved GPU utilization for large language model inference workloads.



### IMPORTANT

Disaggregated prefill-decode is a Technology Preview feature on OpenShift Container Platform and Azure Kubernetes Service (AKS). On CoreWeave Kubernetes Service (CKS), disaggregated prefill-decode is generally available (GA).

### Prerequisites

- Distributed Inference with llm-d is deployed on your Kubernetes cluster with NVIDIA GPUs.
- RDMA-capable networking such as RoCE or InfiniBand is available for optimal NIXL KV cache transfer performance between nodes. If RDMA is not available, NIXL will fall back to using TCP for KV cache transfers, which is not a recommended configuration.



### NOTE

Disaggregated prefill-decode with RDMA has been validated on the following platforms and instance types:

**Table 16.1. Validated platforms for disaggregated prefill-decode serving with RDMA**

| Platform                           | Instance types                                                                     | RDMA transport      |
|------------------------------------|------------------------------------------------------------------------------------|---------------------|
| OpenShift Container Platform       | Bare-metal nodes with NVIDIA A100, H100, or B200 GPUs and ConnectX-6 or later NICs | InfiniBand, RoCE v2 |
| Azure Kubernetes Service (AKS)     | NDasrA100_v4, NDm_A100_v4, ND-H100-v5                                              | InfiniBand          |
| CoreWeave Kubernetes Service (CKS) | gd-8xh100ib-i128, gd-8xh200ib-i128, b200-8x, b300-8x                               | InfiniBand          |

Other instance types with InfiniBand or RoCE networking might work but have not been validated.

- You have sufficient GPU capacity available for both prefill and decode pod pools.
- You have installed the Kubernetes CLI (**kubectl**).

### Procedure



1. AKS only: install THE NVIDIA GPU Operator and Network Operator and configure the RDMA shared device plugin for InfiniBand NICs. For setup instructions, see [Azure AKS RDMA InfiniBand setup](#) and [RDMA shared device plugin configuration](#).
2. AKS only: determine the GPU-NIC PCI topology for your AKS instance type. AKS virtual machines present a flat PCI topology, which prevents NIXL from automatically discovering which GPUs and NICs are aligned. Without this mapping, KV cache transfers might use suboptimal GPU-NIC paths, significantly reducing performance.  
Run the cluster validation tool's RDMA checks to discover the topology.

After validation completes, extract the GPU-NIC PCI mapping for each node from the report:

```
$ kubectl get cm rhaii-validate-report -n rhaii-validation \
 -o jsonpath='{.data.report.json}' | \
 jq -c '.nodes[] | [.node,
 (.results[]|select(.name=="gpu_nic_topology").details.gpu_nic_pcie_mapping)]'
```

Set the mapping as environment variables on all containers in the **LLMInferenceService** CR:

```
env:
 - name: VLLM_GPU_NIC_PCIE_MAPPING
 value: "<gpu_pci_addr>=<nic_pci_addr>,<gpu_pci_addr>=<nic_pci_addr>,..."
 - name: VLLM_NIC_SELECTION_VARS
 value: "UCX_NET_DEVICES:1,NCCL_IB_HCA:1,NVSHMEM_HCA_LIST:1"
```

where:

- **VLLM\_GPU\_NIC\_PCIE\_MAPPING** specifies the PCI address mapping between each GPU and its topology-aligned NIC. Use the mapping from the cluster validation report.
  - **VLLM\_NIC\_SELECTION\_VARS** enables NIC selection for UCX, NCCL, and NVSHMEM based on the PCI mapping.
3. AKS only: enable the Node Resource Interface (NRI) on all GPU nodes and deploy the ulimit adjuster plugin. AKS worker nodes have a default maximum locked memory limit of 64 KiB per container, which is insufficient for NIXL KV cache transfers.  
On each GPU node in the cluster, enable NRI in the containerd configuration and restart the runtime. Open a debug pod on the node:

```
$ kubectl debug node/<gpu_node_name>
```

From the debug pod, run **chroot /host** to access the node filesystem.

Add the following section to **/etc/containerd/config.toml**:

```
[plugins."io.containerd.nri.v1.nri"]
 disable = false
```

Restart containerd to apply the change, then exit the debug pod:

```
systemctl restart containerd
exit
```

Repeat for every GPU node, then deploy the ulimit adjuster plugin:

```
$ kubectl apply -k https://github.com/containerd/nri/contrib/kustomize/ulimit-adjuster
```

Add the following annotation to the **LLMInferenceService** CR pod template for both prefill and decode containers to set the memory lock limit:

```
metadata:
 annotations:
 ulimits.nri.containerd.io/container.main: |
 - type: memlock
 hard: 17179869184
 soft: 17179869184
```

- OpenShift Container Platform only: set the **memlock** ulimit on GPU worker nodes to **unlimited**. RDMA memory registration pins pages into physical memory and fails if the memlock limit is too low. You can set this by applying a **MachineConfig** that configures CRI-O with **default\_ulimits = ["memlock=-1:-1"]** on GPU worker nodes. You can verify the current setting by running **oc debug node/<node\_name> -- chroot /host bash -c 'ulimit -l'**.



#### NOTE

For a full **MachineConfig** CR example that sets **default\_ulimits**, see [Configuring a cluster for RDMA](#).

- Create an **LLMInferenceService** CR with a **spec.prefill** section to enable disaggregated serving mode. Including **spec.prefill** separates the deployment into independent pools, where **spec.prefill** configures the dedicated prefill pod pool and **spec.template** defines the decode pod pool.

The following example shows a minimal disaggregated **LLMInferenceService** configuration by using **Qwen/Qwen3-0.6B** to validate the P/D pipeline. This model is small enough to run with 1 GPU per side, making it suitable for verifying that disaggregated serving, NIXL, and RDMA are functioning correctly.



#### NOTE

A model this small does not benefit from disaggregation in production. The performance advantages of separating prefill and decode emerge with larger models where the two phases have meaningfully different compute and memory profiles. After validating the pipeline, replace the model and resource configuration to match your production workload.

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
 name: pd-smoke-test
spec:
 model:
 name: Qwen/Qwen3-0.6B
 uri: hf://Qwen/Qwen3-0.6B
 replicas: 1
 template:
 containers:
 - name: main
```

```

env:
 - name: VLLM_NIXL_SIDE_CHANNEL_HOST
 valueFrom:
 fieldRef:
 fieldPath: status.podIP
 - name: VLLM_ADDITIONAL_ARGS
 value: --tensor-parallel-size 1 --max-model-len 16000 --block-size 128 --no-disable-
hybrid-kv-cache-manager --kv-transfer-config
{"kv_connector":"NixlConnector","kv_role":"kv_both"}
ports:
 - containerPort: 5600
 name: nixl
 protocol: TCP
resources:
 limits:
 cpu: 4
 memory: 16Gi
 nvidia.com/gpu: 1
 rdma/ib: 1
 requests:
 cpu: 2
 memory: 8Gi
 nvidia.com/gpu: 1
 rdma/ib: 1
volumeMounts:
 - name: shm
 mountPath: /dev/shm
volumes:
 - name: shm
 emptyDir:
 medium: Memory
 sizeLimit: 4Gi
prefill:
 replicas: 1
 template:
 containers:
 - name: main
 env:
 - name: VLLM_NIXL_SIDE_CHANNEL_HOST
 valueFrom:
 fieldRef:
 fieldPath: status.podIP
 - name: VLLM_ADDITIONAL_ARGS
 value: --tensor-parallel-size 1 --max-model-len 16000 --gpu-memory-utilization 0.88
--block-size 128 --no-disable-hybrid-kv-cache-manager --kv-transfer-config
{"kv_connector":"NixlConnector","kv_role":"kv_both"}
ports:
 - containerPort: 5600
 name: nixl
 protocol: TCP
resources:
 limits:
 cpu: 4
 memory: 16Gi
 nvidia.com/gpu: 1
 rdma/ib: 1

```

```

 requests:
 cpu: 2
 memory: 8Gi
 nvidia.com/gpu: 1
 rdma/ib: 1
 securityContext:
 capabilities:
 add:
 - IPC_LOCK
 volumeMounts:
 - name: shm
 mountPath: /dev/shm
 volumes:
 - name: shm
 emptyDir:
 medium: Memory
 sizeLimit: 4Gi
 router:
 gateway: {}
 route: {}
 scheduler:
 template:
 containers:
 - name: main
 - name: tokenizer

```

where:

- **spec.template** specifies the decode pod configuration. Decode pods generate output tokens and are optimized for memory bandwidth.
- **spec.prefill** specifies the prefill pod configuration. Prefill pods process input prompts to build the KV cache and are optimized for compute throughput. The **gpu-memory-utilization** is set slightly lower than the default to leave headroom for KV cache transfers.
- **VLLM\_NIXL\_SIDE\_CHANNEL\_HOST** specifies the pod IP for NIXL peer discovery. NIXL uses this address for side-channel coordination between prefill and decode pods.
- **kv\_connector: NixlConnector** specifies the NVIDIA NIXL library for zero-copy KV cache transfer over RDMA between pods.
- **block-size 128** specifies the KV cache block size. This value must match across all prefill and decode pods.
- **rdma/ib** specifies the RDMA device plugin resource name.

6. Apply the **LLMInferenceService** CR to your cluster.

7. Monitor the deployment progress:

```
$ kubectl get llmisvc <llmisvc_name> -n <namespace> -w
```

## Verification

1. Verify that separate prefill and decode pods are running:

–

```
$ kubectl get pods -n <namespace> -l app.kubernetes.io/name=<llmisvc_name>
```

The output shows pods with both **workload** and **workload-prefill** component labels.

2. Verify that RDMA resources are allocated to the pods:

```
$ kubectl get pods -n <namespace> -l app.kubernetes.io/name=<llmisvc_name> \
-o jsonpath='{range .items[*]}.{.metadata.name}{"\t"}{.spec.containers[0].resources.limits}
{"\n"}{end}'
```

3. Send a test inference request through the gateway to confirm end-to-end functionality.  
In a separate terminal, port-forward to the gateway service:

```
$ kubectl port-forward -n <gateway_namespace> svc/<gateway_service> 8080:80
```

Verify with a model serving request.

4. Verify that KV cache transfers are succeeding by checking the decode pod logs:

```
$ kubectl logs -n <namespace> <decode_pod_name> | grep "KV Transfer"
```

Successful KV cache transfers produce log output similar to the following:

```
(APIServer pid=428) INFO 07-13 16:33:58 [metrics.py:103] KV Transfer metrics: Num
successful transfers=1, Avg xfer time (ms)=3.056, P90 xfer time (ms)=3.056, Avg MB per
transfer=14.0, Throughput (MB/s)=4581.152
```

5. Optionally, verify that NIXL KV cache transfer metrics are active by checking the decode pod metrics endpoint:

```
$ kubectl port-forward -n <namespace> <decode_pod_name> 8000:8000 &
$ PF_PID=$!
$ curl -sk https://localhost:8000/metrics | grep "vllm:nixl_bytes_transferred_count"
$ kill $PF_PID
```

The **vllm:nixl\_bytes\_transferred** metric is a Prometheus histogram. A nonzero **\_count** value confirms that KV cache transfers between prefill and decode pods have completed successfully. The **\_sum** value shows the total bytes transferred across all requests.

## 16.1. NIXLCONNECTOR KV CACHE TRANSFER PARAMETERS

You can configure NIXL KV cache transfer behavior by passing a JSON object to the **--kv-transfer-config** argument in the **VLLM\_ADDITIONAL\_ARGS** environment variable of the **LLMInferenceService** CR template containers. The **NixlConnector** is the vLLM plugin that manages high-speed GPU memory transfers between prefill and decode pods in disaggregated serving deployments.

### --kv-transfer-config arguments

The **--kv-transfer-config** argument accepts the following JSON parameters:

**Table 16.2.** NixlConnector kv-transfer-config parameters

| Parameter                        | Type        | Default     | Description                                                                                                                                                                                                                                                                            |
|----------------------------------|-------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>kv_connector</b>              | String      | None        | Specifies the KV cache connector plugin to use. Set to <b>NixlConnector</b> for NIXL-based GPU memory transfers.                                                                                                                                                                       |
| <b>kv_role</b>                   | String      | None        | Specifies the role of the pod in the disaggregated topology. Set to <b>kv_both</b> so that each pod can both send and receive KV cache data.                                                                                                                                           |
| <b>kv_load_failure_policy</b>    | String      | <b>fail</b> | Specifies the behavior when a KV cache transfer fails. Set to <b>fail</b> to reject the request and return an error to the client. Set to <b>recompute</b> to fall back to local prefill computation on the decode pod, which avoids errors but reduces the benefit of disaggregation. |
| <b>kv_connector_extra_config</b> | JSON object | <b>{}</b>   | Specifies additional configuration for the <b>NixlConnector</b> . This object accepts transport-specific parameters.                                                                                                                                                                   |

### KV role configuration

In a disaggregated Distributed Inference with llm-d deployment, set **kv\_role** to **kv\_both** on all pods. This value allows each pod to both send and receive KV cache data, which is the configuration used by the Distributed Inference with llm-d routing and scheduling layer.

### KV load failure policy options

The **kv\_load\_failure\_policy** parameter controls how the system responds when a NIXL KV cache transfer fails between prefill and decode pods.

**Table 16.3. kv\_load\_failure\_policy comparison**

| Policy           | Behavior                                                                                                                                                | Recommended use                                                                                                                                                                   |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>fail</b>      | The request fails immediately with an error. The client must retry. This is the default policy.                                                         | Production workloads where you prefer explicit failure detection over silent performance degradation. Use this policy when your client applications can handle retries.           |
| <b>recompute</b> | The decode pod falls back to local prefill computation. The request completes, but with higher latency because the decode pod repeats the prefill work. | Workloads where request completion is more important than optimal latency. Use this policy during rolling upgrades or in environments where transient NIXL failures are expected. |

### NIXL compatibility hash

The **NixlConnector** generates a compatibility hash for each pod based on the model configuration,

tensor parallelism settings, and vLLM version. Two pods can exchange KV cache data only when their hashes match. Hash mismatches commonly occur during rolling upgrades when new pods run a different vLLM version or configuration than existing pods.

The compatibility hash includes the following fields:

- vLLM version
- NixlConnector version
- Model name
- Model data type and KV cache data type
- Number of KV heads and head size
- Number of hidden layers
- Attention backend name
- Hybrid memory allocator enabled
- Cross-layer blocks enabled

The hash does not include **kv\_role**, tensor parallelism degree, block size, replica count, or environment variables unrelated to model serving. Tensor parallelism and block size are intentionally excluded to support heterogeneous configurations between prefill and decode pods.

### NIXL networking environment variables

You can configure the following environment variables for NIXL networking in the **LLMInferenceService** CR template:

**Table 16.4. NIXL environment variables**

| Variable                           | Default          | Description                                                                                                                                                                                                                                                                                                   |
|------------------------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VLLM_NIXL_SIDE_CHANNEL_PORT</b> | <b>5600</b>      | Specifies the TCP port used for NIXL side-channel communication between pods. Each pod listens on this port to exchange metadata and coordinate KV cache transfers.                                                                                                                                           |
| <b>VLLM_NIXL_SIDE_CHANNEL_HOST</b> | <b>localhost</b> | Specifies the hostname or IP address for the NIXL side-channel listener. You must set this to the pod's IP address by using a <b>fieldRef</b> to <b>status.podIP</b> so that other pods can connect for KV cache transfers. The default value of <b>localhost</b> does not allow connections from other pods. |

### Additional resources

- [vLLM NixlConnector usage guide](#)

## CHAPTER 17. MONITOR LLM-D DEPLOYMENTS

You can monitor your llm-d deployments by using built-in observability dashboards and Prometheus metrics. Monitoring deployments helps you track traffic health, latency, resource use, and inference performance.

On OpenShift with the Cluster Observability Operator (COO) installed, Distributed Inference with llm-d deployments include built-in observability dashboards in the **Observe & Monitor** section. These dashboards provide pre-built views for monitoring traffic health, latency, and resource use without deploying a separate Grafana instance. You can also use Prometheus to collect metrics from Distributed Inference with llm-d deployments and build custom observability dashboards. You can import community Grafana dashboards to monitor token throughput, cache use, routing decisions, and inference performance. On OpenShift, the controller automatically creates **PodMonitor** and **ServiceMonitor** resources to scrape metrics from vLLM engines and the inference scheduler. On other Kubernetes platforms, you must configure Prometheus scrape targets manually to collect metrics from the vLLM and inference scheduler endpoints.

### 17.1. METRICS FOR LLM-D INFERENCE DEPLOYMENTS

Distributed Inference with llm-d deployments expose Prometheus metrics that you can use to monitor inference performance, track resource usage, and detect operational issues. On OpenShift, the controller automatically creates **PodMonitor** and **ServiceMonitor** resources to collect these metrics when you deploy an **LLMInferenceService**. On other Kubernetes platforms, you must configure Prometheus scrape targets manually to collect metrics from the vLLM engine and inference scheduler endpoints. You can build custom dashboards for service level objective (SLO) compliance and capacity planning.

{llm-d} components expose four distinct categories of Prometheus metrics:

#### vLLM engine metrics

The vLLM model servers expose metrics on port **8000** at the **/metrics** endpoint. These metrics are pod-level measurements of GPU execution time on individual model server replicas. They cover token throughput, request latency distributions, key-value (KV) cache utilization, request queue depth, and model loading status. vLLM metrics provide the foundation for understanding individual model server performance and resource consumption.

#### Inference scheduler metrics

The Endpoint Picker (EPP), also known as the inference scheduler, exposes metrics at the **/metrics** endpoint on the **metrics** service port. These metrics cover request routing decisions, endpoint scoring, scheduling latency, plugin processing times, and prefix cache indexing performance. EPP metrics help you understand how the scheduler distributes requests across model server replicas and how efficiently it routes traffic.

#### Service-level request metrics

The inference scheduler exposes higher-level metrics that track end-to-end inference performance across the service. These metrics are service-level measurements that include scheduler queue wait time and network latency between the scheduler and model server, reflecting actual user-perceived latency. They use the **llm\_d\_epp\_** prefix and include time to first token (TTFT), time per output token (TPOT), inter-token latency (ITL), request error rates, and request duration distributions. Service-level request metrics provide the basis for SLO-based alerting and compliance monitoring.

#### NIXL KV cache transfer metrics

In disaggregated serving deployments, the vLLM engine pods also expose NIXL-specific metrics on port **8000** at the **/metrics** endpoint. These metrics track KV cache transfer operations between prefill and decode pods, including transfer duration, bytes transferred, failed transfers, failed notifications,



and expired KV requests. NIXL metrics are only present when prefill/decode disaggregation is active. Use NIXL metrics to monitor the health of KV cache transfers, detect transfer failures, and verify that decode pods receive prompt tokens from prefill pods rather than computing them locally.

These four metric categories work together to provide a complete picture of your Distributed Inference with llm-d deployment health. Use vLLM metrics to monitor individual model server capacity, EPP metrics to evaluate routing efficiency, inference objective metrics to track overall service quality, and NIXL metrics to verify KV cache transfer health in disaggregated deployments.

## 17.2. VERIFY METRICS COLLECTION FOR LLM-D DEPLOYMENTS

On OpenShift, the controller automatically creates **PodMonitor** and **ServiceMonitor** resources to scrape metrics from vLLM engines and the inference scheduler when you deploy an **LLMInferenceService**. On other Kubernetes platforms, the controller does not create these resources. You must configure Prometheus scrape targets manually to collect metrics.

The following scrape targets expose llm-d metrics:

### vLLM engine metrics

Exposed on port **8000** at the **/metrics** endpoint on each model server pod.

### EPP and inference scheduler metrics

Exposed at the **/metrics** endpoint on the **metrics** service port of the scheduler service.

### Prerequisites

- You have a deployed Distributed Inference with llm-d inference deployment on Red Hat OpenShift AI 3.5.
- You have cluster administrator access.
- You have a Prometheus-based monitoring stack configured in your cluster.  
On OpenShift, enable user workload monitoring. The Prometheus Operator is included automatically. See [Configuring user workload monitoring](#).

On other Kubernetes platforms, you must have the [Prometheus Operator](#) installed if you want to use **PodMonitor** and **ServiceMonitor** resources, or a standalone Prometheus deployment configured with static or service-discovery-based scrape targets.

### Procedure

On OpenShift:

1. Verify that the **PodMonitor** resource for vLLM engine metrics exists in your deployment namespace:

```
$ kubectl get podmonitors -n <NAMESPACE> -l app.kubernetes.io/component=llm-monitoring
```

Replace **<NAMESPACE>** with the namespace where your llm-d deployment runs.

The output shows a **PodMonitor** resource with a name that begins with **kserve-llm-isvc-vllm-engine**. This resource scrapes vLLM metrics from port **8000** on all model server pods.

2. Verify that the **ServiceMonitor** resource for inference scheduler metrics exists in your deployment namespace:

```
$ kubectl get servicemonitors -n <NAMESPACE> -l app.kubernetes.io/component=llm-monitoring
```

The output shows a **ServiceMonitor** resource with a name that begins with **kserve-llm-isvc-scheduler**. This resource scrapes EPP and inference scheduler metrics from the **metrics** port on the scheduler service.

On other Kubernetes platforms without the Prometheus Operator:

1. Configure your Prometheus instance to scrape the following targets in your deployment namespace:

| Component                   | Port        | Path            | Pod selector                                                                                                                            |
|-----------------------------|-------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| vLLM engine                 | <b>8000</b> | <b>/metrics</b> | Pods with the label <b>app: &lt;POOL_NAME&gt;</b> , where <b>&lt;POOL_NAME&gt;</b> is the name of your <b>InferencePool</b> resource.   |
| EPP and inference scheduler | <b>9090</b> | <b>/metrics</b> | The EPP service, selected by the label <b>app: &lt;EPP_NAME&gt;</b> , where <b>&lt;EPP_NAME&gt;</b> is the name of your EPP deployment. |

On all platforms:

1. Verify that Prometheus is scraping vLLM metrics by running the following PromQL query against your Prometheus instance:

```
vllm:num_requests_running{namespace="<NAMESPACE>"}
```

Replace **<NAMESPACE>** with your deployment namespace.

On OpenShift, run this query from the web console by navigating to **Observe > Metrics**.

On other Kubernetes platforms, run this query by using your Prometheus web UI or API.

If the query returns results, Prometheus is successfully scraping vLLM metrics.

2. Verify that Prometheus is scraping EPP and inference scheduler metrics by running the following PromQL query:

```
llm_d_inference_scheduler_disagg_decision_total{namespace="<NAMESPACE>"}
```

Replace **<NAMESPACE>** with your deployment namespace.

If the query returns results, Prometheus is successfully scraping inference scheduler metrics.



## NOTE

Metrics might take up to 60 seconds to appear after the **LLMInferenceService** is deployed. If metrics do not appear, verify that your monitoring stack is configured to scrape metrics from the deployment namespace. On OpenShift, verify that user workload monitoring is enabled and that **PodMonitor** and **ServiceMonitor** resources are present. On other platforms, verify that the Prometheus scrape configuration targets the correct namespace, ports, and label selectors.

## 17.3. BUILT-IN OBSERVABILITY DASHBOARDS FOR LLM-D DEPLOYMENTS

Red Hat OpenShift AI 3.5 includes built-in observability dashboards for Distributed Inference with llm-d deployments. The dashboards appear in the OpenShift AI **Observe & Monitor** section and offer pre-built views for monitoring traffic health, latency, and resource utilization without deploying a separate Grafana instance.

The kserve-module Operator deploys the dashboards automatically as **PersesDashboard** custom resources when you install the Cluster Observability Operator (COO) on your OpenShift cluster. COO manages the Red Hat build of Perses server instance that renders the dashboards.



## NOTE

You must install COO manually from OperatorHub. OpenShift AI does not install COO automatically.

### 17.3.1. Default dashboards

#### LLM Traffic

Shows cluster-level request health indicators such as request rate, error rate, and token consumption. Use this dashboard to answer "Is my inference service healthy and handling traffic correctly?"

#### LLM Performance

Shows latency and cache diagnostics such as time to first token (TTFT) percentiles, inter-token latency (ITL), end-to-end latency, and KV cache hit rate. Use this dashboard to answer "Are my users experiencing acceptable latency?" and "Are my models performing as well as expected?"

#### LLM Utilization

Shows per-replica resource consumption such as GPU utilization, CPU utilization, memory utilization, KV cache utilization, and request queue depth. Use this dashboard to answer "Are my replicas adequately resourced?"

### 17.3.2. Drill-down investigation workflow

The three dashboards support a structured investigation workflow for diagnosing issues with your Distributed Inference with llm-d deployments:

1. Start with **LLM Traffic** to detect error spikes, throughput drops, or abnormal token patterns that indicate a problem.
2. Move to **LLM Performance** to investigate whether the deployment meets latency targets and whether KV cache hit rates explain any latency regressions.

3. Drill into **LLM Utilization** to diagnose per-replica GPU, CPU, memory, and KV cache saturation as root causes.

### 17.3.3. Common diagnostic patterns

The following patterns help you interpret metric combinations across dashboards.

#### Latency issue patterns

- High TTFT with normal inter-token latency indicates a prefill bottleneck from prompt processing or KV cache misses.
- High inter-token latency with normal TTFT indicates a decode bottleneck from token generation contention.
- Normal TTFT and inter-token latency with high end-to-end latency indicates scheduling delays. Check request queue depth in **LLM Utilization**.

#### Queue saturation

In **LLM Utilization**, compare running requests and waiting requests per pod. A steady running count with waiting requests near zero is healthy. Growing waiting requests indicate the pod cannot keep up. If queue depth is high across all pods, scale up. If queue depth is high on only one pod, investigate whether that node has a hardware issue.

#### KV cache pressure

A sudden drop in KV cache hit rate in **LLM Performance** often correlates with a TTFT regression. If KV cache usage in **LLM Performance** is at 100%, cache evictions have started. If preemption rates from the PromQL drill-down queries are high alongside low cache hit rates, memory pressure is the root cause.

#### Pod outlier detection

If aggregate metrics in **LLM Traffic** look healthy but users report intermittent slowness, check per-pod metrics in **LLM Utilization**. A single pod with higher latency, lower cache hit rate, or higher queue depth than its peers indicates unbalanced routing or a hardware issue on that node.

### 17.3.4. Built-in dashboards compared to community Grafana dashboards

OpenShift AI provides two approaches for monitoring Distributed Inference with llm-d deployments with dashboards. Both approaches source from the same underlying Prometheus metrics.

#### Built-in Perses dashboards

Deployed automatically on OpenShift when COO is installed. Rendered in the **Observe & Monitor** section. Require no separate infrastructure.

#### Community Grafana dashboards

Require a separately deployed Grafana instance and manual import of dashboard JSON files. Available on any Kubernetes platform. Offer a broader panel set including diagnostic drill-down and operational views.

On OpenShift with COO installed, use the built-in dashboards as your primary monitoring path. Community Grafana dashboards remain available for non-OpenShift platforms, environments where Grafana is already deployed, or when you need supplementary diagnostic views beyond the built-in panels.

## 17.4. ACCESS LLM-D OBSERVABILITY DASHBOARDS

When your llm-d deployment shows unexpected behavior or performance degradation, access the built-in observability dashboards to diagnose the root cause using traffic, performance, and utilization metrics.

## Prerequisites

- You have installed the Cluster Observability Operator (COO) from OperatorHub. For more information, see [Installing the Cluster Observability Operator](#).
- If your COO version is earlier than 1.5, verify that you have cluster administrator privileges. COO 1.5 or later does not require administrator privileges.
- You have enabled the observability dashboard in OpenShift AI. For more information, see [Enable the observability dashboard in the UI](#).
- You have at least one **LLMInferenceService** deployed and serving traffic.
- You have enabled metrics collection for your Distributed Inference with llm-d deployment.

## Procedure

1. Verify that COO is installed:

```
$ oc get subscription cluster-observability-operator -n openshift-cluster-observability-operator
```

2. Verify that the Perses server is running:

```
$ oc get pods -n openshift-cluster-observability-operator -l app.kubernetes.io/name=perses
```

3. Log in to the OpenShift AI dashboard as a cluster administrator.
4. Click **Observe & Monitor** → **Dashboard**.
5. From the namespace filter, select the namespace where your Distributed Inference with llm-d deployment is running.
6. From the model name filter, select a specific model or leave as **All** to view data for all models.
7. Adjust the time range as needed for your investigation window.
8. Use the drill-down investigation workflow to diagnose issues:
  - a. In the **LLM Traffic** dashboard, review request rate, error rate, and token throughput to identify anomalies.
  - b. In the **LLM Performance** dashboard, review time to first token (TTFT) percentiles, inter-token latency, and KV cache hit rate to diagnose latency issues.
  - c. In the **LLM Utilization** dashboard, review per-replica GPU, CPU, memory, and KV cache utilization to identify resource bottlenecks.
9. For more information about queries, see [PromQL queries for llm-d monitoring](#).

## Verification

- All three dashboards display data for the selected namespace: LLM Traffic, LLM Performance, and LLM Utilization.
- If the panels on the dashboards show **No data**, verify the following conditions:
  - Metrics collection is active for your Distributed Inference with llm-d deployment.
  - The correct namespace is selected.
  - Your **LLMInferenceService** is actively serving requests.



#### NOTE

If the **Error Rate** panel shows **No data** but other panels display data, no errors have occurred. This is expected behavior.

## 17.5. DASHBOARD PANELS AND METRICS FOR LLM-D OBSERVABILITY

The three built-in observability dashboards for Distributed Inference with llm-d deployments contain panels that visualize Prometheus metrics from vLLM engines and the Endpoint Picker (EPP), also known as the inference scheduler. The following tables describe what each panel measures and how to interpret its data.

You can filter by namespace (**exported\_namespace** label) and model name ( **model\_name** label). Selecting a filter limits all panels to the selected namespace or model.

### 17.5.1. LLM Traffic dashboard

The LLM Traffic dashboard provides cluster-level request health indicators for your llm-d deployments.

Table 17.1. LLM Traffic panels

| Panel        | Metrics                                                           | What it answers                                         | Interpretation guidance                                                                                                                                    |
|--------------|-------------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Request Rate | Derived from <b>vllm:request_success_total</b> , request counters | How many requests per second is my deployment handling? | A stable or growing rate matching expected load is healthy. A sudden drop might indicate upstream issues or routing failures.                              |
| Error Rate   | <b>inference_model_request_error_total</b>                        | Are requests failing?                                   | An error rate near zero is healthy. Spikes in error rate require immediate investigation. Correlate with latency metrics in the LLM Performance dashboard. |

| Panel            | Metrics                                                       | What it answers                    | Interpretation guidance                                                                                                                                                                 |
|------------------|---------------------------------------------------------------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Token Throughput | <b>vllm:prompt_tokens_total, vllm:generation_tokens_total</b> | How much work is the system doing? | Token volume consistent with workload patterns is healthy. An unchanged token count despite active clients might indicate that the system rejects requests before they reach the model. |

### 17.5.2. LLM Performance dashboard

The LLM Performance dashboard provides latency percentiles and cache diagnostics for your llm-d deployments.

**Table 17.2. LLM Performance panels**

| Panel                      | Metrics                                                        | What it answers                                            | Interpretation guidance                                                                                                                                                                                                                                                      |
|----------------------------|----------------------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Time to First Token (TTFT) | <b>vllm:time_to_first_token_seconds_bucket</b> (P50, P95, P99) | How long do users wait before the model starts responding? | TTFT within your service-level objective (SLO) threshold is healthy. A spiking P99 might indicate prefill queue saturation or KV cache misses. Correlate with KV Cache Hit Rate: a drop in cache hits often precedes a TTFT regression.                                      |
| End-to-End Latency         | <b>vllm:e2e_request_latency_seconds_bucket</b> (P50, P95, P99) | How long does the full request lifecycle take?             | Stable latency matching expected generation lengths is healthy. Increasing latency without increased output length suggests a bottleneck. If TTFT and inter-token latency are both normal, check request queue depth in the LLM Utilization dashboard for scheduling delays. |

| Panel                     | Metrics                                                                         | What it answers                                     | Interpretation guidance                                                                                                                                                                                                   |
|---------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Inter-Token Latency (ITL) | <b>vllm:time_per_output_token_seconds_bucket</b> (P50, P95)                     | Is token generation smooth for streaming responses? | Consistent ITL indicates smooth streaming. High variance causes choppy user experience and might indicate decode-phase contention.                                                                                        |
| KV Cache Hit Rate         | <b>vllm:prefix_cache_hits_total</b> ,<br><b>vllm:prefix_cache_queries_total</b> | Is prefix caching working effectively?              | A hit rate above 60-80% is healthy for workloads with repeated prefixes. A sudden drop in hit rate correlates with TTFT regressions.                                                                                      |
| KV Cache Usage            | <b>vllm:kv_cache_usage_perc</b>                                                 | How full is the KV cache?                           | Usage below 90% is healthy. Usage at 100% indicates evictions have started, which degrades hit rate and increases TTFT. Correlate with the KV Cache Hit Rate panel: usage at 100% causes evictions that degrade hit rate. |

### 17.5.3. LLM Utilization dashboard

The LLM Utilization dashboard provides per-replica resource consumption metrics for your llm-d deployments.

Table 17.3. LLM Utilization panels

| Panel           | Metrics                     | What it answers                                  | Interpretation guidance                                                                                                                                                                                  |
|-----------------|-----------------------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GPU Utilization | <b>dcgm_gpu_utilization</b> | How much GPU capacity does each replica consume? | Utilization between 40-80% is healthy. Sustained utilization above 95% indicates saturation. If one replica is significantly higher than others, investigate routing balance in the inference scheduler. |



| Panel                        | Metrics                                                                | What it answers                                 | Interpretation guidance                                                                                                                                           |
|------------------------------|------------------------------------------------------------------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Request Queue Depth          | <b>vllm:num_requests_running</b> ,<br><b>vllm:num_requests_waiting</b> | Are replicas keeping up with incoming requests? | Running requests at a steady level with waiting requests near zero is healthy. Growing waiting requests indicate the replica cannot process requests fast enough. |
| Per-pod KV Cache Utilization | <b>vllm:kv_cache_usage_perc</b>                                        | Is the KV cache full on specific replicas?      | Per-pod cache usage below 90% is healthy. If specific pods show full cache while others do not, investigate routing and prefix affinity configuration.            |



#### NOTE

GPU utilization metrics require the NVIDIA Data Center GPU Manager (DCGM) exporter. If you have not deployed DCGM, the GPU Utilization panel does not display data.

### 17.5.4. Metric prefixes in dashboard panels

llm-d metrics are available in two locations in the OpenShift UI, each using different metric naming:

#### OpenShift AI dashboard (Observe & Monitor)

The Perses dashboards query metrics from multiple sources:

- **kserve\_vllm::** vLLM engine metrics: TTFT, latency, cache, queue depth. These are recording rules that map to underlying **vllm:** metrics with normalized labels: **exported\_namespace** instead of **namespace**, and **k8s\_pod\_name** instead of **pod**.
- **inference\_model\_request\_error\_total**: Gateway API Inference Extension metrics for error rate. Queried directly, no recording rules.
- **llm\_d\_epp\_\***: EPP/scheduler metrics for scheduling latency and pool health. Queried directly, no recording rules.
- **accelerator\_gpu\_utilization**: GPU compute utilization. Queried directly.

#### OpenShift Console (Observe > Dashboards)

The Performance dashboards query raw Prometheus metric names such as **vllm\_gpu\_cache\_usage\_perc** and **vllm\_num\_requests\_waiting** directly, without recording rules.

## 17.6. PROMQL QUERIES FOR LLM-D MONITORING

You can use the following PromQL queries to monitor your Distributed Inference with llm-d deployments. The queries are organized into two groups: immediate failure and saturation indicators that drive operational alerts, and diagnostic drill-down queries that support deeper investigation.

### 17.6.1. Failure and saturation indicators

Use the following queries for dashboards and alerts that track service health and capacity limits.

**Table 17.4. Failure and saturation PromQL queries**

| Operational concern                             | PromQL query                                                                                                                               |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Overall error rate                              | <code>sum(rate(llm_d_epp_request_error_total[5m])) /<br/>sum(rate(llm_d_epp_request_total[5m]))</code>                                     |
| Per-model error rate                            | <code>sum by(model_name)<br/>(rate(llm_d_epp_request_error_total[5m])) / sum<br/>by(model_name) (rate(llm_d_epp_request_total[5m]))</code> |
| Error rate by error code                        | <code>sum by(error_code)<br/>(rate(llm_d_epp_request_error_total[5m]))</code>                                                              |
| Request rate by model                           | <code>sum by(model_name, target_model_name)<br/>(rate(llm_d_epp_request_total[5m]))</code>                                                 |
| Overall latency P99                             | <code>histogram_quantile(0.99, sum by(le)<br/>(rate(llm_d_epp_request_duration_seconds_bucket[5m])))</code>                                |
| Overall latency P90                             | <code>histogram_quantile(0.90, sum by(le)<br/>(rate(llm_d_epp_request_duration_seconds_bucket[5m])))</code>                                |
| Model-specific TTFT P99                         | <code>histogram_quantile(0.99, sum by(le, model_name)<br/>(rate(vllm:time_to_first_token_seconds_bucket[5m])))</code>                      |
| Model-specific time per output token (TPOT) P99 | <code>histogram_quantile(0.99, sum by(le, model_name)<br/>(rate(vllm:inter_token_latency_seconds_bucket[5m])))</code>                      |
| Streaming ITL P99                               | <code>histogram_quantile(0.99, sum by(le)<br/>(rate(llm_d_epp_request_streaming_itl_seconds_bucket[5m])<br/>)</code>                       |
| Streaming ITL P90                               | <code>histogram_quantile(0.90, sum by(le)<br/>(rate(llm_d_epp_request_streaming_itl_seconds_bucket[5m])<br/>)</code>                       |
| EPP end-to-end latency P99                      | <code>histogram_quantile(0.99, sum by(le)<br/>(rate(llm_d_epp_scheduler_e2e_duration_seconds_bucket[5<br/>m])))</code>                     |
| Plugin processing latency P99                   | <code>histogram_quantile(0.99, sum by(le, plugin_type)<br/>(rate(llm_d_epp_plugin_duration_seconds_bucket[5m])))</code>                    |
| Scheduler health                                | <code>avg_over_time(up{job=~".*epp.*"}[5m])</code>                                                                                         |

| Operational concern                   | PromQL query                                                                  |
|---------------------------------------|-------------------------------------------------------------------------------|
| Request preemptions per vLLM instance | <b>sum by(pod, instance)</b><br><b>(rate(vllm:num_preemptions_total[5m]))</b> |

### 17.6.2. Diagnostic drill-down queries

Use the following queries for deeper investigation when failure and saturation indicators signal issues.

**Table 17.5. Model serving and scaling queries**

| Operational concern                   | PromQL query                                                                                                                     |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| KV cache utilization by pod           | <b>avg by(pod, model_name) (vllm:kv_cache_usage_perc)</b>                                                                        |
| Request queue length by pod           | <b>sum by(pod, model_name) (vllm:num_requests_waiting)</b>                                                                       |
| Model throughput in tokens per second | <b>sum by(model_name, pod)</b><br><b>(rate(vllm:prompt_tokens_total[5m]) +</b><br><b>rate(vllm:generation_tokens_total[5m]))</b> |
| Generation token rate by pod          | <b>sum by(model_name, pod)</b><br><b>(rate(vllm:generation_tokens_total[5m]))</b>                                                |
| Active requests per pod               | <b>avg by(pod) (vllm:num_requests_running)</b>                                                                                   |

**Table 17.6. Routing and load balancing queries**

| Operational concern               | PromQL query                                                                                               |
|-----------------------------------|------------------------------------------------------------------------------------------------------------|
| Request distribution per instance | <b>sum by(pod)</b><br><b>(rate(llm_d_epp_request_total{target_model_name!=""}[5m]))</b>                    |
| Token distribution across pods    | <b>sum by(pod) (rate(vllm:prompt_tokens_total[5m]) +</b><br><b>rate(vllm:generation_tokens_total[5m]))</b> |
| Routing decision latency P99      | <b>histogram_quantile(0.99, sum by(le)</b><br><b>(rate(llm_d_epp_plugin_duration_seconds_bucket[5m])))</b> |

**Table 17.7. Prefix caching queries**

| Operational concern   | PromQL query                                                                                                  |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| Prefix cache hit rate | <b>sum(rate(vllm:prefix_cache_hits_total[5m])) /</b><br><b>sum(rate(vllm:prefix_cache_queries_total[5m]))</b> |

| Operational concern                | PromQL query                                                                                                          |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Per-instance prefix cache hit rate | <b>sum by(pod) (rate(vllm:prefix_cache_hits_total[5m])) / sum by(pod) (rate(vllm:prefix_cache_queries_total[5m]))</b> |
| KV cache utilization percentage    | <b>avg by(pod, model_name) (vllm:kv_cache_usage_perc * 100)</b>                                                       |
| EPP prefix indexer size            | <b>llm_d_epp_prefix_indexer_size</b>                                                                                  |
| EPP prefix indexer hit ratio P90   | <b>histogram_quantile(0.90, sum by(le) (rate(llm_d_epp_prefix_indexer_hit_ratio_bucket[5m])))</b>                     |

Table 17.8. Prefill/decode disaggregation queries

| Operational concern                  | PromQL query                                                                                                                                                           |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prefill worker utilization           | <b>avg by(pod) (vllm:num_requests_running{pod=~".*prefill.*"})</b>                                                                                                     |
| Decode worker KV cache utilization   | <b>avg by(pod) (vllm:kv_cache_usage_perc{pod=~".*kserve.*",pod!~".*prefill.*"})</b>                                                                                    |
| Prefill queue length                 | <b>sum by(pod) (vllm:num_requests_waiting{pod=~".*prefill.*"})</b>                                                                                                     |
| Disaggregation decision rate by type | <b>sum by(decision_type) (rate(llm_d_inference_scheduler_disagg_decision_total[5m]))</b>                                                                               |
| Disaggregation decision ratio        | <b>sum(rate(llm_d_inference_scheduler_disagg_decision_total{decision_type="prefill-decode"}[5m])) / sum(rate(llm_d_inference_scheduler_disagg_decision_total[5m]))</b> |

Table 17.9. NIXL KV cache transfer queries

| Operational concern               | PromQL query                                                                                   |
|-----------------------------------|------------------------------------------------------------------------------------------------|
| NIXL transfer rate                | <b>sum(rate(vllm:nixl_xfer_time_seconds_count[5m]))</b>                                        |
| NIXL average transfer duration    | <b>rate(vllm:nixl_xfer_time_seconds_sum[5m]) / rate(vllm:nixl_xfer_time_seconds_count[5m])</b> |
| NIXL bytes transferred per second | <b>sum(rate(vllm:nixl_bytes_transferred_sum[5m]))</b>                                          |
| NIXL failed transfer rate         | <b>sum(rate(vllm:nixl_num_failed_transfers_total[5m]))</b>                                     |

| Operational concern              | PromQL query                                                                                                                                    |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| NIXL failed notification rate    | <code>sum(rate(vllm:nixl_num_failed_notifications_total[5m]))</code>                                                                            |
| NIXL failed transfer ratio       | <code>sum(rate(vllm:nixl_num_failed_transfers_total[5m])) /<br/>sum(rate(vllm:nixl_xfer_time_seconds_count[5m]))</code>                         |
| Prompt token source distribution | <code>sum by(source)<br/>(rate(vllm:prompt_tokens_by_source_total[5m]))</code>                                                                  |
| Decode queue depth               | <code>sum by(pod)<br/>(vllm:num_requests_waiting{pod=~".*kserve.*",pod!~".*prefill.*"})</code>                                                  |
| Prefill latency P99              | <code>histogram_quantile(0.99, sum by(le)<br/>(rate(vllm:time_to_first_token_seconds_bucket{pod=~".*prefill.*"}[5m])))</code>                   |
| Decode inter-token latency P99   | <code>histogram_quantile(0.99, sum by(le)<br/>(rate(vllm:inter_token_latency_seconds_bucket{pod=~".*kserve.*",pod!~".*prefill.*"}[5m])))</code> |

## 17.7. IMPORT GRAFANA DASHBOARDS FOR LLM-D

You can import community Grafana dashboards to get a baseline monitoring view of your Distributed Inference with llm-d deployments. The dashboards provide prebuilt visualizations for vLLM performance, failure and saturation indicators, diagnostic drill-downs, KV cache performance, and prefill/decode disaggregation metrics.



### NOTE

On OpenShift with the Cluster Observability Operator installed, built-in observability dashboards appear automatically in the OpenShift AI **Observe and Monitor** section without a separate Grafana instance. The community Grafana dashboards described in this procedure remain available for non-OpenShift Kubernetes platforms, environments where Grafana is already deployed, or when you need supplementary diagnostic views beyond the built-in panels.

### Prerequisites

- You have a Grafana instance deployed and connected to a Prometheus data source that collects llm-d metrics. Grafana is not included with OpenShift. You must deploy Grafana separately, for example, by using the Grafana community Operator.
- Metrics collection is enabled for your Distributed Inference with llm-d deployment.

### Procedure

1. Download the community Grafana dashboard JSON files from the llm-d project. The following dashboards are available:

### llm-d vLLM Overview

General vLLM metrics overview for monitoring llm-d inference servers. Download from [llm-d-vllm-overview.json](#).

### llm-d Failure and Saturation Indicators

Key failure and saturation indicators for identifying system issues and capacity constraints. Download from [llm-d-failure-saturation-dashboard.json](#).

### llm-d Diagnostic Drill-Down

Detailed diagnostic metrics for investigating performance issues. Download from [llm-d-diagnostic-drilldown-dashboard.json](#).

### llm-d Performance Dashboard

Performance metrics including KV cache usage. Download from [llm-d-performance-kv-cache.json](#).

### P/D Coordinator Metrics

Prefill/decode disaggregation performance metrics, including vLLM end-to-end latency, prefill duration, decode duration, and phase breakdown. Download from [llm-d-pd-coordinator-metrics.json](#).

### WVA Operational Dashboard

Operational metrics for the workload variant autoscaler (WVA) controller, including models processed and optimization cycle duration. Download from [operational-dashboard.json](#).

2. In the Grafana web interface, navigate to **Dashboards > New > Import**.
3. Upload a dashboard JSON file or paste the JSON content into the import dialog.
4. Select the Prometheus data source that collects metrics from your llm-d deployment.
5. Click **Import** to create the dashboard.
6. Repeat the import process for each dashboard JSON file.
7. After you import the WVA Operational Dashboard, configure the following settings:
  - Under **Datasource**, select the Prometheus data source that collects WVA metrics.
  - Under **namespace\_label**, leave the value set to **exported\_namespace**.  
For more information about namespace labels in WVA metrics, see [Understanding namespace labels in metrics](#).

### Verification

- Navigate to a newly imported dashboard and verify that the panels display data from your llm-d deployment. If a "No data" message is displayed, verify that the Prometheus data source is correctly configured and that a cluster administrator has enabled user workload monitoring. On non-OpenShift platforms, verify that Prometheus scrape targets are configured for your llm-d namespace.



#### NOTE

In disconnected environments, download the dashboard JSON files from a connected network and transfer them to a workstation with access to the Grafana instance. The dashboards do not reference external data sources, so they function in disconnected environments after Prometheus and Grafana are available.

## 17.8. VLLM METRICS FOR LLM-D

The vLLM model servers in Distributed Inference with llm-d deployments expose Prometheus metrics on port **8000** at the **/metrics** endpoint. You can use these metrics to monitor token throughput, request latency, KV cache usage, and request queue depth for individual model server replicas.

### Token throughput metrics

Table 17.10. Token throughput metrics

| Metric name                             | Type      | Labels            | Description                                                                                                   |
|-----------------------------------------|-----------|-------------------|---------------------------------------------------------------------------------------------------------------|
| <b>vllm:prompt_tokens_total</b>         | counter   | <b>model_name</b> | The total number of prompt tokens processed by the model server.                                              |
| <b>vllm:generation_tokens_total</b>     | counter   | <b>model_name</b> | The total number of generation tokens produced by the model server.                                           |
| <b>vllm:e2e_request_latency_seconds</b> | histogram | <b>model_name</b> | The end-to-end request latency distribution in seconds, measured from request receipt to response completion. |

### Latency metrics

Table 17.11. Latency metrics

| Metric name                             | Type      | Labels            | Description                                                                                                                             |
|-----------------------------------------|-----------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>vllm:time_to_first_token_seconds</b> | histogram | <b>model_name</b> | The time to first token (TTFT) distribution in seconds. This measures the latency from request submission to the first generated token. |
| <b>vllm:inter_token_latency_seconds</b> | histogram | <b>model_name</b> | The inter-token latency distribution in seconds. This measures the time between consecutive output tokens.                              |

### KV cache metrics

Table 17.12. KV cache metrics

| Metric name                     | Type  | Labels            | Description                                                                      |
|---------------------------------|-------|-------------------|----------------------------------------------------------------------------------|
| <b>vllm:kv_cache_usage_perc</b> | gauge | <b>model_name</b> | The GPU KV cache utilization as a percentage between <b>0.0</b> and <b>1.0</b> . |

| Metric name                       | Type    | Labels            | Description                                                                                                                                                                       |
|-----------------------------------|---------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>vllm:num_preemptions_total</b> | counter | <b>model_name</b> | The total number of request preemptions caused by KV cache pressure. Preemptions occur when the cache is full and running requests must be evicted to make room for new requests. |

## Request queue metrics

Table 17.13. Request queue metrics

| Metric name                         | Type    | Labels                                     | Description                                                                                      |
|-------------------------------------|---------|--------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>vllm:num_requests_running</b>    | gauge   | <b>model_name</b>                          | The current number of requests being actively processed by the model server.                     |
| <b>vllm:num_requests_waiting</b>    | gauge   | <b>model_name</b>                          | The current number of requests waiting in the queue for processing.                              |
| <b>vllm:request_successes_total</b> | counter | <b>engine, model_name, finished_reason</b> | The total number of successfully completed inference requests, broken down by completion reason. |

## Prefix cache metrics

Table 17.14. Prefix cache metrics

| Metric name                            | Type    | Labels            | Description                                                                                                                                    |
|----------------------------------------|---------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>vllm:prefix_cache_hits_total</b>    | counter | <b>model_name</b> | The total number of prefix cache hits. A cache hit occurs when a request prompt matches cached key-value data, avoiding redundant computation. |
| <b>vllm:prefix_cache_queries_total</b> | counter | <b>model_name</b> | The total number of prefix cache queries. Use this metric with <b>prefix_cache_hits_total</b> to calculate the cache hit rate.                 |

## 17.9. EPP AND INFERENCE SCHEDULER METRICS FOR LLM-D

The Endpoint Picker (EPP), or the inference scheduler, in Distributed Inference with llm-d deployments expose Prometheus metrics at the **/metrics** endpoint on the **metrics** service port. You can use these metrics to monitor request routing decisions, scheduling latency, inference performance, and prefix cache indexing performance.





## NOTE

Metrics in Distributed Inference with llm-d use the **llm\_d\_epp\_** prefix. The **inference\_objective\_**, **inference\_extension\_**, and **inference\_pool\_** metric prefixes are deprecated. These prefixes remain available for backward compatibility.

### Inference objective metrics

The service-level request metrics track end-to-end inference performance at the Endpoint Picker (EPP). These metrics measure user-perceived latency, which includes scheduler queue wait time and network latency between the scheduler and the model server. These metrics use the **llm\_d\_epp\_request\_** prefix.

Table 17.15. Service-level request metrics

| Metric name                                     | Type      | Labels                                                                                               | Description                                                                                                                                                                                                                                                                 |
|-------------------------------------------------|-----------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_request_total</b>                  | counter   | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b>                       | The total number of inference requests processed by the scheduler.                                                                                                                                                                                                          |
| <b>llm_d_epp_request_error_total</b>            | counter   | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority,</b><br><b>error_code</b> | The total number of inference request errors, broken down by error code.                                                                                                                                                                                                    |
| <b>llm_d_epp_request_duration_seconds</b>       | histogram | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b>                       | The end-to-end inference request duration distribution in seconds.                                                                                                                                                                                                          |
| <b>llm_d_epp_request_ttft_seconds</b>           | histogram | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b>                       | The time to first token (TTFT) distribution in seconds, measured from request receipt to first response byte. For non-streaming requests, this equals the total request duration.                                                                                           |
| <b>llm_d_epp_request_streaming_tpot_seconds</b> | histogram | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b>                       | The average time per output token (TPOT) distribution in seconds for streaming requests, computed as (end-to-end latency - TTFT) / (output tokens - 1).                                                                                                                     |
| <b>llm_d_epp_request_streaming_itl_seconds</b>  | histogram | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b>                       | The inter-token latency (ITL) distribution in seconds for streaming requests, measured as the time between consecutive response body chunks. Use ITL to detect decode-phase lag spikes that TPOT averaging can mask. The EPP emits this metric only for streaming requests. |

| Metric name                            | Type      | Labels                                                                         | Description                                      |
|----------------------------------------|-----------|--------------------------------------------------------------------------------|--------------------------------------------------|
| <b>llm_d_epp_request_input_tokens</b>  | histogram | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b> | The input token count distribution per request.  |
| <b>llm_d_epp_request_output_tokens</b> | histogram | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b> | The output token count distribution per request. |
| <b>llm_d_epp_request_running</b>       | gauge     | <b>model_name,</b><br><b>target_model_name,</b><br><b>fairness_id,priority</b> | The current number of active inference requests. |

### Scheduling and routing metrics

The scheduling and routing metrics track the internal performance of the Endpoint Picker. These metrics use the **llm\_d\_epp\_** prefix.

Table 17.16. Scheduling and routing metrics

| Metric name                                     | Type      | Labels                                                                                        | Description                                                                                                                                                                       |
|-------------------------------------------------|-----------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_scheduler_e2e_duration_seconds</b> | histogram | None                                                                                          | The end-to-end scheduling duration in seconds, measured from request receipt to endpoint selection.                                                                               |
| <b>llm_d_epp_scheduler_attempts_total</b>       | counter   | <b>status,</b><br><b>target_model_name,</b><br><b>endpoint_name,</b><br><b>namespace,port</b> | The total number of scheduling attempts, broken down by status: <b>success</b> or <b>failure</b> .                                                                                |
| <b>llm_d_epp_plugin_duration_seconds</b>        | histogram | <b>extension_point,</b><br><b>plugin_type,</b><br><b>plugin_name</b>                          | The plugin processing duration in seconds, broken down by extension point, plugin type, and plugin name. Use this metric to identify slow plugins that impact scheduling latency. |
| <b>llm_d_epp_model_rewrite_decisions_total</b>  | counter   | <b>model_rewrite_name,</b><br><b>model_name,</b><br><b>target_model</b>                       | The total number of model rewrite decisions made by the scheduler.                                                                                                                |

### Prefix cache indexer metrics

The prefix cache indexer metrics track KV cache prefix matching performance in the scheduler. These metrics use the **llm\_d\_epp\_** prefix.

Table 17.17. Prefix cache indexer metrics

| Metric name                                | Type      | Labels | Description                                                                                                                                           |
|--------------------------------------------|-----------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_prefix_in_dexer_size</b>      | gauge     | None   | The current size of the prefix cache index maintained by the scheduler.                                                                               |
| <b>llm_d_epp_prefix_in_dexer_hit_ratio</b> | histogram | None   | The prefix cache hit ratio distribution. A higher hit ratio indicates that requests frequently match cached prefixes, reducing redundant computation. |
| <b>llm_d_epp_prefix_in_dexer_hit_bytes</b> | histogram | None   | The prefix cache hit size distribution in bytes. Larger hit sizes indicate more effective prefix reuse.                                               |

### Inference pool metrics

The inference pool metrics track aggregate statistics across all endpoints in a pool. These metrics use the **llm\_d\_epp\_** prefix.

Table 17.18. Inference pool metrics

| Metric name                                   | Type  | Labels                             | Description                                                                        |
|-----------------------------------------------|-------|------------------------------------|------------------------------------------------------------------------------------|
| <b>llm_d_epp_average_kv_cache_utilization</b> | gauge | <b>name</b>                        | The average KV cache utilization across all endpoints in the inference pool.       |
| <b>llm_d_epp_average_queue_size</b>           | gauge | <b>name</b>                        | The average request queue size across all endpoints in the inference pool.         |
| <b>llm_d_epp_average_running_requests</b>     | gauge | <b>name</b>                        | The average number of running requests across all endpoints in the inference pool. |
| <b>llm_d_epp_ready_endpoints</b>              | gauge | <b>name</b>                        | The number of ready endpoints in the inference pool.                               |
| <b>llm_d_epp_per_endpoint_queue_size</b>      | gauge | <b>name, model_server_endpoint</b> | The request queue size for a specific endpoint in the inference pool.              |

### Flow control metrics

The flow control metrics track priority-based queuing, request dispatching, and pool saturation when flow control is enabled. These metrics use the **llm\_d\_epp\_flow\_control\_** prefix.

Table 17.19. Flow control metrics

| Metric name                                                    | Type      | Labels                                                                               | Description                                                                                                                                                                                                                 |
|----------------------------------------------------------------|-----------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>llm_d_epp_flow_control_queue_size</b>                       | gauge     | <b>fairness_id, priority, inference_pool, model_name, target_model_name</b>          | The current number of requests actively held in the flow control queue, broken down by fairness group and priority band.                                                                                                    |
| <b>llm_d_epp_flow_control_queue_bytes</b>                      | gauge     | <b>fairness_id, priority, inference_pool, model_name, target_model_name</b>          | The current total size in bytes of requests actively held in the flow control queue, broken down by fairness group and priority band.                                                                                       |
| <b>llm_d_epp_flow_control_pool_saturation</b>                  | gauge     | <b>inference_pool</b>                                                                | The current saturation level of the inference pool, ranging from 0.0 (empty) to 1.0 (fully saturated). When saturation reaches the configured threshold, dispatch halts for all priority bands.                             |
| <b>llm_d_epp_flow_control_requests_total</b>                   | counter   | <b>outcome, priority, inference_pool</b>                                             | The total number of requests processed by the flow control layer, broken down by outcome:<br><b>Dispatched, RejectedCapacity, RejectedNoEndpoints, RejectedOther, EvictedTTL, EvictedContextCancelled, or EvictedOther.</b> |
| <b>llm_d_epp_flow_control_request_queue_duration_seconds</b>   | histogram | <b>fairness_id, priority, outcome, inference_pool, model_name, target_model_name</b> | The distribution of total time that requests wait in the flow control layer, measured from enqueue to final outcome.                                                                                                        |
| <b>llm_d_epp_flow_control_dispatch_cycle_duration_seconds</b>  | histogram | None                                                                                 | The distribution of time taken for each internal dispatch cycle in the flow control layer. Use this metric to identify dispatch processing bottlenecks.                                                                     |
| <b>llm_d_epp_flow_control_request_enqueue_duration_seconds</b> | histogram | <b>fairness_id, priority, outcome</b>                                                | The distribution of time taken to enqueue requests into the flow control layer.                                                                                                                                             |

## 17.10. MONITOR BATCH INFERENCE WORKLOADS

You can monitor batch inference workloads by collecting Prometheus metrics from the batch gateway components and configuring alerts for batch processing failures. The batch gateway API server, processor, and garbage collector expose metrics that you can use to track job throughput, queue depth, processing latency, and error rates.

## Prerequisites

- The batch inference subsystem is deployed and configured. For more information, see [Configure batch inference for Distributed Inference with llm-d](#).
- You have a Prometheus-based monitoring stack configured in your cluster. On OpenShift, enable user workload monitoring. See [Configuring user workload monitoring](#).
- You have cluster administrator access.

## Procedure

1. Verify that the **ServiceMonitor** resource for the batch gateway API server exists:

```
$ oc get servicemonitors -n <batch_gateway_namespace> -l
app.kubernetes.io/component=apiserver
```

Replace **<batch\_gateway\_namespace>** with the namespace where the batch gateway is deployed.

The **ServiceMonitor** scrapes API server metrics from port 8081. The Batch Gateway Operator creates this resource automatically during deployment. If the **ServiceMonitor** does not exist, verify that the Batch Gateway Operator deployment completed successfully.

2. Verify that the **PodMonitor** resource for the batch processor exists:

```
$ oc get podmonitors -n <batch_gateway_namespace> -l
app.kubernetes.io/component=processor
```

The **PodMonitor** scrapes processor metrics from port 9090. The Batch Gateway Operator creates this resource automatically during deployment. If the **PodMonitor** does not exist, verify that the Batch Gateway Operator deployment completed successfully.

3. Verify that the **PodMonitor** resource for the garbage collector exists:

```
$ oc get podmonitors -n <batch_gateway_namespace> -l app.kubernetes.io/component=gc
```

The **PodMonitor** scrapes garbage collector metrics from port 9091. The Batch Gateway Operator creates this resource automatically during deployment. If the **PodMonitor** does not exist, verify that the Batch Gateway Operator deployment completed successfully.



### NOTE

For PromQL queries and metric names, see the [batch gateway metrics implementation](#) in the repository. The metric definitions are in the **apiserver/metrics/**, **processor/metrics/**, and **gc/metrics/** subdirectories.

4. Optional: Import pre-built Grafana dashboards for batch gateway components. The batch gateway includes **ConfigMap** resources containing Grafana dashboard JSON definitions. Import these dashboards into your Grafana instance to visualize batch processing metrics.

```
$ oc get configmaps -n <batch_gateway_namespace> -l grafana_dashboard=1
```

## Verification

- Verify that the **ServiceMonitor** and **PodMonitor** resources exist and that Prometheus is configured to scrape them.

## Additional resources

- [Batch gateway repository](#)

## 17.11. CAPACITY PLANNING FOR DISAGGREGATED INFERENCE

You can use per-phase metrics from disaggregated inference deployments to identify scaling needs, detect transfer failures, and make data-driven decisions about prefill and decode pool sizing. Effective capacity planning requires monitoring the utilization balance between phases and the health of NIXL KV cache transfers.

### Utilization imbalance detection

In a disaggregated deployment, prefill and decode pods have different resource consumption patterns. Monitoring the utilization of each phase independently reveals when one side becomes a bottleneck.

Compare prefill queue depth against decode KV cache utilization to identify imbalances:

- High prefill queue depth with low decode KV cache utilization indicates that the prefill pool is undersized relative to the decode pool. Scale the prefill pool up.
- Low prefill queue depth with high decode KV cache utilization indicates that the decode pool is under pressure while prefill capacity is idle. Scale the decode pool up.
- Both metrics consistently high indicates that the overall deployment needs more capacity across both pools.

Use the following PromQL queries to compare utilization:

```
Prefill queue depth
sum by(pod) (vllm:num_requests_waiting{pod=~".*prefill.*"})

Decode KV cache utilization
avg by(pod) (vllm:kv_cache_usage_perc{pod=~".*kserve.*",pod!~".*prefill.*"})
```

### KV cache transfer health

NIXL KV cache transfer failures can indicate networking issues, NIXL hash incompatibilities during upgrades, or resource contention. A healthy disaggregated deployment shows zero or near-zero failed transfers.

Use the following query to calculate the handoff failure rate:

```
sum(rate(vllm:nixl_num_failed_transfers_total[5m]))
/ sum(rate(vllm:nixl_xfer_time_seconds_count[5m]))
```

If the failure rate is sustained, investigate NIXL connectivity and hash compatibility between pods.

### Prompt token source distribution

The **vllm:prompt\_tokens\_by\_source\_total** metric tracks how decode pods receive their prompt tokens. In a healthy disaggregated deployment, decode pods receive the majority of prompt tokens through external KV transfer from prefill pods rather than through local computation.

The metric reports three sources:

- **external\_kv\_transfer**: Tokens received from prefill pods through NIXL. This should be the dominant source in disaggregated mode.
- **local\_compute**: Tokens computed locally by the decode pod. A high proportion of local compute tokens indicates that KV cache transfers are failing or that the **kv\_load\_failure\_policy** is set to **recompute**.
- **local\_cache\_hit**: Tokens served from the decode pod's local KV cache from a previous request.

Use the following query to check the distribution:

```
sum by(source) (rate(vllm:prompt_tokens_by_source_total[5m]))
```

If **local\_compute** exceeds **external\_kv\_transfer** on decode pods, investigate NIXL transfer failures and verify that the **NixlConnector** configuration is correct.

### Independent scaling decisions

In disaggregated deployments, you can scale each phase independently. Use the following metric signals to determine when to scale each pool:

- **Scale prefill up** when **vllm:num\_requests\_waiting** on prefill pods consistently exceeds 5 requests per pod over a sustained period.
- **Scale decode up** when **vllm:kv\_cache\_usage\_perc** on decode pods exceeds 85% and **vllm:num\_requests\_waiting** on decode pods is greater than zero.
- **Scale prefill down** when prefill pods show consistently low utilization with **vllm:num\_requests\_running** near zero and no queued requests.

When scaling, consider maintaining at least 4 replicas per side if you need to perform rolling updates. With fewer than 4 replicas, rolling updates can deadlock because of GPU-exclusive resource scheduling. For deployments that do not require zero-downtime updates, 2 replicas per side is sufficient.

## 17.12. CONFIGURE DISTRIBUTED TRACING FOR DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENTS

Configure distributed tracing for LLM distributed inference (Distributed Inference with llm-d) deployments using the Red Hat build of Tempo. Distributed tracing provides end-to-end visibility into inference requests, enabling performance debugging, latency analysis, and system health monitoring across the inference scheduler, routing sidecar, and vLLM model server.

### 17.12.1. Distributed tracing architecture for Distributed Inference with llm-d

Distributed tracing for Distributed Inference with llm-d records request flows across the gateway, inference scheduler, routing sidecar, and vLLM model server to help you identify latency and troubleshoot inference workloads.

### 17.12.1.1. Four-component tracing pipeline

The distributed tracing pipeline consists of four components:

#### vLLM model server instrumentation

The vLLM model server emits trace data using the OpenTelemetry SDK.

#### Inference scheduler instrumentation

The inference scheduler emits trace data using the OpenTelemetry SDK. The scheduler creates spans for request routing decisions and forwards requests with trace context headers to enable parent-child span relationships across components.

#### Routing sidecar instrumentation

The routing sidecar emits traces for disaggregated inference operations, including prefill-decode coordination and KV cache transfers between pods. The sidecar propagates trace context to ensure end-to-end visibility across distributed inference stages.

#### Tempo storage with Jaeger UI

Tempo receives traces directly from the inference components, stores trace data with configurable retention policies, and exposes the Jaeger Query Frontend for searching, visualizing, and analyzing traces.

All instrumented components export traces directly to Tempo over gRPC using OTLP. == Data flow

Traces flow through the system in the following sequence:

1. A client sends an inference request to the Gateway endpoint over HTTPS.
2. The inference scheduler receives the request, creates a child span, selects a vLLM replica, and forwards the request with trace context headers.
3. In disaggregated prefill and decode deployments, the routing sidecar coordinates request handling and KV cache transfer between the prefill and decode pods. Its spans record these operations for each inference request.
4. The vLLM model server receives the request, creates child spans for tokenization and model inference operations, and returns the response.
5. All components export their spans directly to Tempo over gRPC (port 4317) using OTLP.
6. Tempo ingests the spans and stores them in the configured backend. You can use in-memory storage for development and testing or supported object storage for persistent production storage.
7. Users access the Jaeger UI to search for traces by service name, time range, or trace ID.

### 17.12.1.2. Span hierarchy

A typical inference request generates the following span structure:

```
Root Span:
├── Child Span: inference-server-decode
│ └── Child Span: llm_request
```

Each span includes timing data, service name, and trace context. This hierarchy enables latency analysis at each stage of the inference pipeline.



### 17.12.1.3. Why distributed tracing matters for LLM inference

Distributed tracing addresses key observability challenges in LLM inference deployments:

#### Latency diagnosis

Identifies which component contributes most to request latency across the gateway, scheduler, and vLLM model server.

#### Bottleneck identification

Reveals slow operations within vLLM (tokenization, model forward pass, tensor operations).

#### Request flow visualization

Shows the complete path of inference requests through distributed components.

#### Error correlation

Links errors across services by trace ID, simplifying troubleshooting.

#### Performance tuning

Provides data for optimizing scheduling decisions, model loading, and GPU utilization.

## 17.12.2. Install distributed tracing prerequisite Operators

Install the Red Hat build of Tempo Operator required for distributed tracing support in Distributed Inference with llm-d deployments. The Tempo Operator manages the lifecycle of trace storage and query components.

### Prerequisites

- You have an OpenShift Container Platform 4.19.9+ cluster.
- You have **cluster-admin** access.
- You have installed the OpenShift CLI (`oc`).
- You have configured access to your cluster.

### Procedure

1. Install the Red Hat build of Tempo Operator:
  - a. On OpenShift 4.20 and later, install from the **software catalog**
    - On OpenShift 4.19, install from the **OperatorHub**.
  - b. Search for **Red Hat build of Tempo**
  - c. Select the Operator from the **redhat-operators** catalog.
  - d. Click **Install**.
  - e. On the **Install Operator** page, accept the default settings and click **Install**.
2. Verify the Tempo Operator installation:

```
$ oc get csv -n openshift-operators | grep tempo
```

## Verification

- The Tempo Operator CSV shows **Succeeded** status.
- The Tempo Operator pod is running in the **openshift-operators** namespace:

```
$ oc get pods -n openshift-operators | grep tempo
```

## Additional resources

- [Enabling Distributed Inference with llm-d](#)

### 17.12.3. Deploy a Tempo instance for trace storage

You can store and inspect distributed traces from Distributed Inference with llm-d deployments by configuring a **TempoMonolithic** instance with a Jaeger user interface.

## Prerequisites

- You have installed the Red Hat build of Tempo Operator.
- You have **cluster-admin** access.
- You have installed the OpenShift CLI (`oc`).

## Procedure

1. Create a namespace for the Tempo deployment:

```
$ oc new-project tempo-system
```

2. Save the following YAML to a file named **tempo-monolithic.yaml**. Create a **TempoMonolithic** custom resource:

```
apiVersion: tempo.grafana.com/v1alpha1
kind: TempoMonolithic
metadata:
 name: tempo-llmd
 namespace: tempo-system
spec:
 storage:
 traces:
 backend: memory
 jaegerui:
 enabled: true
 route:
 enabled: true
 ingestion:
 otlp:
 grpc:
 enabled: true
 http:
 enabled: true
```

where:

### **storage.traces.backend**

Specifies the trace storage backend. The value **memory** provides temporary, in-memory storage for development and testing.

### **jaegerui.enabled**

Enables the Jaeger Query Frontend for trace visualization.

### **jaegerui.route.enabled**

Creates a **Route** for external access to the Jaeger user interface.

### **ingestion.otlp.grpc.enabled**

Enables ingestion of telemetry data by using OTLP over gRPC.

### **ingestion.otlp.http.enabled**

Enables ingestion of telemetry data by using OTLP over HTTP.



### **NOTE**

The **memory** backend does not persist traces after the Tempo pod restarts. For production environments, configure a supported persistent storage backend.

3. Apply the **TempoMonolithic** custom resource:

```
$ oc apply -f tempo-monolithic.yaml
```

4. Wait for the Tempo instance to be ready:

```
$ oc get tempomonolithic tempo-llmd -n tempo-system -w
```

Wait until the **READY** status shows **True**.

5. Retrieve the Jaeger user interface route URL:

```
$ oc get route -n tempo-system
```

Example output:

| NAME                | HOST/PORT                                         |
|---------------------|---------------------------------------------------|
| tempo-llmd-jaegerui | tempo-llmd-jaegerui-tempo-system.apps.example.com |

## **Verification**

1. Verify that the **TempoMonolithic** custom resource shows **READY=True**:

```
$ oc get tempomonolithic tempo-llmd -n tempo-system
```

The output shows **READY** status as **True**.

2. Verify that Tempo pods are running in the **tempo-system** namespace:

—

```
$ oc get pods -n tempo-system
```

All pods show **Running** status.

3. Verify that the Jaeger user interface route is accessible:

```
$ oc get route tempo-llmd-jaegerui -n tempo-system -o jsonpath='{.spec.host}'
```

The output shows the route hostname.

#### 17.12.4. Enable distributed tracing for LLMInferenceService

Enable distributed tracing for an **LLMInferenceService** by using the declarative **spec.tracing** API. Distributed tracing provides end-to-end visibility into inference request flows from the Gateway through the scheduler to the vLLM model server.

##### Prerequisites

- You have deployed a Tempo instance for trace storage. For more information, see [Deploy a Tempo instance for trace storage](#).
- You have an **LLMInferenceService** resource deployed.
- You have installed the OpenShift CLI (``oc``).
- You have cluster administrator access.

##### Procedure

1. Save the following YAML to a file named **llminferenceservice-tracing.yaml**. Create an **LLMInferenceService** custom resource with distributed tracing enabled:

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
 name: traced-llm-inference
 namespace: llm-inference
spec:
 tracing:
 exporterEndpoint: "http://tempo-tracing:4317"
 sampler: "parentbased_traceidratio"
 samplerArg: "0.05"
 exporter: "otlp"
 model:
 uri: hf://Qwen/Qwen2.5-7B-Instruct
 name: Qwen/Qwen2.5-7B-Instruct
 replicas: 1
 router:
 scheduler: {}
 route: {}
 gateway: {}
 template:
 containers:
 - name: main
 image: registry.redhat.io/rhaii-early-access/vllm-cuda-rhel9:3.5.0-ea.1-1780065492
```

```

imagePullPolicy: Always
resources:
 limits:
 cpu: "4"
 memory: 32Gi
 nvidia.com/gpu: "1"
 requests:
 cpu: "2"
 memory: 16Gi
 nvidia.com/gpu: "1"

```

where:

### **spec.tracing.exporterEndpoint**

Specifies the Tempo endpoint for trace export. Use the cluster-internal service DNS name **tempo-tracing:4317**.

### **spec.tracing.sampler**

Specifies the sampling strategy. Use **parentbased\_traceidratio** for probabilistic sampling.

### **spec.tracing.samplerArg**

Specifies the sampling rate as a ratio between 0 and 1. A value of **0.05** captures 5% of traces. For production deployments, use 1% to 5% to reduce trace volume and storage costs.

### **spec.tracing.exporter**

Specifies the trace export protocol. Use **otlp** for OTLP over gRPC.

2. Apply the **LLMInferenceService** custom resource:

```
$ oc apply -f llminferenceservice-tracing.yaml
```

3. Wait for the **LLMInferenceService** resource to be ready:

```
$ oc get llmsvc traced-llm-inference -n llm-inference -w
```

Wait until **READY=True**.

4. Verify that the tracing configuration is applied:

```
$ oc get llmsvc traced-llm-inference -n llm-inference -o jsonpath='{.spec.tracing}'
```

## **Verification**

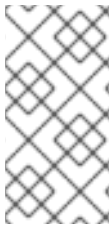
1. Verify that the **LLMInferenceService** resource shows **READY=True**:

```
$ oc get llmsvc traced-llm-inference -n llm-inference
```

2. Verify that the scheduler and vLLM pods are running:

```
$ oc get pods -n llm-inference
```

All pods show **Running** status.

**NOTE**

The configuration uses **parentbased\_traceidratio** sampling with a 5% sampling rate (**samplerArg: "0.05"**). This balances observability with trace volume and storage costs. For higher-traffic production deployments, consider reducing the sampling rate to 1-2%. For more information, see [Configure distributed tracing sampling rates](#).

**Next steps**

- [Verify distributed tracing deployment](#)

**17.12.5. Distributed tracing sampling strategies**

OpenTelemetry sampling strategies control what percentage of traces are captured and stored. Choosing the right sampling approach balances observability coverage with performance overhead and storage costs.

Understanding sampling strategies helps you configure appropriate trace capture rates for development and production environments.

**17.12.5.1. Why sample traces**

Sampling reduces the volume of trace data without completely disabling observability:

**Performance impact**

Capturing every trace adds CPU overhead for span creation, serialization, and network export. Sampling reduces this overhead proportionally.

**Storage costs**

Storing all traces in high-traffic environments requires significant object storage capacity. Sampling reduces storage requirements and costs.

**Network overhead**

Exporting traces to the OpenTelemetry Collector consumes network bandwidth. Sampling reduces outbound traffic from inference pods.

**Query performance**

Searching through millions of traces degrades Jaeger UI responsiveness. Sampling keeps trace volume within manageable limits.

**17.12.5.2. Sampling strategies**

OpenTelemetry supports multiple sampling strategies, each suited to different use cases.

**17.12.5.2.1. Always-on sampling**

Always-on sampling captures 100% of traces.

**Use case**

Development, debugging, and low-traffic environments where complete observability is required.

**Performance impact**

High storage and network overhead. All inference requests generate trace data. For low request volumes, the performance overhead might be acceptable.

**Configuration**

```
tracing:
 sampler: parentbased_always_on
```

#### 17.12.5.2.2. Probabilistic sampling

Probabilistic sampling captures a configurable percentage of traces based on trace ID hash.

##### Use case

Production environments with moderate traffic where statistical sampling provides sufficient coverage.

##### Sampling rate

Configured as a decimal between 0.0 and 1.0. For example, **0.1** means 10% of traces are sampled.

##### Consistent sampling

The same trace ID always produces the same sampling decision, ensuring complete traces (not partial).

##### Configuration

```
tracing:
 sampler: "parentbased_traceidratio"
 samplerArg: "0.1"
```

#### 17.12.5.2.3. Parent-based sampling

Parent-based samplers make sampling decisions based on whether the parent span was sampled:

##### parentbased\_always\_on

If the parent span was sampled, this span is sampled. If there is no parent span, the trace is always sampled.

##### parentbased\_always\_off

If the parent span was sampled, this span is sampled. If there is no parent span, the trace is not sampled.

##### parentbased\_traceidratio

If the parent span was sampled, this span is sampled. If there is no parent span, probabilistic sampling is used based on trace ID.

This ensures trace completeness across distributed components.

#### 17.12.5.3. Coordinating sampling across components

Both the inference scheduler and vLLM should use the same sampling strategy and rate to ensure trace completeness:

```
spec:
 tracing:
 sampler: "parentbased_traceidratio"
 samplerArg: "0.05"
```

Mismatched sampling rates can result in incomplete traces where some spans are missing.

## Additional resources

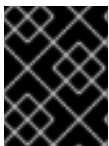
- [Configure distributed tracing sampling rates](#)
- [Distributed tracing sampling parameters](#)

### 17.12.6. Configure distributed tracing sampling rates

Configure OpenTelemetry trace sampling rates to balance observability needs with performance overhead and storage costs. Probabilistic sampling reduces trace volume in production deployments while maintaining statistical coverage.

#### Prerequisites

- You understand distributed tracing sampling strategies. For more information, see [Distributed tracing sampling strategies](#).
- You have deployed an **LLMInferenceService** resource with distributed tracing enabled.
- You have installed the OpenShift CLI (``oc``).
- You have cluster administrator access.



#### IMPORTANT

Configure the scheduler and vLLM components with the same sampling settings. Different settings can produce incomplete traces with missing spans.

#### Procedure

1. Choose a sampling strategy for your environment:
  - **Development and testing:** Use **parentbased\_always\_on** to sample all traces.
  - **Production:** Use **parentbased\_traceidratio** with an initial sampling rate of 1% to 5%.
2. Save the following YAML to a file named **llminferenceservice-tracing.yaml**. Update the **LLMInferenceService** resource to use probabilistic sampling. The following example configures a 5% sampling rate:

```
apiVersion: serving.kserve.io/v1alpha2
kind: LLMInferenceService
metadata:
 name: traced-llm-inference
 namespace: llm-inference
spec:
 tracing:
 exporterEndpoint: "http://tempo-tracing:4317"
 sampler: "parentbased_traceidratio"
 samplerArg: "0.05"
 exporter: "otlp"
```

3. Apply the configuration:

```
$ oc apply -f llminferenceservice-tracing.yaml
```



#### 4. Monitor trace volume and adjust the sampling rate as needed:

##### a. Check Tempo storage usage:

```
$ oc exec -n tempo-system <tempo_pod> -- df -h
```

where:

```
__<tempo_pod>__
```

Specifies the name of the Tempo pod.

##### b. In the Jaeger UI, review the number and distribution of sampled traces.

##### c. Adjust the **samplerArg** value based on observed trace volume.

### Verification

#### 1. Confirm that the sampling configuration is present in the **LLMInferenceService** resource:

```
$ oc get llminferenceservice traced-llm-inference -n llm-inference -o yaml
```

#### 2. Verify that the **spec.tracing.sampler** field is set to **parentbased\_traceidratio** and that the **spec.tracing.samplerArg** field is set to **"0.05"**.

#### 3. Generate inference requests and confirm that traces appear in the tracing interface at approximately the configured sampling rate.



### NOTE

Each sampled trace adds approximately 1-2 KB of data and minimal CPU overhead for trace context propagation. Monitor vLLM and scheduler CPU and memory usage when enabling tracing.

### Additional resources

- [Distributed tracing sampling parameters](#)

## 17.12.7. Verify distributed tracing deployment

Verify that distributed tracing is working correctly by generating test inference traffic and viewing traces in Jaeger UI. Successful verification confirms that traces flow end-to-end from the inference components to Tempo storage.

### Prerequisites

- You have deployed a Tempo instance for trace storage. For more information, see [Deploy a Tempo instance for trace storage](#).
- You have enabled distributed tracing for your **LLMInferenceService** and the resource shows **READY=True**. For more information, see [Enable distributed tracing for LLMInferenceService](#).
- The Jaeger user interface is accessible through a route or port forwarding.

## Procedure

1. Send a test inference request to generate trace data:

```
$ SERVICE_URL=$(oc get llm_svc <llm_svc_name> -n <namespace> \
 -o jsonpath={.status.url})

$ curl -X POST "${SERVICE_URL}/v1/chat/completions" \
 -H "Content-Type: application/json" \
 -d { "model": "<model_name>", "messages": [{"role": "user", "content": "What is
 Kubernetes?"}], "max_tokens": 100 }
```

where:

- **<llm\_svc\_name>**: Specifies the name of your tracing-enabled **LLMInferenceService** resource.
- **<namespace>**: Specifies the namespace where your **LLMInferenceService** is deployed.
- **<model\_name>**: Specifies the model name as defined in your **LLMInferenceService** resource.



### NOTE

If you configured trace sampling, send multiple requests to ensure at least one trace is captured. The number of requests needed is **1 / sample\_rate**. For example, with a 5% sample rate, send at least 20 requests to see a trace in Jaeger UI.

2. Access the Jaeger UI by using port forwarding:

```
$ oc port-forward -n tempo-system svc/tempo-llmd-query-frontend 16686:16686
```

3. Open a web browser to **http://localhost:16686**.

4. Search for traces:

- a. In the **Service** dropdown, select the model server service or the EPP service. Look for service names such as **inference-server-decode** for the vLLM model server or **llm-d-epp** for the Endpoint Picker.
- b. Set the **Lookback** time to **Last 15 minutes**.
- c. Click **Find Traces**.

5. Examine a trace to verify end-to-end spans:

- a. Click a trace from the search results.
- b. Verify the trace has the following spans:
  - Scheduler span showing request routing
  - vLLM span showing model inference
- c. Check that all spans share the same Trace ID.

- d. Verify span attributes include service names.
6. Verify trace context propagation:
    - a. Expand each span in the trace view.
    - b. Verify parent-child relationships between spans.
    - c. Confirm span timing data is accurate.

### 17.12.8. Troubleshoot distributed tracing errors

Diagnose and resolve common issues with distributed tracing for Distributed Inference with llm-d deployments. This reference covers the most frequently encountered problems and their solutions.

#### 17.12.8.1. No traces appearing in Jaeger UI

##### Symptoms:

- Inference requests succeed but no traces appear in Jaeger UI
- Jaeger UI shows no services or traces

##### Diagnosis:

1. Check Tempo logs for receiver errors:

```
$ oc logs -n tempo-system -l app.kubernetes.io/name=tempo-monolithic | grep -i error
```

2. Verify the **spec.tracing** configuration:

```
$ oc get llm_svc traced-llm-inference -n llm-inference -o yaml | grep -A 4 tracing:
```

3. Check network connectivity from vLLM to Tempo:

```
$ oc exec -n llm-inference <vllm-pod> -- \
 curl -v http://tempo-tracing:4317
```

##### Resolution:

- If the **spec.tracing** configuration is missing, update the `LLMInferenceService` custom resource with the correct tracing parameters.
- If network connectivity fails, check `NetworkPolicy` resources and ensure Tempo is running.

### 17.12.9. Distributed tracing sampling parameters

OpenTelemetry sampling parameters control which traces an **LLMInferenceService** resource records and exports, enabling you to manage trace volume across Distributed Inference with llm-d components.

#### 17.12.9.1. sampler field

| Sampler value                   | Description                                                            | Use case                                                                      |
|---------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>always_on</b>                | Samples 100% of traces.                                                | Development, debugging, low-traffic environments.                             |
| <b>always_off</b>               | Does not sample any traces.                                            | Temporarily stop trace collection without removing the tracing configuration. |
| <b>traceidratio</b>             | Samples based on trace ID hash (probabilistic).                        | Production environments requiring consistent sampling.                        |
| <b>parentbased_always_on</b>    | Always samples if parent span is sampled, otherwise always samples.    | Ensures complete traces when distributed tracing spans services.              |
| <b>parentbased_traceidratio</b> | Always samples if parent span is sampled, otherwise uses traceidratio. | Production environments with consistent sampling across services.             |

Use **parentbased\_traceidratio** for production Distributed Inference with llm-d deployments to ensure consistent sampling decisions across scheduler and vLLM components.

#### 17.12.9.2. samplerArg field

For **traceidratio** and **parentbased\_traceidratio** samplers:

| Value        | Sampling rate | Description                              |
|--------------|---------------|------------------------------------------|
| <b>1.0</b>   | 100%          | Samples all traces.                      |
| <b>0.5</b>   | 50%           | Samples approximately 1 in 2 traces.     |
| <b>0.1</b>   | 10%           | Samples approximately 1 in 10 traces.    |
| <b>0.01</b>  | 1%            | Samples approximately 1 in 100 traces.   |
| <b>0.001</b> | 0.1%          | Samples approximately 1 in 1,000 traces. |
| <b>0.0</b>   | 0%            | Does not sample any traces.              |

Format: Decimal value between **0.0** and **1.0** (inclusive).

### 17.12.9.3. Configuration example

The following example configures a 5% parent-based sampling rate in the **spec.tracing** section of an **LLMInferenceService** custom resource:

```
spec:
 tracing:
 sampler: "parentbased_traceidratio"
 samplerArg: "0.05"
```

#### Additional resources

- [Configure distributed tracing sampling rates](#)
- [Distributed tracing sampling strategies](#)
- [OpenTelemetry sampling specification](#)

#### Additional resources

- [Enabling Distributed Inference with llm-d](#)
- [vLLM OpenTelemetry documentation](#)
- [Red Hat OpenShift distributed tracing platform documentation](#)
- [Tempo documentation](#)

## CHAPTER 18. COLLECT DISTRIBUTED INFERENCE WITH LLM-D DIAGNOSTIC DATA FROM OPENSHIFT

You can collect comprehensive diagnostic data from your Distributed Inference with llm-d deployment on OpenShift by running the **oc adm must-gather** command with the OpenShift AI must-gather image. You can gather cluster configuration, component status, logs, and metrics in a single diagnostic bundle to send to Red Hat Support for case analysis.

### Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- The Distributed Inference with llm-d application and its dependencies are deployed on the cluster.

### Procedure

1. Run the must-gather command with the OpenShift AI must-gather image:

```
$ oc adm must-gather \
 --image=registry.redhat.io/rhoai/odh-must-gather-rhel9:v3.5 \
 -- "export COMPONENT=llm-d; /usr/bin/gather"
```

The command creates a directory in the current path that contains the diagnostic bundle.

2. Optional: To also collect Workload Variant Autoscaler (WVA) diagnostics, add the **ENABLE\_WVA=true** environment variable:

```
$ oc adm must-gather \
 --image=registry.redhat.io/rhoai/odh-must-gather-rhel9:v3.5 \
 -- "export COMPONENT=llm-d; export ENABLE_WVA=true; /usr/bin/gather"
```

3. Optional: To collect observability stack diagnostics, specify the monitoring namespace:

```
$ oc adm must-gather \
 --image=registry.redhat.io/rhoai/odh-must-gather-rhel9:v3.5 \
 -- "export COMPONENT=llm-d; export MONITORING_NAMESPACE=redhat-ods-monitoring; /usr/bin/gather"
```

4. Locate the must-gather output directory:

```
$ ls -d must-gather.local.*
```

5. Create a compressed archive of the diagnostic bundle:

```
$ tar -czf must-gather.tar.gz must-gather.local.*
```

6. Attach the **must-gather.tar.gz** file to your Red Hat Support case.

### Verification

- Verify the diagnostic bundle contents:

```
$ tar -tzf must-gather.tar.gz | head -20
```

The output should list diagnostic files organized by collection category, including cluster configuration, component status, logs, and metrics.

- Extract and inspect the diagnostic bundle structure:

```
$ mkdir -p must-gather-output
$ tar -xzf must-gather.tar.gz -C must-gather-output
$ ls -R must-gather-output
```

## CHAPTER 19. TROUBLESHOOT DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENT ISSUES

Diagnose and resolve issues with Distributed Inference with llm-d model deployments by verifying component configurations and connectivity.

### Prerequisites

- You have cluster administrator permissions for OpenShift.
- You have the OpenShift CLI (`oc`) installed.
- You have Distributed Inference with llm-d enabled on your cluster.
- You have installed Red Hat Connectivity Link on your cluster.

### Procedure

1. Verify the Kuadrant object exists in Red Hat Connectivity Link:

```
$ oc get kuadrant -n kuadrant-system
```

The command returns a Kuadrant instance:

```
NAME AGE
kuadrant 10m
```

2. Verify the Authorino object configuration and service annotation:

- a. Check the Authorino object:

```
$ oc get authorino -n kuadrant-system -o yaml
```

Verify that TLS is enabled in the **listener.tls** section:

```
spec:
 listener:
 tls:
 enabled: true
 certSecretRef:
 name: authorino-server-cert
```

- b. Check the Authorino service annotation:

```
$ oc get svc authorino-authorino-authorization -n kuadrant-system -o yaml | grep serving-
cert-secret-name
```

The command returns the annotation:

```
service.beta.openshift.io/serving-cert-secret-name: authorino-server-cert
```

3. Restart the controller pods:



If configuration changes do not take effect, restart the controller pods:

```
$ oc delete pod -n redhat-ods-applications -l app=odh-model-controller
$ oc delete pod -n redhat-ods-applications -l control-plane=kserve-controller-manager
```

4. Verify the global authentication policy was created:

```
$ oc get authpolicy -n openshift-ingress
```

The command returns the global **AuthPolicy** custom resource:

```
NAME AGE
openshift-ai-inference-authn 10m
```

## Verification

1. Verify the LeaderWorkerSet Operator is installed and available:
  - a. In the web console, navigate to **Ecosystem → Installed Operators**.
  - b. Select the **openshift-lws-operator** project from the **Project** list.
  - c. Verify that the LeaderWorkerSet Operator shows a green checkmark with **Condition: Available** status.
2. Verify Red Hat Connectivity Link components are installed:
  - a. In the web console, navigate to **Ecosystem → Installed Operators**.
  - b. Select the **kuadrant-system** project from the **Project** list.
  - c. Verify that all four operators show green checkmarks with **Succeeded, Up to date** status:
    - Authorino Operator
    - DNS Operator
    - Limitador
    - Red Hat Connectivity Link
3. Verify the Kuadrant instance is ready:
  - a. In the web console, navigate to **Ecosystem → Installed Operators**.
  - b. Select the **kuadrant-system** project from the **Project** list.
  - c. Click **Red Hat Connectivity Link**
  - d. Click the **Kuadrant** tab.
  - e. Verify that the **kuadrant** instance shows **Condition: Ready** with a green checkmark.
4. Verify the Authorino service annotation exists:
  - a. In the web console, navigate to **Networking → Services**.

- b. Select the **kuadrant-system** project from the **Project** list.
- c. Click the **authorino-authorino-authorization** service.
- d. Verify that the annotation **service.beta.openshift.io/serving-cert-secret-name: authorino-server-cert** exists.

#### Additional resources

- [Cluster administrator prerequisites for Distributed Inference with llm-d](#)
- [Configuring authentication for Distributed Inference with llm-d](#)

## 19.1. COMMON ISSUES WITH DISTRIBUTED INFERENCE WITH LLM-D DEPLOYMENTS

Reference guide for common issues and solutions when deploying Distributed Inference with llm-d models.

### 19.1.1. Common issues and solutions

| Issue                                   | Possible cause                                  | Solution                                                                                                                                                                                                                                                  |
|-----------------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication not working              | Components configured in wrong order            | Verify you followed the cluster administrator prerequisites in the correct order. If not, delete and recreate resources following the documented sequence.                                                                                                |
| HTTP 403 Forbidden errors               | Missing or incorrect RBAC permissions           | Verify the service account has <b>get</b> permission on the <b>LLMInferenceService</b> resource. Run <b>oc auth can-i get llminferenceservices/&lt;model-name&gt; -n &lt;namespace&gt; --as=system:serviceaccount:&lt;namespace&gt;:&lt;sa-name&gt;</b> . |
| Gateway not routing traffic             | Incorrect namespace allowedRoutes configuration | Review the Gateway <b>allowedRoutes</b> section. Ensure your model deployment namespace is listed in the <b>values</b> array.                                                                                                                             |
| Model deployment stuck in pending state | LeaderWorkerSet Operator not configured         | Verify the LeaderWorkerSet Operator is installed and the <b>cluster</b> instance shows <b>Condition: Available</b> . See the cluster administrator prerequisites.                                                                                         |

| Issue                                                                   | Possible cause                                    | Solution                                                                                                                                                                                                        |
|-------------------------------------------------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No external access to model                                             | Gateway or GatewayClass missing                   | Verify the GatewayClass and Gateway resources exist and show <b>Ready</b> status. Run <b>oc get gatewayclass openshift-ai-inference</b> and <b>oc get gateway openshift-ai-inference -n openshift-ingress</b> . |
| AuthPolicy not created for model                                        | Red Hat Connectivity Link not properly configured | Follow the debugging checklist to verify Kuadrant and Authorino configuration. Check that the Kuadrant instance shows <b>Condition: Ready</b> .                                                                 |
| Token authentication not working after manual ServiceAccount creation   | Missing required annotations or labels            | Verify the Secret has the <b>kubernetes.io/service-account.name</b> annotation and the Role, RoleBinding, and Secret have the <b>opendatahub.io/dashboard: "true"</b> label.                                    |
| Models become inaccessible after Red Hat Connectivity Link installation | Prerequisite steps not completed in order         | Delete and recreate the Gateway, Kuadrant instance, and Authorino configuration following the documented order. Restart controller pods after recreating resources.                                             |

### Additional resources

- [Cluster administrator prerequisites for Distributed Inference with llm-d](#)
- [Configuring authentication for Distributed Inference with llm-d](#)